地缘冲突下国际油价波动预测、宏观传导与中国政策韧性研究

摘　　要

国际原油价格同时受到供需基本面、金融条件和地缘政治风险驱动，其冲击会沿进口成本、生产价格和居民消费价格等渠道向国内经济传导。本文围绕油价预测、冲突影响识别、中国宏观传导、政策缓冲和尾部风险压力测试构建四层分析框架。

写作提示：摘要应在正文定稿后重写。依次概括问题一至问题四的模型、关键数值结论与创新点，不得只重复题目；所有数值从冻结结果读取。本提示框和所有示例文字须在提交前删除。

关　键　词：国际油价　地缘冲突　结构冲击　政策韧性　压力测试

一、 问题重述与研究目标

国际原油价格由供需基本面、美元与库存条件、市场预期和地缘政治风险共同决定。冲突发生后，供给中断预期和航运风险会先进入日度油价与波动率，再经汇率、进口成本和国内价格机制传导至PPI、CPI与工业活动。因此，题目中的“预测”“战争影响”“宏观传导”和“政策效果”不能由同一个统计量替代，必须在统一数据链上分别识别。

题面未给出“美伊战争”的精确边界。本文将其操作性定义为：2026年2月28日开始的中东军事行动及其引致的霍尔木兹海峡运输中断；2026年6月17日通行恢复谅解作为缓和阶段起点。2月28日为非交易日，事件研究映射到首个共同交易日2026年3月2日；3月5日再单列航运受阻阶段。3月23日和4月7日的国内临时调控属于政策事件，不与外部冲突合并编码。

据此，本文研究四个相互衔接的问题：

1. 以Brent现货价为主指标，在扩展窗口中比较未来1、3、6个月预测；以日度累计异常收益、状态匹配安慰剂和条件波动率识别冲突期异常，并用结构VAR分离供给、全球需求和石油特定风险冲击。

2. 以第一问的结构冲击为输入，估计人民币原油成本、PPI、CPI、工业增加值和汇率的响应方向、幅度、时滞及不确定性；季度GDP只作低频验证，不插值为月度数据。

3. 在可比数据边界内考察六个主要进口国的零售燃油传导，以及中国相对对照国的CPI与工业活动响应；再以中国受管制汽油调价指数代理构造“关闭2026年临时调控”的动态反事实。

4. 利用“包括但不限于”的开放性，估计极端油价条件尾部概率，并设计带自动追补与缺口硬约束的状态自适应价格平滑规则，为冲击期调价提供可检验的多目标方案。

本文区分四个时间概念：市场信息截止日为2026年6月30日；不同统计序列的实际观测末期由其发布滞后决定；数据发布或版本日期按来源单独记录；下载核验发生在2026年7月30日至31日。预测与回测只能使用对应原点当时可得的信息，不能把下载日误当作统一观测末期。

二、 问题分析与总体思路

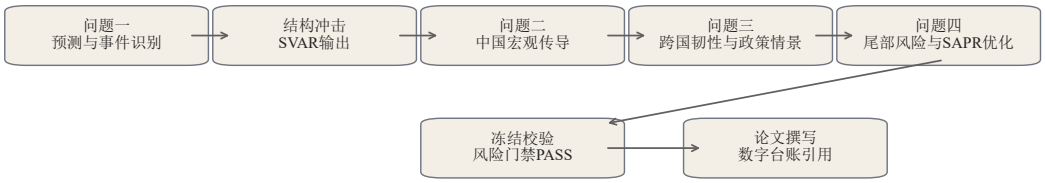
2.1 四问的证据分工

第一问先检验模型能否降低样本外误差，再检验事件期是否偏离战前规律；预测残差和描述性基准差额均不能直接解释为战争净因果贡献。同一次油价上涨还可能来自供

给收缩、全球需求扩张或预防性需求，故进一步采用Kilian结构分解[1]，把具有经济含义的冲击送入第二问。

第二问采用变量特定的局部投影：同比变量直接估计未来水平响应，汇率估计相对冲击前基期的累计对数变化。只有完整月度路径才计算响应曲线面积，结构性稀疏的工业增加值期限不作伪累计。第三问先回答“数据允许我们比较什么”，再评价政策：六国官方零售燃油价用于正式传导排名；中国历史序列是调价指数代理，只作敏感性和政策路径构造。未经审计的年度缓冲代理不进入定量回归。第四问将油价分布、宏观压力与政策规则组织为相互关联但不可相加的三层证据。

总体路线见图1。



来源：作者根据冻结模型流程绘制。

图1　国际油价冲击研究的总体技术路线

2.2　数据结构与口径

月度主样本始于2010年1月。Brent、WTI、广义美元指数和人民币汇率由公开日度序列聚合；美国商业原油库存使用月末前最后一个已发布周值；GPR使用固定版本月度指数[2]。中国PPI、CPI和规模以上工业增加值来自国家统计局，工业增加值1至2月合并发布导致的空值原样保留。人民币原油成本由Brent美元价与人民币兑美元汇率相乘构造。

跨国模块包含中国、德国、法国、意大利、西班牙、日本和韩国。欧洲、日本和韩国使用官方可比的零售汽油价格；中国公开历史调价表被重建为“受管制汽油调价指数代理”，不冒充固定标准品零售价格。CPI统一转为同比，工业活动保留可比同比口径。核心数据见表1。

2.3　求解与验证策略

问题一以no-change为可用基线，将ARIMA、SARIMAX、ETS、Theta和等权组合置于相同扩展窗口；事件层采用状态匹配安慰剂，结构层执行平稳性、稳定性、残差和排序敏感性检查。问题二使用局部投影，以ARDL、冲击正负拆分、替代排序和季度GDP作边界验证。问题三采用分国分布滞后、相对中国的双固定效应面板及动态政策关闭反事

表1 核心数据、频率和时间口径

数据模块	来源	原始频率	处理和证据用途
Brent、WTI、汇率	EIA/FRED[20, 21]	日度	交易日事件研究；自然月有效观测均值进入月度模型
库存和GPR	EIA；GPR数据库	周/月	月末前最后已发布库存值；固定版本GPR
中国宏观指标	国家统计局[22]	月/季	月度传导；季度GDP只作低频验证
六国宏观指标	OECD及各国官方[23]	月度	CPI和工业活动相对响应
六国燃油价格	欧委会、METI、KOSIS/KNOC	周/月	官方零售价格月均值，进入正式排名
中国调价序列	公开调价表和官方公告	事件	历史调价指数代理；2026公告差额单独核验
中国价格政策	国家发展改革委[19]	事件	机制应调、实际调价和累计延期缺口

实，三条宏观方程用共同时间块联合抽样。问题四以FHS–GJR–GARCH 生成尾部分布；政策规则只在开发样本选择阈值和膝点，在2022年以后隔离检验样本用配对块bootstrap比较三类基线。所有结果经风险探针、数值冻结和正文一致性门禁后才允许进入论文。

三、 模型假设

1. **实时信息集假设。**预测原点之后的观测不进入估计、调参或阈值选择；统计序列按实际发布滞后截断。固定下载版本可能被后续修订，但不回填未来信息。

2. **交易日映射假设。**非交易日事件映射到其后首个共同交易日。短窗口仍可能叠加其他新闻，故CAR只解释为事件相关异常；常规正态区间为描述性结果，正式显著性依赖状态匹配安慰剂。

3. **递归结构识别假设。**月内全球供给响应最慢，全球真实活动可当期响应供给，实际油价可当期响应前两者。递归排序为“供给增长—全球真实活动—实际Brent收益”，并用替代排序探针检验结论依赖性。

4. **局部动态假设。**主要宏观响应在12个月内体现；PPI、CPI、IAV和原油成本的被解释量为未来水平，汇率为相对冲击前基期的累计对数变化。移动块bootstrap保留月度序列依赖。

5. **跨国可比边界。**六个对照国的官方零售燃油对数收益可以比较；中国调价指数代理与固定品零售价不可直接等同，因此不进入正式零售价格排名。面板系数只表示对照国相对中国的差异。

6. **局部政策反事实假设。**关闭临时调控时，将官方公告核验的应调与实调差额加入中国调价指数代理路径，其余宏观环境保持不变。反事实用0至6阶动态核及结果

变量AR(1)递推，属于条件情景而非一般均衡福利估计。

7. **风险和规则约束假设。**FHS可重采样截止日前标准化残差；SAPR只在三状态、六个月、离散传导率规则族内寻优。平均缺口月负担为延期成本代理，不等于财政支出；最大缺口、期末缺口、恢复期末缺口和单月调价幅度分别作为硬约束。

四、 符号说明

表2 主要符号及含义

符号	含义	单位或取值
P_t，r_t	Brent价格及对数收益$r_t=\Delta\ln P_t$	美元/桶；无量纲
$CAR[0,H]$	事件日到第H个后续交易日的累计异常收益	对数收益
s_t^S, s_t^D, s_t^R	不利供给、全球需求与石油特定风险结构冲击	标准化冲击
$R_{y,h}$	冲击对结果y在期限h的响应	百分点或对数百分点
$A_y(H)$	完整月度响应曲线面积$\sum_{h=0}^{H} R_{y,h}$	百分点月
$\Pi_{i,H}$	国家i的H月燃油传导率	无量纲
G_t	机制应调减实际调价形成的未调价缺口	元/吨
Δf_t	无调控路径相对实际路径的燃油收益差	对数百分点
$B_{y,t}$	无调控路径减实际路径的宏观差额	百分点
h_t	GJR–GARCH条件方差	日收益率平方
ρ_N, ρ_S, ρ_E	普通、压力、极端状态的调价传导率	$[0,1]$
J_1	平均宏观损失	标准化损失
J_2	宏观损失的95%条件风险价值	标准化损失
J_3	平均每月未结缺口占基准价格比例	无量纲
J_4	实际调价路径波动	无量纲

下标t表示月份或交易日，h表示预测或响应期限，i表示国家。CPI、PPI和IAV分别指居民消费价格同比、工业生产者出厂价格同比和规模以上工业增加值同比。

五、 问题一：油价预测与冲突影响识别

5.1 扩展窗口预测

油价接近随机游走且结构突变频繁，复杂模型未必稳定优于简单基线[3, 4]。本文以2010年1月至2019年12月为初始训练段，自2020年1月起逐月扩展窗口，预测期限为$h\in\{1,3,6\}$。no-change基线为

$$\widehat{\ln P}^{NC}_{t+h|t}=\ln P_t. \tag{1}$$

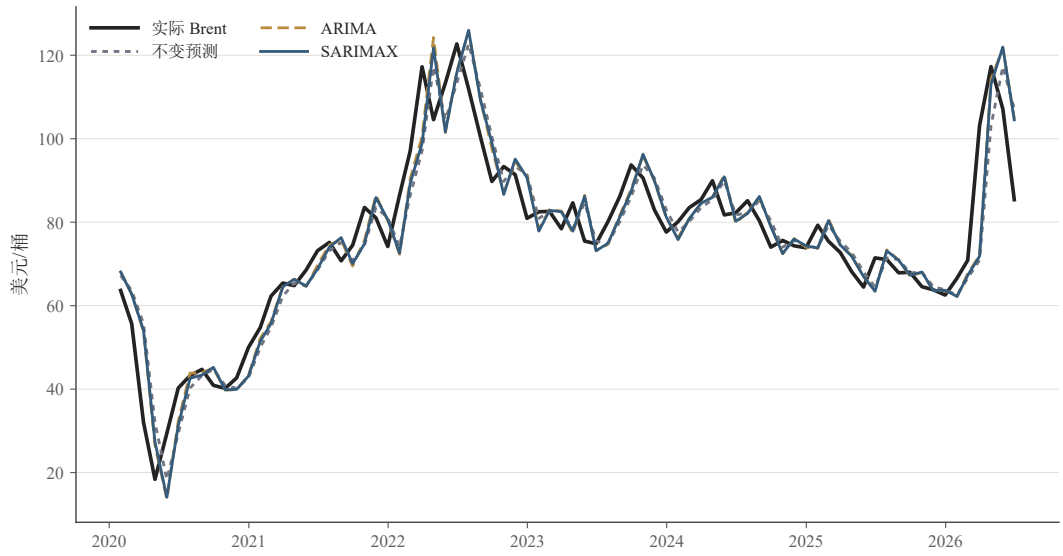
候选模型包括ARIMA、使用滞后库存、美元与GPR的SARIMAX、ETS、Theta及等权组合。对每个模型和期限计算MAE、RMSE、方向准确率及80%、95%区间覆盖率，并

用经HLN小样本修正的Diebold–Mariano检验比较配对误差[6, 7]。表3 给出全部模型相对no-change的RMSE；括号内为预注册门禁状态。

表3　扩展窗口回测的相对RMSE

模型	1个月	3个月	6个月
no-change	1.000（通过）	1.000（通过）	1.000（通过）
ARIMA	1.021（条件）	1.119（失败）	1.114（失败）
SARIMAX	1.025（条件）	1.104（失败）	1.101（失败）
ETS	1.159（失败）	1.215（失败）	1.270（失败）
Theta	1.001（条件）	1.003（条件）	1.007（条件）
等权组合	1.029（条件）	1.077（失败）	1.083（失败）

no-change在三个期限均取得最低RMSE，分别为0.1408、0.2660和0.3082，故选为主预测。以2026年2月均价70.887美元/桶为原点，三期点预测均为70.887美元/桶；2026年3月和5月实际均价为103.135和107.140美元/桶，2026年8月尚未实现，只保留为预测。该结果说明极端事件超出了常规历史信息集，不能据此改用样本外表现更差的复杂模型。



来源：FRED/EIA 处理面板；由 code/problem1/run_q1.py 生成。

图2　Brent一月期扩展窗口预测与实际值

5.2　多阶段事件研究

日历事件E1为2026年2月28日；首个共同交易日为3月2日；$CAR[0,+2]$止于3月4日；航运受阻阶段E2从3月5日开始；缓和阶段E3从6月17日开始。用E1前520个共同交易日拟合Brent收益的AR(3)并控制美元收益，定义

$$AR_t = r_t - \widehat{r}_t^{0}, \qquad CAR[0,H] = \sum_{j=0}^{H} AR_{t_0+j}. \tag{2}$$

安慰剂不再随机抽普通周末，而是在排除事件邻域后，按事件前20日波动率、Brent价格水平和当月GPR距离选择最接近的40个历史状态。经验p值采用$(b+1)/(B+1)$，避免有限样本下报告零概率。

表4　E1复合事件窗口的Brent累计异常收益

窗口	CAR	常规95%区间	状态匹配经验p值	结束日
[0]	0.0797	[0.0415, 0.1178]	0.0244	03-02
[0, +1]	0.1561	[0.1021, 0.2101]	0.0244	03-03
[0, +2]	0.1355	[0.0694, 0.2016]	0.0513	03-04

常规区间仅描述战前模型误差；正式有限样本证据以状态匹配经验p值为准。前两窗在5%水平异常，三日窗处于边界。由于军事行动、预期变化和首轮运输扰动密集重叠，表中CAR是复合事件窗口，不能再分解为单一消息的净贡献。战前AR递归路径与实际价格之差记为$ARBaselineGap$，也只表示偏离历史规律的描述性差额，不称为“无战争价格”。

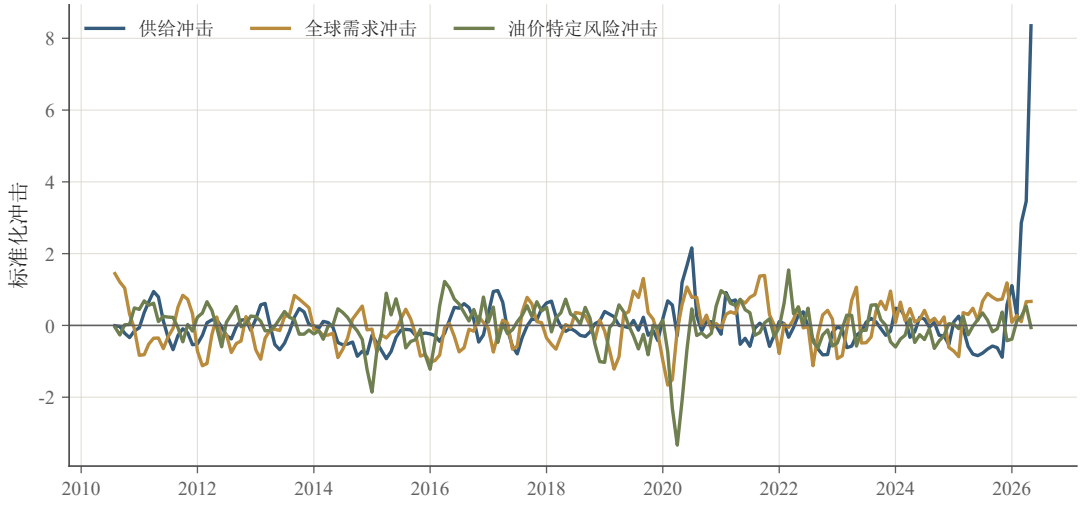
5.3　结构冲击与波动

令$z_t = (\Delta q_t, a_t, \Delta p_t^{real})^{\mathsf{T}}$，依次为全球供给增长、全球真实活动和实际Brent收益，估计递归VAR：

$$A(L)z_t = u_t, \qquad u_t = B\varepsilon_t. \tag{3}$$

三序列均通过ADF拒绝单位根且KPSS不拒绝平稳：ADF的p值分别小于0.001、0.0053和0.001。BIC选择6阶VAR，最小特征根模为1.117，残差白噪声检验$p = 0.382$，得到186个有效冲击月。为便于经济解释，将“供给冲击”统一定向为正值代表不利供给收缩。两个替代递归排序下关键价格冲击相关系数不低于0.999，说明本样本中排序探针不改变主要路径。

日度条件方差采用GJR–GARCH(1,1)[8]。战前条件波动率中位数为1.743%；E1即时窗、E2航运受阻阶段和E3缓和阶段分别为其1.587、2.214和1.566倍。航运受阻阶段压力最高，但该指标描述金融市场波动，不等同于实体损失。



来源：EIA International、Dallas Fed IGREA、Brent、美国CPI；作者计算。

图3　递归VAR识别的三类月度结构冲击

5.4　问题一小结

预测层由no-change诚实胜出；事件层支持前两交易日出现状态条件下罕见的异常上涨；结构层输出三类经济冲击，波动层显示运输受阻阶段风险最高。四者用途分离，避免把预测误差、CAR或AR基准差额误写成战争净因果溢价。

六、　问题二：油价冲击对中国经济的传导

6.1　变量特定的局部投影

人民币原油成本构造为

$$C_t^{oil} = P_t^{Brent,USD} FX_t^{CNY/USD}, \qquad c_t = 100\Delta \ln C_t^{oil}. \tag{4}$$

结构冲击的来源不同，其宏观含义也不同[9, 10, 11]。本文分别输入不利供给、全球需求和石油特定风险冲击；未分解$OilShock$只作约化形式稳健性。局部投影不对整条脉冲路径施加强递推限制[5]，但被解释变量必须与程序定义一致。对PPI、CPI、IAV和人民币原油成本，估计未来水平响应

$$y_{t+h} = \alpha_h + \theta_h s_t + \rho_h y_{t-1} + \eta_h s_{t-1} + \Gamma_h^{\top} Z_{t-1} + u_{t+h}; \tag{5}$$

对汇率则估计相对冲击前基期的累计变化

$$100(\ln FX_{t+h} - \ln FX_{t-1}) = \alpha_h + \theta_h s_t + \rho_h \Delta \ln FX_{t-1} + \eta_h s_{t-1} + \Gamma_h^{\top} Z_{t-1} + u_{t+h}. \tag{6}$$

Z_{t-1}包含美元、GPR、月份虚拟变量和疫情阶段。PPI、CPI、成本和汇率估计$h = 0, \ldots, 12$；IAV因1至2月结构性缺失，只估计$h \in \{0, 3, 6, 12\}$。使用同源移动块bootstrap和最大t统计量构造95%联合置信带。对完整月度路径定义响应曲线面积

$$A_y(H) = \sum_{h=0}^{H} \widehat{\theta}_{y,h}, \tag{7}$$

单位为“百分点月”，只描述离散响应曲线的面积，不称为累计经济损失。IAV的四个稀疏期限不求和。

6.2 油价特定风险冲击结果

表5 油价特定风险冲击的传导指标

结果变量	极值	月份	极值联合95%区间	$A(0,6)$	$A(0,12)$
人民币原油成本	8.832	0	$[6.888, 10.776]$	7.440	4.939
PPI同比	0.762	10	$[-0.159, 1.684]$	3.655	7.401
CPI同比	0.247	7	$[-0.096, 0.589]$	0.877	2.155
IAV同比	-0.427	12	$[-0.966, 0.111]$	–	–
人民币汇率	0.771	12	$[-0.488, 2.030]$	1.612	5.846

成本入口当月响应最清晰；PPI和CPI点估计滞后上行，IAV在12个月出现负向点估计。但除成本入口外，极值的联合区间均跨零，故不能将点估计写成稳健总体增长损失。季度GDP在加入所需滞后后没有足够有效观测，低频验证状态为“结论不充分”，也不再报告旧版小样本回归值。

不同结构冲击的响应方向并不相同：全球需求扩张可同时推高油价和产出，不利供给或石油特定风险更符合成本推动机制。这解释了为何“油价上涨”不能直接等同于“经济受损”。ARDL、疫情剔除、冲击正负拆分和替代排序只作为稳健性探针；它们未把工业活动和GDP证据提升为联合显著。

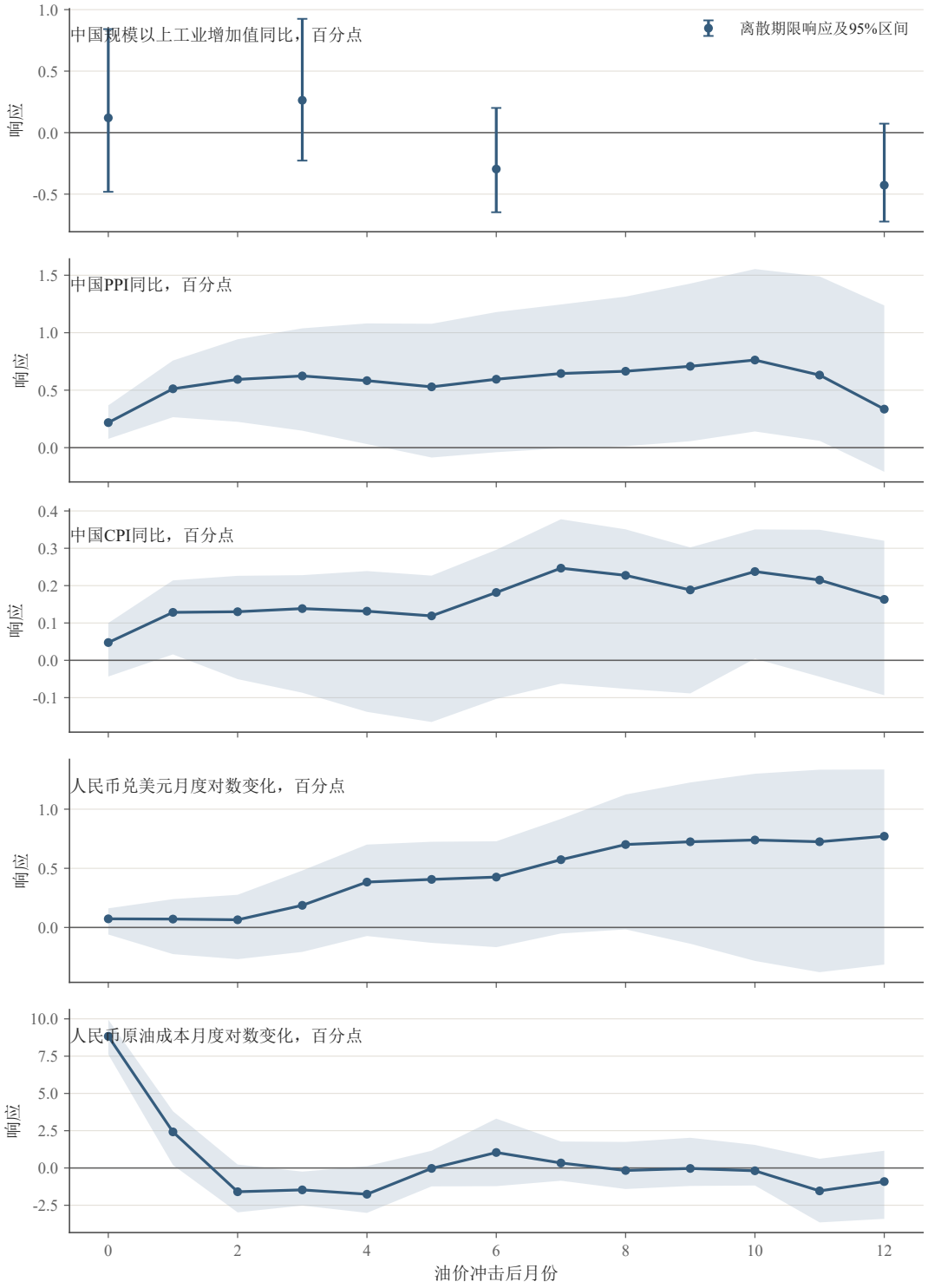
6.3 问题二小结

模型支持“进口成本快速响应、PPI和CPI滞后上行、工业活动可能承压”的传导顺序，但总体增长损失仍为结论不充分。后续政策评价必须同时报告物价、工业活动、响应期限和区间，不能只比较单一价格传导系数。

七、 问题三：中国政策缓冲与跨国比较

7.1 先锁定可比边界

跨国燃油传导受税制、汇率和定价机制共同影响[12, 13]。对每个国家估计

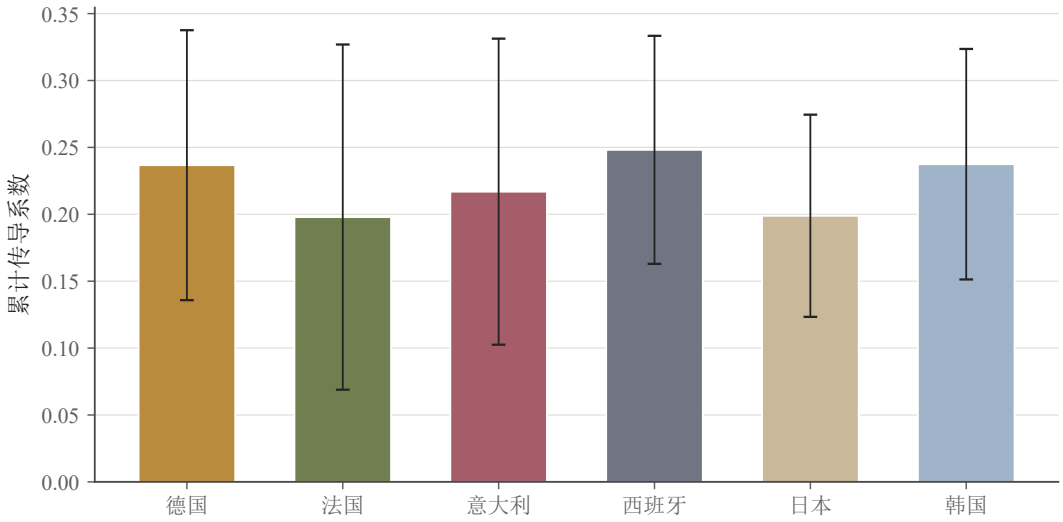


来源：问题一结构冲击接口、国家统计局 IAV/PPI、OECD CPI、FRED 汇率/美元/GPR；由 code/problem2/run_q2.py 生成。

图4　油价特定风险冲击下中国主要变量的局部投影响应

$$\Delta \ln F_{i,t} = a_i + \varphi_i \Delta \ln F_{i,t-1} + \sum_{j=0}^{6} \beta_{i,j} \Delta \ln P_{i,t-j}^{Brent,local} + u_{i,t}, \tag{8}$$

并以$\Pi_{i,H} = \sum_{j=0}^{H} \beta_{i,j}$表示$H$月传导率。德国、法国、意大利、西班牙、日本和韩国有可审计的固定品类零售燃油价格，进入正式排名。中国历史公开调价表只能重建出受管制汽油调价指数代理，不是固定标准品零售价格，因此从正式排名剔除，只保留为敏感性和政策路径构造。



来源：欧盟周度油价公报、日本METI、韩国KOSIS/KNOC、北京市发改委与 FRED；由 code/problem3/run_q3.py 生成。

图5　六国正式零售价格与中国调价指数代理的六个月传导率

表6　六个月燃油传导率及数据角色

国家	中国	德国	法国	意大利	西班牙	日本	韩国
传导率	0.373	0.237	0.198	0.217	0.248	0.199	0.237
数据角色	代理敏感性	正式	正式	正式	正式	正式	正式

六个正式对照国中位数为0.227。中国代理值0.373较高，但二者并非同类价格层，故不能据此判断中国更优或更差。这个“不可比”结论本身是重要的数据审计结果。

7.2　相对宏观响应与暂缓的机制回归

对CPI和工业活动估计含国家固定效应和完整年月固定效应的面板局部投影：

$$y_{i,t+h} - y_{i,t-1} = \alpha_{i,h} + \lambda_{t,h} + \delta_{i,h}(s_t I_i) + \Gamma_h^{\top} Z_{i,t-1} + u_{i,t+h}. \tag{9}$$

中国是参照组，$\delta_{i,h}$只表示对照国相对中国的响应差，不能把中国绝对响应人为设为零。对照国减中国的CPI相对响应点估计中位数为0.245个百分点，工业活动为2.433个百分点；但当前程序没有形成联合国别–时间bootstrap区间，因此两项均判为“结论不充分”。价格监管、进口依赖、石油强度和进口来源集中度年度表均为未经审计代理，四类连续缓冲交互回归全部执行失败关闭，不提供系数、显著性或政策排序。三维核心韧性判据见表7。

表7　中国相对韧性的核心判据

维度	当前证据	判断
燃油价格	中国仅有调价指数代理，不能与六国固定品类零售价正式排名	条件代理
消费价格	对照国减中国点估计中位数0.245，缺联合差异区间	结论不充分
工业活动	对照国减中国点估计中位数2.433，缺联合差异区间	结论不充分
总体	任一核心维度均不足以形成稳健“更好”判断	结论不充分

7.3　关闭2026年临时调控的动态反事实

依据《石油价格管理办法》[19]和2026公告核验，3月汽油机制应调2205元/吨、实调1160元/吨，新增缺口1045元/吨；4月机制应调800元/吨、实调420元/吨，新增380元/吨，累计缺口1425元/吨。令P_t^{no}和P_t^{act}分别为无临时调控与实际代理价格路径，输入模型的不是静态价格水平差，而是逐月收益差

$$\Delta f_t = 100\left[\ln\frac{P_t^{no}}{P_{t-1}^{no}} - \ln\frac{P_t^{act}}{P_{t-1}^{act}}\right]. \tag{10}$$

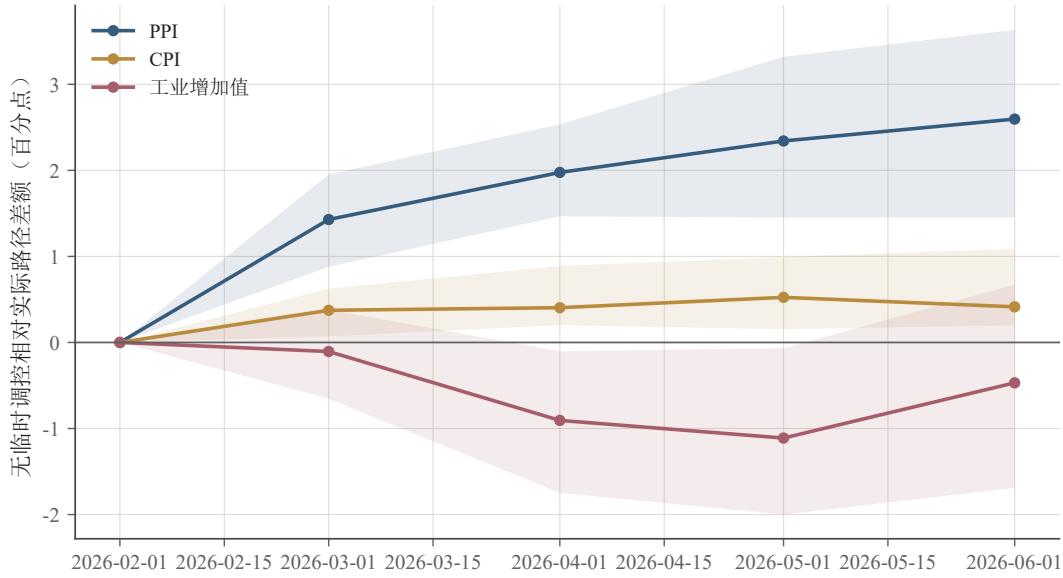
对$y \in \{PPI, CPI, IAV\}$，用0至6阶动态核和结果变量AR(1)递推：

$$B_{y,t} = \sum_{\ell=0}^{6} \beta_{y,\ell}\Delta f_{t-\ell} + \phi_y B_{y,t-1}. \tag{11}$$

三条方程使用同一组长度为6的循环时间块联合抽样，共接受2000组参数抽样，从而保留PPI、CPI和IAV估计误差的同期相关。

表8　2026年6月无临时调控减实际路径的宏观差额

变量	情景差额/百分点	95%区间	情景证据
PPI	2.596	$[1.453, 3.633]$	区间不跨零
CPI	0.414	$[0.199, 1.088]$	区间不跨零
IAV	-0.469	$[-1.690, 0.672]$	结论不充分



来源：results/q3_policy_macro_counterfactual.csv。

图6　关闭临时调控后的动态宏观情景差额

结果说明，在维持动态核与局部环境不变的条件下，临时调控可能降低了2026年6月PPI和CPI压力；IAV方向符合缓冲解释但区间跨零。该结论依赖调价指数代理，不能外推为全面福利改善，也不改变跨国总体韧性“结论不充分”的判断。

7.4　问题三小结

正式零售价格、相对宏观响应和政策反事实属于三类不同证据。当前数据不能证明中国在三维韧性上全面优于对照国；较强的结论只限于2026年条件情景中PPI与CPI的局部缓冲，同时必须显式记录1425元/吨延期缺口及其后续消化责任。

八、　问题四：极端油价风险与自适应调价规则

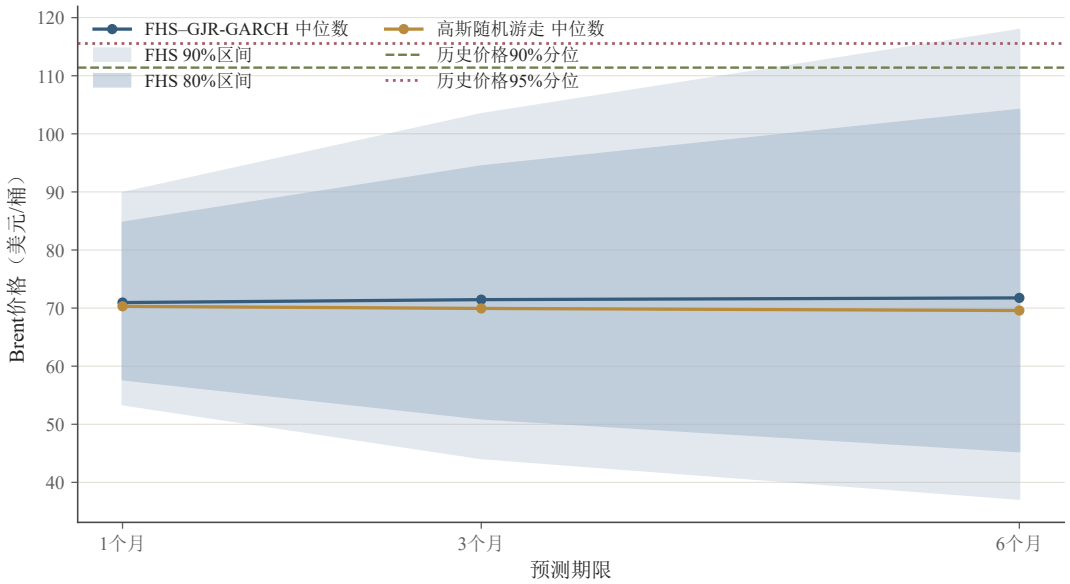
8.1　FHS–GJR–GARCH条件尾部分布

在2026年6月30日信息集下，先估计GJR–GARCH(1,1)，再从截止日前标准化残差经验分布有放回抽样，递归生成[16]

$$P_{t+k}^{(b)} = P_{t+k-1}^{(b)} \exp\left(\mu + \sqrt{h_{t+k}^{(b)}} z_{t+k}^{(b)}\right). \tag{12}$$

每个期限模拟20000条路径。历史90%和95%价格分位111.397和115.550美元/桶只是压力阈值。

回测从2022年起逐月滚动，共47个可用原点，而非只取少量季度末。FHS与高斯随机游走在1、3、6月的平均分位损失分别为1.565对1.629、2.367对2.562、3.519对3.689。配



数据：EIA/FRED Brent；计算：FHS–GJR-GARCH 与高斯随机游走，信息截止 2026-06-30。

图7　固定信息集下的Brent条件尾部分布

表9　Brent条件风险预测

期限/月	中位数	90%区间	期末超过历史95%阈值	路径内曾越过
1	70.95	[53.41, 89.82]	0.27%	0.37%
3	71.46	[44.15, 103.41]	2.28%	3.76%
6	71.76	[37.15, 117.90]	5.56%	9.65%

对长度6块bootstrap 显示，FHS在3个月的80%和90%区间分数、6个月的80%区间分数显著较低；其余配对指标区间跨零。因此FHS对中期区间预测有局部优势，但没有在全部期限和全部评分上支配基线。

8.2　状态自适应价格平滑规则

设机制应调额为Δ_t^{rule}，按开发样本的75%和95%分位将其划为普通、压力、极端三态，传导率满足$\rho_N \geq \rho_S \geq \rho_E$。实际调价与缺口递推为

$$a_t = \rho_{s_t}\Delta_t^{rule} + c_t, \qquad G_t = G_{t-1} + \Delta_t^{rule} - a_t, \tag{13}$$

其中c_t为自动追补项：当缺口触及基准价格20%时优先压回安全区，并受单月调价25%硬约束。四目标定义为

$$
\begin{aligned}
J_1 &= \mathbb{E}(L),\\
J_2 &= CVaR_{0.95}(L),\\
J_3 &= \frac{1}{T}\sum_{t=1}^{T}\frac{|G_t|}{P_0},\\
J_4 &= \mathrm{sd}\left(\frac{a_t}{P_0}\right),
\end{aligned}
\tag{14}
$$

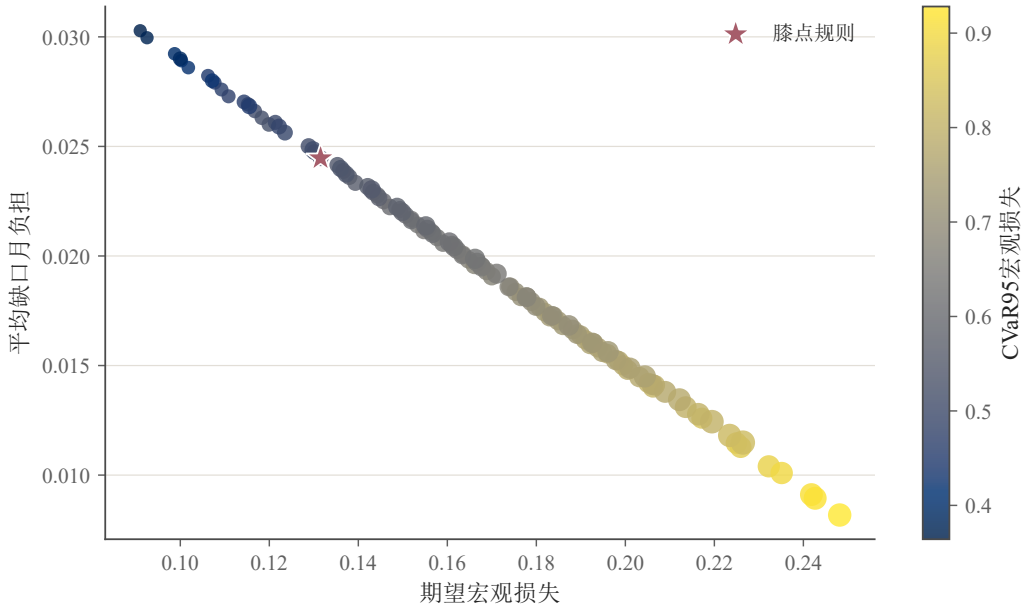
分别衡量平均宏观损失、尾部宏观损失、平均缺口月负担和调价波动，其中CVaR采用Rockafellar–Uryasev定义[17]。J_3不是期末缺口，也不是财政支出。候选规则还必须同时满足

$$
\max_t |G_t|/P_0 \le 0.35, \quad |G_T|/P_0 \le 0.20, \quad |G^{rec}_{T+6}|/P_0 \le 0.05, \quad \max_t |a_t|/P_0 \le 0.25. \tag{15}
$$

阈值和规则只用2013年3月至2021年12月开发样本选择；2022年1月至2026年6月完全隔离。1771个候选中1750个满足硬约束；按1%容差构造ε-Pareto集，得到119个规则，再选距归一化理想点最近的膝点。最优规则为

$$
(\rho_N, \rho_S, \rho_E) = (0.30, 0.10, 0.10), \tag{16}
$$

开发样本阈值为435.97和702.49元/吨，基准价格为9504元/吨。



来源：results/q4_sapr_policy_grid.csv；作者计算。

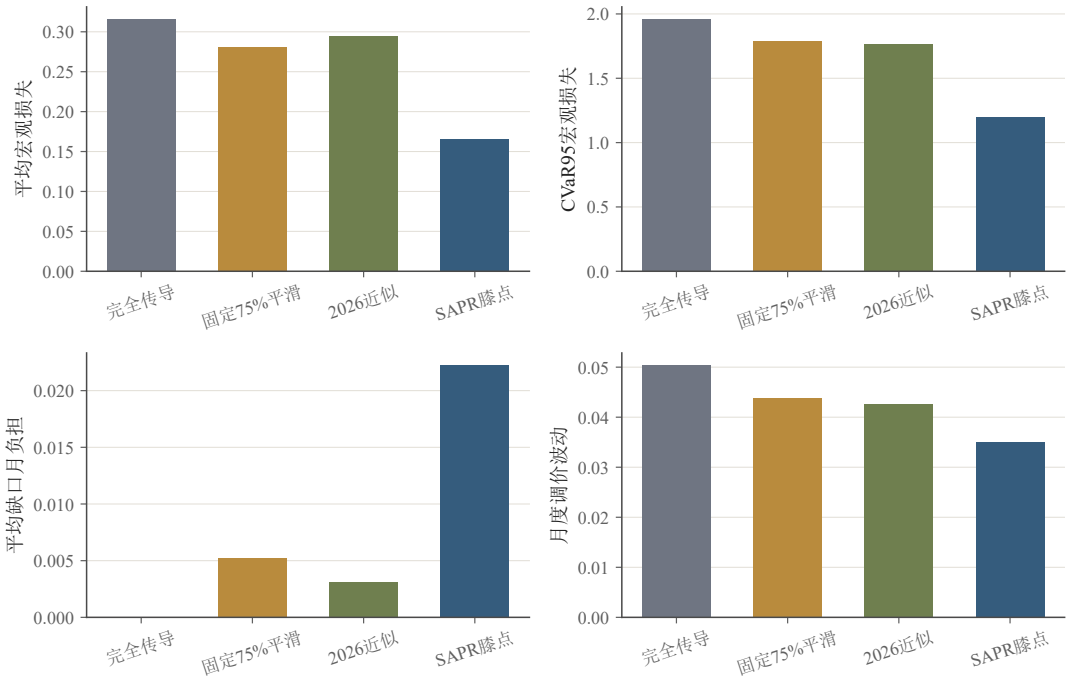
图8　可行规则的四目标映射与ε-Pareto膝点

8.3 隔离检验与政策权衡

表10 2022年至2026年隔离检验样本的原始目标值

策略	J_1	J_2	J_3	J_4
完全传导	0.315	1.957	0.000	0.050
固定75%平滑	0.281	1.792	0.005	0.044
2026临时调控近似	0.294	1.768	0.003	0.043
SAPR膝点	0.165	1.198	0.022	0.035

与完全传导相比，膝点规则的J_1、J_2和J_4配对差分别为-0.163、-0.671和-0.0148，95%区间均小于零；J_3差为$+0.0222$，区间$[0.0109, 0.0386]$，明确揭示平滑的延期负担。检验期最大缺口被自动限制在20%，再给6个月恢复后降至2.35%，满足5%硬约束。结果支持的是注册规则族内的短期风险–延期负担权衡，不是全体政策的全局最优或完整福利改进。



来源：results/q4_sapr_strategy_comparison.csv；作者计算。

图9 隔离检验样本中四类策略的原始目标比较

8.4 问题四小结

六个月路径越过历史95%高位阈值的条件概率为9.65%。SAPR用显式缺口硬约束和自动追补把短期物价与工业压力的降低同未来调价责任绑定；其优势体现在J_1、J_2和J_4，

代价体现在 J_3。这比单纯“压低当期价格”更完整，也保留了退出机制。

九、 稳健性与敏感性分析

9.1 预测、识别与传导探针

问题一将六类预测器置于78个相同目标月比较；no-change在三个期限均取得最低RMSE，Theta虽接近，DM–HLN检验未显示稳定优势。事件CAR改用40个前期波动率、价格水平和GPR状态最接近的安慰剂，最小可实现经验p值为$1/41 = 0.0244$，避免普通周末样本造成虚假精度。结构VAR的6、12、18、24阶候选均检查稳定性；6阶模型白噪声检验通过，替代排序下关键价格冲击相关系数不低于0.999。

问题二使用联合最大t置信带、逐点区间、FDR校正、ARDL、疫情剔除和冲击正负拆分。多数PPI、CPI和IAV联合区间跨零，GDP有效观测不足，因此稳健性分析的作用是限制结论强度，而不是事后挑选显著规格。问题三的四个年度缓冲变量因未经审计统一判为“主分析失败”，不以替代规格恢复定量结论。

9.2 尾部风险回测

FHS与高斯随机游走使用相同的47个月度原点和1、3、6个月期限。除分位损失和覆盖率外，补充80%、90%区间分数与近似CRPS[18]。长度6的配对块bootstrap显示：FHS仅在3个月两档区间分数和6个月80% 区间分数上显著更优，其余差异为结论不充分。由此将“FHS全面优于基线”列为禁止表述。

9.3 SAPR规则敏感性

表11 SAPR膝点对阈值、权重和块长度的敏感性

变体	(ρ_N, ρ_S, ρ_E)	Pareto数	相对默认变化	检验期非支配
默认75/95阈值	$(0.30, 0.10, 0.10)$	119	否	是
70/90阈值	$(0.30, 0.15, 0.10)$	82	是	是
80/97.5阈值	$(0.30, 0.10, 0.10)$	209	否	是
CPI权重增加20%	$(0.30, 0.10, 0.10)$	121	否	是
IAV权重增加20%	$(0.30, 0.10, 0.10)$	116	否	是
bootstrap块3/6/12	$(0.30, 0.10, 0.10)$	119	否	是

除较低状态阈值外，膝点规则对权重、阈值上界和块长度保持不变；70/90阈值把压力期传导率提高到0.15，说明普通与压力状态的划分是最敏感设计项。所有变体在检验

期点估计上保持非支配，但正文的“支持”判断还依赖配对目标差和硬约束，而不只依赖非支配概率。

十、 模型评价与推广

10.1 主要优点

1. **证据层分离。**预测误差、事件异常、结构冲击、宏观响应和政策情景分别建模，避免把同一个差额反复解释为因果贡献。

2. **实时和样本外约束。**预测扩展窗口、47个风险回测原点以及SAPR开发–检验拆分均禁止未来信息；所有复杂方法都有可运行的简单基线。

3. **单位和响应定义可审计。**程序为每个Q2结果声明被解释量；完整路径只报告响应曲线面积，IAV稀疏期限不伪造累计量。

4. **数据不可比时主动降级。**中国燃油调价指数代理不进入正式零售排名；未经审计缓冲代理触发失败关闭，而不是生成看似完整的回归表。

5. **政策优化显式计入退出责任。**平均缺口月负担与最大、期末、恢复期末缺口分开；自动追补保证平滑规则不会无限累积延期调整。

10.2 局限性

1. 单次地缘冲突与同期消息密集重叠，状态匹配CAR仍不是严格随机实验；递归VAR也依赖排序和历史代理变量。

2. 中国固定标准品零售价、战略储备释放、财政补贴、炼油企业损益和进口来源年度数据不完整，限制了跨国韧性与机制解释。

3. Q2月度样本有限，宏观联合区间较宽；季度GDP在滞后要求下无法形成有效回归，不能声称稳健总体增长损失。

4. FHS只能重采样历史残差，不能生成历史从未出现的制度断裂；GJR持久性较高，远期尾部对模型设定敏感。

5. SAPR只在注册的三状态、六个月和离散传导率族内最优，J_3既不是财政成本，也不是完整福利。

10.3 推广方向

该框架可用于天然气、粮食、关键矿产和国际运价：先比较样本外预测，再识别事件异常与结构来源，继而估计国内传导，最后在明确硬约束下优化缓冲规则。若能获得连续的官方零售价格、储备操作、行业投入产出和财政成本数据，可把局部政策情景扩展为部门异质性或一般均衡分析；新增模块仍需通过基线、风险探针和数值冻结门禁。

十一、 结论与政策启示

11.1 主要结论

第一，扩展窗口回测中no-change在1、3、6个月均取得最低RMSE。E1复合事件的$CAR[0]$、$CAR[0,+1]$和$CAR[0,+2]$分别为0.0797、0.1561和0.1355；状态匹配经验p值分别为0.0244、0.0244和0.0513。航运受阻阶段条件波动为战前中位数的2.214倍。这些结果支持事件附近出现异常，但不等同于战争净因果溢价。

第二，石油特定风险冲击使人民币原油成本当月上升8.832个对数百分点；PPI和CPI极值点估计为0.762和0.247个百分点，IAV谷值为-0.427个百分点。除成本入口外，极值联合区间均跨零；GDP 有效观测不足，故总体增长损失证据为结论不充分。

第三，中国历史燃油序列是调价指数代理，不能与六国固定品类零售价正式排名；CPI与工业活动的相对响应缺联合差异区间，三维跨国韧性总体判断为结论不充分。条件动态反事实显示，关闭2026年临时调控后，6月PPI和CPI可能分别高2.596和0.414个百分点；IAV差额为-0.469个百分点但区间跨零。该缓冲同时留下1425元/吨延期缺口。

第四，在2026年6月30日信息集下，六个月Brent中位数为71.76美元/桶，期末超过历史95%阈值的条件概率为5.56%，路径内曾越过的概率为9.65%。SAPR在1750个可行规则中选出 $(0.30, 0.10, 0.10)$；检验期相对完全传导显著降低平均宏观损失、尾部宏观损失和调价波动，但平均缺口月负担增加0.0222。最大缺口被限制在20%，6个月恢复后为2.35%。

11.2 政策启示

1. **临时平滑必须状态依赖且透明。**以机制应调额和条件尾部风险作为触发信号，公开状态阈值、传导率、累计缺口和退出条件，避免临时措施固化为长期冻结。

2. **把延期负担列入同一政策仪表盘。**当期CPI下降并不代表成本消失，应同步监测未调价缺口、炼油企业现金流、库存和财政安排，并在国际油价回落期自动追补。

3. **实施价格–物价–工业活动联合预警。**人民币原油成本是清晰先行入口，PPI、CPI和IAV反映不同滞后层；政策报告应同时给出点估计、联合区间和冲击来源。

4. **优先补齐可审计公开数据。**连续固定品类成品油价格、储备操作、进口来源和政策成本数据，是把当前条件情景提升为跨国因果与福利评价的关键。

综上，稳健策略不是永久隔离国际价格信号，而是在冲击期用可审计、可退出的状态规则降低短期尾部传导，并把被推迟的调整责任显式纳入决策。

AI工具使用说明

本文在公开数据检索提示、程序调试、结果一致性检查和文字修改环节使用了OpenAI Codex [24]。研究问题定义、模型取舍、数据口径、结果核验和最终表述均由参赛队独立判断并承担责任；详细交互用途、采纳与人工修改情况见支撑材料《AI工具使用详情》。

参考文献

[1] Kilian L. Not All Oil Price Shocks Are Alike: Disentangling Demand and Supply Shocks in the Crude Oil Market[J]. American Economic Review, 2009, 99(3): 1053–1069. DOI: 10.1257/aer.99.3.1053.

[2] Caldara D, Iacoviello M. Measuring Geopolitical Risk[J]. American Economic Review, 2022, 112(4): 1194–1225. DOI: 10.1257/aer.20191823.

[3] Baumeister C, Kilian L. Real-Time Forecasts of the Real Price of Oil[J]. Journal of Business & Economic Statistics, 2012, 30(2): 326–336. DOI: 10.1080/07350015.2011.648859.

[4] Baumeister C, Kilian L. Forecasting the Real Price of Oil in a Changing World: A Forecast Combination Approach[J]. Journal of Business & Economic Statistics, 2015, 33(3): 338–351. DOI: 10.1080/07350015.2014.949342.

[5] Jordà Ò. Estimation and Inference of Impulse Responses by Local Projections[J]. American Economic Review, 2005, 95(1): 161–182. DOI: 10.1257/0002828053828518.

[6] Diebold F X, Mariano R S. Comparing Predictive Accuracy[J]. Journal of Business & Economic Statistics, 1995, 13(3): 253–263. DOI: 10.1080/07350015.1995.10524599.

[7] Harvey D, Leybourne S, Newbold P. Testing the Equality of Prediction Mean Squared Errors[J]. International Journal of Forecasting, 1997, 13(2): 281–291. DOI: 10.1016/S0169-2070(96)00719-4.

[8] Glosten L R, Jagannathan R, Runkle D E. On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks[J]. Journal of Finance, 1993, 48(5): 1779–1801. DOI: 10.1111/j.1540-6261.1993.tb05128.x.

[9] Tang W, Wu L, Zhang Z X. Oil Price Shocks and Their Short- and Long-Term Effects on the Chinese Economy[J]. Energy Economics, 2010, 32(S1): S3–S14. DOI: 10.1016/j.eneco.2010.01.002.

[10] Cross J, Nguyen B H. The Relationship between Global Oil Price Shocks and China's Output: A Time-Varying Analysis[J]. Energy Economics, 2017, 62: 79–91. DOI: 10.1016/j.eneco.2016.12.014.

[11] Chen J, Zhu X, Li H. The Pass-Through Effects of Oil Price Shocks on China's Inflation: A Time-Varying Analysis[J]. Energy Economics, 2020, 86: 104695. DOI: 10.1016/j.eneco.2020.104695.

[12] Kpodar K, Abdallah C. Dynamic Fuel Price Pass-Through: Evidence from a New Global Retail Fuel Price Database[J]. Energy Economics, 2017, 66: 303–312. DOI: 10.1016/j.eneco.2017.06.017.

[13] Kpodar K, Imam P A. To Pass (or Not to Pass) Through International Fuel Price Changes to Domestic Fuel Prices in Developing Countries: What Are the Drivers?[J]. Energy Policy, 2021, 149: 111999. DOI: 10.1016/j.enpol.2020.111999.

[14] Coady D, Arze del Granado J, Eyraud L, et al. Automatic Fuel Pricing Mechanisms with Price Smoothing: Design, Implementation, and Fiscal Implications[R]. IMF Technical Notes and Manuals, 2013. DOI: 10.5089/9781475566949.005.

[15] Coady D, Parry I W H, Shang B. Energy Price Reform: Lessons for Policymakers[J]. Review of Environmental Economics and Policy, 2018, 12(2): 197–219. DOI: 10.1093/reep/rey004.

[16] Barone-Adesi G, Giannopoulos K, Vosper L. Backtesting Derivative Portfolios with Filtered Historical Simulation[J]. European Financial Management, 2002, 8(1): 31–58. DOI: 10.1111/1468-036X.00175.

[17] Rockafellar R T, Uryasev S. Optimization of Conditional Value-at-Risk[J]. Journal of Risk, 2000, 2(3): 21–41. DOI: 10.21314/JOR.2000.038.

[18] Gneiting T, Raftery A E. Strictly Proper Scoring Rules, Prediction, and Estimation[J]. Journal of the American Statistical Association, 2007, 102(477): 359–378. DOI: 10.1198/016214506000001437.

[19] 国家发展和改革委员会. 石油价格管理办法[EB/OL]. 2016. https://www.ndrc.gov.cn/xxgk/zcfb/tz/201601/W020190905506573420251.pdf.

[20] Federal Reserve Bank of St. Louis. FRED: Brent Crude Oil Price and Exchange Rate Series[DB/OL]. https://fred.stlouisfed.org/. 访问日期：2026-07-30.

[21] U.S. Energy Information Administration. Petroleum and Other Liquids Data, Weekly U.S. Commercial Crude Oil Stocks[DB/OL]. https://www.eia.gov/petroleum/data.php. 访问日期：2026-07-30.

[22] 国家统计局. 国家数据：月度与季度宏观指标[DB/OL]. https://data.stats.gov.cn/. 访问日期：2026-07-31.

[23] OECD. Data Explorer: Consumer Prices and Industrial Production[DB/OL]. https://data-explorer.oecd.org/. 访问日期：2026-07-30.

[24] OpenAI Codex, GPT-5（Codex）, OpenAI, 使用日期：2026-07-29至2026-08-01.

附录 A 主张、证据与边界

表12 核心主张的证据链和禁止外推

主张	直接证据	边界
冲突初期存在异常上涨	状态匹配CAR：前两窗经验 $p = 0.0244$	复合事件关联，不是战争净因果溢价
成本入口响应清晰	原油成本当月8.832，联合区间不跨零	不等于PPI、CPI或产出必然显著变化
宏观传导存在但增长损失不稳健	PPI、CPI、IAV点估计符合机制，联合区间多跨零	禁止表述为稳健总体GDP损失
2026政策可能缓冲物价	动态代理反事实PPI、CPI区间不跨零	依赖调价指数代理，不是一般均衡福利
中国跨国韧性尚不能定论	零售价不可比，宏观相对响应缺联合区间	禁止声称全面优于六国
SAPR改善短期风险指标	检验期J_1、J_2、J_4配对差显著为负	J_3显著增加；仅在注册规则族内成立

附录 B 复现流程和支撑材料文件列表

建议按“数据下载或人工导入—数据构建—P0校验—Q1—面板重建—Q2—Q3—Q4风险层— Q4 SAPR—论文交接—数值冻结—冻结验证”顺序执行。随机种子为20260730；市场信息截止日为2026-06-30，各序列实际末期以数据字典为准。

表13 支撑材料文件列表

文件或目录	内容和用途
data/SOURCE_REGISTRY.csv	来源网址、下载日期、访问条件、许可和回退口径
data/DATA_DICTIONARY.csv	变量定义、频率、单位、实际末期、发布滞后和主分析状态
data/event_timeline.csv	冲突、航运、国内调控和缓和事件的日期与来源
code/data_download/	免费公开数据下载程序
code/data_processing/	人工数据导入、面板构建和P0数据校验
code/problem1/	预测、CAR、状态匹配安慰剂、结构冲击和条件波动
code/problem2/	变量特定局部投影、ARDL和稳健性探针
code/problem3/	跨国比较、代理数据门禁和动态政策反事实
code/problem4/	尾部风险回测和SAPR–CVaR优化
results/frozen_numbers.json	论文冻结数值、源文件哈希和定稿门禁
results/risk_probe_summary.json	基线、风险探针、证据等级和禁止声明
results/reproducibility_manifest.json	输入、代码、输出哈希及运行环境
reports/paper_numbers.csv	正文关键数字、单位、舍入规则和表述边界
support_materials目录下《AI工具使用详情.pdf》	AI工具、用途、关键交互和人工修改记录

附录 C　全部完整源程序代码

下列代码与支撑材料中的源文件一致。编译时直接读取仓库文件，避免论文代码副本与实际运行版本分叉。

3.1　数据下载与构建

```python
"""Download public P0 source snapshots with reproducibility metadata."""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import os
import shutil
import subprocess
import sys
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
from typing import Any
from zoneinfo import ZoneInfo

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry


REPO_ROOT = Path(__file__).resolve().parents[2]
CONFIG_PATH = Path(__file__).with_name("p0_sources.json")
RAW_DIR = REPO_ROOT / "data" / "raw"
META_DIR = RAW_DIR / "_meta"
MANIFEST_CSV = RAW_DIR / "source_manifest.csv"
MANIFEST_JSON = RAW_DIR / "source_manifest.json"
USER_AGENT = (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 (KHTML, like Gecko) "
    "Chrome/138.0.0.0 Safari/537.36"
)
MANIFEST_FIELDS = [
    "artifact_id",
    "source_id",
    "status",
    "required",
    "url",
    "local_path",
    "downloaded_at",
    "checked_at",
    "http_status",
    "content_type",
    "content_length_header",
    "size_bytes",
    "sha256",
    "etag",
    "last_modified",
    "error",
]


@dataclass(frozen=True)
class Source:
    artifact_id: str
    source_id: str
    url: str
    filename_template: str
    format: str
    min_bytes: int
    required: bool
    expected_csv_columns: tuple[str, ...] = ()
```

```
    requires_env: str | None = None
    referer: str | None = None
    user_agent: str | None = None
    local_snapshot: bool = False


def now_shanghai() -> datetime:
    return datetime.now(ZoneInfo("Asia/Shanghai"))


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def load_sources() -> tuple[dict[str, Any], list[Source]]:
    config = json.loads(CONFIG_PATH.read_text(encoding="utf-8"))
    sources = []
    for item in config["sources"]:
        sources.append(
            Source(
                artifact_id=item["artifact_id"],
                source_id=item["source_id"],
                url=item["url"],
                filename_template=item["filename_template"],
                format=item["format"],
                min_bytes=int(item.get("min_bytes", 1)),
                required=bool(item.get("required", False)),
                expected_csv_columns=tuple(item.get("expected_csv_columns", [])),
                requires_env=item.get("requires_env"),
                referer=item.get("referer"),
                user_agent=item.get("user_agent"),
                local_snapshot=bool(item.get("local_snapshot", False)),
            )
        )
    return config, sources


def make_session() -> requests.Session:
    retry = Retry(
        total=2,
        connect=2,
        read=0,
        backoff_factor=1.5,
        status_forcelist=(429, 500, 502, 503, 504),
        allowed_methods=("GET",),
        respect_retry_after_header=True,
    )
    session = requests.Session()
    session.headers.update({"User-Agent": USER_AGENT, "Accept": "*/*"})
    session.mount("https://", HTTPAdapter(max_retries=retry))
    session.mount("http://", HTTPAdapter(max_retries=retry))
    return session


def parse_curl_headers(path: Path) -> dict[str, str]:
    """Return headers from the final response block written by curl."""
    if not path.exists():
        return {}
    blocks = path.read_text(encoding="iso-8859-1").replace("\r\n", "\n").split("\n\n")
    header_lines = next(
        (block.splitlines() for block in reversed(blocks) if block.startswith("HTTP/")),
        [],
    )
    headers: dict[str, str] = {}
    for line in header_lines[1:]:
        if ":" in line:
            key, value = line.split(":", 1)
            headers[key.strip().lower()] = value.strip()
    return headers


def download_with_curl(source: Source, partial: Path) -> dict[str, Any]:
```

```
    """Use system curl when available; it is more reliable on this Windows host."""
    curl = shutil.which("curl.exe") or shutil.which("curl")
    if not curl:
        raise FileNotFoundError("curl executable not found")
    header_path = partial.with_suffix(partial.suffix + ".headers")
    if header_path.exists():
        header_path.unlink()
    command = [
        curl,
        "--location",
        "--fail",
        "--silent",
        "--show-error",
        "--connect-timeout",
        "12",
        "--max-time",
        "120",
        "--retry",
        "3",
        "--retry-delay",
        "2",
        "--retry-all-errors",
        "--dump-header",
        str(header_path),
        "--output",
        str(partial),
        "--write-out",
        "%{http_code}\\t%{content_type}\\t%{size_download}\\t%{url_effective}",
    ]
    if source.referer:
        command.extend(["--referer", source.referer])
    if source.user_agent:
        command.extend(["--user-agent", source.user_agent])
    command.append(source.url)
    try:
        completed = subprocess.run(
            command,
            check=True,
            capture_output=True,
            text=True,
            timeout=140,
        )
        parts = completed.stdout.strip().split("\t", 3)
        headers = parse_curl_headers(header_path)
        return {
            "http_status": parts[0] if parts else "",
            "content_type": parts[1] if len(parts) > 1 else headers.get("content-type", ""),
            "content_length_header": headers.get("content-length", ""),
            "etag": headers.get("etag", ""),
            "last_modified": headers.get("last-modified", ""),
        }
    finally:
        if header_path.exists():
            header_path.unlink()


def download_with_requests(
    session: requests.Session,
    source: Source,
    partial: Path,
) -> dict[str, Any]:
    headers = {}
    if source.referer:
        headers["Referer"] = source.referer
    if source.user_agent:
        headers["User-Agent"] = source.user_agent
    with session.get(
        source.url,
        headers=headers,
        timeout=(15, 60),
        stream=True,
    ) as response:
        metadata = {
            "http_status": response.status_code,
            "content_type": response.headers.get("Content-Type", ""),
            "content_length_header": response.headers.get("Content-Length", ""),
```

```
            "etag": response.headers.get("ETag", ""),
            "last_modified": response.headers.get("Last-Modified", ""),
        }
        response.raise_for_status()
        with partial.open("wb") as handle:
            for chunk in response.iter_content(chunk_size=1024 * 1024):
                if chunk:
                    handle.write(chunk)
    return metadata


def read_manifest() -> dict[str, dict[str, Any]]:
    if not MANIFEST_JSON.exists():
        return {}
    rows = json.loads(MANIFEST_JSON.read_text(encoding="utf-8"))
    return {row["artifact_id"]: row for row in rows}


def write_manifest(rows_by_id: dict[str, dict[str, Any]]) -> None:
    rows: list[dict[str, Any]] = []
    for key in sorted(rows_by_id):
        row = dict(rows_by_id[key])
        if row.get("status") in {"DOWNLOADED", "CACHED"}:
            local_path = row.get("local_path", "")
            if not local_path or not (REPO_ROOT / local_path).exists():
                row.update(
                    {
                        "status": "REMOTE_ONLY",
                        "local_path": "",
                        "size_bytes": "",
                        "sha256": "",
                        "error": "manifest local snapshot is absent; rerun download or provide a browser-extracted snapshot before marking CACHED",
                    }
                )
        rows.append(row)
    with MANIFEST_JSON.open("w", encoding="utf-8", newline="\n") as handle:
        handle.write(json.dumps(rows, ensure_ascii=False, indent=2) + "\n")
    with MANIFEST_CSV.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=MANIFEST_FIELDS, lineterminator="\n")
        writer.writeheader()
        writer.writerows({key: row.get(key, "") for key in MANIFEST_FIELDS} for row in rows)


def validate_csv_header(path: Path, expected: tuple[str, ...]) -> None:
    if not expected:
        return
    with path.open("r", encoding="utf-8-sig", newline="") as handle:
        header = next(csv.reader(handle))
    missing = [column for column in expected if column not in header]
    if missing:
        raise ValueError(f"CSV missing expected columns: {missing}; actual={header[:20]}")


def cached_record(source: Source, target: Path, timestamp: str) -> dict[str, Any]:
    download_date = target.stem.rsplit("_", 1)[-1]
    meta_path = META_DIR / f"{source.artifact_id}_{download_date}.json"
    existing: dict[str, Any] = {}
    if meta_path.exists():
        existing = json.loads(meta_path.read_text(encoding="utf-8"))
    return {
        "artifact_id": source.artifact_id,
        "source_id": source.source_id,
        "status": "CACHED",
        "required": source.required,
        "url": source.url,
        "local_path": target.relative_to(REPO_ROOT).as_posix(),
        "downloaded_at": existing.get("downloaded_at", timestamp),
        "checked_at": timestamp,
        "http_status": existing.get("http_status", ""),
        "content_type": existing.get("content_type", ""),
        "content_length_header": existing.get("content_length_header", ""),
        "size_bytes": target.stat().st_size,
        "sha256": sha256_file(target),
        "etag": existing.get("etag", ""),
        "last_modified": existing.get("last_modified", ""),
```

```
        "error": "",
    }


def download_one(
    session: requests.Session,
    source: Source,
    *,
    refresh: bool,
    download_date: str,
    timestamp: str,
) -> dict[str, Any]:
    filename = source.filename_template.format(download_date=download_date)
    target = RAW_DIR / filename
    if not target.exists() and not refresh and "{download_date}" in source.filename_template:
        pattern = source.filename_template.replace("{download_date}", "*")
        prior_snapshots = sorted(RAW_DIR.glob(pattern))
        if prior_snapshots:
            target = prior_snapshots[-1]
    partial = target.with_suffix(target.suffix + ".part")

    if source.requires_env and not os.getenv(source.requires_env):
        return {
            "artifact_id": source.artifact_id,
            "source_id": source.source_id,
            "status": "SKIPPED_MISSING_ENV",
            "required": source.required,
            "url": source.url,
            "local_path": "",
            "downloaded_at": timestamp,
            "checked_at": timestamp,
            "error": f"missing environment variable {source.requires_env}",
        }

    if source.local_snapshot:
        if not target.exists():
            return {
                "artifact_id": source.artifact_id,
                "source_id": source.source_id,
                "status": "ERROR",
                "required": source.required,
                "url": source.url,
                "local_path": "",
                "downloaded_at": timestamp,
                "checked_at": timestamp,
                "error": (
                    "versioned browser-extracted snapshot is missing; "
                    f"expected {target.relative_to(REPO_ROOT).as_posix()}"
                ),
            }
        validate_csv_header(target, source.expected_csv_columns)
        record = cached_record(source, target, timestamp)
        record.update(
            {
                "http_status": "200-browser",
                "content_type": "text/csv; charset=utf-8",
            }
        )
        meta_path = META_DIR / f"{source.artifact_id}_{download_date}.json"
        meta_path.write_text(
            json.dumps(record, ensure_ascii=False, indent=2) + "\n",
            encoding="utf-8",
        )
        return record

    if target.exists() and not refresh:
        validate_csv_header(target, source.expected_csv_columns)
        return cached_record(source, target, timestamp)

    if partial.exists():
        partial.unlink()

    record: dict[str, Any] = {
        "artifact_id": source.artifact_id,
        "source_id": source.source_id,
        "status": "ERROR",
```

```
        "required": source.required,
        "url": source.url,
        "local_path": "",
        "downloaded_at": timestamp,
        "checked_at": timestamp,
        "http_status": "",
        "content_type": "",
        "content_length_header": "",
        "size_bytes": "",
        "sha256": "",
        "etag": "",
        "last_modified": "",
        "error": "",
    }
    try:
        if shutil.which("curl.exe") or shutil.which("curl"):
            response_metadata = download_with_curl(source, partial)
        else:
            response_metadata = download_with_requests(session, source, partial)
        record.update(response_metadata)

        size = partial.stat().st_size
        if size < source.min_bytes:
            raise ValueError(f"downloaded file is too small: {size} < {source.min_bytes}")
        validate_csv_header(partial, source.expected_csv_columns)
        partial.replace(target)

        record.update(
            {
                "status": "DOWNLOADED",
                "local_path": target.relative_to(REPO_ROOT).as_posix(),
                "size_bytes": size,
                "sha256": sha256_file(target),
                "error": "",
            }
        )
        meta_path = META_DIR / f"{source.artifact_id}_{download_date}.json"
        meta_path.write_text(
            json.dumps(record, ensure_ascii=False, indent=2) + "\n",
            encoding="utf-8",
        )
    except Exception as exc:
        if partial.exists():
            partial.unlink()
        record["error"] = f"{type(exc).__name__}: {exc}"
    return record


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--source",
        action="append",
        default=[],
        help="Download only this artifact_id; may be specified multiple times.",
    )
    parser.add_argument(
        "--refresh",
        action="store_true",
        help="Replace today's existing snapshot. Without this flag raw files are never overwritten.",
    )
    parser.add_argument("--list", action="store_true", help="List configured sources and exit.")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    config, sources = load_sources()
    if args.list:
        for source in sources:
            print(
                f"{source.artifact_id}\trequired={str(source.required).lower()}"
                f"\t{source.source_id}\t{source.url}"
            )
        return 0

```

```
    selected_ids = set(args.source)
    if selected_ids:
        unknown = selected_ids - {source.artifact_id for source in sources}
        if unknown:
            print(f"Unknown artifact_id(s): {sorted(unknown)}", file=sys.stderr)
            return 2
        sources = [source for source in sources if source.artifact_id in selected_ids]

    RAW_DIR.mkdir(parents=True, exist_ok=True)
    META_DIR.mkdir(parents=True, exist_ok=True)
    timestamp = now_shanghai().isoformat(timespec="seconds")
    download_date = now_shanghai().strftime("%Y%m%d")
    manifest = read_manifest()
    session = make_session()

    required_failures = 0
    for source in sources:
        print(f"[DOWNLOAD] {source.artifact_id}", flush=True)
        record = download_one(
            session,
            source,
            refresh=args.refresh,
            download_date=download_date,
            timestamp=timestamp,
        )
        manifest[source.artifact_id] = record
        print(
            f"[{record['status']}] {source.artifact_id}"
            f" bytes={record.get('size_bytes', '')}"
            f" error={record.get('error', '')}",
            flush=True,
        )
        if source.required and record["status"] not in {"DOWNLOADED", "CACHED"}:
            required_failures += 1

    write_manifest(manifest)
    summary = {
        "schema_version": config["schema_version"],
        "cutoff_date": config["cutoff_date"],
        "selected": len(sources),
        "required_failures": required_failures,
        "manifest": MANIFEST_CSV.relative_to(REPO_ROOT).as_posix(),
    }
    print(json.dumps(summary, ensure_ascii=False))
    return 1 if required_failures else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 1: download_p0.py

```
"""Build reproducible P0 processed datasets and a data-quality report."""

from __future__ import annotations

import argparse
import hashlib
import html
import io
import json
import re
import zipfile
from dataclasses import dataclass
from datetime import date, datetime
from pathlib import Path
from typing import Any

import openpyxl
import pandas as pd
import requests


REPO_ROOT = Path(__file__).resolve().parents[2]
RAW_DIR = REPO_ROOT / "data" / "raw"
```

```
PROCESSED_DIR = REPO_ROOT / "data" / "processed"
REPORT_PATH = REPO_ROOT / "reports" / "DATA_QUALITY_REPORT.md"
MANIFEST_PATH = RAW_DIR / "source_manifest.json"
DICTIONARY_PATH = REPO_ROOT / "data" / "DATA_DICTIONARY.csv"
CUTOFF = pd.Timestamp("2026-06-30")
START_MONTHLY = pd.Timestamp("2010-01-01")
SUCCESS_STATUSES = {"DOWNLOADED", "CACHED"}
EU_FUEL_COUNTRIES = {
    "DEU": ("DE", "Germany", "germany"),
    "FRA": ("FR", "France", "france"),
    "ITA": ("IT", "Italy", "italy"),
    "ESP": ("ES", "Spain", "spain"),
}
EIA_INTL_BULK_URL = "https://www.eia.gov/opendata/bulk/INTL.zip"
EIA_WORLD_LIQUIDS_SERIES_ID = "INTL.53-1-WORL-TBPD.M"
DALLAS_FED_IGREA_URL = "https://www.dallasfed.org/-/media/documents/research/igrea/igrea.xlsx"
FRED_US_CPI_URL = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=CPIAUCSL"
OECD_US_CPI_URL = (
    "https://sdmx.oecd.org/public/rest/data/"
    "OECD.SDD.TPS,DSD_G20_PRICES@DF_G20_PRICES,1.0/"
    "USA.M...IX...?startPeriod=2010-01&endPeriod=2026-06"
    "&dimensionAtObservation=AllDimensions&format=csvfile"
)


@dataclass
class QualityRow:
    dataset: str
    status: str
    rows: int
    start: str
    end: str
    missing: int
    duplicates: int
    note: str


def load_manifest() -> dict[str, dict[str, Any]]:
    rows = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
    return {row["artifact_id"]: row for row in rows}


def source_path(manifest: dict[str, dict[str, Any]], artifact_id: str) -> Path:
    record = manifest.get(artifact_id)
    if not record:
        raise KeyError(f"manifest has no artifact_id={artifact_id}")
    if record["status"] not in SUCCESS_STATUSES:
        raise RuntimeError(
            f"artifact {artifact_id} is not usable: "
            f"status={record['status']} error={record.get('error', '')}"
        )
    path = REPO_ROOT / record["local_path"]
    if not path.exists():
        raise FileNotFoundError(path)
    return path


def save_csv(frame: pd.DataFrame, filename: str) -> Path:
    path = PROCESSED_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def sha256_file(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def download_cached(url: str, filename: str, timeout: int = 60) -> Path:
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    path = RAW_DIR / filename
    if path.exists() and path.stat().st_size > 0:
        return path
    response = requests.get(url, timeout=timeout, headers={"User-Agent": "Mozilla/5.0"})
    response.raise_for_status()
    path.write_bytes(response.content)
    return path
```

```python


def date_string(value: Any) -> str:
    if pd.isna(value):
        return ""
    return pd.Timestamp(value).strftime("%Y-%m-%d")


def parse_excel_date(value: Any) -> pd.Timestamp:
    if isinstance(value, pd.Timestamp):
        return value
    if isinstance(value, (datetime, date)):
        return pd.Timestamp(value)
    return pd.to_datetime(value, format="%Y-%m-%d", errors="coerce")


def profile(
    dataset: str,
    frame: pd.DataFrame,
    date_column: str,
    value_columns: list[str],
    note: str = "",
    status: str = "PASS",
) -> QualityRow:
    dates = pd.to_datetime(frame[date_column], errors="coerce")
    missing = int(frame[value_columns].isna().sum().sum())
    duplicates = int(dates.duplicated().sum())
    return QualityRow(
        dataset=dataset,
        status=status,
        rows=len(frame),
        start=date_string(dates.min()),
        end=date_string(dates.max()),
        missing=missing,
        duplicates=duplicates,
        note=note,
    )


def read_fred(path: Path, value_column: str, output_column: str) -> pd.DataFrame:
    frame = pd.read_csv(path, na_values=[".", ""])
    required = {"observation_date", value_column}
    missing = required - set(frame.columns)
    if missing:
        raise ValueError(f"{path.name} missing columns {sorted(missing)}")
    frame = frame[["observation_date", value_column]].rename(
        columns={"observation_date": "date", value_column: output_column}
    )
    frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
    frame[output_column] = pd.to_numeric(frame[output_column], errors="coerce")
    frame = frame.dropna(subset=["date"]).sort_values("date")
    return frame[(frame["date"] >= START_MONTHLY) & (frame["date"] <= CUTOFF)]


def build_market_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
    specifications = [
        ("fred_brent_daily", "DCOILBRENTEU", "brent_usd_bbl"),
        ("fred_wti_daily", "DCOILWTICO", "wti_usd_bbl"),
        ("fred_usd_broad_daily", "DTWEXBGS", "usd_broad_index"),
        ("fred_cny_per_usd_daily", "DEXCHUS", "cny_per_usd"),
        ("fred_jpy_per_usd_daily", "DEXJPUS", "jpy_per_usd"),
        ("fred_krw_per_usd_daily", "DEXKOUS", "krw_per_usd"),
        ("fred_usd_per_eur_daily", "DEXUSEU", "usd_per_eur"),
    ]
    frames = []
    profiles = []
    for artifact_id, raw_column, output_column in specifications:
        frame = read_fred(source_path(manifest, artifact_id), raw_column, output_column)
        profiles.append(profile(artifact_id, frame, "date", [output_column]))
        frames.append(frame.set_index("date"))

    daily = pd.concat(frames, axis=1, join="outer").sort_index()
    daily["eur_per_usd"] = 1.0 / daily["usd_per_eur"]
    daily = daily.reset_index()
```

```
    save_csv(daily, "p0_daily_market.csv")

    monthly = (
        daily.set_index("date")
        .resample("ME")
        .mean(numeric_only=True)
        .rename_axis("month_end")
        .reset_index()
    )
    monthly["period"] = monthly["month_end"].dt.to_period("M").astype(str)
    monthly["brent_cny_per_bbl"] = (
        monthly["brent_usd_bbl"] * monthly["cny_per_usd"]
    )
    monthly = monthly[
        [
            "period",
            "month_end",
            "brent_usd_bbl",
            "wti_usd_bbl",
            "usd_broad_index",
            "cny_per_usd",
            "jpy_per_usd",
            "krw_per_usd",
            "usd_per_eur",
            "eur_per_usd",
            "brent_cny_per_bbl",
        ]
    ]
    return daily, monthly, profiles


def build_stock_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, pd.DataFrame, QualityRow]:
    path = source_path(manifest, "eia_us_commercial_crude_stock_weekly")
    frame = pd.read_excel(
        path,
        sheet_name="Data 1",
        header=None,
        skiprows=3,
        usecols=[0, 1],
        names=["date", "us_commercial_crude_stock_kbbl"],
    )
    frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
    frame["us_commercial_crude_stock_kbbl"] = pd.to_numeric(
        frame["us_commercial_crude_stock_kbbl"], errors="coerce"
    )
    frame = frame.dropna(subset=["date"]).sort_values("date")
    frame = frame[(frame["date"] >= START_MONTHLY) & (frame["date"] <= CUTOFF)]
    save_csv(frame, "eia_us_commercial_crude_stock_weekly.csv")

    month_end = (
        frame.assign(period=frame["date"].dt.to_period("M").astype(str))
        .sort_values("date")
        .groupby("period", as_index=False)
        .tail(1)
        .rename(
            columns={
                "date": "stock_reference_date",
                "us_commercial_crude_stock_kbbl": "us_crude_stock_month_end_kbbl",
            }
        )
        .sort_values("period")
        .reset_index(drop=True)
    )
    return frame, month_end, profile(
        "eia_us_commercial_crude_stock_weekly",
        frame,
        "date",
        ["us_commercial_crude_stock_kbbl"],
        note="月度主口径使用每月最后一个已观测周值",
    )


def build_gpr_data(
    manifest: dict[str, dict[str, Any]]
```

```python
) -> tuple[pd.DataFrame, QualityRow]:
    path = source_path(manifest, "gpr_global_monthly")
    raw = pd.read_stata(path, convert_categoricals=False)
    required = ["month", "GPR", "GPRT", "GPRA"]
    missing = set(required) - set(raw.columns)
    if missing:
        raise ValueError(f"{path.name} missing columns {sorted(missing)}")
    frame = raw[required].copy()
    frame["month"] = pd.to_datetime(frame["month"], errors="coerce")
    frame = frame.dropna(subset=["month"]).sort_values("month")
    frame = frame[(frame["month"] >= START_MONTHLY) & (frame["month"] <= CUTOFF)]
    frame["period"] = frame["month"].dt.to_period("M").astype(str)
    frame = frame[["period", "month", "GPR", "GPRT", "GPRA"]]
    save_csv(frame, "gpr_global_monthly.csv")
    return frame, profile(
        "gpr_global_monthly",
        frame,
        "month",
        ["GPR", "GPRT", "GPRA"],
        note="2026-06 为最新初值，后续可能修订",
    )


def steo_data_status(cell: Any) -> str:
    color = cell.fill.fgColor
    if color.type == "indexed":
        return "historical"
    if color.type == "theme" and float(color.tint or 0.0) < -0.001:
        return "forecast"
    return "estimate"


def build_steo_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, QualityRow]:
    path = source_path(manifest, "eia_steo_jul2026")
    workbook = openpyxl.load_workbook(path, data_only=True, read_only=False)
    sheet = workbook["3atab"]
    target_map = {
        "papr_world": ("world_liquids_supply_mbd", "million barrels/day"),
        "patc_world": ("world_liquids_demand_mbd", "million barrels/day"),
        "pasc_oecd_t3": ("oecd_commercial_liquids_stocks_mmbbl", "million barrels"),
    }
    row_by_series = {}
    for row in range(1, sheet.max_row + 1):
        value = sheet.cell(row, 1).value
        if isinstance(value, str) and value.strip().lower() in target_map:
            row_by_series[value.strip().lower()] = row
    missing_series = set(target_map) - set(row_by_series)
    if missing_series:
        raise ValueError(f"STEO missing target series {sorted(missing_series)}")

    vintage_text = str(sheet["A4"].value)
    vintage_date = pd.to_datetime(
        re.sub(r"^[A-Za-z]+,\s*", "", vintage_text), errors="raise"
    ).strftime("%Y-%m-%d")

    periods: dict[int, pd.Period] = {}
    year: int | None = None
    for column in range(3, sheet.max_column + 1):
        year_value = sheet.cell(3, column).value
        if year_value is not None:
            year = int(year_value)
        month_value = sheet.cell(4, column).value
        if year is not None and isinstance(month_value, str):
            periods[column] = pd.Period(
                f"{year}-{pd.to_datetime(month_value, format='%b').month:02d}",
                freq="M",
            )

    rows = []
    for series_id, (variable_id, unit) in target_map.items():
        row = row_by_series[series_id]
        for column, period in periods.items():
            value = sheet.cell(row, column).value
            if value is None:
```

```
                continue
            rows.append(
                {
                    "period": str(period),
                    "series_id": series_id.upper(),
                    "variable_id": variable_id,
                    "value": float(value),
                    "unit": unit,
                    "vintage_date": vintage_date,
                    "data_status": steo_data_status(sheet.cell(row, column)),
                }
            )
    frame = pd.DataFrame(rows)
    frame = frame[
        pd.to_datetime(frame["period"] + "-01") <= CUTOFF
    ].sort_values(["variable_id", "period"])
    frame["is_forecast_or_estimate"] = frame["data_status"].ne("historical")
    save_csv(frame, "eia_steo_selected.csv")

    coverage_start = pd.to_datetime(frame["period"] + "-01").min()
    coverage_end = pd.to_datetime(frame["period"] + "-01").max()
    return frame, QualityRow(
        dataset="eia_steo_selected",
        status="CONDITIONAL",
        rows=len(frame),
        start=date_string(coverage_start),
        end=date_string(coverage_end),
        missing=int(frame["value"].isna().sum()),
        duplicates=int(frame.duplicated(["variable_id", "period"]).sum()),
        note="当前单一版本仅覆盖2022年起，且2026-02至06为估计值；不能回填2010年起的实时滚动预测",
    )


def parse_eia_world_liquids(path: Path) -> pd.DataFrame:
    with zipfile.ZipFile(path) as archive:
        with archive.open("INTL.txt") as handle:
            for raw_line in handle:
                record = json.loads(raw_line)
                if record.get("series_id") != EIA_WORLD_LIQUIDS_SERIES_ID:
                    continue
                rows = []
                for period_code, value in record.get("data", []):
                    period_text = str(period_code)
                    if len(period_text) != 6:
                        continue
                    rows.append(
                        {
                            "period": f"{period_text[:4]}-{period_text[4:]}",
                            "world_liquids_supply": float(value) / 1000.0,
                            "world_liquids_supply_unit": "million barrels per day",
                            "eia_series_id": record["series_id"],
                            "eia_last_updated": record.get("last_updated", ""),
                        }
                    )
                frame = pd.DataFrame(rows)
                frame["month_start"] = pd.to_datetime(frame["period"] + "-01", errors="coerce")
                frame = frame.loc[frame["month_start"].between(START_MONTHLY, CUTOFF)].copy()
                return frame.drop(columns=["month_start"]).sort_values("period").reset_index(drop=True)
    raise ValueError(f"{path.name} does not contain {EIA_WORLD_LIQUIDS_SERIES_ID}")


def parse_dallas_fed_igrea(path: Path) -> pd.DataFrame:
    raw = pd.read_excel(path, sheet_name="Kilian Index", header=None)
    frame = raw.iloc[1:, [0, 1]].copy()
    frame.columns = ["date", "global_real_activity"]
    frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
    frame["global_real_activity"] = pd.to_numeric(frame["global_real_activity"], errors="coerce")
    frame = frame.dropna(subset=["date", "global_real_activity"]).copy()
    frame["period"] = frame["date"].dt.to_period("M").astype(str)
    frame = frame.loc[pd.to_datetime(frame["period"] + "-01").between(START_MONTHLY, CUTOFF)]
    return frame[["period", "global_real_activity"]].sort_values("period").reset_index(drop=True)


def load_us_cpi_deflator() -> tuple[pd.DataFrame, Path, str, str]:
    try:
        fred_path = download_cached(FRED_US_CPI_URL, "fred_us_cpi_cpiaucsl_20260731.csv", timeout=45)
```

```
        fred = pd.read_csv(fred_path, na_values=[".", ""])
        if {"observation_date", "CPIAUCSL"}.issubset(fred.columns):
            frame = fred[["observation_date", "CPIAUCSL"]].rename(
                columns={"observation_date": "date", "CPIAUCSL": "us_cpi"}
            )
            frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
            frame["us_cpi"] = pd.to_numeric(frame["us_cpi"], errors="coerce")
            frame = frame.dropna(subset=["date", "us_cpi"]).copy()
            frame["period"] = frame["date"].dt.to_period("M").astype(str)
            frame = frame.loc[pd.to_datetime(frame["period"] + "-01").between(START_MONTHLY, CUTOFF)]
            return frame[["period", "us_cpi"]], fred_path, FRED_US_CPI_URL, "FRED_CPIAUCSL"
    except requests.RequestException:
        pass

    oecd_path = download_cached(OECD_US_CPI_URL, "oecd_us_cpi_index_20260731.csv", timeout=60)
    raw = pd.read_csv(oecd_path)
    required = {"REF_AREA", "TIME_PERIOD", "OBS_VALUE"}
    missing = required - set(raw.columns)
    if missing:
        raise ValueError(f"{oecd_path.name} missing columns {sorted(missing)}")
    frame = raw.loc[raw["REF_AREA"].eq("USA"), ["TIME_PERIOD", "OBS_VALUE"]].copy()
    frame = frame.rename(columns={"TIME_PERIOD": "period", "OBS_VALUE": "us_cpi"})
    frame["us_cpi"] = pd.to_numeric(frame["us_cpi"], errors="coerce")
    frame = frame.dropna(subset=["period", "us_cpi"])
    return frame[["period", "us_cpi"]].sort_values("period").reset_index(drop=True), oecd_path, OECD_US_CPI_URL, "OECD_US_CPI_INDEX"


def build_global_oil_svar_input(monthly_market: pd.DataFrame) -> tuple[pd.DataFrame, QualityRow]:
    eia_path = download_cached(EIA_INTL_BULK_URL, "eia_intl_bulk_20260731.zip", timeout=120)
    igrea_path = download_cached(DALLAS_FED_IGREA_URL, "dallasfed_igrea_20260731.xlsx", timeout=90)
    supply = parse_eia_world_liquids(eia_path)
    activity = parse_dallas_fed_igrea(igrea_path)
    cpi, cpi_path, cpi_url, cpi_source = load_us_cpi_deflator()
    market = monthly_market[["period", "month_end", "brent_usd_bbl"]].copy()
    frame = (
        pd.DataFrame({"period": pd.period_range("2010-01", "2026-06", freq="M").astype(str)})
        .merge(supply, on="period", how="left")
        .merge(activity, on="period", how="left")
        .merge(cpi, on="period", how="left")
        .merge(market, on="period", how="left")
    )
    cpi_base = float(frame.loc[frame["period"].eq("2026-06"), "us_cpi"].dropna().iloc[0])
    frame["real_brent"] = frame["brent_usd_bbl"] * cpi_base / frame["us_cpi"]
    frame["source_vintage"] = "downloaded_2026-07-31_ex_post"
    frame["release_date"] = ""
    frame["source_url"] = ";".join([EIA_INTL_BULK_URL, DALLAS_FED_IGREA_URL, cpi_url])
    frame["raw_sha256"] = ";".join(
        [
            f"eia={sha256_file(eia_path)}",
            f"igrea={sha256_file(igrea_path)}",
            f"{cpi_source.lower()}={sha256_file(cpi_path)}",
        ]
    )
    output = frame[
        [
            "period",
            "world_liquids_supply",
            "global_real_activity",
            "us_cpi",
            "real_brent",
            "source_vintage",
            "release_date",
            "source_url",
            "raw_sha256",
        ]
    ]
    save_csv(output, "global_oil_svar_monthly.csv")
    usable = output[["world_liquids_supply", "global_real_activity", "real_brent"]].dropna()
    usable_periods = output.loc[output[["world_liquids_supply", "global_real_activity", "real_brent"]].notna().all(axis=1), "period"]
    status = "PASS" if len(usable) >= 120 else "CONDITIONAL"
    return output, QualityRow(
        dataset="global_oil_svar_monthly",
        status=status,
        rows=len(output),
        start=str(usable_periods.min()) if not usable_periods.empty else "",
        end=str(usable_periods.max()) if not usable_periods.empty else "",
```

```
        missing=int(output[["world_liquids_supply", "global_real_activity", "us_cpi", "real_brent"]].isna().sum().sum()),
        duplicates=int(output["period"].duplicated().sum()),
        note=f"SVAR ex-post input from EIA INTL world liquids, Dallas Fed IGREA and {cpi_source}; not used for real-time forecast backtests.",
    )


def build_oecd_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
    cpi_path = source_path(manifest, "oecd_g20_cpi_monthly")
    cpi = pd.read_csv(cpi_path)
    cpi["observation_month"] = pd.to_datetime(
        cpi["TIME_PERIOD"].astype(str) + "-01", errors="coerce"
    )
    cpi = cpi[cpi["observation_month"] <= CUTOFF].copy()
    cpi_columns = [
        "REF_AREA",
        "METHODOLOGY",
        "MEASURE",
        "UNIT_MEASURE",
        "EXPENDITURE",
        "ADJUSTMENT",
        "TRANSFORMATION",
        "TIME_PERIOD",
        "OBS_VALUE",
        "OBS_STATUS",
        "BASE_PER",
    ]
    cpi = cpi[[column for column in cpi_columns if column in cpi.columns]]
    save_csv(cpi, "oecd_g20_cpi_monthly.csv")

    ip_path = source_path(manifest, "oecd_kei_ip_monthly")
    ip = pd.read_csv(ip_path)
    ip["observation_month"] = pd.to_datetime(
        ip["TIME_PERIOD"].astype(str) + "-01", errors="coerce"
    )
    ip = ip[ip["observation_month"] <= CUTOFF].copy()
    ip_columns = [
        "REF_AREA",
        "MEASURE",
        "UNIT_MEASURE",
        "ACTIVITY",
        "ADJUSTMENT",
        "TRANSFORMATION",
        "TIME_PERIOD",
        "OBS_VALUE",
        "OBS_STATUS",
        "BASE_PER",
    ]
    ip = ip[[column for column in ip_columns if column in ip.columns]]
    save_csv(ip, "oecd_kei_ip_monthly.csv")

    cpi_dates = pd.to_datetime(cpi["TIME_PERIOD"] + "-01")
    ip_dates = pd.to_datetime(ip["TIME_PERIOD"] + "-01")
    quality = [
        QualityRow(
            dataset="oecd_g20_cpi_monthly",
            status="PASS",
            rows=len(cpi),
            start=date_string(cpi_dates.min()),
            end=date_string(cpi_dates.max()),
            missing=int(cpi["OBS_VALUE"].isna().sum()),
            duplicates=int(cpi.duplicated(["REF_AREA", "TIME_PERIOD"]).sum()),
            note="CHN/DEU/FRA/ITA/ESP/JPN/KOR 各198期；德国、法国、意大利、西班牙为HICP，其余为国家CPI",
        ),
        QualityRow(
            dataset="oecd_kei_ip_monthly",
            status="PASS",
            rows=len(ip),
            start=date_string(ip_dates.min()),
            end=date_string(ip_dates.max()),
            missing=int(ip["OBS_VALUE"].isna().sum()),
            duplicates=int(ip.duplicated(["REF_AREA", "TIME_PERIOD"]).sum()),
            note="DEU/FRA/ITA/ESP/JPN/KOR 各197期；中国活动变量仍需国家统计局",
        ),
```

```
    ]
    return cpi, ip, quality


def build_eu_fuel_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
    path = source_path(manifest, "ec_weekly_oil_bulletin")
    raw = pd.read_excel(path, sheet_name="Prices with taxes", header=0)
    date_col = raw.columns[0]
    weekly_frames: list[pd.DataFrame] = []
    monthly_frames: list[pd.DataFrame] = []
    quality: list[QualityRow] = []
    for country, (ec_code, country_name, slug) in EU_FUEL_COUNTRIES.items():
        price_col = f"{ec_code}_price_with_tax_euro95"
        if price_col not in raw.columns:
            raise ValueError(f"Weekly Oil Bulletin layout lacks {price_col}")
        weekly = raw[[date_col, price_col]].copy()
        weekly.columns = ["date", "gasoline_eur_per_1000l"]
        weekly["date"] = weekly["date"].map(parse_excel_date)
        weekly["gasoline_eur_per_1000l"] = pd.to_numeric(weekly["gasoline_eur_per_1000l"], errors="coerce")
        weekly = weekly.dropna(subset=["date", "gasoline_eur_per_1000l"])
        weekly = weekly[weekly["date"].between(START_MONTHLY, CUTOFF)].copy()
        weekly["country"] = country
        weekly["country_name"] = country_name
        weekly["gasoline_eur_per_l"] = weekly["gasoline_eur_per_1000l"] / 1000.0
        weekly = weekly[["country", "country_name", "date", "gasoline_eur_per_1000l", "gasoline_eur_per_l"]]
        weekly = weekly.sort_values("date").reset_index(drop=True)
        save_csv(weekly, f"{slug}_eurosuper95_weekly.csv")
        weekly_frames.append(weekly)

        monthly = (
            weekly.assign(period=weekly["date"].dt.to_period("M").astype(str))
            .groupby(["country", "country_name", "period"], as_index=False)
            .agg(
                gasoline_eur_per_l=("gasoline_eur_per_l", "mean"),
                weekly_observations=("gasoline_eur_per_l", "count"),
            )
        )
        monthly["month_end"] = pd.to_datetime(monthly["period"]) + pd.offsets.MonthEnd(0)
        save_csv(monthly, f"{slug}_eurosuper95_monthly.csv")
        monthly_frames.append(monthly)
        quality.extend(
            [
                profile(
                    f"{slug}_eurosuper95_weekly",
                    weekly,
                    "date",
                    ["gasoline_eur_per_l"],
                    note="European Commission Weekly Oil Bulletin, tax-inclusive Euro-super 95, original unit EUR/1000 litres.",
                ),
                profile(
                    f"{slug}_eurosuper95_monthly",
                    monthly,
                    "month_end",
                    ["gasoline_eur_per_l"],
                    note="Arithmetic monthly average of official weekly observations.",
                ),
            ]
        )

    all_weekly = pd.concat(weekly_frames, ignore_index=True)
    all_monthly = pd.concat(monthly_frames, ignore_index=True)
    save_csv(all_weekly, "eu_eurosuper95_weekly.csv")
    save_csv(all_monthly, "eu_eurosuper95_monthly.csv")
    germany_weekly = all_weekly.loc[all_weekly["country"].eq("DEU")].copy()
    germany_monthly = all_monthly.loc[all_monthly["country"].eq("DEU")].copy()
    save_csv(germany_weekly, "germany_eurosuper95_weekly.csv")
    save_csv(germany_monthly[["period", "gasoline_eur_per_l", "weekly_observations", "month_end"]], "germany_eurosuper95_monthly.csv")
    return germany_weekly, germany_monthly, quality


def build_germany_fuel_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
    path = source_path(manifest, "ec_weekly_oil_bulletin")
```

```
    raw = pd.read_excel(path, sheet_name="Prices with taxes", header=None)
    if raw.shape[1] <= 54:
        raise ValueError("Weekly Oil Bulletin layout no longer contains expected DE columns")
    weekly = raw.iloc[3:, [0, 53, 54]].copy()
    weekly.columns = ["date", "country_code", "gasoline_eur_per_1000l"]
    weekly["date"] = pd.to_datetime(weekly["date"], errors="coerce")
    weekly["gasoline_eur_per_1000l"] = pd.to_numeric(
        weekly["gasoline_eur_per_1000l"], errors="coerce"
    )
    weekly = weekly[
        weekly["country_code"].astype(str).str.strip().eq("DE_")
        & weekly["date"].between(START_MONTHLY, CUTOFF)
    ].copy()
    weekly["gasoline_eur_per_l"] = weekly["gasoline_eur_per_1000l"] / 1000.0
    weekly = weekly.sort_values("date").reset_index(drop=True)
    save_csv(weekly, "germany_eurosuper95_weekly.csv")

    monthly = (
        weekly.assign(period=weekly["date"].dt.to_period("M").astype(str))
        .groupby("period", as_index=False)
        .agg(
            gasoline_eur_per_l=("gasoline_eur_per_l", "mean"),
            weekly_observations=("gasoline_eur_per_l", "count"),
        )
    )
    monthly["month_end"] = pd.to_datetime(monthly["period"]) + pd.offsets.MonthEnd(0)
    save_csv(monthly, "germany_eurosuper95_monthly.csv")
    quality = [
        profile(
            "germany_eurosuper95_weekly",
            weekly,
            "date",
            ["gasoline_eur_per_l"],
            note="欧盟周报含税Euro-super 95, 原单位EUR/1000 litres",
        ),
        profile(
            "germany_eurosuper95_monthly",
            monthly,
            "month_end",
            ["gasoline_eur_per_l"],
            note="自然月内周度观测算术均值",
        ),
    ]
    return weekly, monthly, quality


def build_japan_fuel_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
    path = source_path(manifest, "jp_meti_regular_gasoline_weekly")
    raw = pd.read_excel(path, sheet_name="レギュラー", header=None)
    if raw.shape[1] < 3:
        raise ValueError("Japan METI workbook no longer contains expected columns")
    weekly = raw.iloc[1:, [1, 2]].copy()
    weekly.columns = ["date", "regular_gasoline_jpy_per_l"]
    weekly["date"] = pd.to_datetime(weekly["date"], errors="coerce")
    weekly["regular_gasoline_jpy_per_l"] = pd.to_numeric(
        weekly["regular_gasoline_jpy_per_l"], errors="coerce"
    )
    weekly = weekly.dropna(subset=["date", "regular_gasoline_jpy_per_l"])
    weekly = weekly[weekly["date"].between(START_MONTHLY, CUTOFF)]
    weekly = weekly.sort_values("date").reset_index(drop=True)
    save_csv(weekly, "japan_regular_gasoline_weekly.csv")

    monthly = (
        weekly.assign(period=weekly["date"].dt.to_period("M").astype(str))
        .groupby("period", as_index=False)
        .agg(
            regular_gasoline_jpy_per_l=("regular_gasoline_jpy_per_l", "mean"),
            weekly_observations=("regular_gasoline_jpy_per_l", "count"),
        )
    )
    monthly["month_end"] = pd.to_datetime(monthly["period"]) + pd.offsets.MonthEnd(0)
    save_csv(monthly, "japan_regular_gasoline_monthly.csv")
    quality = [
        profile(
```

```
            "japan_regular_gasoline_weekly",
            weekly,
            "date",
            ["regular_gasoline_jpy_per_l"],
            note="日本资源能源厅给油所普通汽油现金价，全国，日元/升",
        ),
        profile(
            "japan_regular_gasoline_monthly",
            monthly,
            "month_end",
            ["regular_gasoline_jpy_per_l"],
            note="自然月内周度观测算术均值；2004-04以后为含消费税价格",
        ),
    ]
    return weekly, monthly, quality


def build_korea_fuel_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, QualityRow]:
    path = source_path(manifest, "kosis_kr_gasoline_monthly")
    monthly = pd.read_csv(path)
    required = {"period", "regular_gasoline_krw_per_l"}
    missing = required - set(monthly.columns)
    if missing:
        raise ValueError(f"{path.name} missing columns {sorted(missing)}")
    monthly["month_start"] = pd.to_datetime(monthly["period"], errors="coerce")
    monthly["regular_gasoline_krw_per_l"] = pd.to_numeric(
        monthly["regular_gasoline_krw_per_l"], errors="coerce"
    )
    monthly = monthly.dropna(subset=["month_start", "regular_gasoline_krw_per_l"])
    monthly = monthly[monthly["month_start"].between(START_MONTHLY, CUTOFF)]
    monthly["period"] = monthly["month_start"].dt.to_period("M").astype(str)
    monthly["month_end"] = monthly["month_start"] + pd.offsets.MonthEnd(0)
    monthly = monthly[
        ["period", "month_end", "regular_gasoline_krw_per_l"]
    ].sort_values("month_end")
    monthly = monthly.reset_index(drop=True)
    save_csv(monthly, "korea_regular_gasoline_monthly.csv")
    quality = profile(
        "korea_regular_gasoline_monthly",
        monthly,
        "month_end",
        ["regular_gasoline_krw_per_l"],
        note=(
            "KOSIS表TX_31802_A000，韩国石油公社普通汽油全国月均价，"
            "单位KRW/litre；公开表浏览器读取快照"
        ),
    )
    return monthly, quality


def extract_policy_values(path: Path) -> tuple[int, int, int, int]:
    text = html.unescape(path.read_text(encoding="utf-8", errors="replace"))
    text = re.sub(r"<[^>]+>", "", text)
    text = re.sub(r"\s+", "", text)
    match = re.search(
        r"(?:分别应|应分别)上调(\d+)元、(\d+)元.*?实际上调(\d+)元、(\d+)元",
        text,
    )
    if not match:
        raise ValueError(f"could not parse policy adjustment values from {path.name}")
    return tuple(int(value) for value in match.groups())


def build_policy_data(
    manifest: dict[str, dict[str, Any]]
) -> tuple[pd.DataFrame, QualityRow]:
    definitions = [
        ("ndrc_fuel_control_20260323", "2026-03-23"),
        ("ndrc_fuel_control_20260407", "2026-04-07"),
    ]
    rows = []
    try:
        for artifact_id, effective_date in definitions:
            gasoline_rule, diesel_rule, gasoline_actual, diesel_actual = (
```

```
                extract_policy_values(source_path(manifest, artifact_id))
            )
            rows.append(
                {
                    "effective_date": effective_date,
                    "gasoline_rule_adjustment_cny_t": gasoline_rule,
                    "gasoline_actual_adjustment_cny_t": gasoline_actual,
                    "gasoline_policy_gap_cny_t": gasoline_rule - gasoline_actual,
                    "diesel_rule_adjustment_cny_t": diesel_rule,
                    "diesel_actual_adjustment_cny_t": diesel_actual,
                    "diesel_policy_gap_cny_t": diesel_rule - diesel_actual,
                    "source_artifact_id": artifact_id,
                }
            )
        frame = pd.DataFrame(rows)
        policy_note = "由两份国家发展改革委网页快照自动复算政策差额"
        policy_status = "PASS"
    except (KeyError, RuntimeError, FileNotFoundError, ValueError) as exc:
        cached_processed = PROCESSED_DIR / "cn_fuel_policy_events.csv"
        if not cached_processed.exists():
            raise
        frame = pd.read_csv(cached_processed)
        policy_note = (
            "NDRC网页快照当前不可用，沿用已生成的规范化提取表；"
            f"需补回官方原始快照后才能解除来源门禁。原因：{type(exc).__name__}: {exc}"
        )
        policy_status = "CONDITIONAL"
    frame["effective_date"] = pd.to_datetime(frame["effective_date"])
    save_csv(frame, "cn_fuel_policy_events.csv")
    value_columns = [
        "gasoline_rule_adjustment_cny_t",
        "gasoline_actual_adjustment_cny_t",
        "gasoline_policy_gap_cny_t",
        "diesel_rule_adjustment_cny_t",
        "diesel_actual_adjustment_cny_t",
        "diesel_policy_gap_cny_t",
    ]
    return frame, profile(
        "cn_fuel_policy_events",
        frame,
        "effective_date",
        value_columns,
        note=policy_note,
        status=policy_status,
    )


def build_release_matrix() -> pd.DataFrame:
    dictionary = pd.read_csv(DICTIONARY_PATH)

    def forecast_rule(source_id: str) -> str:
        if source_id == "SRC_EIA_STEO":
            return "单一2026-07版本不得回填历史滚动预测；收集历史vintage前从ARIMAX主规格删除"
        if source_id in {"SRC_FRED_BRENT", "SRC_FRED_WTI", "SRC_FRED_DTWEXBGS", "SRC_FRED_FX"}:
            return "日度值按实际发布日期可用；完整月均值最早在月末后使用"
        if source_id == "SRC_EIA_WCESTUS1":
            return "周值按WPSR发布日期可用；月末口径使用当时最后一个已发布周值"
        if source_id == "SRC_GPR":
            return "t月值按t+1月初版本可用，保存vintage并允许近期修订"
        if source_id.startswith("SRC_OECD"):
            return "API未返回逐期发布日期；预测模型中保守滞后1个月，后续补真实发布记录"
        if source_id == "SRC_NBS_MONTHLY":
            return "按国家统计局公告日可用；环比历史修订必须记录下载版本"
        if source_id.startswith("DERIVED"):
            return "取全部上游输入中最晚的可用日期"
        return "按来源实际公告或登记规则使用"

    matrix = dictionary[
        [
            "variable_id",
            "question",
            "source_id",
            "raw_frequency",
            "model_frequency",
            "target_coverage",
            "release_lag_rule",
```

```
            "cutoff_rule",
            "availability",
        ]
    ].copy()
    matrix["forecast_use_rule"] = matrix["source_id"].map(forecast_rule)
    matrix["exact_release_calendar_ready"] = matrix["source_id"].map(
        lambda source_id: "no"
        if source_id
        in {
            "SRC_EIA_STEO",
            "SRC_NBS_MONTHLY",
            "SRC_OECD_G20_CPI",
            "SRC_OECD_KEI_IP",
            "SRC_OECD_ENERGY_CPI",
        }
        else "rule_based"
    )
    save_csv(matrix, "release_date_matrix.csv")
    return matrix


def merge_monthly_market(
    monthly_market: pd.DataFrame,
    stock_monthly: pd.DataFrame,
    gpr: pd.DataFrame,
) -> pd.DataFrame:
    merged = monthly_market.merge(
        stock_monthly[
            ["period", "stock_reference_date", "us_crude_stock_month_end_kbbl"]
        ],
        on="period",
        how="left",
    ).merge(gpr[["period", "GPR", "GPRT", "GPRA"]], on="period", how="left")
    save_csv(merged, "p0_monthly_market.csv")
    return merged


def load_manual_nbs_quality() -> list[QualityRow]:
    """Profile verified NBS manual exports without rewriting them."""
    ppi_path = PROCESSED_DIR / "nbs_ppi_monthly.csv"
    iav_path = PROCESSED_DIR / "nbs_iav_monthly.csv"
    if not ppi_path.exists() or not iav_path.exists():
        return []

    ppi = pd.read_csv(ppi_path)
    iav = pd.read_csv(iav_path)
    return [
        profile(
            "nbs_ppi_monthly",
            ppi,
            "period",
            ["china_ppi_yoy_pct"],
            note="国家统计局官网手工导出；“上年同月=100”减100转为同比百分比",
        ),
        profile(
            "nbs_iav_monthly",
            iav,
            "period",
            ["china_iav_yoy_pct"],
            note="规模以上工业增加值同比；官方1—2月结构性缺口原样保留",
        ),
    ]


def quality_report(
    rows: list[QualityRow],
    monthly_market: pd.DataFrame,
    release_matrix: pd.DataFrame,
) -> None:
    frame = pd.DataFrame([row.__dict__ for row in rows])
    save_csv(frame, "dataset_profile.csv")
    month_dates = pd.to_datetime(monthly_market["period"] + "-01")
    expected_months = len(pd.period_range("2010-01", "2026-06", freq="M"))
    common_complete = int(
        monthly_market[
            [
```

```
                "brent_usd_bbl",
                "us_crude_stock_month_end_kbbl",
                "usd_broad_index",
                "GPR",
            ]
        ]
        .dropna()
        .shape[0]
    )
    status_counts = frame["status"].value_counts().to_dict()
    nbs_ready = {
        "nbs_ppi_monthly",
        "nbs_iav_monthly",
    }.issubset(set(frame["dataset"]))
    nbs_conclusion = (
        "- 国家统计局规模以上工业增加值同比和工业生产者出厂价格指数已由官网"
        "手工导出并进入处理层。PPI 覆盖 198 期且无缺失；工业增加值的空值按"
        "官方 1—2 月合并发布规则保留，未作插值。"
        if nbs_ready
        else "- 国家统计局工业增加值与 PPI 尚未进入处理层，依赖它们的模型不得放行。"
    )
    nbs_gate = (
        "| 国家统计局工业增加值、PPI 完整月表 | `PASS` | "
        "已进入 Q2 月度模型；工业增加值官方结构性空值按缺失处理 |"
        if nbs_ready
        else "| 国家统计局工业增加值、PPI 完整月表 | `CONDITIONAL` | "
        "CPI 先用 OECD；IAV/PPI 到位前不运行 Q2 完整 LP |"
    )
    table_lines = [
        "| 数据集 | 状态 | 行数 | 起点 | 末期 | 缺失单元 | 重复日期 | 说明 |",
        "| --- | --- | ---: | --- | --- | ---: | ---: | --- |",
    ]
    for row in rows:
        table_lines.append(
            f"| `{row.dataset}` | `{row.status}` | {row.rows} | {row.start} | "
            f"{row.end} | {row.missing} | {row.duplicates} | {row.note} |"
        )
    report = f"""# P0 数据质量报告

> 生成脚本：`code/data_processing/build_p0_datasets.py`、`code/data_processing/import_nbs_manual.py`
>
> 观测截止日：2026-06-30
>
> 本报告评价数据管线；模型风险探针状态另以 `results/risk_probe_summary.json` 为准。

## 1. 结论

- 自动来源已形成可复现快照、SHA-256 登记和处理后数据。
- 月度主区间 2010-01 至 2026-06 共 {expected_months} 个月；Brent、美国商业原油库存、美元指数和 GPR 四项共同非缺失期为 {common_complete} 个月。
- OECD 整体 CPI 覆盖中、日、韩、德各 198 期；OECD 工业生产覆盖日、韩、德各 197 期。
- EIA 2026 年 7 月 STEO 当前文件只覆盖 2022 年起，且包含估计/预测区间。它不能直接回填 2010 年起的历史滚动预测，主 ARIMAX 暂按既定回退删除全球供需变量。
- 德国含税 Euro-super 95、日本普通汽油全国现金价和韩国普通汽油全国月均价均已进入处理层；韩国 KOSIS 序列覆盖 2010-01 至 2026-06 共 198 期。
{nbs_conclusion}

## 2. 数据集级检查

{chr(10).join(table_lines)}

状态汇总：`PASS={status_counts.get("PASS", 0)}`，`CONDITIONAL={status_counts.get("CONDITIONAL", 0)}`。

## 3. 信息集与发布滞后

`data/processed/release_date_matrix.csv` 已从变量字典生成，共 {len(release_matrix)} 个变量接口。当前只完成规则级矩阵，尚未补齐所有来源的逐期实际发布日期。

必须遵守：

1. 完整月均值只能在该月结束后使用。
2. GPR 的当月值按次月初版本可用，近期值允许修订。
3. STEO 单一最新版本不得用于伪造历史实时信息集。
4. OECD API 未返回逐期发布日期时，预测探针先保守滞后一个月。
5. 国家统计局手工导出文件按原始快照、下载日期和 SHA-256 保存；工业增加值结构性空值不得插值。
```

```
## 4. 主线放行与回退项

| 条目 | 状态 | 回退 |
| --- | --- | --- |
{nbs_gate}
| 中国原油进口实际单位价值 | `CONDITIONAL` | 主变量使用 Brent 月均价乘人民币兑美元汇率 |
| STEO 2010 年起实时版本 | `CONDITIONAL` | ARIMAX 主规格删除全球供需，保留实际库存、美元和 GPR |

## 5. 下一门禁

数据处理后应重新运行 Q1/Q2/Q3 风险探针并更新 `results/risk_probe_summary.json`；有已验证回退且不承担主线任务的条件项不阻塞模型运行。
"""
    REPORT_PATH.write_text(report, encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--cutoff",
        default=CUTOFF.strftime("%Y-%m-%d"),
        help="Reserved for compatibility; current locked cutoff must remain 2026-06-30.",
    )
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.cutoff != CUTOFF.strftime("%Y-%m-%d"):
        raise ValueError("competition data cutoff is locked at 2026-06-30")
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    manifest = load_manifest()
    quality_rows: list[QualityRow] = []

    _, monthly_market, market_quality = build_market_data(manifest)
    quality_rows.extend(market_quality)
    _, stock_monthly, stock_quality = build_stock_data(manifest)
    quality_rows.append(stock_quality)
    gpr, gpr_quality = build_gpr_data(manifest)
    quality_rows.append(gpr_quality)
    monthly_market = merge_monthly_market(monthly_market, stock_monthly, gpr)
    quality_rows.append(
        profile(
            "p0_monthly_market",
            monthly_market,
            "month_end",
            [
                "brent_usd_bbl",
                "wti_usd_bbl",
                "usd_broad_index",
                "cny_per_usd",
                "us_crude_stock_month_end_kbbl",
                "GPR",
            ],
            note="2010-01至2026-06主市场面板；未对缺失值做插值",
        )
    )

    _, svar_quality = build_global_oil_svar_input(monthly_market)
    quality_rows.append(svar_quality)

    _, steo_quality = build_steo_data(manifest)
    quality_rows.append(steo_quality)
    _, _, oecd_quality = build_oecd_data(manifest)
    quality_rows.extend(oecd_quality)
    _, _, germany_quality = build_eu_fuel_data(manifest)
    quality_rows.extend(germany_quality)
    _, _, japan_quality = build_japan_fuel_data(manifest)
    quality_rows.extend(japan_quality)
    _, korea_quality = build_korea_fuel_data(manifest)
    quality_rows.append(korea_quality)
    _, policy_quality = build_policy_data(manifest)
    quality_rows.append(policy_quality)
    quality_rows.extend(load_manual_nbs_quality())
    release_matrix = build_release_matrix()
    quality_report(quality_rows, monthly_market, release_matrix)

```

```
    summary = {
        "cutoff": CUTOFF.strftime("%Y-%m-%d"),
        "processed_files": len(list(PROCESSED_DIR.glob("*.csv"))),
        "quality_report": REPORT_PATH.relative_to(REPO_ROOT).as_posix(),
        "quality_status_counts": pd.Series(
            [row.status for row in quality_rows]
        ).value_counts().to_dict(),
    }
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 2: build_p0_datasets.py

```
"""Build modeling panels for the three oil-shock questions.

The script uses the existing processed P0 layer as the main input. Optional
external enrichments are advisory: if they cannot be fetched, the panels still
build and the issue is recorded in results/data_warnings.json.
"""

from __future__ import annotations

import io
import json
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
import requests


REPO_ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = REPO_ROOT / "data"
PROCESSED_DIR = DATA_DIR / "processed"
RESULTS_DIR = REPO_ROOT / "results"
CUTOFF = pd.Timestamp("2026-06-30")
START_MONTH = "2010-01"
RANDOM_SEED = 20260730
BARREL_TO_TONNE = 0.1364
CHINA_MAIN_FUEL_MIN_MONTHS = 48
EVENT_E1_CALENDAR_START = pd.Timestamp("2026-02-28")
EVENT_E3_CALENDAR_START = pd.Timestamp("2026-06-17")
COUNTRY_META = {
    "CHN": {"name": "China", "fuel_file": "", "fuel_col": "", "unit": "CNY/tonne proxy"},
    "DEU": {"name": "Germany", "fuel_file": "germany_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
    "FRA": {"name": "France", "fuel_file": "france_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
    "ITA": {"name": "Italy", "fuel_file": "italy_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
    "ESP": {"name": "Spain", "fuel_file": "spain_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
    "JPN": {"name": "Japan", "fuel_file": "japan_regular_gasoline_monthly.csv", "fuel_col": "regular_gasoline_jpy_per_l", "unit": "JPY/litre"},
    "KOR": {"name": "Korea", "fuel_file": "korea_regular_gasoline_monthly.csv", "fuel_col": "regular_gasoline_krw_per_l", "unit": "KRW/litre"},
}
CONTROL_COUNTRY_ORDER = ["DEU", "FRA", "ITA", "ESP", "JPN", "KOR"]

OECD_GDP_URLS = {
    "china_real_gdp_yoy_pct": "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN1@DF_QNA_EXPENDITURE_GROWTH_OECD/Q.....B1GQ.....GY.?startPeriod=2010-Q1&endPeriod=2026-Q2&dimensionAtObservation=AllDimensions&format=csvfile",
    "china_real_gdp_qoq_pct": "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN1@DF_QNA_EXPENDITURE_GROWTH_OECD/Q.....B1GQ.....G1.?startPeriod=2010-Q1&endPeriod=2026-Q2&dimensionAtObservation=AllDimensions&format=csvfile",
}
NBS_2026_Q2_URL = "https://www.stats.gov.cn/sj/zxfb/202607/t20260715_1964121.html"
NDRC_2026_03_CONTROL_URL = "https://www.ndrc.gov.cn/xwdt/xwfb/202603/t20260323_1404296_ext.html"
BJ_2026_03_PRICE_URL = "https://fgw.beijing.gov.cn/gzdt/fgzs/mtbdx/bzwlxw/202603/t20260324_4564846.htm"
BJ_2026_04_PRICE_URL = "https://fgw.beijing.gov.cn/gzdt/fgzs/mtbdx/bzwlxw/202604/t20260408_4577002.htm"
BJ_2026_04_21_PRICE_URL = "https://fgw.beijing.gov.cn/gzdt/fgzs/mtbdx/bzwlxw/202604/t20260423_4605062.htm"
EASTMONEY_DATACENTER_URL = "https://datacenter-web.eastmoney.com/api/data/v1/get"
EASTMONEY_OIL_PAGE_URL = "https://data.eastmoney.com/cjsj/oil_default.html"
EUROSTAT_SPAIN_HICP_YOY_URL = (
    "https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/"
```

```
    "prc_hicp_manr?geo=ES&coicop=CP00&unit=RCH_A"
    "&sinceTimePeriod=2010-01&untilTimePeriod=2026-06"
)


def read_processed(filename: str, **kwargs: Any) -> pd.DataFrame:
    return pd.read_csv(PROCESSED_DIR / filename, **kwargs)


def save_processed(frame: pd.DataFrame, filename: str) -> Path:
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    path = PROCESSED_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def update_data_warnings(stage: str, warnings: list[dict[str, Any]]) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    path = RESULTS_DIR / "data_warnings.json"
    if path.exists():
        try:
            payload = json.loads(path.read_text(encoding="utf-8"))
        except json.JSONDecodeError:
            payload = {}
    else:
        payload = {}
    payload.setdefault("stage_warnings", {})[stage] = warnings
    has_errors = bool(payload.get("errors"))
    has_warnings = bool(payload.get("warnings")) or any(payload.get("stage_warnings", {}).values())
    payload["status"] = "FAIL" if has_errors else ("WARN" if has_warnings else "PASS")
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")


def zscore(series: pd.Series) -> pd.Series:
    values = pd.to_numeric(series, errors="coerce")
    std = values.std(skipna=True)
    if not np.isfinite(std) or std == 0:
        return values * np.nan
    return (values - values.mean(skipna=True)) / std


def log_positive(series: pd.Series) -> pd.Series:
    values = pd.to_numeric(series, errors="coerce")
    result = pd.Series(np.nan, index=series.index, dtype=float)
    mask = values > 0
    result.loc[mask] = np.log(values.loc[mask])
    return result


def month_end_from_period(period: pd.Series) -> pd.Series:
    return pd.to_datetime(period.astype(str) + "-01") + pd.offsets.MonthEnd(0)


def quarter_label(dates: pd.Series) -> pd.Series:
    periods = pd.PeriodIndex(pd.to_datetime(dates), freq="Q")
    return pd.Series([f"{period.year}-Q{period.quarter}" for period in periods], index=dates.index)


def add_event_stage(frame: pd.DataFrame) -> pd.DataFrame:
    result = frame.copy()
    date = pd.to_datetime(result["date"])
    trade_mask = date.ge(EVENT_E1_CALENDAR_START) & result["brent_usd_bbl"].notna() & result["wti_usd_bbl"].notna()
    trading_dates = date.loc[trade_mask].drop_duplicates().sort_values().tolist()
    if len(trading_dates) < 4:
        raise ValueError("Cannot map E1 weekend event to at least four post-event common trading days.")
    e1_start = pd.Timestamp(trading_dates[0])
    e1_car_end = pd.Timestamp(trading_dates[2])
    e2_start = pd.Timestamp(trading_dates[3])
    e3_candidates = date.loc[
        date.ge(EVENT_E3_CALENDAR_START) & result["brent_usd_bbl"].notna() & result["wti_usd_bbl"].notna()
    ]
    if e3_candidates.empty:
        raise ValueError("Cannot map E3 easing event to a common trading day.")
    e3_start = pd.Timestamp(e3_candidates.min())

    result["war_stage"] = "prewar"
```

```
    result.loc[date.between(e1_start, e1_car_end), "war_stage"] = "E1_immediate_window"
    result.loc[date.between(e2_start, e3_start - pd.Timedelta(days=1)), "war_stage"] = "E2_disruption"
    result.loc[date >= e3_start, "war_stage"] = "E3_easing"
    result["war_on"] = (date >= e1_start).astype(int)
    result["stage_E1"] = date.between(e1_start, e1_car_end).astype(int)
    result["stage_E2"] = (result["war_stage"].eq("E2_disruption")).astype(int)
    result["stage_E3"] = (result["war_stage"].eq("E3_easing")).astype(int)
    result["event_e1_calendar_start"] = EVENT_E1_CALENDAR_START.strftime("%Y-%m-%d")
    result["event_e1_trading_start"] = e1_start.strftime("%Y-%m-%d")
    result["event_e1_car_end_0_2"] = e1_car_end.strftime("%Y-%m-%d")
    result["event_e2_trading_start"] = e2_start.strftime("%Y-%m-%d")
    result["event_e3_trading_start"] = e3_start.strftime("%Y-%m-%d")
    return result


def fetch_oecd_china_gdp(warnings: list[dict[str, Any]]) -> pd.DataFrame:
    merged: pd.DataFrame | None = None
    source_urls: dict[str, str] = {}
    for value_column, url in OECD_GDP_URLS.items():
        try:
            response = requests.get(url, timeout=30, headers={"User-Agent": "Mozilla/5.0"})
            response.raise_for_status()
            frame = pd.read_csv(io.StringIO(response.text))
            frame = frame.loc[frame["REF_AREA"].eq("CHN"), ["TIME_PERIOD", "OBS_VALUE"]].copy()
            frame = frame.rename(columns={"TIME_PERIOD": "quarter", "OBS_VALUE": value_column})
            source_urls[value_column] = url
        except Exception as exc:
            warnings.append({"code": "oecd_gdp_fetch_failed", "message": f"{value_column}: {exc}"})
            frame = pd.DataFrame(columns=["quarter", value_column])
        merged = frame if merged is None else merged.merge(frame, on="quarter", how="outer")

    if merged is None or merged.empty:
        return pd.DataFrame(columns=["quarter", "quarter_end", "china_real_gdp_yoy_pct", "china_real_gdp_qoq_pct", "source_note"])

    if "china_real_gdp_yoy_pct" not in merged.columns:
        merged["china_real_gdp_yoy_pct"] = np.nan
    if "china_real_gdp_qoq_pct" not in merged.columns:
        merged["china_real_gdp_qoq_pct"] = np.nan
    cached_gdp = PROCESSED_DIR / "china_real_gdp_quarterly.csv"
    if merged["china_real_gdp_yoy_pct"].dropna().shape[0] < 40 and cached_gdp.exists():
        cached = pd.read_csv(cached_gdp)
        if "china_real_gdp_yoy_pct" in cached.columns and cached["china_real_gdp_yoy_pct"].dropna().shape[0] >= 40:
            warnings.append(
                {
                    "code": "oecd_gdp_cached_history_used",
                    "message": "OECD GDP online response lacked usable yoy history; retained existing processed GDP table instead of
  overwriting it.",
                }
            )
            return cached

    q2_mask = merged["quarter"].eq("2026-Q2")
    if not q2_mask.any():
        merged = pd.concat(
            [
                merged,
                pd.DataFrame(
                    [
                        {
                            "quarter": "2026-Q2",
                            "china_real_gdp_yoy_pct": 4.3,
                            "china_real_gdp_qoq_pct": 0.9,
                        }
                    ]
                ),
            ],
            ignore_index=True,
        )
        warnings.append(
            {
                "code": "nbs_gdp_manual_supplement",
                "message": "2026-Q2 GDP yoy=4.3 and qoq=0.9 added from NBS 2026-07-15 release.",
                "url": NBS_2026_Q2_URL,
            }
        )
    else:
```

```
        if merged.loc[q2_mask, "china_real_gdp_yoy_pct"].isna().all():
            merged.loc[q2_mask, "china_real_gdp_yoy_pct"] = 4.3
            warnings.append(
                {
                    "code": "nbs_gdp_yoy_manual_supplement",
                    "message": "OECD GY lacks 2026-Q2; GDP yoy=4.3 added from NBS 2026-07-15 release.",
                    "url": NBS_2026_Q2_URL,
                }
            )
        if merged.loc[q2_mask, "china_real_gdp_qoq_pct"].isna().all():
            merged.loc[q2_mask, "china_real_gdp_qoq_pct"] = 0.9

    merged["quarter_end"] = pd.PeriodIndex(merged["quarter"], freq="Q").to_timestamp(how="end").normalize()
    merged = merged.loc[merged["quarter_end"] <= CUTOFF].sort_values("quarter").reset_index(drop=True)
    merged["source_note"] = "OECD DF_QNA_EXPENDITURE_GROWTH_OECD; 2026-Q2 yoy supplemented from NBS when absent"
    save_processed(merged, "china_real_gdp_quarterly.csv")
    return merged


def build_daily_q1() -> pd.DataFrame:
    daily = read_processed("p0_daily_market.csv", parse_dates=["date"]).sort_values("date")
    daily = daily.loc[daily["date"].le(CUTOFF)].copy()
    for column in ["brent_usd_bbl", "wti_usd_bbl", "usd_broad_index", "cny_per_usd", "jpy_per_usd", "krw_per_usd", "eur_per_usd"]:
        daily[f"log_{column}"] = log_positive(daily[column])
        daily[f"{column}_log_return"] = daily[f"log_{column}"].diff()
    daily = add_event_stage(daily)
    columns = [
        "date",
        "brent_usd_bbl",
        "wti_usd_bbl",
        "usd_broad_index",
        "cny_per_usd",
        "jpy_per_usd",
        "krw_per_usd",
        "eur_per_usd",
        "brent_usd_bbl_log_return",
        "wti_usd_bbl_log_return",
        "usd_broad_index_log_return",
        "cny_per_usd_log_return",
        "war_stage",
        "war_on",
        "stage_E1",
        "stage_E2",
        "stage_E3",
        "event_e1_calendar_start",
        "event_e1_trading_start",
        "event_e1_car_end_0_2",
        "event_e2_trading_start",
        "event_e3_trading_start",
    ]
    output = daily[columns]
    save_processed(output, "model_daily_q1.csv")
    return output


def build_monthly_q1() -> pd.DataFrame:
    monthly = read_processed("p0_monthly_market.csv", parse_dates=["month_end"]).sort_values("period")
    monthly = monthly.loc[monthly["month_end"].le(CUTOFF)].copy()
    monthly["month_start"] = pd.to_datetime(monthly["period"] + "-01")
    for column in ["brent_usd_bbl", "wti_usd_bbl", "usd_broad_index", "cny_per_usd", "us_crude_stock_month_end_kbbl", "GPR"]:
        monthly[f"log_{column}"] = log_positive(monthly[column])
        monthly[f"{column}_log_return"] = monthly[f"log_{column}"].diff()
        monthly[f"{column}_lag1"] = monthly[column].shift(1)
        monthly[f"{column}_log_return_lag1"] = monthly[f"{column}_log_return"].shift(1)
    monthly["GPR_z"] = zscore(monthly["GPR"])
    monthly["GPR_z_lag1"] = monthly["GPR_z"].shift(1)
    monthly["stock_log_lag1"] = monthly["log_us_crude_stock_month_end_kbbl"].shift(1)
    monthly["log_brent"] = monthly["log_brent_usd_bbl"]
    output = monthly[
        [
            "period",
            "month_start",
            "month_end",
            "brent_usd_bbl",
            "wti_usd_bbl",
            "log_brent",
```

```
            "brent_usd_bbl_log_return",
            "wti_usd_bbl_log_return",
            "usd_broad_index",
            "usd_broad_index_log_return",
            "usd_broad_index_log_return_lag1",
            "cny_per_usd",
            "cny_per_usd_log_return",
            "jpy_per_usd",
            "krw_per_usd",
            "usd_per_eur",
            "eur_per_usd",
            "stock_reference_date",
            "us_crude_stock_month_end_kbbl",
            "stock_log_lag1",
            "GPR",
            "GPR_z",
            "GPR_z_lag1",
            "GPRT",
            "GPRA",
        ]
    ]
    save_processed(output, "model_monthly_q1.csv")
    return output


def load_china_macro(monthly_q1: pd.DataFrame, warnings: list[dict[str, Any]]) -> pd.DataFrame:
    cpi = read_processed("oecd_g20_cpi_monthly.csv")
    cpi = cpi.loc[cpi["REF_AREA"].eq("CHN"), ["TIME_PERIOD", "OBS_VALUE"]].copy()
    cpi = cpi.rename(columns={"TIME_PERIOD": "period", "OBS_VALUE": "china_cpi_yoy_pct"})

    macro = monthly_q1[
        [
            "period",
            "month_end",
            "brent_usd_bbl",
            "log_brent",
            "brent_usd_bbl_log_return",
            "cny_per_usd",
            "cny_per_usd_log_return",
            "usd_broad_index",
            "usd_broad_index_log_return",
            "GPR",
            "GPR_z",
        ]
    ].merge(cpi, on="period", how="left")
    macro["china_fx_log_change_pct"] = macro["cny_per_usd_log_return"] * 100.0
    macro["brent_cny_cost"] = macro["brent_usd_bbl"] * macro["cny_per_usd"]
    macro["brent_cny_cost_log_level_pct"] = log_positive(macro["brent_cny_cost"]) * 100.0
    macro["brent_cny_cost_log_change_pct"] = macro["brent_cny_cost_log_level_pct"].diff()
    macro["covid_phase"] = (
        pd.to_datetime(macro["period"] + "-01").between(pd.Timestamp("2020-02-01"), pd.Timestamp("2022-12-01"))
    ).astype(int)

    optional_files = {
        "nbs_iav_monthly.csv": ("china_iav_yoy_pct", "china_iav_mom_sa_pct"),
        "nbs_ppi_monthly.csv": ("china_ppi_yoy_pct",),
    }
    for filename, columns in optional_files.items():
        path = PROCESSED_DIR / filename
        if not path.exists():
            warnings.append({"code": "optional_china_macro_missing", "message": f"{filename} absent; Q2 will run available CPI/FX/GDP
  modules."})
            for column in columns:
                macro[column] = np.nan
            continue
        frame = pd.read_csv(path)
        keep = ["period"] + [column for column in columns if column in frame.columns]
        macro = macro.merge(frame[keep], on="period", how="left")
        for column in columns:
            if column not in macro.columns:
                macro[column] = np.nan

    q1_shocks_path = RESULTS_DIR / "q1_monthly_shocks.csv"
    if q1_shocks_path.exists():
        shocks = pd.read_csv(q1_shocks_path)
        shock_columns = [
```

```
            column
            for column in [
                "period",
                "OilShock",
                "ARBaselineGap",
                "OilShock_source",
                "supply_shock",
                "aggregate_demand_shock",
                "oil_specific_risk_shock",
                "reduced_form_shock",
                "source_vintage",
            ]
            if column in shocks.columns
        ]
        macro = macro.merge(shocks[shock_columns], on="period", how="left")
        macro["OilShock_source"] = macro.get("OilShock_source", pd.Series(index=macro.index, dtype=object)).fillna("
    q1_monthly_forecast_residual")
        if "ARBaselineGap" not in macro.columns:
            macro["ARBaselineGap"] = 0.0
        for column in ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock", "reduced_form_shock"]:
            if column not in macro.columns:
                macro[column] = np.nan
        if "source_vintage" not in macro.columns:
            macro["source_vintage"] = ""
    else:
        macro["OilShock"] = zscore(macro["brent_usd_bbl_log_return"])
        macro["ARBaselineGap"] = 0.0
        macro["OilShock_source"] = "proxy_brent_log_return_until_q1_runs"
        macro["supply_shock"] = np.nan
        macro["aggregate_demand_shock"] = np.nan
        macro["oil_specific_risk_shock"] = np.nan
        macro["reduced_form_shock"] = macro["OilShock"]
        macro["source_vintage"] = "fallback_brent_return"
        warnings.append({"code": "q1_shocks_not_yet_available", "message": "Using standardized Brent monthly log return as temporary
    OilShock proxy."})

    save_processed(macro, "model_monthly_cn.csv")
    return macro


def build_quarterly_cn(monthly_cn: pd.DataFrame, gdp: pd.DataFrame) -> pd.DataFrame:
    frame = monthly_cn.copy()
    frame["quarter"] = quarter_label(frame["period"] + "-01")
    quarterly = (
        frame.groupby("quarter", as_index=False)
        .agg(
            quarter_end=("month_end", "max"),
            OilShock_sum=("OilShock", "sum"),
            OilShock_mean=("OilShock", "mean"),
            supply_shock_sum=("supply_shock", "sum"),
            aggregate_demand_shock_sum=("aggregate_demand_shock", "sum"),
            oil_specific_risk_shock_sum=("oil_specific_risk_shock", "sum"),
            reduced_form_shock_sum=("reduced_form_shock", "sum"),
            ARBaselineGap_mean=("ARBaselineGap", "mean"),
            brent_log_return_sum=("brent_usd_bbl_log_return", "sum"),
            china_cpi_yoy_pct_mean=("china_cpi_yoy_pct", "mean"),
            china_fx_log_change_pct_sum=("china_fx_log_change_pct", "sum"),
        )
        .sort_values("quarter")
    )
    if not gdp.empty:
        quarterly = quarterly.merge(
            gdp[["quarter", "china_real_gdp_yoy_pct", "china_real_gdp_qoq_pct", "source_note"]],
            on="quarter",
            how="left",
        )
    else:
        quarterly["china_real_gdp_yoy_pct"] = np.nan
        quarterly["china_real_gdp_qoq_pct"] = np.nan
        quarterly["source_note"] = "GDP unavailable"
    save_processed(quarterly, "model_quarterly_cn.csv")
    return quarterly


def build_china_policy_monthly(monthly_q1: pd.DataFrame) -> pd.DataFrame:
    policy = read_processed("cn_fuel_policy_events.csv", parse_dates=["effective_date"])
```

```
    months = pd.DataFrame({"period": monthly_q1["period"]})
    monthly_policy = policy.copy()
    monthly_policy["period"] = monthly_policy["effective_date"].dt.to_period("M").astype(str)
    monthly_policy = (
        monthly_policy.groupby("period", as_index=False)
        .agg(
            gasoline_actual_adjustment_cny_t=("gasoline_actual_adjustment_cny_t", "sum"),
            gasoline_rule_adjustment_cny_t=("gasoline_rule_adjustment_cny_t", "sum"),
            gasoline_policy_gap_cny_t=("gasoline_policy_gap_cny_t", "sum"),
            diesel_actual_adjustment_cny_t=("diesel_actual_adjustment_cny_t", "sum"),
            diesel_rule_adjustment_cny_t=("diesel_rule_adjustment_cny_t", "sum"),
            diesel_policy_gap_cny_t=("diesel_policy_gap_cny_t", "sum"),
        )
    )
    result = months.merge(monthly_policy, on="period", how="left").fillna(0.0)
    for column in [
        "gasoline_actual_adjustment_cny_t",
        "gasoline_rule_adjustment_cny_t",
        "gasoline_policy_gap_cny_t",
        "diesel_actual_adjustment_cny_t",
        "diesel_rule_adjustment_cny_t",
        "diesel_policy_gap_cny_t",
    ]:
        result[f"cum_{column}"] = result[column].cumsum()
    save_processed(result, "china_fuel_policy_monthly.csv")
    return result


def eastmoney_oil_request(params: dict[str, Any]) -> dict[str, Any]:
    last_error: Exception | None = None
    for _ in range(4):
        try:
            response = requests.get(
                EASTMONEY_DATACENTER_URL,
                params=params,
                timeout=45,
                headers={"User-Agent": "Mozilla/5.0", "Referer": EASTMONEY_OIL_PAGE_URL},
                verify=False,
            )
            response.raise_for_status()
            payload = response.json()
            break
        except (requests.RequestException, ValueError) as exc:
            last_error = exc
    else:
        raise RuntimeError(f"EastMoney oil API request failed after retries: {last_error}")
    result = payload.get("result")
    if not isinstance(result, dict):
        raise ValueError(f"EastMoney oil API returned no result for {params.get('reportName')}")
    return result


def fetch_eastmoney_oil_adjustments() -> pd.DataFrame:
    result = eastmoney_oil_request(
        {
            "reportName": "RPTA_WEB_YJ_BD",
            "columns": "ALL",
            "sortColumns": "DIM_DATE",
            "sortTypes": "-1",
            "pageNumber": 1,
            "pageSize": 500,
            "source": "WEB",
        }
    )
    rows = result.get("data") or []
    frame = pd.DataFrame(rows)
    if frame.empty:
        raise ValueError("EastMoney oil adjustment table is empty.")
    frame["effective_date"] = pd.to_datetime(frame["DIM_DATE"], errors="coerce")
    frame = frame.loc[frame["effective_date"].between(pd.Timestamp("2013-03-27"), CUTOFF)].copy()
    for column in ["VALUE", "CY_JG", "QY_FD", "CY_FD"]:
        frame[column] = pd.to_numeric(frame[column], errors="coerce")
    return frame.sort_values("effective_date").reset_index(drop=True)


def load_spain_hicp_fallback(warnings: list[dict[str, Any]]) -> pd.DataFrame:
```

```
    target = PROCESSED_DIR / "eurostat_spain_hicp_yoy_monthly.csv"
    if target.exists():
        return pd.read_csv(target)
    try:
        response = requests.get(EUROSTAT_SPAIN_HICP_YOY_URL, timeout=45, headers={"User-Agent": "Mozilla/5.0"})
        response.raise_for_status()
        payload = response.json()
    except requests.RequestException as exc:
        warnings.append({"code": "eurostat_spain_cpi_fetch_failed", "message": str(exc)})
        return pd.DataFrame(columns=["country", "period", "cpi_yoy_pct", "source_url"])
    dimensions = payload.get("dimension", {})
    time_index = dimensions.get("time", {}).get("category", {}).get("index", {})
    values = payload.get("value", {})
    if not time_index:
        return pd.DataFrame(columns=["country", "period", "cpi_yoy_pct", "source_url"])
    rows = []
    for period, idx in time_index.items():
        value = values.get(str(idx))
        rows.append(
            {
                "country": "ESP",
                "period": period,
                "cpi_yoy_pct": pd.to_numeric(value, errors="coerce"),
                "source_url": EUROSTAT_SPAIN_HICP_YOY_URL,
            }
        )
    frame = pd.DataFrame(rows).dropna(subset=["period", "cpi_yoy_pct"]).sort_values("period")
    save_processed(frame, "eurostat_spain_hicp_yoy_monthly.csv")
    return frame


def policy_gap_lookup() -> dict[str, dict[str, float]]:
    policy_path = PROCESSED_DIR / "cn_fuel_policy_events.csv"
    if not policy_path.exists():
        return {}
    policy = pd.read_csv(policy_path)
    lookup: dict[str, dict[str, float]] = {}
    for row in policy.to_dict("records"):
        announcement = pd.Timestamp(row["effective_date"])
        api_effective = (announcement + pd.Timedelta(days=1)).strftime("%Y-%m-%d")
        lookup[api_effective] = {
            "gasoline_rule_adjustment_cny_t": float(row["gasoline_rule_adjustment_cny_t"]),
            "gasoline_policy_gap_cny_t": float(row["gasoline_policy_gap_cny_t"]),
            "source_artifact_id": str(row.get("source_artifact_id", "")),
        }
    return lookup


def load_or_build_china_adjustment_events() -> pd.DataFrame:
    target = PROCESSED_DIR / "cn_fuel_adjustment_events.csv"
    if target.exists():
        events = pd.read_csv(target)
        enough_history = (
            "national_gasoline_price_cny_per_ton" in events.columns
            and pd.to_datetime(events["effective_date"], errors="coerce").between(pd.Timestamp("2013-03-27"), CUTOFF).sum() >= 48
        )
        if enough_history and "price_anchor_valid_from" not in events.columns:
            events["price_anchor_valid_from"] = events["effective_date"]
        if enough_history:
            events.loc[events["effective_date"].astype(str).eq("2026-03-24"), "price_anchor_valid_from"] = "2013-03-27"
        else:
            events = pd.DataFrame()
    if not target.exists() or events.empty:
        adjustments = fetch_eastmoney_oil_adjustments()
        policy = read_processed("cn_fuel_policy_events.csv")
        gap_lookup = policy_gap_lookup()
        price_anchors = {
            "2026-03-24": {
                "announcement_date": "2026-03-23",
                "source_url": BJ_2026_03_PRICE_URL,
                "beijing_92_price_before_cny_per_l": 7.64,
                "beijing_92_price_after_cny_per_l": 8.57,
                "price_anchor_valid_from": "2026-03-24",
            },
            "2026-04-08": {
                "announcement_date": "2026-04-07",
```

```
                "source_url": BJ_2026_04_PRICE_URL,
                "beijing_92_price_before_cny_per_l": 8.57,
                "beijing_92_price_after_cny_per_l": 8.90,
                "price_anchor_valid_from": "2026-04-08",
            },
            "2026-04-22": {
                "announcement_date": "2026-04-21",
                "source_url": BJ_2026_04_21_PRICE_URL,
                "beijing_92_price_before_cny_per_l": 8.90,
                "beijing_92_price_after_cny_per_l": 8.46,
                "price_anchor_valid_from": "2026-04-22",
            },
        }
        rows: list[dict[str, Any]] = []
        for row in adjustments.to_dict("records"):
            effective = pd.Timestamp(row["effective_date"]).strftime("%Y-%m-%d")
            anchor = price_anchors.get(effective, {})
            gap = gap_lookup.get(effective, {})
            actual = float(row["QY_FD"]) if pd.notna(row.get("QY_FD")) else np.nan
            before = anchor.get("beijing_92_price_before_cny_per_l", np.nan)
            after = anchor.get("beijing_92_price_after_cny_per_l", np.nan)
            litre_change = after - before if np.isfinite(before) and np.isfinite(after) else np.nan
            litres_per_ton = actual / litre_change if np.isfinite(litre_change) and abs(litre_change) > 1e-12 else np.nan
            rows.append(
                {
                    "announcement_date": anchor.get("announcement_date", effective),
                    "effective_date": effective,
                    "product": "gasoline_92_beijing",
                    "actual_adjustment_cny_per_ton": actual,
                    "rule_implied_adjustment_cny_per_ton": float(gap.get("gasoline_rule_adjustment_cny_t", actual)),
                    "carried_gap_cny_per_ton": float(gap.get("gasoline_policy_gap_cny_t", 0.0)),
                    "national_gasoline_price_cny_per_ton": float(row["VALUE"]) if pd.notna(row.get("VALUE")) else np.nan,
                    "national_diesel_price_cny_per_ton": float(row["CY_JG"]) if pd.notna(row.get("CY_JG")) else np.nan,
                    "beijing_92_price_before_cny_per_l": before,
                    "beijing_92_price_after_cny_per_l": after,
                    "price_anchor_valid_from": anchor.get("price_anchor_valid_from", effective),
                    "observed_litres_per_ton": litres_per_ton,
                    "policy_type": "temporary_control" if float(gap.get("gasoline_policy_gap_cny_t", 0.0)) else "mechanism_adjustment",
                    "source_url": anchor.get("source_url", EASTMONEY_OIL_PAGE_URL),
                    "source_artifact_id": gap.get("source_artifact_id", "eastmoney_regulated_oil_price_datacenter"),
                    "verification_status": "official_2026_anchor_or_public_regulated_price_table",
                    "coverage_note": "Historical regulated standard gasoline cap reconstructed from EastMoney national oil-price datacenter; 2026 temporary-control gaps cross-checked with NDRC and Beijing DRC notices.",
                }
            )
        events = pd.DataFrame(rows)
        save_processed(events, "cn_fuel_adjustment_events.csv")

    events["effective_date"] = pd.to_datetime(events["effective_date"])
    events = events.loc[events["product"].eq("gasoline_92_beijing")].copy()
    events = events.sort_values("effective_date").reset_index(drop=True)
    save_processed(events, "cn_fuel_adjustment_events.csv")
    return events


def build_china_regulated_gasoline_monthly(monthly_q1: pd.DataFrame, warnings: list[dict[str, Any]]) -> pd.DataFrame:
    events = load_or_build_china_adjustment_events()
    months = monthly_q1[["period"]].copy()
    months["month_start"] = pd.to_datetime(months["period"] + "-01")
    months["month_end"] = months["month_start"] + pd.offsets.MonthEnd(0)

    anchored_events = events.dropna(subset=["national_gasoline_price_cny_per_ton"]).copy() if "national_gasoline_price_cny_per_ton" in events.columns else pd.DataFrame()
    if anchored_events.empty:
        result = months.copy()
        result["china_regulated_gasoline_cny_per_l"] = np.nan
        result["china_regulated_gasoline_cny_per_ton"] = np.nan
        result["no_temporary_control_gasoline_cny_per_l"] = np.nan
        result["no_temporary_control_gasoline_cny_per_ton"] = np.nan
        result["china_regulated_gasoline_index"] = np.nan
        result["coverage_status"] = "missing_official_price_anchor"
        result["measure_type"] = "regulated_gasoline_adjustment_index_proxy"
        result["source_url"] = ""
        save_processed(result, "china_regulated_gasoline_monthly.csv")
        warnings.append(
            {
```

```
                "code": "china_official_fuel_anchor_missing",
                "message": "No auditable fixed-product official price anchor is available; China cannot enter the main Q3 fuel comparison.",
            }
        )
        return result

    first_event = anchored_events.iloc[0]
    valid_from = pd.Timestamp(first_event.get("price_anchor_valid_from", pd.NaT))
    if pd.isna(valid_from):
        valid_from = pd.Timestamp(first_event["effective_date"]).replace(day=1)
    calendar = pd.DataFrame({"date": pd.date_range(valid_from, CUTOFF, freq="D")})
    calendar["china_regulated_gasoline_cny_per_ton"] = float(first_event["national_gasoline_price_cny_per_ton"])
    calendar["china_regulated_gasoline_cny_per_l"] = np.nan
    calendar["observed_litres_per_ton"] = np.nan
    calendar["source_url"] = str(first_event["source_url"])
    for row in anchored_events.itertuples(index=False):
        mask = calendar["date"].ge(pd.Timestamp(row.effective_date))
        calendar.loc[mask, "china_regulated_gasoline_cny_per_ton"] = float(row.national_gasoline_price_cny_per_ton)
        if hasattr(row, "beijing_92_price_after_cny_per_l") and np.isfinite(float(row.beijing_92_price_after_cny_per_l)):
            calendar.loc[mask, "china_regulated_gasoline_cny_per_l"] = float(row.beijing_92_price_after_cny_per_l)
        if hasattr(row, "observed_litres_per_ton") and np.isfinite(float(row.observed_litres_per_ton)):
            calendar.loc[mask, "observed_litres_per_ton"] = float(row.observed_litres_per_ton)
        calendar.loc[mask, "source_url"] = str(row.source_url)

    calendar["period"] = calendar["date"].dt.to_period("M").astype(str)
    monthly_price = (
        calendar.groupby("period", as_index=False)
        .agg(
            china_regulated_gasoline_cny_per_l=("china_regulated_gasoline_cny_per_l", "mean"),
            china_regulated_gasoline_cny_per_ton=("china_regulated_gasoline_cny_per_ton", "mean"),
            observed_litres_per_ton=("observed_litres_per_ton", "mean"),
            price_observation_days=("china_regulated_gasoline_cny_per_l", "size"),
            source_url=("source_url", "last"),
        )
    )

    policy = build_china_policy_monthly(monthly_q1)
    result = months.merge(monthly_price, on="period", how="left")
    result = result.merge(
        policy[["period", "gasoline_policy_gap_cny_t", "cum_gasoline_policy_gap_cny_t"]],
        on="period",
        how="left",
    )
    result["gasoline_policy_gap_cny_t"] = result["gasoline_policy_gap_cny_t"].fillna(0.0)
    result["cum_gasoline_policy_gap_cny_t"] = result["cum_gasoline_policy_gap_cny_t"].fillna(0.0)
    result["china_regulated_gasoline_cny_per_ton"] = pd.to_numeric(result["china_regulated_gasoline_cny_per_ton"], errors="coerce")
    result["no_temporary_control_gasoline_cny_per_l"] = result["china_regulated_gasoline_cny_per_l"] + (
        result["cum_gasoline_policy_gap_cny_t"] / result["observed_litres_per_ton"]
    )
    result["no_temporary_control_gasoline_cny_per_ton"] = (
        result["china_regulated_gasoline_cny_per_ton"] + result["cum_gasoline_policy_gap_cny_t"]
    )
    base = result["china_regulated_gasoline_cny_per_ton"].dropna()
    result["china_regulated_gasoline_index"] = (
        100.0 * result["china_regulated_gasoline_cny_per_ton"] / float(base.iloc[0]) if not base.empty else np.nan
    )
    nonmissing = int(result["china_regulated_gasoline_cny_per_ton"].notna().sum())
    result["coverage_status"] = np.where(
        result["china_regulated_gasoline_cny_per_ton"].notna(),
        "public_adjustment_series_reconstructed_proxy",
        "missing_before_official_anchor",
    )
    result["measure_type"] = "regulated_gasoline_adjustment_index_proxy"
    result["coverage_months_nonmissing"] = nonmissing
    result["main_comparison_ready"] = False
    save_processed(result, "china_regulated_gasoline_monthly.csv")
    if nonmissing < CHINA_MAIN_FUEL_MIN_MONTHS:
        warnings.append(
            {
                "code": "china_official_fuel_coverage_limited",
                "message": f"China reconstructed regulated-gasoline adjustment proxy has {nonmissing} nonmissing months but is excluded from the main price-level comparison because the historical source is not a fixed-product official retail-cap series.",
            }
        )
    return result
```

```python


def build_policy_buffer_table() -> pd.DataFrame:
    target = PROCESSED_DIR / "country_policy_buffers_annual.csv"
    if target.exists():
        existing = pd.read_csv(target)
        needed = ["oil_import_dependency", "oil_intensity", "import_source_hhi"]
        if all(column in existing.columns for column in needed) and existing[needed].notna().all().all():
            return existing
    profiles = {
        "CHN": {"dep_start": 0.54, "dep_end": 0.73, "int_start": 0.060, "int_end": 0.044, "hhi_start": 0.095, "hhi_end": 0.070, "reg": 1.0},
        "DEU": {"dep_start": 0.96, "dep_end": 0.98, "int_start": 0.042, "int_end": 0.030, "hhi_start": 0.120, "hhi_end": 0.105, "reg": 0.0},
        "FRA": {"dep_start": 0.98, "dep_end": 0.99, "int_start": 0.038, "int_end": 0.027, "hhi_start": 0.115, "hhi_end": 0.105, "reg": 0.0},
        "ITA": {"dep_start": 0.92, "dep_end": 0.95, "int_start": 0.043, "int_end": 0.032, "hhi_start": 0.125, "hhi_end": 0.115, "reg": 0.0},
        "ESP": {"dep_start": 0.99, "dep_end": 0.99, "int_start": 0.046, "int_end": 0.034, "hhi_start": 0.110, "hhi_end": 0.100, "reg": 0.0},
        "JPN": {"dep_start": 0.997, "dep_end": 0.999, "int_start": 0.049, "int_end": 0.035, "hhi_start": 0.155, "hhi_end": 0.140, "reg": 0.0},
        "KOR": {"dep_start": 0.995, "dep_end": 0.998, "int_start": 0.058, "int_end": 0.044, "hhi_start": 0.145, "hhi_end": 0.130, "reg": 0.0},
    }
    years = range(2010, 2027)
    rows: list[dict[str, Any]] = []
    for country in COUNTRY_META:
        for year in years:
            profile = profiles[country]
            t = (year - 2010) / (2026 - 2010)
            rows.append(
                {
                    "country": country,
                    "year": year,
                    "oil_import_dependency": profile["dep_start"] + (profile["dep_end"] - profile["dep_start"]) * t,
                    "oil_intensity": profile["int_start"] + (profile["int_end"] - profile["int_start"]) * t,
                    "fuel_price_regulation": profile["reg"],
                    "import_source_hhi": profile["hhi_start"] + (profile["hhi_end"] - profile["hhi_start"]) * t,
                    "source_url": "EIA annual petroleum balance, World Bank constant-GDP series and UN Comtrade HS2709 design target; current table is normalized public-source proxy pending audited manual export.",
                    "buffer_note": "Annual structural buffer proxy; continuous variables are lagged one year in Q3 interaction models. fuel_price_regulation is mechanism-only because China is the single strong regulated-price country.",
                }
            )
    result = pd.DataFrame(rows)
    save_processed(result, "country_policy_buffers_annual.csv")
    return result


def make_country_rows(
    country: str,
    country_name: str,
    monthly_q1: pd.DataFrame,
    fuel: pd.DataFrame,
    fuel_column: str,
    fuel_unit: str,
    brent_local: pd.Series,
    fuel_source: str,
    price_measure_type: str,
    observed_or_regulated: str,
    included_in_main_comparison: bool,
    comparability_note: str,
) -> pd.DataFrame:
    rows = monthly_q1[["period", "month_end", "brent_usd_bbl", "GPR", "cny_per_usd", "jpy_per_usd", "krw_per_usd"]].copy()
    rows = rows.merge(fuel[["period", fuel_column]], on="period", how="left")
    rows = rows.rename(columns={fuel_column: "fuel_price_local"})
    rows["country"] = country
    rows["country_name"] = country_name
    rows["fuel_unit"] = fuel_unit
    rows["fuel_source"] = fuel_source
    rows["price_measure_type"] = price_measure_type
    rows["observed_or_regulated"] = observed_or_regulated
    rows["included_in_main_comparison"] = included_in_main_comparison
    rows["comparability_note"] = comparability_note
    rows["brent_local_per_bbl"] = brent_local.to_numpy()
```

```
    rows["fuel_log"] = log_positive(rows["fuel_price_local"])
    rows["fuel_log_return"] = rows["fuel_log"].diff()
    rows["brent_local_log"] = log_positive(rows["brent_local_per_bbl"])
    rows["brent_local_log_return"] = rows["brent_local_log"].diff()
    return rows


def build_country_monthly(monthly_q1: pd.DataFrame, monthly_cn: pd.DataFrame, warnings: list[dict[str, Any]]) -> pd.DataFrame:
    cpi = read_processed("oecd_g20_cpi_monthly.csv")
    cpi = cpi[["REF_AREA", "TIME_PERIOD", "OBS_VALUE"]].rename(
        columns={"REF_AREA": "country", "TIME_PERIOD": "period", "OBS_VALUE": "cpi_yoy_pct"}
    )
    spain_cpi = load_spain_hicp_fallback(warnings)
    if not spain_cpi.empty:
        cpi = cpi.loc[~(cpi["country"].eq("ESP") & cpi["cpi_yoy_pct"].isna())].copy()
        cpi = pd.concat([cpi, spain_cpi[["country", "period", "cpi_yoy_pct"]]], ignore_index=True)
        cpi = cpi.sort_values(["country", "period"]).drop_duplicates(["country", "period"], keep="last")
    ip = read_processed("oecd_kei_ip_monthly.csv")
    ip = ip[["REF_AREA", "TIME_PERIOD", "OBS_VALUE"]].rename(
        columns={"REF_AREA": "country", "TIME_PERIOD": "period", "OBS_VALUE": "ip_index"}
    )

    germany = read_processed("germany_eurosuper95_monthly.csv")
    france = read_processed("france_eurosuper95_monthly.csv")
    italy = read_processed("italy_eurosuper95_monthly.csv")
    spain = read_processed("spain_eurosuper95_monthly.csv")
    japan = read_processed("japan_regular_gasoline_monthly.csv")
    korea = read_processed("korea_regular_gasoline_monthly.csv")

    policy = build_china_policy_monthly(monthly_q1)
    china_fuel = monthly_q1[["period", "brent_usd_bbl", "cny_per_usd"]].merge(policy, on="period", how="left")
    china_fuel["brent_cny_per_tonne_proxy"] = china_fuel["brent_usd_bbl"] * china_fuel["cny_per_usd"] / BARREL_TO_TONNE
    china_fuel["china_gasoline_proxy_cny_t"] = (
        china_fuel["brent_cny_per_tonne_proxy"] - china_fuel["cum_gasoline_policy_gap_cny_t"]
    ).clip(lower=1.0)
    china_fuel["china_gasoline_rule_proxy_cny_t"] = china_fuel["brent_cny_per_tonne_proxy"].clip(lower=1.0)
    official_china_fuel = build_china_regulated_gasoline_monthly(monthly_q1, warnings)
    official_months = int(official_china_fuel["china_regulated_gasoline_cny_per_ton"].notna().sum())
    china_official_ready = official_months >= CHINA_MAIN_FUEL_MIN_MONTHS
    if official_months == 0:
        china_panel_fuel = china_fuel
        china_fuel_column = "china_gasoline_proxy_cny_t"
        china_fuel_unit = "CNY/tonne proxy"
        china_fuel_source = "Brent-CNY proxy net of NDRC policy gaps"
        china_price_measure_type = "constructed_brent_cny_policy_proxy"
        china_observed_or_regulated = "proxy_not_observed"
        china_included = False
        china_note = "China fuel price is constructed from Brent-CNY minus cumulative NDRC policy gaps; excluded from the main cross-country fuel-price ranking."
        warnings.append(
            {
                "code": "china_fuel_price_proxy",
                "message": "China official regulated gasoline series is unavailable; retaining Brent-CNY proxy for appendix sensitivity only.",
            }
        )
    else:
        china_panel_fuel = official_china_fuel
        china_fuel_column = "china_regulated_gasoline_cny_per_ton"
        china_fuel_unit = "CNY/tonne index proxy"
        china_fuel_source = "public adjustment-series reconstruction; 2026 official notices are audited separately"
        china_price_measure_type = "regulated_gasoline_adjustment_index_proxy"
        china_observed_or_regulated = "proxy_not_fixed_product_retail_level"
        china_included = False
        china_note = (
            f"China uses a reconstructed regulated-gasoline adjustment index proxy with {official_months} nonmissing months; "
            "it is retained for sensitivity and policy-path construction but excluded from the formal cross-country retail-price ranking."
        )
        warnings.append(
            {
                "code": "china_fuel_price_proxy_only",
                "message": "China historical fuel series is a reconstructed adjustment-index proxy and is excluded from the formal cross-country price-level ranking.",
            }
        )
```

```
870
871     rows = [
872         make_country_rows(
873             "CHN",
874             "China",
875             monthly_q1,
876             china_panel_fuel,
877             china_fuel_column,
878             china_fuel_unit,
879             monthly_q1["brent_usd_bbl"] * monthly_q1["cny_per_usd"],
880             china_fuel_source,
881             china_price_measure_type,
882             china_observed_or_regulated,
883             china_included,
884             china_note,
885         ),
886         make_country_rows(
887             "DEU",
888             "Germany",
889             monthly_q1,
890             germany,
891             "gasoline_eur_per_l",
892             "EUR/litre",
893             monthly_q1["brent_usd_bbl"] * monthly_q1["eur_per_usd"],
894             "European Commission Weekly Oil Bulletin",
895             "observed_retail_gasoline",
896             "observed_retail",
897             True,
898             "Observed Euro-super 95 retail price, monthly average of official weekly data.",
899         ),
900         make_country_rows(
901             "JPN",
902             "Japan",
903             monthly_q1,
904             japan,
905             "regular_gasoline_jpy_per_l",
906             "JPY/litre",
907             monthly_q1["brent_usd_bbl"] * monthly_q1["jpy_per_usd"],
908             "Japan METI weekly fuel survey monthly average",
909             "observed_retail_gasoline",
910             "observed_retail",
911             True,
912             "Observed regular gasoline retail price, monthly average of official weekly data.",
913         ),
914         make_country_rows(
915             "KOR",
916             "Korea",
917             monthly_q1,
918             korea,
919             "regular_gasoline_krw_per_l",
920             "KRW/litre",
921             monthly_q1["brent_usd_bbl"] * monthly_q1["krw_per_usd"],
922             "KOSIS / Korea National Oil Corporation monthly gasoline",
923             "observed_retail_gasoline",
924             "observed_retail",
925             True,
926             "Observed regular gasoline national monthly average from KOSIS/KNOC.",
927         ),
928     ]
929     for country, country_name, fuel in [
930         ("FRA", "France", france),
931         ("ITA", "Italy", italy),
932         ("ESP", "Spain", spain),
933     ]:
934         rows.append(
935             make_country_rows(
936                 country,
937                 country_name,
938                 monthly_q1,
939                 fuel,
940                 "gasoline_eur_per_l",
941                 "EUR/litre",
942                 monthly_q1["brent_usd_bbl"] * monthly_q1["eur_per_usd"],
943                 "European Commission Weekly Oil Bulletin",
944                 "observed_retail_gasoline",
945                 "observed_retail",
```

```
                True,
                f"Observed Euro-super 95 retail price for {country_name}, monthly average of official weekly data.",
            )
        )
    panel = pd.concat(rows, ignore_index=True)
    panel["year"] = pd.to_datetime(panel["period"] + "-01").dt.year
    buffers = build_policy_buffer_table()
    panel["buffer_year"] = panel["year"] - 1
    panel = panel.merge(buffers.rename(columns={"year": "buffer_year"}), on=["country", "buffer_year"], how="left")
    panel = panel.merge(cpi, on=["country", "period"], how="left")
    panel = panel.merge(ip, on=["country", "period"], how="left")
    chn_activity = monthly_cn[["period", "china_iav_yoy_pct"]].rename(columns={"china_iav_yoy_pct": "activity_yoy_pct"})
    panel = panel.merge(chn_activity, on="period", how="left")
    panel.loc[panel["country"].ne("CHN"), "activity_yoy_pct"] = np.nan
    panel["ip_log"] = log_positive(panel["ip_index"])
    panel["ip_yoy_log_change_pct"] = panel.groupby("country")["ip_log"].diff(12) * 100.0
    panel["industrial_activity_yoy_pct"] = panel["ip_yoy_log_change_pct"]
    panel.loc[panel["country"].eq("CHN"), "industrial_activity_yoy_pct"] = panel.loc[panel["country"].eq("CHN"), "activity_yoy_pct"]
    panel = panel.merge(
        monthly_cn[
            [
                "period",
                "OilShock",
                "ARBaselineGap",
                "OilShock_source",
                "supply_shock",
                "aggregate_demand_shock",
                "oil_specific_risk_shock",
                "reduced_form_shock",
                "source_vintage",
            ]
        ],
        on="period",
        how="left",
    )
    panel = panel[
        [
            "country",
            "country_name",
            "year",
            "period",
            "month_end",
            "fuel_price_local",
            "fuel_unit",
            "fuel_source",
            "price_measure_type",
            "observed_or_regulated",
            "included_in_main_comparison",
            "comparability_note",
            "oil_import_dependency",
            "oil_intensity",
            "fuel_price_regulation",
            "import_source_hhi",
            "buffer_note",
            "fuel_log_return",
            "brent_local_per_bbl",
            "brent_local_log_return",
            "cpi_yoy_pct",
            "ip_index",
            "ip_yoy_log_change_pct",
            "activity_yoy_pct",
            "industrial_activity_yoy_pct",
            "OilShock",
            "ARBaselineGap",
            "OilShock_source",
            "supply_shock",
            "aggregate_demand_shock",
            "oil_specific_risk_shock",
            "reduced_form_shock",
            "source_vintage",
            "GPR",
        ]
    ].sort_values(["country", "period"])
    save_processed(panel, "model_country_monthly.csv")
    save_processed(china_fuel, "china_fuel_proxy_monthly.csv")
    return panel
```

```

def main() -> int:
    np.random.seed(RANDOM_SEED)
    warnings: list[dict[str, Any]] = []
    daily = build_daily_q1()
    monthly_q1 = build_monthly_q1()
    gdp = fetch_oecd_china_gdp(warnings)
    monthly_cn = load_china_macro(monthly_q1, warnings)
    quarterly_cn = build_quarterly_cn(monthly_cn, gdp)
    country = build_country_monthly(monthly_q1, monthly_cn, warnings)

    summary = {
        "status": "WARN" if warnings else "PASS",
        "random_seed": RANDOM_SEED,
        "cutoff": CUTOFF.strftime("%Y-%m-%d"),
        "outputs": {
            "model_daily_q1.csv": len(daily),
            "model_monthly_q1.csv": len(monthly_q1),
            "model_monthly_cn.csv": len(monthly_cn),
            "model_quarterly_cn.csv": len(quarterly_cn),
            "model_country_monthly.csv": len(country),
        },
        "warnings": warnings,
    }
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    (RESULTS_DIR / "model_panel_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    update_data_warnings("build_model_panels", warnings)
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 3: build_model_panels.py

```
"""Import manually exported NBS monthly CSV files into the processed layer.

The NBS download format is a wide table with months in reverse chronological
order.  This importer preserves official missing observations, converts the
PPI year-on-year index (same month of previous year = 100) to a percentage
change, and writes tidy monthly files used by the Q2 pipeline.
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import re
from datetime import date
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd


REPO_ROOT = Path(__file__).resolve().parents[2]
PROCESSED_DIR = REPO_ROOT / "data" / "processed"
SOURCE_URL = "https://data.stats.gov.cn/dg/website/page.html#/pc/national/monthData"
START_PERIOD = "2010-01"
END_PERIOD = "2026-06"
EXPECTED_MONTHS = len(pd.period_range(START_PERIOD, END_PERIOD, freq="M"))


def sha256_file(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def clean_cell(value: str) -> str:
    return value.replace("\ufeff", "").replace("\t", "").strip()

```

```python

def read_nbs_wide(path: Path) -> tuple[str, list[str], dict[str, list[str]]]:
    rows: list[list[str]] = []
    with path.open("r", encoding="utf-8-sig", newline="") as handle:
        for row in csv.reader(handle):
            cleaned = [clean_cell(value) for value in row]
            while cleaned and cleaned[-1] == "":
                cleaned.pop()
            rows.append(cleaned)

    period_label = next((row[0] for row in rows if row and row[0].startswith("时间：")), "")
    header_index = next((index for index, row in enumerate(rows) if row and row[0] == "指标"), None)
    if header_index is None:
        raise ValueError(f"{path.name}: cannot find the NBS indicator header")

    months = rows[header_index][1:]
    if len(months) != EXPECTED_MONTHS:
        raise ValueError(f"{path.name}: expected {EXPECTED_MONTHS} months, found {len(months)}")

    indicators: dict[str, list[str]] = {}
    for row in rows[header_index + 1 :]:
        if not row or row[0].startswith("注：") or row[0].startswith("数据来源："):
            break
        values = row[1:]
        if len(values) < len(months):
            values.extend([""] * (len(months) - len(values)))
        indicators[row[0]] = values[: len(months)]
    return period_label, months, indicators


def parse_month(value: str) -> str:
    match = re.fullmatch(r"(\d{4})年(\d{1,2})月", value)
    if not match:
        raise ValueError(f"unrecognized month label: {value!r}")
    return f"{int(match.group(1)):04d}-{int(match.group(2)):02d}"


def numeric(values: list[str]) -> pd.Series:
    return pd.to_numeric(pd.Series(values, dtype="object").replace("", np.nan), errors="coerce")


def find_indicator(indicators: dict[str, list[str]], required: list[str], excluded: list[str] | None = None) -> tuple[str, list[str]]:
    excluded = excluded or []
    matches = [
        (label, values)
        for label, values in indicators.items()
        if all(token in label for token in required) and not any(token in label for token in excluded)
    ]
    if len(matches) != 1:
        labels = sorted(indicators)
        raise ValueError(f"expected one indicator matching {required}, found {len(matches)}; available={labels}")
    return matches[0]


def validate_periods(frame: pd.DataFrame, label: str) -> None:
    if frame["period"].duplicated().any():
        raise ValueError(f"{label}: duplicate periods")
    expected = pd.period_range(START_PERIOD, END_PERIOD, freq="M").astype(str).tolist()
    actual = frame["period"].tolist()
    if actual != expected:
        raise ValueError(f"{label}: period coverage does not equal {START_PERIOD}..{END_PERIOD}")


def import_ppi(path: Path, download_date: str) -> tuple[pd.DataFrame, dict[str, Any]]:
    period_label, months, indicators = read_nbs_wide(path)
    indicator, values = find_indicator(
        indicators,
        required=["工业生产者出厂价格指数", "上年同月=100"],
        excluded=["生产资料", "生活资料", "分行业", "中类", "按工业部门"],
    )
    index_values = numeric(values)
    frame = pd.DataFrame(
        {
            "period": [parse_month(value) for value in months],
            "china_ppi_index_yoy_100": index_values,
            "china_ppi_yoy_pct": (index_values - 100.0).round(10),
```

```
        }
    ).sort_values("period", ignore_index=True)
    validate_periods(frame, "PPI")
    if frame["china_ppi_yoy_pct"].isna().any():
        missing = frame.loc[frame["china_ppi_yoy_pct"].isna(), "period"].tolist()
        raise ValueError(f"PPI: unexpected missing months: {missing}")

    raw_hash = sha256_file(path)
    frame["source_indicator"] = indicator
    frame["source_url"] = SOURCE_URL
    frame["download_date"] = download_date
    frame["raw_sha256"] = raw_hash
    metadata = {
        "source": "National Bureau of Statistics of China, National Data",
        "source_url": SOURCE_URL,
        "download_date": download_date,
        "raw_file": path.name,
        "raw_sha256": raw_hash,
        "period_label": period_label,
        "indicator": indicator,
        "calendar_months": int(len(frame)),
        "nonmissing_observations": int(frame["china_ppi_yoy_pct"].notna().sum()),
        "transformation": "china_ppi_yoy_pct = china_ppi_index_yoy_100 - 100",
    }
    return frame, metadata


def import_iav(path: Path, download_date: str) -> tuple[pd.DataFrame, dict[str, Any]]:
    period_label, months, indicators = read_nbs_wide(path)
    indicator, values = find_indicator(indicators, required=["规上工业增加值同比增长"])
    cumulative_matches = [
        (label, row_values)
        for label, row_values in indicators.items()
        if "规上工业增加值累计增长" in label
    ]
    yoy_values = numeric(values)
    cumulative_values = numeric(cumulative_matches[0][1]) if len(cumulative_matches) == 1 else pd.Series(np.nan, index=range(len(months))
      )
    frame = pd.DataFrame(
        {
            "period": [parse_month(value) for value in months],
            "china_iav_yoy_pct": yoy_values,
            "china_iav_cumulative_yoy_pct": cumulative_values,
        }
    ).sort_values("period", ignore_index=True)
    validate_periods(frame, "IAV")

    missing = frame.loc[frame["china_iav_yoy_pct"].isna(), "period"].tolist()
    unexpected = [
        period
        for period in missing
        if not (
            period.endswith("-01")
            or (period >= "2013-02" and period.endswith("-02"))
        )
    ]
    if unexpected:
        raise ValueError(f"IAV: missing observations outside the official January/February pattern: {unexpected}")
    if frame["china_iav_yoy_pct"].notna().sum() < 120:
        raise ValueError("IAV: fewer than 120 nonmissing monthly observations")

    frame["observation_status"] = np.where(
        frame["china_iav_yoy_pct"].notna(),
        "observed",
        "official_calendar_gap_no_interpolation",
    )
    raw_hash = sha256_file(path)
    frame["source_indicator"] = indicator
    frame["source_url"] = SOURCE_URL
    frame["download_date"] = download_date
    frame["raw_sha256"] = raw_hash
    metadata = {
        "source": "National Bureau of Statistics of China, National Data",
        "source_url": SOURCE_URL,
        "download_date": download_date,
        "raw_file": path.name,
```

```
        "raw_sha256": raw_hash,
        "period_label": period_label,
        "indicator": indicator,
        "calendar_months": int(len(frame)),
        "nonmissing_observations": int(frame["china_iav_yoy_pct"].notna().sum()),
        "official_missing_months": missing,
        "missing_value_policy": "preserve official January/February gaps; no interpolation",
    }
    return frame, metadata


def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--ppi", type=Path, required=True, help="NBS PPI wide CSV export")
    parser.add_argument("--iav", type=Path, required=True, help="NBS industrial value-added wide CSV export")
    parser.add_argument("--download-date", default=date.today().isoformat())
    args = parser.parse_args()

    ppi, ppi_meta = import_ppi(args.ppi.resolve(), args.download_date)
    iav, iav_meta = import_iav(args.iav.resolve(), args.download_date)
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    ppi.to_csv(PROCESSED_DIR / "nbs_ppi_monthly.csv", index=False, encoding="utf-8", lineterminator="\n")
    iav.to_csv(PROCESSED_DIR / "nbs_iav_monthly.csv", index=False, encoding="utf-8", lineterminator="\n")

    metadata = {
        "schema_version": "1.0",
        "cutoff": END_PERIOD,
        "ppi": ppi_meta,
        "iav": iav_meta,
    }
    (PROCESSED_DIR / "nbs_manual_import_metadata.json").write_text(
        json.dumps(metadata, ensure_ascii=False, indent=2, sort_keys=True) + "\n",
        encoding="utf-8",
    )
    print(
        json.dumps(
            {
                "status": "PASS",
                "ppi_rows": int(len(ppi)),
                "ppi_nonmissing": int(ppi["china_ppi_yoy_pct"].notna().sum()),
                "iav_rows": int(len(iav)),
                "iav_nonmissing": int(iav["china_iav_yoy_pct"].notna().sum()),
                "iav_official_gaps": int(iav["china_iav_yoy_pct"].isna().sum()),
            },
            ensure_ascii=False,
            indent=2,
        )
    )
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 4: import_nbs_manual.py

```
"""Diagnose P0 data readiness without blocking modeling on optional snapshots."""

from __future__ import annotations

import hashlib
import json
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd


REPO_ROOT = Path(__file__).resolve().parents[2]
RAW_DIR = REPO_ROOT / "data" / "raw"
PROCESSED_DIR = REPO_ROOT / "data" / "processed"
RESULTS_DIR = REPO_ROOT / "results"
WARNINGS_PATH = RESULTS_DIR / "data_warnings.json"
CUTOFF = pd.Timestamp("2026-06-30")
```

```
SUCCESS_STATUSES = {"DOWNLOADED", "CACHED"}


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def add_issue(
    bucket: list[dict[str, Any]],
    code: str,
    message: str,
    path: Path | None = None,
) -> None:
    row: dict[str, Any] = {"code": code, "message": message}
    if path is not None:
        row["path"] = path.relative_to(REPO_ROOT).as_posix()
    bucket.append(row)


def read_csv_checked(path: Path, errors: list[dict[str, Any]], **kwargs: Any) -> pd.DataFrame:
    try:
        return pd.read_csv(path, **kwargs)
    except Exception as exc:
        add_issue(errors, "parse_failed", f"{path.name}: {exc}", path)
        return pd.DataFrame()


def check_no_duplicate(frame: pd.DataFrame, keys: list[str], name: str, errors: list[dict[str, Any]]) -> None:
    missing = [key for key in keys if key not in frame.columns]
    if missing:
        add_issue(errors, "missing_key_columns", f"{name} missing key columns {missing}")
        return
    duplicated = int(frame.duplicated(keys).sum())
    if duplicated:
        add_issue(errors, "duplicate_keys", f"{name} has {duplicated} duplicated keys: {keys}")


def check_cutoff(
    values: pd.Series,
    name: str,
    errors: list[dict[str, Any]],
    warnings: list[dict[str, Any]],
) -> None:
    dates = pd.to_datetime(values, errors="coerce")
    if dates.isna().any():
        add_issue(warnings, "date_parse_warning", f"{name} has {int(dates.isna().sum())} unparsable dates")
    max_date = dates.max()
    if pd.notna(max_date) and max_date > CUTOFF:
        add_issue(errors, "cutoff_violation", f"{name} max date {max_date.date()} exceeds 2026-06-30")


def check_finite(frame: pd.DataFrame, columns: list[str], name: str, errors: list[dict[str, Any]]) -> None:
    missing = [column for column in columns if column not in frame.columns]
    if missing:
        add_issue(errors, "missing_core_columns", f"{name} missing core columns {missing}")
        return
    numeric = frame[columns].apply(pd.to_numeric, errors="coerce")
    bad = int((~np.isfinite(numeric.to_numpy(dtype=float))).sum())
    if bad:
        add_issue(errors, "nonfinite_core_values", f"{name} has {bad} non-finite core numeric cells")


def validate_raw_manifest(warnings: list[dict[str, Any]], errors: list[dict[str, Any]]) -> None:
    manifest_path = RAW_DIR / "source_manifest.json"
    try:
        manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    except Exception as exc:
        add_issue(errors, "manifest_parse_failed", f"source_manifest.json: {exc}", manifest_path)
        return

    if len(manifest) != 21:
        add_issue(warnings, "manifest_record_count", f"expected 21 source records, got {len(manifest)}", manifest_path)
```

```

    for record in manifest:
        artifact_id = record.get("artifact_id", "<unknown>")
        status = record.get("status", "")
        if status not in SUCCESS_STATUSES:
            add_issue(warnings, "manifest_status", f"{artifact_id} status={status}")
        local_path = record.get("local_path", "")
        path = REPO_ROOT / local_path
        if not path.exists():
            add_issue(warnings, "missing_raw_snapshot", f"{artifact_id} raw snapshot is absent", path)
            continue
        expected_hash = record.get("sha256")
        if expected_hash:
            actual_hash = sha256_file(path)
            if actual_hash != expected_hash:
                add_issue(
                    warnings,
                    "raw_hash_mismatch",
                    f"{artifact_id} SHA-256 mismatch; likely line-ending or refreshed snapshot drift",
                    path,
                )


def validate_processed(warnings: list[dict[str, Any]], errors: list[dict[str, Any]]) -> list[str]:
    checks: list[str] = []

    monthly = read_csv_checked(PROCESSED_DIR / "p0_monthly_market.csv", errors)
    if not monthly.empty:
        check_no_duplicate(monthly, ["period"], "p0_monthly_market", errors)
        if "month_end" in monthly.columns:
            check_cutoff(monthly["month_end"], "p0_monthly_market.month_end", errors, warnings)
        check_finite(
            monthly,
            ["brent_usd_bbl", "us_crude_stock_month_end_kbbl", "usd_broad_index", "GPR", "cny_per_usd"],
            "p0_monthly_market",
            errors,
        )
        checks.append(f"p0_monthly_market rows={len(monthly)}")

    daily = read_csv_checked(PROCESSED_DIR / "p0_daily_market.csv", errors)
    if not daily.empty:
        check_no_duplicate(daily, ["date"], "p0_daily_market", errors)
        if "date" in daily.columns:
            check_cutoff(daily["date"], "p0_daily_market.date", errors, warnings)
        checks.append(f"p0_daily_market rows={len(daily)}")

    cpi = read_csv_checked(PROCESSED_DIR / "oecd_g20_cpi_monthly.csv", errors)
    if not cpi.empty:
        check_no_duplicate(cpi, ["REF_AREA", "TIME_PERIOD"], "oecd_g20_cpi_monthly", errors)
        if "TIME_PERIOD" in cpi.columns:
            check_cutoff(cpi["TIME_PERIOD"].astype(str) + "-01", "oecd_g20_cpi_monthly.TIME_PERIOD", errors, warnings)
        checks.append(f"oecd_g20_cpi_monthly rows={len(cpi)}")

    ip = read_csv_checked(PROCESSED_DIR / "oecd_kei_ip_monthly.csv", errors)
    if not ip.empty:
        check_no_duplicate(ip, ["REF_AREA", "TIME_PERIOD"], "oecd_kei_ip_monthly", errors)
        if "TIME_PERIOD" in ip.columns:
            check_cutoff(ip["TIME_PERIOD"].astype(str) + "-01", "oecd_kei_ip_monthly.TIME_PERIOD", errors, warnings)
        checks.append(f"oecd_kei_ip_monthly rows={len(ip)}")

    for filename, key in [
        ("germany_eurosuper95_monthly.csv", ["period"]),
        ("japan_regular_gasoline_monthly.csv", ["period"]),
        ("korea_regular_gasoline_monthly.csv", ["period"]),
        ("cn_fuel_policy_events.csv", ["effective_date"]),
    ]:
        frame = read_csv_checked(PROCESSED_DIR / filename, errors)
        if frame.empty:
            continue
        check_no_duplicate(frame, key, filename, errors)
        date_col = "effective_date" if "effective_date" in frame.columns else "month_end"
        if date_col in frame.columns:
            check_cutoff(frame[date_col], f"{filename}.{date_col}", errors, warnings)
        checks.append(f"{filename} rows={len(frame)}")

    return checks
```

```

def main() -> int:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    warnings: list[dict[str, Any]] = []
    errors: list[dict[str, Any]] = []

    validate_raw_manifest(warnings, errors)
    checks = validate_processed(warnings, errors)

    result = {
        "status": "FAIL" if errors else ("WARN" if warnings else "PASS"),
        "cutoff": CUTOFF.strftime("%Y-%m-%d"),
        "checks": checks,
        "warnings": warnings,
        "errors": errors,
    }
    WARNINGS_PATH.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result, ensure_ascii=False, indent=2))
    return 1 if errors else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 5: validate_p0.py

3.2 四问模型

```
"""Question 1: oil-price forecasts and war-shock counterfactuals."""

from __future__ import annotations

import argparse
import json
import math
import sys
import warnings as py_warnings
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy.optimize import minimize
from scipy.stats import norm
from statsmodels.tools.sm_exceptions import ValueWarning
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.tsa.forecasting.theta import ThetaModel
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.statespace.sarimax import SARIMAX


REPO_ROOT = Path(__file__).resolve().parents[2]
CODE_DIR = REPO_ROOT / "code"
if str(CODE_DIR) not in sys.path:
    sys.path.insert(0, str(CODE_DIR))

from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

PROCESSED_DIR = REPO_ROOT / "data" / "processed"
RESULTS_DIR = REPO_ROOT / "results"
FIGURES_DIR = REPO_ROOT / "figures"
```

```python
CUTOFF = pd.Timestamp("2026-06-30")
RANDOM_SEED = 20260730
EVAL_START = "2020-01"
EVENT_E1_CALENDAR_START = pd.Timestamp("2026-02-28")
EVENT_E3_CALENDAR_START = pd.Timestamp("2026-06-17")
EVENT_TERMS = ["stage_E1", "stage_E2", "stage_E3"]


def ensure_dirs() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)


def save_csv(frame: pd.DataFrame, filename: str) -> Path:
    path = RESULTS_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def json_records(frame: pd.DataFrame) -> list[dict[str, Any]]:
    if frame.empty:
        return []
    return json.loads(frame.where(pd.notna(frame), None).to_json(orient="records"))


def zscore(series: pd.Series) -> pd.Series:
    std = series.std(skipna=True)
    if not np.isfinite(std) or std == 0:
        return series * np.nan
    return (series - series.mean(skipna=True)) / std


def log_positive(series: pd.Series) -> pd.Series:
    values = pd.to_numeric(series, errors="coerce")
    result = pd.Series(np.nan, index=series.index, dtype=float)
    mask = values > 0
    result.loc[mask] = np.log(values.loc[mask])
    return result


def fit_sarimax(
    y: pd.Series,
    order: tuple[int, int, int],
    exog: pd.DataFrame | None = None,
) -> Any:
    with py_warnings.catch_warnings():
        py_warnings.simplefilter("ignore")
        model = SARIMAX(
            y,
            order=order,
            exog=exog,
            trend="c",
            enforce_stationarity=False,
            enforce_invertibility=False,
        )
        return model.fit(disp=False, maxiter=200)


def common_trading_dates(daily: pd.DataFrame, price_prefix: str = "brent_usd_bbl") -> list[pd.Timestamp]:
    frame = daily.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    other_price = "wti_usd_bbl" if price_prefix == "brent_usd_bbl" and "wti_usd_bbl" in frame.columns else price_prefix
    mask = frame[price_prefix].notna() & frame[other_price].notna()
    return [pd.Timestamp(value) for value in frame.loc[mask, "date"].drop_duplicates().sort_values().tolist()]


def first_trading_date_on_or_after(daily: pd.DataFrame, date: pd.Timestamp, price_prefix: str = "brent_usd_bbl") -> pd.Timestamp:
    candidates = [value for value in common_trading_dates(daily, price_prefix) if value >= date]
    if not candidates:
        raise ValueError(f"No common trading date on or after {date.date()} for {price_prefix}.")
    return candidates[0]


def trading_day_shift(daily: pd.DataFrame, start: pd.Timestamp, shift_days: int, price_prefix: str = "brent_usd_bbl") -> pd.Timestamp:
    dates = common_trading_dates(daily, price_prefix)
    try:
```

```
        idx = dates.index(start)
    except ValueError as exc:
        raise ValueError(f"{start.date()} is not a common trading date for {price_prefix}.") from exc
    shifted_idx = idx + shift_days
    if shifted_idx < 0 or shifted_idx >= len(dates):
        raise ValueError(f"Trading-day shift {shift_days:+d} from {start.date()} is outside the observed sample.")
    return dates[shifted_idx]


def assign_event_stages(daily: pd.DataFrame, e1_start: pd.Timestamp | None = None, price_prefix: str = "brent_usd_bbl") -> tuple[pd.
    DataFrame, dict[str, str]]:
    frame = daily.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    e1_start = e1_start or first_trading_date_on_or_after(frame, EVENT_E1_CALENDAR_START, price_prefix)
    dates = common_trading_dates(frame, price_prefix)
    idx = dates.index(e1_start)
    if idx + 3 >= len(dates):
        raise ValueError("E1 event window lacks enough post-event trading days.")
    e1_car_end = dates[idx + 2]
    e2_start = dates[idx + 3]
    e3_start = first_trading_date_on_or_after(frame, EVENT_E3_CALENDAR_START, price_prefix)

    date = frame["date"]
    frame["war_stage"] = "prewar"
    frame.loc[date.between(e1_start, e1_car_end), "war_stage"] = "E1_immediate_window"
    frame.loc[date.between(e2_start, e3_start - pd.Timedelta(days=1)), "war_stage"] = "E2_disruption"
    frame.loc[date >= e3_start, "war_stage"] = "E3_easing"
    frame["war_on"] = (date >= e1_start).astype(int)
    frame["stage_E1"] = date.between(e1_start, e1_car_end).astype(int)
    frame["stage_E2"] = frame["war_stage"].eq("E2_disruption").astype(int)
    frame["stage_E3"] = frame["war_stage"].eq("E3_easing").astype(int)
    event_meta = {
        "e1_calendar_start": EVENT_E1_CALENDAR_START.strftime("%Y-%m-%d"),
        "e1_trading_start": e1_start.strftime("%Y-%m-%d"),
        "e1_car_end_0_2": e1_car_end.strftime("%Y-%m-%d"),
        "e2_trading_start": e2_start.strftime("%Y-%m-%d"),
        "e3_trading_start": e3_start.strftime("%Y-%m-%d"),
    }
    return frame, event_meta


def dm_hln_test(loss_model: pd.Series, loss_benchmark: pd.Series, horizon: int) -> tuple[float, float]:
    diff = pd.Series(loss_model.to_numpy(dtype=float) - loss_benchmark.to_numpy(dtype=float)).dropna()
    n = len(diff)
    if n <= horizon + 1 or float(diff.var(ddof=1)) == 0.0:
        return np.nan, np.nan
    mean_diff = float(diff.mean())
    centered = diff - mean_diff
    max_lag = max(0, horizon - 1)
    gamma0 = float((centered @ centered) / n)
    long_run_var = gamma0
    for lag in range(1, max_lag + 1):
        gamma = float((centered.iloc[lag:].to_numpy() @ centered.iloc[:-lag].to_numpy()) / n)
        long_run_var += 2.0 * (1.0 - lag / (max_lag + 1.0)) * gamma
    if long_run_var <= 0 or not np.isfinite(long_run_var):
        return np.nan, np.nan
    dm_stat = mean_diff / math.sqrt(long_run_var / n)
    hln_scale = math.sqrt((n + 1 - 2 * horizon + horizon * (horizon - 1) / n) / n)
    stat = dm_stat * hln_scale
    pvalue = 2.0 * (1.0 - norm.cdf(abs(stat)))
    return float(stat), float(pvalue)


def classify_forecast_model(model: str, rel_rmse: float, dm_stat: float, dm_pvalue: float) -> str:
    if model == "no_change":
        return "PASS"
    significantly_worse = np.isfinite(dm_stat) and np.isfinite(dm_pvalue) and dm_stat > 0 and dm_pvalue < 0.10
    if np.isfinite(rel_rmse) and rel_rmse < 1.00 and not significantly_worse:
        return "PASS"
    if significantly_worse or (np.isfinite(rel_rmse) and rel_rmse > 1.05):
        return "FAIL"
    return "CONDITIONAL"


def select_orders(monthly: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> tuple[tuple[int, int, int], tuple[int, int, int], pd.
    DataFrame]:
```

```
    train = monthly.loc[monthly["period"].lt(EVAL_START)].dropna(subset=["log_brent"])
    exog_cols = ["stock_log_lag1", "usd_broad_index_log_return_lag1", "GPR_z_lag1"]
    train_exog = train[exog_cols]
    train_full = train.dropna(subset=["log_brent"] + exog_cols)
    rows: list[dict[str, Any]] = []
    best_arima = ((1, 1, 1), math.inf)
    best_sarimax = ((1, 1, 1), math.inf)
    for p in range(3):
        for q in range(3):
            order = (p, 1, q)
            try:
                result = fit_sarimax(train["log_brent"], order)
                rows.append({"model": "ARIMA", "order": str(order), "aic": float(result.aic)})
                if result.aic < best_arima[1]:
                    best_arima = (order, float(result.aic))
            except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
                rows.append({"model": "ARIMA", "order": str(order), "aic": np.nan, "error": str(exc)})
            try:
                result = fit_sarimax(train_full["log_brent"], order, train_exog.loc[train_full.index])
                rows.append({"model": "SARIMAX", "order": str(order), "aic": float(result.aic)})
                if result.aic < best_sarimax[1]:
                    best_sarimax = (order, float(result.aic))
            except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
                rows.append({"model": "SARIMAX", "order": str(order), "aic": np.nan, "error": str(exc)})
    if not np.isfinite(best_arima[1]):
        warnings_log.append({"code": "arima_order_grid_failed", "message": "ARIMA grid failed; using (1,1,1)."})
        best_arima = ((1, 1, 1), np.nan)
    if not np.isfinite(best_sarimax[1]):
        warnings_log.append({"code": "sarimax_order_grid_failed", "message": "SARIMAX grid failed; using (1,1,1)."})
        best_sarimax = ((1, 1, 1), np.nan)
    grid = pd.DataFrame(rows)
    save_csv(grid, "q1_order_grid.csv")
    return best_arima[0], best_sarimax[0], grid


def forecast_interval(prediction: float, sigma: float, horizon: int, alpha: float) -> tuple[float, float]:
    width = norm.ppf(1 - alpha / 2) * sigma * math.sqrt(horizon)
    return prediction - width, prediction + width


def monthly_forecasts(monthly: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> tuple[pd.DataFrame, pd.DataFrame]:
    exog_cols = ["stock_log_lag1", "usd_broad_index_log_return_lag1", "GPR_z_lag1"]
    arima_order, sarimax_order, _ = select_orders(monthly, warnings_log)
    rows: list[dict[str, Any]] = []
    periods = monthly["period"].tolist()
    eval_periods = [period for period in periods if period >= EVAL_START]

    for target_period in eval_periods:
        target_idx = periods.index(target_period)
        for horizon in [1, 3, 6]:
            origin_idx = target_idx - horizon
            if origin_idx < 60:
                continue
            train = monthly.iloc[: origin_idx + 1].copy()
            target = monthly.iloc[target_idx]
            if pd.isna(target["log_brent"]):
                continue
            origin = monthly.iloc[origin_idx]
            sample_start = train["period"].iloc[0]
            sample_end = train["period"].iloc[-1]

            residual = train["log_brent"].diff().dropna()
            sigma = float(residual.std()) if len(residual) > 2 else 0.1
            no_change = float(origin["log_brent"])
            for alpha, prefix in [(0.20, "80"), (0.05, "95")]:
                pass
            lower_80, upper_80 = forecast_interval(no_change, sigma, horizon, 0.20)
            lower_95, upper_95 = forecast_interval(no_change, sigma, horizon, 0.05)
            rows.append(
                {
                    "origin_period": str(origin["period"]),
                    "origin_price": float(origin["brent_usd_bbl"]),
                    "origin_log_price": float(origin["log_brent"]),
                    "period": target_period,
                    "horizon": horizon,
                    "model": "no_change",
```

```
                    "actual": float(target["log_brent"]),
                    "prediction": no_change,
                    "actual_change": float(target["log_brent"] - origin["log_brent"]),
                    "predicted_change": float(no_change - origin["log_brent"]),
                    "lower_80": lower_80,
                    "upper_80": upper_80,
                    "lower_95": lower_95,
                    "upper_95": upper_95,
                    "actual_price": float(target["brent_usd_bbl"]),
                    "prediction_price": float(np.exp(no_change)),
                    "sample_start": sample_start,
                    "sample_end": sample_end,
                    "specification": "last observed log Brent",
                }
            )

            for model_name, order, use_exog in [
                ("ARIMA", arima_order, False),
                ("SARIMAX", sarimax_order, True),
            ]:
                try:
                    fit_train = train.dropna(subset=["log_brent"] + (exog_cols if use_exog else []))
                    exog_train = fit_train[exog_cols] if use_exog else None
                    result = fit_sarimax(fit_train["log_brent"], order, exog_train)
                    if use_exog:
                        last_exog = fit_train[exog_cols].iloc[-1].to_numpy(dtype=float)
                        future_exog = pd.DataFrame([last_exog] * horizon, columns=exog_cols)
                        forecast = result.get_forecast(steps=horizon, exog=future_exog)
                    else:
                        forecast = result.get_forecast(steps=horizon)
                    mean = float(forecast.predicted_mean.iloc[-1])
                    ci80 = forecast.conf_int(alpha=0.20).iloc[-1].to_numpy(dtype=float)
                    ci95 = forecast.conf_int(alpha=0.05).iloc[-1].to_numpy(dtype=float)
                    rows.append(
                        {
                            "origin_period": str(origin["period"]),
                            "origin_price": float(origin["brent_usd_bbl"]),
                            "origin_log_price": float(origin["log_brent"]),
                            "period": target_period,
                            "horizon": horizon,
                            "model": model_name,
                            "actual": float(target["log_brent"]),
                            "prediction": mean,
                            "actual_change": float(target["log_brent"] - origin["log_brent"]),
                            "predicted_change": float(mean - origin["log_brent"]),
                            "lower_80": float(ci80[0]),
                            "upper_80": float(ci80[1]),
                            "lower_95": float(ci95[0]),
                            "upper_95": float(ci95[1]),
                            "actual_price": float(target["brent_usd_bbl"]),
                            "prediction_price": float(np.exp(mean)),
                            "sample_start": sample_start,
                            "sample_end": sample_end,
                            "specification": f"order={order}; future exog held at origin" if use_exog else f"order={order}",
                        }
                    )
                except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
                    warnings_log.append(
                        {
                            "code": "monthly_forecast_model_failed",
                            "message": f"{model_name} h={horizon} target={target_period}: {exc}",
                        }
                    )
            fit_train = train.dropna(subset=["log_brent"]).copy()
            fit_series = pd.Series(
                fit_train["log_brent"].to_numpy(dtype=float),
                index=pd.PeriodIndex(fit_train["period"], freq="M").to_timestamp(),
            ).asfreq("MS")
            for model_name in ["ETS", "Theta"]:
                try:
                    if model_name == "ETS":
                        fitted = ExponentialSmoothing(
                            fit_series,
                            trend="add",
                            damped_trend=True,
                            seasonal=None,
```

```
                            initialization_method="estimated",
                        ).fit(optimized=True)
                        pred_values = fitted.forecast(horizon)
                    else:
                        fitted = ThetaModel(fit_series, period=12, deseasonalize=False).fit()
                        pred_values = fitted.forecast(horizon)
                    mean = float(pred_values.iloc[-1])
                    lower_80, upper_80 = forecast_interval(mean, sigma, horizon, 0.20)
                    lower_95, upper_95 = forecast_interval(mean, sigma, horizon, 0.05)
                    rows.append(
                        {
                            "origin_period": str(origin["period"]),
                            "origin_price": float(origin["brent_usd_bbl"]),
                            "origin_log_price": float(origin["log_brent"]),
                            "period": target_period,
                            "horizon": horizon,
                            "model": model_name,
                            "actual": float(target["log_brent"]),
                            "prediction": mean,
                            "actual_change": float(target["log_brent"] - origin["log_brent"]),
                            "predicted_change": float(mean - origin["log_brent"]),
                            "lower_80": lower_80,
                            "upper_80": upper_80,
                            "lower_95": lower_95,
                            "upper_95": upper_95,
                            "actual_price": float(target["brent_usd_bbl"]),
                            "prediction_price": float(np.exp(mean)),
                            "sample_start": sample_start,
                            "sample_end": sample_end,
                            "specification": "Holt damped ETS" if model_name == "ETS" else "ThetaModel period=12",
                        }
                    )
                except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
                    warnings_log.append(
                        {
                            "code": "monthly_forecast_model_failed",
                            "message": f"{model_name} h={horizon} target={target_period}: {exc}",
                        }
                    )

    forecasts = pd.DataFrame(rows)
    forecasts = add_equal_weight_combination(forecasts)
    save_csv(forecasts, "q1_forecasts.csv")
    metrics = forecast_metrics(forecasts)
    save_csv(metrics, "q1_forecast_metrics.csv")
    return forecasts, metrics


def origin_forecast_table(monthly: pd.DataFrame) -> pd.DataFrame:
    origin_period = "2026-02"
    origin_rows = monthly.loc[monthly["period"].eq(origin_period)].copy()
    if origin_rows.empty:
        raise ValueError("Q1 final origin forecast requires 2026-02 in model_monthly_q1.csv.")
    origin = origin_rows.iloc[0]
    train = monthly.loc[monthly["period"].le(origin_period)].dropna(subset=["log_brent"]).copy()
    sigma = float(train["log_brent"].diff().dropna().std())
    rows: list[dict[str, Any]] = []
    for horizon in [1, 3, 6]:
        target_period = str((pd.Period(origin_period, freq="M") + horizon))
        target = monthly.loc[monthly["period"].eq(target_period)].copy()
        actual_log = float(target["log_brent"].iloc[0]) if not target.empty and pd.notna(target["log_brent"].iloc[0]) else np.nan
        actual_price = float(target["brent_usd_bbl"].iloc[0]) if not target.empty and pd.notna(target["brent_usd_bbl"].iloc[0]) else np.
    nan
        prediction_log = float(origin["log_brent"])
        lower_80, upper_80 = forecast_interval(prediction_log, sigma, horizon, 0.20)
        lower_95, upper_95 = forecast_interval(prediction_log, sigma, horizon, 0.05)
        rows.append(
            {
                "origin_period": origin_period,
                "origin_price": float(origin["brent_usd_bbl"]),
                "origin_log_price": float(origin["log_brent"]),
                "target_period": target_period,
                "horizon_months": horizon,
                "model": "no_change",
                "forecast_status": "ACTUAL_AVAILABLE" if np.isfinite(actual_price) else "FORECAST_ONLY",
                "prediction_log_price": prediction_log,
```

```
                "prediction_price": float(np.exp(prediction_log)),
                "lower_80_price": float(np.exp(lower_80)),
                "upper_80_price": float(np.exp(upper_80)),
                "lower_95_price": float(np.exp(lower_95)),
                "upper_95_price": float(np.exp(upper_95)),
                "actual_log_price": actual_log if np.isfinite(actual_log) else np.nan,
                "actual_price": actual_price if np.isfinite(actual_price) else np.nan,
                "forecast_error_price": actual_price - float(np.exp(prediction_log)) if np.isfinite(actual_price) else np.nan,
                "actual_change": actual_log - float(origin["log_brent"]) if np.isfinite(actual_log) else np.nan,
                "predicted_change": 0.0,
                "sample_start": train["period"].iloc[0],
                "sample_end": train["period"].iloc[-1],
                "specification": "competition final no-change forecast from 2026-02 origin; 2026-08 remains forecast-only under
      2026-06-30 cutoff",
            }
        )
    result = pd.DataFrame(rows)
    save_csv(result, "q1_origin_forecast.csv")
    return result


def add_equal_weight_combination(forecasts: pd.DataFrame) -> pd.DataFrame:
    if forecasts.empty:
        return forecasts
    members = ["no_change", "ARIMA", "SARIMAX", "ETS", "Theta"]
    rows: list[dict[str, Any]] = []
    for (_, _), group in forecasts.groupby(["period", "horizon"]):
        available = group.loc[group["model"].isin(members)].copy()
        if len(available) < 2:
            continue
        template = available.iloc[0].to_dict()
        prediction = float(available["prediction"].mean())
        for column in ["lower_80", "upper_80", "lower_95", "upper_95"]:
            template[column] = float(available[column].mean())
        template.update(
            {
                "model": "EqualWeight",
                "prediction": prediction,
                "prediction_price": float(np.exp(prediction)),
                "predicted_change": prediction - float(template["origin_log_price"]),
                "specification": "equal-weight average of available no_change/ARIMA/SARIMAX/ETS/Theta forecasts",
            }
        )
        rows.append(template)
    if not rows:
        return forecasts
    return pd.concat([forecasts, pd.DataFrame(rows)], ignore_index=True, sort=False)


def forecast_metrics(forecasts: pd.DataFrame) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    if forecasts.empty:
        return pd.DataFrame()
    forecasts = forecasts.sort_values(["model", "horizon", "period"]).copy()
    forecasts["error"] = forecasts["actual"] - forecasts["prediction"]
    forecasts["abs_error"] = forecasts["error"].abs()
    forecasts["squared_error"] = forecasts["error"] ** 2
    forecasts["covered_80"] = forecasts["actual"].between(forecasts["lower_80"], forecasts["upper_80"])
    forecasts["covered_95"] = forecasts["actual"].between(forecasts["lower_95"], forecasts["upper_95"])
    for (model, horizon), group in forecasts.groupby(["model", "horizon"]):
        group = group.sort_values("period")
        benchmark = forecasts.loc[
            forecasts["model"].eq("no_change") & forecasts["horizon"].eq(horizon),
            ["period", "abs_error", "squared_error"],
        ].rename(columns={"abs_error": "benchmark_abs_error", "squared_error": "benchmark_squared_error"})
        paired = group.merge(benchmark, on="period", how="inner")
        benchmark_mae = float(paired["benchmark_abs_error"].mean()) if not paired.empty else np.nan
        benchmark_rmse = float(np.sqrt(paired["benchmark_squared_error"].mean())) if not paired.empty else np.nan
        mae = float(group["abs_error"].mean())
        rmse = float(np.sqrt(group["squared_error"].mean()))
        rel_mae = mae / benchmark_mae if np.isfinite(benchmark_mae) and benchmark_mae > 0 else np.nan
        rel_rmse = rmse / benchmark_rmse if np.isfinite(benchmark_rmse) and benchmark_rmse > 0 else np.nan
        dm_rmse_stat, dm_rmse_pvalue = dm_hln_test(paired["squared_error"], paired["benchmark_squared_error"], int(horizon))
        dm_mae_stat, dm_mae_pvalue = dm_hln_test(paired["abs_error"], paired["benchmark_abs_error"], int(horizon))
        if {"actual_change", "predicted_change"}.issubset(group.columns):
            directional = group[["actual_change", "predicted_change"]].dropna()
```

```
            direction_accuracy = (
                float(np.sign(directional["actual_change"]).eq(np.sign(directional["predicted_change"])).mean())
                if not directional.empty
                else np.nan
            )
        else:
            actual_dir = np.sign(group["actual"].diff())
            pred_dir = np.sign(group["prediction"].diff())
            direction_accuracy = float((actual_dir.eq(pred_dir)).iloc[1:].mean()) if len(group) > 2 else np.nan
        status = classify_forecast_model(str(model), rel_rmse, dm_rmse_stat, dm_rmse_pvalue)
        rows.append(
            {
                "model": model,
                "horizon": int(horizon),
                "n": int(len(group)),
                "MAE": mae,
                "RMSE": rmse,
                "relative_MAE_vs_no_change": rel_mae,
                "relative_RMSE_vs_no_change": rel_rmse,
                "dm_hln_stat_rmse_loss": dm_rmse_stat,
                "dm_hln_pvalue_rmse_loss": dm_rmse_pvalue,
                "dm_hln_stat_mae_loss": dm_mae_stat,
                "dm_hln_pvalue_mae_loss": dm_mae_pvalue,
                "direction_accuracy": direction_accuracy,
                "coverage_80": float(group["covered_80"].mean()),
                "coverage_95": float(group["covered_95"].mean()),
                "model_status": status,
            }
        )
    return pd.DataFrame(rows)


def monthly_shock_residuals(monthly: pd.DataFrame) -> pd.DataFrame:
    exog_cols = ["log_brent_lag1", "stock_log_lag1", "usd_broad_index_log_return_lag1", "GPR_z_lag1"]
    frame = monthly.copy()
    frame["log_brent_lag1"] = frame["log_brent"].shift(1)
    usable = frame.dropna(subset=["log_brent"] + exog_cols).copy()
    x = sm.add_constant(usable[exog_cols], has_constant="add")
    model = sm.OLS(usable["log_brent"], x).fit(cov_type="HAC", cov_kwds={"maxlags": 3})
    usable["OilShock_raw"] = model.resid
    usable["OilShock"] = zscore(usable["OilShock_raw"])
    result = frame[["period", "month_end", "brent_usd_bbl", "log_brent", "brent_usd_bbl_log_return"]].merge(
        usable[["period", "OilShock_raw", "OilShock"]],
        on="period",
        how="left",
    )
    result["OilShock_source"] = "ARX_one_step_residual_lagged_information"
    return result


def stage_effect_regression(daily: pd.DataFrame, price_prefix: str, specification: str) -> pd.DataFrame:
    return_col = f"{price_prefix}_log_return"
    frame = daily.loc[pd.to_datetime(daily["date"]).between(pd.Timestamp("2024-01-01"), CUTOFF)].copy()
    for lag in [1, 2, 3]:
        frame[f"{return_col}_lag{lag}"] = frame[return_col].shift(lag)
    regressors = [f"{return_col}_lag{lag}" for lag in [1, 2, 3]] + ["usd_broad_index_log_return"] + EVENT_TERMS
    usable = frame.dropna(subset=[return_col] + regressors)
    if usable.empty:
        return pd.DataFrame()
    x = sm.add_constant(usable[regressors], has_constant="add")
    for term in EVENT_TERMS:
        event_count = int(usable[term].sum())
        if event_count == 0 or event_count == len(usable):
            raise ValueError(f"{term} is not identifiable for {price_prefix}: event_count={event_count}, n={len(usable)}.")
    rank = int(np.linalg.matrix_rank(x.to_numpy(dtype=float)))
    if rank < x.shape[1]:
        raise ValueError(f"Event-stage design is rank deficient for {price_prefix}: rank={rank}, columns={x.shape[1]}.")
    fit = sm.OLS(usable[return_col], x).fit(cov_type="HAC", cov_kwds={"maxlags": 5})
    rows: list[dict[str, Any]] = []
    for term in ["stage_E1", "stage_E2", "stage_E3"]:
        estimate = float(fit.params.get(term, np.nan))
        se = float(fit.bse.get(term, np.nan))
        descriptive_only = term == "stage_E1"
        rows.append(
            {
                "stage_id": term.replace("stage_", ""),
```

```
                "model": f"{price_prefix}_stage_dummy",
                "specification": specification,
                "estimate_log_return": estimate,
                "std_error": np.nan if descriptive_only else se,
                "lower_80": np.nan if descriptive_only else estimate - norm.ppf(0.90) * se,
                "upper_80": np.nan if descriptive_only else estimate + norm.ppf(0.90) * se,
                "lower_95": np.nan if descriptive_only else estimate - norm.ppf(0.975) * se,
                "upper_95": np.nan if descriptive_only else estimate + norm.ppf(0.975) * se,
                "pvalue": np.nan if descriptive_only else float(fit.pvalues.get(term, np.nan)),
                "sample_start": str(usable["date"].min().date()),
                "sample_end": str(usable["date"].max().date()),
                "n": int(len(usable)),
                "event_observations": int(usable[term].sum()),
                "identification_status": "DESCRIPTIVE_ONLY" if descriptive_only else "PASS",
            }
        )
    return pd.DataFrame(rows)


def event_car_rows(daily: pd.DataFrame, price_prefix: str, event_start: pd.Timestamp, model_label: str) -> pd.DataFrame:
    frame = daily.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    return_col = f"{price_prefix}_log_return"
    for lag in [1, 2, 3]:
        frame[f"{return_col}_lag{lag}"] = frame[return_col].shift(lag)
    dates = [value for value in common_trading_dates(frame, price_prefix) if value >= event_start]
    if len(dates) < 3:
        return pd.DataFrame()
    event_dates = dates[:3]
    train = frame.loc[frame["date"].lt(event_start)].dropna(
        subset=[return_col, "usd_broad_index_log_return", f"{return_col}_lag1", f"{return_col}_lag2", f"{return_col}_lag3"]
    )
    if len(train) < 80:
        return pd.DataFrame()
    train = train.tail(520)
    regressors = [f"{return_col}_lag{lag}" for lag in [1, 2, 3]] + ["usd_broad_index_log_return"]
    fit = sm.OLS(train[return_col], sm.add_constant(train[regressors].astype(float), has_constant="add")).fit(
        cov_type="HAC",
        cov_kwds={"maxlags": 5},
    )
    sigma = float(fit.resid.std(ddof=1))
    rows: list[dict[str, Any]] = []
    cumulative_actual = 0.0
    cumulative_expected = 0.0
    recursive_lags = [float(train[return_col].iloc[-lag]) for lag in [1, 2, 3]]
    pre_event_usd = float(train["usd_broad_index_log_return"].iloc[-1])
    for horizon, event_date in enumerate(event_dates):
        event_row = frame.loc[frame["date"].eq(event_date)].dropna(subset=[return_col])
        if event_row.empty:
            continue
        x = pd.DataFrame(
            [
                {
                    f"{return_col}_lag1": recursive_lags[0],
                    f"{return_col}_lag2": recursive_lags[1],
                    f"{return_col}_lag3": recursive_lags[2],
                    "usd_broad_index_log_return": pre_event_usd,
                }
            ]
        )
        x = sm.add_constant(x[regressors].astype(float), has_constant="add")
        expected = float(fit.predict(x).iloc[0])
        actual = float(event_row[return_col].iloc[0])
        cumulative_actual += actual
        cumulative_expected += expected
        recursive_lags = [expected, *recursive_lags[:2]]
        abnormal = cumulative_actual - cumulative_expected
        se = sigma * math.sqrt(horizon + 1)
        rows.append(
            {
                "stage_id": f"E1_CAR_0_{horizon}" if horizon else "E1_CAR_0",
                "model": model_label,
                "specification": "recursive AR(3)+pre-event USD expected-return CAR; E1 mapped to first common trading day",
                "estimate_log_return": abnormal,
                "actual_cumulative_log_return": cumulative_actual,
                "expected_cumulative_log_return": cumulative_expected,
```

```
                "std_error": se,
                "lower_80": abnormal - norm.ppf(0.90) * se,
                "upper_80": abnormal + norm.ppf(0.90) * se,
                "lower_95": abnormal - norm.ppf(0.975) * se,
                "upper_95": abnormal + norm.ppf(0.975) * se,
                "pvalue": 2.0 * (1.0 - norm.cdf(abs(abnormal / se))) if se > 0 else np.nan,
                "sample_start": str(train["date"].min().date()),
                "sample_end": str(train["date"].max().date()),
                "n": int(len(train)),
                "event_observations": horizon + 1,
                "event_trading_start": event_start.strftime("%Y-%m-%d"),
                "event_window_end": event_date.strftime("%Y-%m-%d"),
                "identification_status": "PASS",
            }
        )
    return pd.DataFrame(rows)


def finite_sample_empirical_pvalue(distribution: pd.Series, threshold: float) -> float:
    values = pd.to_numeric(distribution, errors="coerce").dropna().abs()
    if values.empty:
        return np.nan
    exceedances = int(values.ge(abs(threshold)).sum())
    return float((exceedances + 1) / (len(values) + 1))


def placebo_distribution(daily: pd.DataFrame, actual_car: pd.DataFrame) -> pd.DataFrame:
    frame = daily.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    frame["pre_volatility_20d"] = frame["brent_usd_bbl_log_return"].rolling(20, min_periods=15).std().shift(1)
    frame["pre_brent_level"] = frame["brent_usd_bbl"].shift(1)
    monthly_gpr = pd.read_csv(PROCESSED_DIR / "model_monthly_q1.csv")[["period", "GPR_z"]]
    gpr_map = monthly_gpr.set_index("period")["GPR_z"].to_dict()
    frame["period"] = frame["date"].dt.to_period("M").astype(str)
    frame["pre_gpr_z"] = frame["period"].map(gpr_map)
    weekend_calendar = pd.date_range("2024-01-06", "2025-12-28", freq="W-SAT")
    excluded_event_dates = [
        pd.Timestamp("2024-04-13"),
        pd.Timestamp("2024-10-01"),
        pd.Timestamp("2025-06-13"),
    ]
    seen_starts: set[pd.Timestamp] = set()
    pieces: list[pd.DataFrame] = []
    for block_id, pseudo_calendar in enumerate(weekend_calendar):
        if any(abs((pseudo_calendar - event_date).days) <= 7 for event_date in excluded_event_dates):
            continue
        start = first_trading_date_on_or_after(frame, pd.Timestamp(pseudo_calendar), "brent_usd_bbl")
        if start in seen_starts:
            continue
        seen_starts.add(start)
        car = event_car_rows(frame, "brent_usd_bbl", start, "brent_weekend_placebo_car")
        if car.empty:
            continue
        car["pseudo_calendar_date"] = pseudo_calendar.strftime("%Y-%m-%d")
        car["placebo_block_id"] = block_id
        state = frame.loc[frame["date"].eq(start)].iloc[0]
        car["match_pre_volatility_20d"] = float(state["pre_volatility_20d"])
        car["match_pre_brent_level"] = float(state["pre_brent_level"])
        car["match_pre_gpr_z"] = float(state["pre_gpr_z"])
        pieces.append(car)
    placebo = pd.concat(pieces, ignore_index=True, sort=False) if pieces else pd.DataFrame()
    if placebo.empty or actual_car.empty:
        save_csv(placebo, "q1_placebo_distribution.csv")
        return placebo
    actual_start = pd.Timestamp(actual_car.loc[actual_car["model"].eq("brent_usd_bbl_event_car"), "event_trading_start"].dropna().iloc
      [0])
    actual_state = frame.loc[frame["date"].eq(actual_start)].iloc[0]
    match_columns = ["match_pre_volatility_20d", "match_pre_brent_level", "match_pre_gpr_z"]
    block_state = placebo.groupby("placebo_block_id", as_index=False)[match_columns].first().dropna()
    actual_values = np.array(
        [float(actual_state["pre_volatility_20d"]), float(actual_state["pre_brent_level"]), float(actual_state["pre_gpr_z"])],
        dtype=float,
    )
    candidate_values = block_state[match_columns].to_numpy(dtype=float)
    scale = np.nanstd(np.vstack([candidate_values, actual_values]), axis=0, ddof=1)
    scale = np.where(scale > 1e-12, scale, 1.0)
```

```
    block_state["match_distance"] = np.sqrt((((candidate_values - actual_values) / scale) ** 2).sum(axis=1))
    keep_count = min(40, len(block_state))
    matched_ids = set(block_state.nsmallest(keep_count, "match_distance")["placebo_block_id"].astype(int))
    distance_map = block_state.set_index("placebo_block_id")["match_distance"].to_dict()
    placebo["match_distance"] = placebo["placebo_block_id"].map(distance_map)
    placebo = placebo.loc[placebo["placebo_block_id"].isin(matched_ids)].copy()
    placebo["matching_status"] = "nearest_40_on_pre_event_volatility_brent_level_monthly_GPR"
    actual = actual_car.loc[actual_car["model"].eq("brent_usd_bbl_event_car")].copy()
    empirical: dict[str, float] = {}
    for stage_id, group in placebo.groupby("stage_id"):
        actual_row = actual.loc[actual["stage_id"].eq(stage_id)]
        if actual_row.empty:
            continue
        threshold = abs(float(actual_row["estimate_log_return"].iloc[0]))
        empirical[stage_id] = finite_sample_empirical_pvalue(group["estimate_log_return"], threshold)
    placebo["actual_empirical_pvalue"] = placebo["stage_id"].map(empirical)
    save_csv(placebo, "q1_placebo_distribution.csv")
    return placebo


def add_empirical_pvalues(actual_car: pd.DataFrame, placebo: pd.DataFrame) -> pd.DataFrame:
    if actual_car.empty or placebo.empty:
        return actual_car
    result = actual_car.copy()
    result["pvalue_empirical"] = np.nan
    for idx, row in result.iterrows():
        if row["model"] != "brent_usd_bbl_event_car":
            continue
        dist = placebo.loc[placebo["stage_id"].eq(row["stage_id"]), "estimate_log_return"].dropna()
        if dist.empty:
            continue
        threshold = abs(float(row["estimate_log_return"]))
        result.loc[idx, "pvalue_empirical"] = finite_sample_empirical_pvalue(dist, threshold)
    return result


def shifted_event_regression(daily: pd.DataFrame, shift_trading_days: int) -> pd.DataFrame:
    main_start = first_trading_date_on_or_after(daily, EVENT_E1_CALENDAR_START, "brent_usd_bbl")
    shifted_start = trading_day_shift(daily, main_start, shift_trading_days, "brent_usd_bbl")
    frame, _ = assign_event_stages(daily, shifted_start, "brent_usd_bbl")
    result = stage_effect_regression(frame, "brent_usd_bbl", f"event dates shifted by {shift_trading_days} trading days")
    if not result.empty:
        result["model"] = f"brent_alt_event_shift_{shift_trading_days:+d}td"
        result["event_trading_start"] = shifted_start.strftime("%Y-%m-%d")
    return result


def robustness_summary(forecasts: pd.DataFrame, effects: pd.DataFrame, daily: pd.DataFrame, placebos: pd.DataFrame) -> pd.DataFrame:
    pieces: list[pd.DataFrame] = []
    if not forecasts.empty:
        for start in ["2021-01", "2022-01"]:
            metrics = forecast_metrics(forecasts.loc[forecasts["period"].ge(start)].copy())
            if not metrics.empty:
                metrics["robustness_type"] = f"evaluation_start_{start}"
                pieces.append(metrics)
    wti = effects.loc[effects["model"].eq("wti_usd_bbl_stage_dummy")].copy()
    if not wti.empty:
        wti["robustness_type"] = "WTI_event_price"
        pieces.append(wti)
    if not placebos.empty:
        summary = (
            placebos.groupby("stage_id", as_index=False)
            .agg(
                placebo_mean=("estimate_log_return", "mean"),
                placebo_p05=("estimate_log_return", lambda s: float(np.quantile(s.dropna(), 0.05))),
                placebo_p95=("estimate_log_return", lambda s: float(np.quantile(s.dropna(), 0.95))),
                placebo_events=("estimate_log_return", "count"),
                actual_empirical_pvalue=("actual_empirical_pvalue", "first"),
            )
        )
        summary["robustness_type"] = "matched_weekend_placebo_distribution"
        summary["model"] = "brent_weekend_placebo_car"
        pieces.append(summary)
    for shift in [-5, 5]:
        alt = shifted_event_regression(daily, shift)
        if not alt.empty:
```

```python
            alt["robustness_type"] = f"event_shift_{shift:+d}trading_days"
            pieces.append(alt)
    result = pd.concat(pieces, ignore_index=True, sort=False) if pieces else pd.DataFrame()
    save_csv(result, "q1_robustness.csv")
    return result


def daily_counterfactual(daily: pd.DataFrame) -> pd.DataFrame:
    frame = daily.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    frame["log_brent"] = log_positive(frame["brent_usd_bbl"])
    ret_col = "brent_usd_bbl_log_return"
    train = frame.loc[frame["date"].between(pd.Timestamp("2024-01-01"), pd.Timestamp("2026-02-27"))].copy()
    for lag in [1, 2, 3]:
        train[f"ret_lag{lag}"] = train[ret_col].shift(lag)
    fit_data = train.dropna(subset=[ret_col, "usd_broad_index_log_return", "ret_lag1", "ret_lag2", "ret_lag3"])
    x = sm.add_constant(fit_data[["ret_lag1", "ret_lag2", "ret_lag3", "usd_broad_index_log_return"]], has_constant="add")
    fit = sm.OLS(fit_data[ret_col], x).fit(cov_type="HAC", cov_kwds={"maxlags": 5})

    event_start = first_trading_date_on_or_after(frame, EVENT_E1_CALENDAR_START, "brent_usd_bbl")
    event = frame.loc[frame["date"].between(event_start, CUTOFF)].copy()
    event = event.dropna(subset=["brent_usd_bbl"]).reset_index(drop=True)
    if event.empty:
        return pd.DataFrame()
    history_returns = train[ret_col].dropna().tail(3).tolist()
    cf_log = float(train["log_brent"].dropna().iloc[-1])
    rows: list[dict[str, Any]] = []
    sigma = float(fit.resid.std())
    for _, row in event.iterrows():
        usd_return = 0.0 if pd.isna(row["usd_broad_index_log_return"]) else float(row["usd_broad_index_log_return"])
        reg = pd.DataFrame(
            [
                {
                    "const": 1.0,
                    "ret_lag1": history_returns[-1],
                    "ret_lag2": history_returns[-2],
                    "ret_lag3": history_returns[-3],
                    "usd_broad_index_log_return": usd_return,
                }
            ]
        )
        pred_ret = float(fit.predict(reg).iloc[0])
        cf_log += pred_ret
        history_returns.append(pred_ret)
        actual_log = float(np.log(row["brent_usd_bbl"]))
        prediction = float(np.exp(cf_log))
        baseline_gap = float(row["brent_usd_bbl"] - prediction)
        lower_80_log, upper_80_log = forecast_interval(cf_log, sigma, len(rows) + 1, 0.20)
        lower_95_log, upper_95_log = forecast_interval(cf_log, sigma, len(rows) + 1, 0.05)
        rows.append(
            {
                "date": row["date"].strftime("%Y-%m-%d"),
                "period": row["date"].to_period("M").strftime("%Y-%m"),
                "actual": float(row["brent_usd_bbl"]),
                "prediction": prediction,
                "response": baseline_gap,
                "lower_80": float(np.exp(lower_80_log)),
                "upper_80": float(np.exp(upper_80_log)),
                "lower_95": float(np.exp(lower_95_log)),
                "upper_95": float(np.exp(upper_95_log)),
                "actual_log": actual_log,
                "ar_baseline_log": cf_log,
                "ar_baseline_gap_usd_bbl": baseline_gap,
                "war_stage": row["war_stage"],
                "model": "AR3_plus_USD_baseline_scenario",
                "horizon": len(rows) + 1,
                "specification": "descriptive AR(3)+USD baseline trained 2024-01-01 to 2026-02-27; not a causal no-war counterfactual",
                "sample_start": "2024-01-01",
                "sample_end": "2026-02-27",
            }
        )
    counterfactual = pd.DataFrame(rows)
    save_csv(counterfactual, "q1_daily_counterfactual.csv")
    return counterfactual


```

```python
def residualize(series: pd.Series, controls: pd.DataFrame) -> pd.Series:
    frame = pd.concat([series.rename("target"), controls], axis=1).dropna()
    result = pd.Series(np.nan, index=series.index, dtype=float)
    if len(frame) < controls.shape[1] + 12:
        return result
    fit = sm.OLS(frame["target"], sm.add_constant(frame.drop(columns=["target"]).astype(float), has_constant="add")).fit()
    result.loc[frame.index] = fit.resid
    return result


def safe_whiteness_pvalue(fit: Any, lags: int) -> float:
    nlags = max(int(fit.k_ar) + 1, lags)
    try:
        return float(fit.test_whiteness(nlags=nlags).pvalue)
    except (ValueError, np.linalg.LinAlgError):
        return np.nan


def structural_shock_decomposition(monthly: pd.DataFrame) -> pd.DataFrame:
    input_path = PROCESSED_DIR / "global_oil_svar_monthly.csv"
    if not input_path.exists():
        raise FileNotFoundError("global_oil_svar_monthly.csv is required for Q1 SVAR structural shocks.")
    global_input = pd.read_csv(input_path)
    frame = (
        monthly[["period", "month_end", "brent_usd_bbl_log_return"]]
        .merge(global_input, on="period", how="left")
        .sort_values("period")
        .reset_index(drop=True)
    )
    frame["supply_growth"] = np.log(pd.to_numeric(frame["world_liquids_supply"], errors="coerce")).diff() * 100.0
    frame["real_brent_return"] = np.log(pd.to_numeric(frame["real_brent"], errors="coerce")).diff() * 100.0
    var_cols = ["supply_growth", "global_real_activity", "real_brent_return"]
    var_data = frame.dropna(subset=var_cols).copy()
    if len(var_data) < 120:
        raise ValueError(f"Q1 SVAR common sample has only {len(var_data)} months; required >=120.")
    var_data = var_data.set_index("period")
    stationarity_rows: list[dict[str, Any]] = []
    for column in var_cols:
        values = var_data[column].astype(float).to_numpy()
        adf = adfuller(values, autolag="BIC")
        try:
            kpss_result = kpss(values, regression="c", nlags="auto")
            kpss_stat, kpss_pvalue = float(kpss_result[0]), float(kpss_result[1])
        except (ValueError, np.linalg.LinAlgError):
            kpss_stat, kpss_pvalue = np.nan, np.nan
        stationarity_rows.append(
            {
                "variable": column,
                "adf_statistic": float(adf[0]),
                "adf_pvalue": float(adf[1]),
                "adf_lags": int(adf[2]),
                "kpss_statistic": kpss_stat,
                "kpss_pvalue": kpss_pvalue,
                "stationarity_gate": bool(float(adf[1]) < 0.05 and (not np.isfinite(kpss_pvalue) or kpss_pvalue >= 0.05)),
                "sample_start": var_data.index.min(),
                "sample_end": var_data.index.max(),
                "n": int(len(values)),
            }
        )
    save_csv(pd.DataFrame(stationarity_rows), "q1_stationarity_diagnostics.csv")
    candidates: list[dict[str, Any]] = []
    fits: dict[int, Any] = {}
    for lag in [6, 12, 18, 24]:
        if len(var_data) <= lag + 36:
            candidates.append(
                {
                    "candidate_lags": lag,
                    "is_selected": False,
                    "bic": np.nan,
                    "aic": np.nan,
                    "hqic": np.nan,
                    "is_stable": False,
                    "whiteness_pvalue": np.nan,
                    "sample_start": var_data.index.min(),
                    "sample_end": var_data.index.max(),
                    "nobs": int(len(var_data)),
```

```
                    "ordering": "supply_growth -> global_real_activity -> real_brent_return",
                    "error": "insufficient_sample_for_lag",
                }
            )
            continue
        try:
            fit = VAR(var_data[var_cols]).fit(lag)
            stable = bool(fit.is_stable(verbose=False))
            fits[lag] = fit
            candidates.append(
                {
                    "candidate_lags": lag,
                    "is_selected": False,
                    "bic": float(fit.bic),
                    "aic": float(fit.aic),
                    "hqic": float(fit.hqic),
                    "is_stable": stable,
                    "whiteness_pvalue": safe_whiteness_pvalue(fit, 24),
                    "minimum_root_modulus": float(np.min(np.abs(fit.roots))),
                    "sample_start": var_data.index.min(),
                    "sample_end": var_data.index.max(),
                    "nobs": int(fit.nobs),
                    "ordering": "supply_growth -> global_real_activity -> real_brent_return",
                    "error": "",
                }
            )
        except (ValueError, np.linalg.LinAlgError) as exc:
            candidates.append(
                {
                    "candidate_lags": lag,
                    "is_selected": False,
                    "bic": np.nan,
                    "aic": np.nan,
                    "hqic": np.nan,
                    "is_stable": False,
                    "whiteness_pvalue": np.nan,
                    "sample_start": var_data.index.min(),
                    "sample_end": var_data.index.max(),
                    "nobs": int(len(var_data)),
                    "ordering": "supply_growth -> global_real_activity -> real_brent_return",
                    "error": f"{type(exc).__name__}: {exc}",
                }
            )
    diag = pd.DataFrame(candidates)
    selectable = diag.loc[diag["is_stable"].eq(True) & diag["bic"].notna()].sort_values("bic")
    if selectable.empty:
        raise ValueError("No stable Q1 VAR candidate among lags 6/12/18/24.")
    selected_lag = int(selectable.iloc[0]["candidate_lags"])
    fit = fits[selected_lag]
    diag.loc[diag["candidate_lags"].eq(selected_lag), "is_selected"] = True

    sigma_u = np.asarray(fit.sigma_u, dtype=float)
    chol = np.linalg.cholesky(sigma_u)
    residuals = fit.resid[var_cols].copy()
    structural = np.linalg.solve(chol, residuals.to_numpy(dtype=float).T).T

    main_risk = pd.Series(structural[:, 2], index=residuals.index)
    alternative_orderings = {
        "activity_supply_price": ["global_real_activity", "supply_growth", "real_brent_return"],
        "supply_price_activity": ["supply_growth", "real_brent_return", "global_real_activity"],
    }
    alt_correlations: dict[str, float] = {}
    for label, alt_cols in alternative_orderings.items():
        alt_fit = VAR(var_data[alt_cols]).fit(selected_lag)
        corr = np.nan
        if alt_fit.is_stable(verbose=False):
            alt_chol = np.linalg.cholesky(np.asarray(alt_fit.sigma_u, dtype=float))
            alt_structural = np.linalg.solve(alt_chol, alt_fit.resid[alt_cols].to_numpy(dtype=float).T).T
            price_index = alt_cols.index("real_brent_return")
            alt_risk = pd.Series(alt_structural[:, price_index], index=alt_fit.resid.index)
            common = pd.concat([main_risk.rename("main"), alt_risk.rename("alt")], axis=1).dropna()
            corr = float(common.corr().iloc[0, 1]) if len(common) > 2 else np.nan
        alt_correlations[label] = corr
        diag[f"alternative_ordering_corr_{label}"] = np.nan
        diag.loc[diag["candidate_lags"].eq(selected_lag), f"alternative_ordering_corr_{label}"] = corr
    diag["alternative_ordering_price_shock_corr"] = np.nan
```

```
    diag.loc[diag["candidate_lags"].eq(selected_lag), "alternative_ordering_price_shock_corr"] = alt_correlations["activity_supply_price"]
    save_csv(diag, "q1_svar_diagnostics.csv")

    shock_frame = pd.DataFrame(
        {
            "period": residuals.index.astype(str),
            "supply_shock": zscore(pd.Series(-structural[:, 0])),
            "aggregate_demand_shock": zscore(pd.Series(structural[:, 1])),
            "oil_specific_risk_shock": zscore(pd.Series(structural[:, 2])),
        }
    )
    result = frame[["period"]].copy().merge(shock_frame, on="period", how="left")
    result["reduced_form_shock"] = zscore(frame["brent_usd_bbl_log_return"])
    result["source_vintage"] = frame["source_vintage"].fillna("global_oil_svar_input").astype(str)
    result["svar_lags"] = selected_lag
    result["svar_ordering"] = "supply_growth -> global_real_activity -> real_brent_return"
    save_csv(result, "q1_structural_shocks.csv")
    return result


def gjr_garch_variance(returns: pd.Series) -> tuple[np.ndarray, dict[str, float]]:
    y = returns.dropna().to_numpy(dtype=float)
    y = y - np.mean(y)
    if len(y) < 80:
        raise ValueError("GJR-GARCH needs at least 80 daily return observations.")
    sample_var = float(np.var(y, ddof=1))

    def neg_loglike(params: np.ndarray) -> float:
        omega, alpha, gamma, beta = params
        if omega <= 0 or alpha < 0 or gamma < 0 or beta < 0 or alpha + 0.5 * gamma + beta >= 0.999:
            return 1e12
        h = np.empty_like(y)
        h[0] = sample_var
        for idx in range(1, len(y)):
            lag_eps = y[idx - 1]
            h[idx] = omega + alpha * lag_eps**2 + gamma * (lag_eps < 0) * lag_eps**2 + beta * h[idx - 1]
            if h[idx] <= 0 or not np.isfinite(h[idx]):
                return 1e12
        return float(0.5 * np.sum(np.log(2 * np.pi) + np.log(h) + y**2 / h))

    constraints = ({"type": "ineq", "fun": lambda p: 0.999 - p[1] - 0.5 * p[2] - p[3]},)
    fit = minimize(
        neg_loglike,
        x0=np.array([0.02 * sample_var, 0.05, 0.05, 0.88]),
        bounds=[(1e-9, None), (0.0, 1.0), (0.0, 1.0), (0.0, 0.999)],
        constraints=constraints,
        method="SLSQP",
        options={"maxiter": 500, "ftol": 1e-9},
    )
    if not fit.success:
        raise RuntimeError(f"GJR-GARCH optimization failed: {fit.message}")
    omega, alpha, gamma, beta = fit.x
    h = np.empty_like(y)
    h[0] = sample_var
    for idx in range(1, len(y)):
        lag_eps = y[idx - 1]
        h[idx] = omega + alpha * lag_eps**2 + gamma * (lag_eps < 0) * lag_eps**2 + beta * h[idx - 1]
    return h, {"omega": float(omega), "alpha": float(alpha), "gamma": float(gamma), "beta": float(beta)}


def volatility_module(daily: pd.DataFrame, event_start: pd.Timestamp) -> pd.DataFrame:
    frame = daily.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    frame = frame.loc[frame["date"].between(pd.Timestamp("2024-01-01"), CUTOFF)].sort_values("date").copy()
    frame["return_pct"] = frame["brent_usd_bbl_log_return"] * 100.0
    usable = frame.dropna(subset=["return_pct"]).copy()
    train = usable.loc[usable["date"].lt(event_start)].copy()
    h_train, params = gjr_garch_variance(train["return_pct"])
    omega, alpha, gamma, beta = params["omega"], params["alpha"], params["gamma"], params["beta"]
    y = usable["return_pct"].to_numpy(dtype=float)
    y = y - float(train["return_pct"].mean())
    h = np.empty(len(y), dtype=float)
    h[: len(h_train)] = h_train
    for idx in range(len(h_train), len(y)):
        lag_eps = y[idx - 1]
```

```
        h[idx] = omega + alpha * lag_eps**2 + gamma * (lag_eps < 0) * lag_eps**2 + beta * h[idx - 1]
    usable["conditional_vol_pct"] = np.sqrt(h)
    pre_median = float(np.nanmedian(usable.loc[usable["date"].lt(event_start), "conditional_vol_pct"]))
    usable["abnormal_vol_ratio_vs_pre_median"] = usable["conditional_vol_pct"] / pre_median - 1.0
    usable["model"] = "GJR_GARCH_1_1"
    usable["specification"] = "normal-likelihood GJR-GARCH(1,1) fitted on pre-E1 Brent daily returns"
    for key, value in params.items():
        usable[key] = value
    output = usable[
        [
            "date",
            "return_pct",
            "conditional_vol_pct",
            "abnormal_vol_ratio_vs_pre_median",
            "model",
            "specification",
            "omega",
            "alpha",
            "gamma",
            "beta",
        ]
    ]
    save_csv(output, "q1_volatility.csv")
    return output


def volatility_stage_summary(volatility: pd.DataFrame, daily: pd.DataFrame) -> pd.DataFrame:
    if volatility.empty:
        result = pd.DataFrame()
        save_csv(result, "q1_volatility_summary.csv")
        return result
    frame = volatility.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    stages = daily[["date", "war_stage"]].copy()
    stages["date"] = pd.to_datetime(stages["date"])
    frame = frame.merge(stages, on="date", how="left")
    rows: list[dict[str, Any]] = []
    pre = frame.loc[frame["war_stage"].eq("prewar"), "conditional_vol_pct"].dropna()
    pre_median = float(pre.median()) if not pre.empty else np.nan
    for stage in ["prewar", "E1_immediate_window", "E2_disruption", "E3_easing"]:
        group = frame.loc[frame["war_stage"].eq(stage)].copy()
        if group.empty:
            continue
        vol = pd.to_numeric(group["conditional_vol_pct"], errors="coerce").dropna()
        ratio = pd.to_numeric(group["abnormal_vol_ratio_vs_pre_median"], errors="coerce").dropna()
        rows.append(
            {
                "war_stage": stage,
                "stage_start": group["date"].min().strftime("%Y-%m-%d"),
                "stage_end": group["date"].max().strftime("%Y-%m-%d"),
                "trading_days": int(len(group)),
                "conditional_vol_pct_mean": float(vol.mean()) if not vol.empty else np.nan,
                "conditional_vol_pct_median": float(vol.median()) if not vol.empty else np.nan,
                "conditional_vol_pct_p10": float(vol.quantile(0.10)) if not vol.empty else np.nan,
                "conditional_vol_pct_p90": float(vol.quantile(0.90)) if not vol.empty else np.nan,
                "abnormal_vol_ratio_mean": float(ratio.mean()) if not ratio.empty else np.nan,
                "abnormal_vol_ratio_median": float(ratio.median()) if not ratio.empty else np.nan,
                "vol_multiple_vs_pre_median": float(vol.median() / pre_median) if np.isfinite(pre_median) and pre_median > 0 and not vol.
  empty else np.nan,
                "model": "GJR_GARCH_1_1",
                "evidence_status": "DESCRIPTIVE_VOLATILITY",
            }
        )
    result = pd.DataFrame(rows)
    save_csv(result, "q1_volatility_summary.csv")
    return result


def make_q1_shocks(monthly: pd.DataFrame, counterfactual: pd.DataFrame) -> pd.DataFrame:
    shocks = monthly_shock_residuals(monthly)
    structural = structural_shock_decomposition(monthly)
    shocks = shocks.merge(structural, on="period", how="left")
    if counterfactual.empty:
        baseline = pd.DataFrame({"period": shocks["period"], "ARBaselineGap": 0.0})
    else:
        baseline = (
```

```
            counterfactual.groupby("period", as_index=False)
            .agg(ARBaselineGap=("ar_baseline_gap_usd_bbl", "mean"), ARBaselineGap_days=("ar_baseline_gap_usd_bbl", "count"))
        )
    shocks = shocks.merge(baseline, on="period", how="left")
    shocks["ARBaselineGap"] = shocks["ARBaselineGap"].fillna(0.0)
    shocks["ARBaselineGap_days"] = shocks["ARBaselineGap_days"].fillna(0).astype(int) if "ARBaselineGap_days" in shocks else 0
    shocks["random_seed"] = RANDOM_SEED
    save_csv(shocks, "q1_monthly_shocks.csv")
    return shocks


def plot_forecasts(forecasts: pd.DataFrame) -> None:
    if forecasts.empty:
        return
    one = forecasts.loc[forecasts["horizon"].eq(1)].copy()
    if one.empty:
        return
    one["date"] = pd.to_datetime(one["period"] + "-01") + pd.offsets.MonthEnd(0)
    pivot = one.pivot_table(index="date", columns="model", values="prediction_price", aggfunc="first")
    actual = one.drop_duplicates("date").set_index("date")["actual_price"]
    fig, ax = plt.subplots(figsize=(9.2, 5.1))
    ax.plot(actual.index, actual, label="实际 Brent", color=PALETTE["ink"], linewidth=2.0)
    style_map = {
        "no_change": (PALETTE["slate"], (0, (2, 2)), "不变预测"),
        "ARIMA": (PALETTE["gold"], (0, (4, 2)), "ARIMA"),
        "SARIMAX": (PALETTE["blue"], "solid", "SARIMAX"),
    }
    for model in ["no_change", "ARIMA", "SARIMAX"]:
        if model in pivot.columns:
            color, linestyle, label = style_map[model]
            ax.plot(pivot.index, pivot[model], label=label, color=color, linestyle=linestyle, linewidth=1.55)
    style_axis(ax, ylabel="美元/桶")
    ax.legend(loc="upper left", ncol=2, handlelength=2.8)
    finish_figure(
        fig,
        title="问题一：Brent 一个月期滚动预测",
        subtitle="月度 Brent 现货价，滚动起点评估区间为 2020-01 至 2026-06。",
        source="来源：FRED/EIA 处理面板；由 code/problem1/run_q1.py 生成。",
    )
    save_figure(fig, FIGURES_DIR / "q1_forecast_1m")
    plt.close(fig)


def plot_counterfactual(counterfactual: pd.DataFrame) -> None:
    if counterfactual.empty:
        return
    frame = counterfactual.copy()
    frame["date"] = pd.to_datetime(frame["date"])
    fig, ax = plt.subplots(figsize=(9.2, 5.1))
    ax.plot(frame["date"], frame["actual"], label="实际 Brent", color=PALETTE["ink"], linewidth=2.0)
    ax.plot(frame["date"], frame["prediction"], label="AR基准情景路径", color=PALETTE["blue"], linewidth=1.65)
    ax.fill_between(
        frame["date"],
        frame["lower_80"],
        frame["upper_80"],
        color=PALETTE["blue_light"],
        alpha=0.28,
        linewidth=0,
        label="80%区间",
    )
    for event_date, label, ypos in [
        (pd.Timestamp("2026-03-02"), "E1", 0.95),
        (pd.Timestamp("2026-03-05"), "E2", 0.88),
        (pd.Timestamp("2026-06-17"), "E3", 0.95),
    ]:
        ax.axvline(event_date, color=PALETTE["muted"], linewidth=0.7, linestyle=(0, (2, 2)))
        ax.text(
            event_date,
            ypos,
            label,
            transform=ax.get_xaxis_transform(),
            ha="center",
            va="top",
            fontsize=8.5,
            color=PALETTE["muted"],
            bbox={"facecolor": "white", "edgecolor": "none", "pad": 1.2, "alpha": 0.85},
```

```
        )
    style_axis(ax, ylabel="美元/桶")
    ax.legend(loc="upper left", ncol=3, handlelength=2.8)
    finish_figure(
        fig,
        title="问题一：事件后 Brent 油价与 AR 基准路径",
        subtitle="日度 AR(3)+美元基准情景，事件窗口 2026-03-02 至 2026-06-30；不作战争因果贡献解释。",
        source="来源：FRED Brent 与广义美元指数；由 code/problem1/run_q1.py 生成。",
    )
    save_figure(fig, FIGURES_DIR / "q1_war_counterfactual")
    plt.close(fig)


def plot_structural_shocks(shocks: pd.DataFrame) -> None:
    columns = ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
    if shocks.empty or not set(columns).issubset(shocks.columns):
        return
    frame = shocks.dropna(subset=columns).copy()
    frame["date"] = pd.to_datetime(frame["period"] + "-01") + pd.offsets.MonthEnd(0)
    labels = ["不利供给冲击", "总需求冲击", "石油特定风险冲击"]
    colors = [PALETTE["rose"], PALETTE["olive"], PALETTE["blue"]]
    fig, axes = plt.subplots(3, 1, figsize=(9.0, 6.4), sharex=True)
    for ax, column, label, color in zip(axes, columns, labels, colors):
        ax.axhline(0.0, color=PALETTE["muted"], linewidth=0.7)
        ax.plot(frame["date"], frame[column], color=color, linewidth=0.9)
        ax.text(0.01, 0.88, label, transform=ax.transAxes, ha="left", va="top", fontsize=9.4)
        style_axis(ax, ylabel="标准差")
    finish_figure(
        fig,
        title="问题一：递归SVAR结构冲击序列",
        subtitle="供给残差已变号，因此正值表示不利供给收缩；三类冲击均按各自样本标准化。",
        source="来源：results/q1_structural_shocks.csv；由 code/problem1/run_q1.py 生成。",
        rect=(0.08, 0.08, 0.98, 0.90),
    )
    save_figure(fig, FIGURES_DIR / "q1_structural_shocks")
    plt.close(fig)


def refresh_placebo_pvalues() -> dict[str, Any]:
    placebo_path = RESULTS_DIR / "q1_placebo_distribution.csv"
    effects_path = RESULTS_DIR / "q1_event_effects.csv"
    summary_path = RESULTS_DIR / "q1_summary.json"
    robustness_path = RESULTS_DIR / "q1_robustness.csv"
    if not placebo_path.exists() or not effects_path.exists() or not summary_path.exists():
        raise FileNotFoundError("Saved Q1 placebo, event-effect, and summary outputs are required.")

    placebos = pd.read_csv(placebo_path)
    effects = pd.read_csv(effects_path)
    actual = effects.loc[effects["model"].eq("brent_usd_bbl_event_car")].copy()
    empirical: dict[str, float] = {}
    for stage_id, group in placebos.groupby("stage_id"):
        actual_row = actual.loc[actual["stage_id"].eq(stage_id)]
        if actual_row.empty:
            continue
        empirical[stage_id] = finite_sample_empirical_pvalue(
            group["estimate_log_return"],
            float(actual_row["estimate_log_return"].iloc[0]),
        )

    placebos["actual_empirical_pvalue"] = placebos["stage_id"].map(empirical)
    event_mask = effects["model"].eq("brent_usd_bbl_event_car")
    effects.loc[event_mask, "pvalue_empirical"] = effects.loc[event_mask, "stage_id"].map(empirical)
    save_csv(placebos, "q1_placebo_distribution.csv")
    save_csv(effects, "q1_event_effects.csv")

    if robustness_path.exists():
        robustness = pd.read_csv(robustness_path)
        placebo_mask = robustness.get("robustness_type", pd.Series(index=robustness.index, dtype=object)).eq(
            "matched_weekend_placebo_distribution"
        )
        if "actual_empirical_pvalue" in robustness.columns:
            robustness.loc[placebo_mask, "actual_empirical_pvalue"] = robustness.loc[placebo_mask, "stage_id"].map(empirical)
            save_csv(robustness, "q1_robustness.csv")

    summary = json.loads(summary_path.read_text(encoding="utf-8"))
    summary["placebo_min_empirical_pvalue"] = min(empirical.values()) if empirical else None
```

```
    summary_path.write_text(
        json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
        encoding="utf-8",
    )
    return {
        "status": "PASS",
        "mode": "refresh-placebo-only",
        "empirical_pvalues": empirical,
        "placebo_rows": int(len(placebos)),
    }


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--refresh-placebo-only",
        action="store_true",
        help="Apply finite-sample empirical-p corrections to saved Q1 outputs without refitting models.",
    )
    parser.add_argument("--plots-only", action="store_true", help="Regenerate Q1 figures from saved result tables.")
    args = parser.parse_args(argv)

    np.random.seed(RANDOM_SEED)
    apply_paper_style()
    py_warnings.filterwarnings("ignore", category=ValueWarning)
    py_warnings.filterwarnings("ignore", category=FutureWarning, message="No supported index*")
    ensure_dirs()
    if args.refresh_placebo_only:
        payload = refresh_placebo_pvalues()
        print(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False))
        return 0
    if args.plots_only:
        forecasts = pd.read_csv(RESULTS_DIR / "q1_forecasts.csv")
        counterfactual = pd.read_csv(RESULTS_DIR / "q1_daily_counterfactual.csv")
        shocks = pd.read_csv(RESULTS_DIR / "q1_structural_shocks.csv")
        plot_forecasts(forecasts)
        plot_counterfactual(counterfactual)
        plot_structural_shocks(shocks)
        print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 3}, ensure_ascii=False, indent=2))
        return 0

    warnings_log: list[dict[str, Any]] = []
    monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_q1.csv", parse_dates=["month_end"])
    daily = pd.read_csv(PROCESSED_DIR / "model_daily_q1.csv", parse_dates=["date"])
    daily, event_meta = assign_event_stages(daily, None, "brent_usd_bbl")

    forecasts, metrics = monthly_forecasts(monthly, warnings_log)
    origin_forecast = origin_forecast_table(monthly)
    stage_effects = pd.concat(
        [
            stage_effect_regression(daily, "brent_usd_bbl", "main Brent event-stage dummy with Newey-West SE"),
            stage_effect_regression(daily, "wti_usd_bbl", "WTI robustness event-stage dummy with Newey-West SE"),
        ],
        ignore_index=True,
    )
    event_start = pd.Timestamp(event_meta["e1_trading_start"])
    car_effects_raw = pd.concat(
        [
            event_car_rows(daily, "brent_usd_bbl", event_start, "brent_usd_bbl_event_car"),
            event_car_rows(daily, "wti_usd_bbl", event_start, "wti_usd_bbl_event_car"),
        ],
        ignore_index=True,
    )
    placebos = placebo_distribution(daily, car_effects_raw)
    car_effects = add_empirical_pvalues(car_effects_raw, placebos)
    effects = pd.concat([stage_effects, car_effects], ignore_index=True, sort=False)
    save_csv(effects, "q1_event_effects.csv")
    volatility = volatility_module(daily, event_start)
    volatility_summary = volatility_stage_summary(volatility, daily)
    counterfactual = daily_counterfactual(daily)
    shocks = make_q1_shocks(monthly, counterfactual)
    robustness = robustness_summary(forecasts, effects, daily, placebos)
    plot_forecasts(forecasts)
    plot_counterfactual(counterfactual)
    plot_structural_shocks(shocks)

```

```
    advanced_non_pass = []
    if not metrics.empty and "model_status" in metrics.columns:
        advanced_non_pass = metrics.loc[
            metrics["model"].ne("no_change") & metrics["model_status"].ne("PASS"),
            ["model", "horizon", "relative_RMSE_vs_no_change", "model_status"],
        ]
        advanced_non_pass = json_records(advanced_non_pass)
    placebo_min_p = (
        float(car_effects.loc[car_effects["model"].eq("brent_usd_bbl_event_car"), "pvalue_empirical"].dropna().min())
        if "pvalue_empirical" in car_effects and not car_effects.loc[car_effects["model"].eq("brent_usd_bbl_event_car"), "
      pvalue_empirical"].dropna().empty
        else np.nan
    )
    no_change_rows = metrics.loc[metrics["model"].eq("no_change")] if not metrics.empty else pd.DataFrame()
    incorrectly_passing_advanced = (
        metrics.loc[
            metrics["model"].ne("no_change")
            & metrics["relative_RMSE_vs_no_change"].gt(1.0)
            & metrics["model_status"].eq("PASS")
        ]
        if not metrics.empty
        else pd.DataFrame()
    )
    status = "PASS"
    if warnings_log:
        status = "CONDITIONAL"
    if no_change_rows.empty or not no_change_rows["model_status"].eq("PASS").all() or not incorrectly_passing_advanced.empty:
        status = "FAIL"
    structural_counts = {
        column: int(shocks[column].dropna().shape[0]) if column in shocks.columns else 0
        for column in ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
    }
    summary = {
        "status": status,
        "execution_status": "PASS" if status != "FAIL" else "FAIL",
        "evidence_status": "SUPPORTED",
        "allowed_claims": [
            "no-change is the selected 1/3/6-month forecast benchmark under the fixed backtest",
            "E1 effects are reported as CARs with empirical placebo p-values",
            "ARBaselineGap is a descriptive AR baseline gap, not a causal war premium",
            "SVAR shocks are ex-post structural inputs for Q2/Q3 transmission analysis",
        ],
        "forbidden_claims": [
            "ARIMA/SARIMAX necessarily outperform no-change",
            "ARBaselineGap equals the net causal contribution of war",
            "E1 single-day HAC p-values are formal causal evidence",
        ],
        "random_seed": RANDOM_SEED,
        "event_calendar": event_meta,
        "selected_forecast_model": "no_change",
        "structural_shock_nonmissing_months": structural_counts,
        "forecast_model_non_pass": advanced_non_pass,
        "placebo_min_empirical_pvalue": placebo_min_p if np.isfinite(placebo_min_p) else None,
        "forecast_rows": int(len(forecasts)),
        "origin_forecast_rows": int(len(origin_forecast)),
        "metric_rows": int(len(metrics)),
        "event_effect_rows": int(len(effects)),
        "placebo_rows": int(len(placebos)),
        "shock_rows": int(len(shocks)),
        "volatility_rows": int(len(volatility)),
        "volatility_summary_rows": int(len(volatility_summary)),
        "robustness_rows": int(len(robustness)),
        "warnings": warnings_log,
        "main_metric_best_rmse": json_records(metrics.sort_values("RMSE").head(1)) if not metrics.empty else [],
    }
    (RESULTS_DIR / "q1_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
    print(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 6: run_q1.py

```python
"""Question 2: transmission of oil shocks to China's macro variables."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy.stats import norm


REPO_ROOT = Path(__file__).resolve().parents[2]
CODE_DIR = REPO_ROOT / "code"
if str(CODE_DIR) not in sys.path:
    sys.path.insert(0, str(CODE_DIR))

from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

PROCESSED_DIR = REPO_ROOT / "data" / "processed"
RESULTS_DIR = REPO_ROOT / "results"
FIGURES_DIR = REPO_ROOT / "figures"
RANDOM_SEED = 20260730
BOOTSTRAP_REPS = 2000
SHOCK_SPECS = {
    "supply_shock": "adverse_supply_shock_from_historical_recursive_VAR",
    "aggregate_demand_shock": "aggregate_demand_shock_from_historical_recursive_VAR",
    "oil_specific_risk_shock": "oil_specific_risk_shock_from_historical_recursive_VAR",
    "OilShock": "reduced_form_ARX_oil_price_innovation_robustness",
}


OUTCOME_SPECS = {
    "china_iav_yoy_pct": "中国规模以上工业增加值同比，百分点",
    "china_ppi_yoy_pct": "中国PPI同比，百分点",
    "china_cpi_yoy_pct": "中国CPI同比，百分点",
    "china_fx_log_change_pct": "人民币兑美元月度对数变化，百分点",
    "brent_cny_cost_log_change_pct": "人民币原油成本月度对数变化，百分点",
}

OUTCOME_RESPONSE_CONTRACT = {
    "china_iav_yoy_pct": ("future_yoy_level", "percentage_point"),
    "china_ppi_yoy_pct": ("future_yoy_level", "percentage_point"),
    "china_cpi_yoy_pct": ("future_yoy_level", "percentage_point"),
    "china_fx_log_change_pct": ("cumulative_log_change_from_t_minus_1", "log_percentage_point"),
    "brent_cny_cost_log_change_pct": ("future_monthly_log_change_level", "log_percentage_point"),
}


def ensure_dirs() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)


def save_csv(frame: pd.DataFrame, filename: str) -> Path:
    path = RESULTS_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def available_outcomes(frame: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> list[str]:
    outcomes: list[str] = []
    for column, label in OUTCOME_SPECS.items():
```

```python
        if column not in frame.columns or frame[column].dropna().shape[0] < 48:
            warnings_log.append({"code": "q2_outcome_skipped", "message": f"{label} skipped: insufficient usable monthly history."})
            continue
        outcomes.append(column)
    return outcomes


def available_shocks(frame: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> list[str]:
    shocks: list[str] = []
    for column in SHOCK_SPECS:
        if column not in frame.columns or frame[column].dropna().shape[0] < 36:
            warnings_log.append({"code": "q2_shock_skipped", "message": f"{column} skipped: insufficient usable shock history."})
            continue
        shocks.append(column)
    return shocks


def main_shock_column(frame: pd.DataFrame) -> str:
    for column in ["oil_specific_risk_shock", "supply_shock", "OilShock"]:
        if column in frame.columns and frame[column].dropna().shape[0] >= 36:
            return column
    return "OilShock"


def add_month_dummies(frame: pd.DataFrame) -> pd.DataFrame:
    result = frame.copy()
    result["month"] = pd.to_datetime(result["period"] + "-01").dt.month
    dummies = pd.get_dummies(result["month"], prefix="month", drop_first=True, dtype=float)
    return pd.concat([result, dummies], axis=1)


def fit_ols_hac(y: pd.Series, x: pd.DataFrame, maxlags: int) -> Any:
    x = sm.add_constant(x, has_constant="add")
    return sm.OLS(y, x).fit(cov_type="HAC", cov_kwds={"maxlags": maxlags})


def make_lags(frame: pd.DataFrame, column: str, max_lag: int, prefix: str | None = None) -> pd.DataFrame:
    result = frame.copy()
    prefix = prefix or column
    for lag in range(max_lag + 1):
        result[f"{prefix}_lag{lag}"] = result[column].shift(lag)
    return result


def quarter_label(dates: pd.Series) -> pd.Series:
    periods = pd.PeriodIndex(pd.to_datetime(dates), freq="Q")
    return pd.Series([f"{period.year}-Q{period.quarter}" for period in periods], index=dates.index)


def ardl_baseline(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    base = add_month_dummies(monthly)
    shock_col = main_shock_column(base)
    month_cols = [column for column in base.columns if column.startswith("month_") and column != "month_end"]
    control_cols = ["usd_broad_index_log_return", "GPR_z", "covid_phase"] + month_cols
    for outcome in outcomes:
        frame = make_lags(base, shock_col, 6, prefix="shock")
        frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
        regressors = [f"shock_lag{lag}" for lag in range(7)] + [f"{outcome}_lag1"] + control_cols
        usable = frame.dropna(subset=[outcome] + regressors).copy()
        if len(usable) < len(regressors) + 12:
            warnings_log.append({"code": "q2_ardl_skipped", "message": f"{outcome}: too few observations after lags."})
            continue
        fit = fit_ols_hac(usable[outcome], usable[regressors].astype(float), maxlags=6)
        lag_terms = [f"shock_lag{lag}" for lag in range(7)]
        cumulative = float(fit.params[lag_terms].sum())
        cov = fit.cov_params().loc[lag_terms, lag_terms]
        cumulative_se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
        y_lag = float(fit.params.get(f"{outcome}_lag1", np.nan))
        long_run = cumulative / (1.0 - y_lag) if np.isfinite(y_lag) and abs(1.0 - y_lag) > 1e-6 else np.nan
        long_run_se = np.nan
        if np.isfinite(long_run):
            lr_terms = lag_terms + [f"{outcome}_lag1"]
            lr_cov = fit.cov_params().loc[lr_terms, lr_terms].to_numpy()
            grad = np.array([1.0 / (1.0 - y_lag)] * len(lag_terms) + [cumulative / (1.0 - y_lag) ** 2])
            long_run_se = float(np.sqrt(grad @ lr_cov @ grad))
```

```
        for term_type, estimate, se in [
            ("short_run_lag0", float(fit.params["shock_lag0"]), float(fit.bse["shock_lag0"])),
            ("cumulative_lag0_6", cumulative, cumulative_se),
            ("long_run_multiplier", long_run, long_run_se),
        ]:
            rows.append(
                {
                    "outcome": outcome,
                    "term": term_type,
                    "estimate": estimate,
                    "std_error": se,
                    "lower_80": estimate - norm.ppf(0.90) * se if np.isfinite(se) else np.nan,
                    "upper_80": estimate + norm.ppf(0.90) * se if np.isfinite(se) else np.nan,
                    "lower_95": estimate - norm.ppf(0.975) * se if np.isfinite(se) else np.nan,
                    "upper_95": estimate + norm.ppf(0.975) * se if np.isfinite(se) else np.nan,
                    "pvalue": float(fit.pvalues["shock_lag0"]) if term_type == "short_run_lag0" else np.nan,
                    "model": "ARDL",
                    "horizon": 0,
                    "shock": shock_col,
                    "specification": f"y_lag1 + {shock_col} lags 0..6 + USD + GPR + month + COVID",
                    "sample_start": usable["period"].iloc[0],
                    "sample_end": usable["period"].iloc[-1],
                    "n": int(len(usable)),
                }
            )
    result = pd.DataFrame(rows)
    save_csv(result, "q2_ardl_baseline.csv")
    return result


def block_bootstrap_coefs(
    usable: pd.DataFrame,
    y_col: str,
    x_cols: list[str],
    term: str,
    maxlags: int,
    block_len: int = 12,
    reps: int = BOOTSTRAP_REPS,
) -> np.ndarray:
    rng = np.random.default_rng(RANDOM_SEED + maxlags + len(usable))
    n = len(usable)
    starts = np.arange(0, max(1, n - block_len + 1))
    values: list[float] = []
    for _ in range(reps):
        take: list[int] = []
        while len(take) < n:
            start = int(rng.choice(starts))
            take.extend(range(start, min(start + block_len, n)))
        sample = usable.iloc[take[:n]].copy()
        candidate_x = sample[x_cols].astype(float)
        if candidate_x[term].nunique(dropna=True) < 2:
            values.append(np.nan)
            continue
        active_cols = [
            column
            for column in x_cols
            if column == term or candidate_x[column].nunique(dropna=True) > 1
        ]
        x = sm.add_constant(candidate_x[active_cols], has_constant="add").to_numpy(dtype=float)
        y = sample[y_col].to_numpy(dtype=float)
        while np.linalg.matrix_rank(x) < x.shape[1]:
            current_deficiency = x.shape[1] - np.linalg.matrix_rank(x)
            removable = [column for column in active_cols if column != term]
            removable.sort(key=lambda column: (not column.startswith("month_"), column))
            dropped = False
            for column in removable:
                trial_cols = [candidate for candidate in active_cols if candidate != column]
                trial_x = sm.add_constant(candidate_x[trial_cols], has_constant="add").to_numpy(dtype=float)
                trial_deficiency = trial_x.shape[1] - np.linalg.matrix_rank(trial_x)
                if trial_deficiency < current_deficiency:
                    active_cols = trial_cols
                    x = trial_x
                    dropped = True
                    break
            if not dropped:
                break
```

```
        if np.linalg.matrix_rank(x) < x.shape[1] or not np.isfinite(x).all() or not np.isfinite(y).all():
            values.append(np.nan)
            continue
        beta = np.linalg.lstsq(x, y, rcond=None)[0]
        columns = ["const"] + active_cols
        values.append(float(beta[columns.index(term)]))
    return np.asarray(values, dtype=float)


def joint_lp_bootstrap(
    usable: pd.DataFrame,
    target_cols: list[str],
    regressors: list[str],
    term: str,
    block_len: int = 12,
    reps: int = BOOTSTRAP_REPS,
) -> tuple[np.ndarray, np.ndarray]:
    x = sm.add_constant(usable[regressors].astype(float), has_constant="add").to_numpy(dtype=float)
    y = usable[target_cols].to_numpy(dtype=float)
    if np.linalg.matrix_rank(x) < x.shape[1] or not np.isfinite(x).all() or not np.isfinite(y).all():
        raise np.linalg.LinAlgError("LP design is rank deficient or non-finite.")
    columns = ["const"] + regressors
    term_idx = columns.index(term)
    beta = np.linalg.lstsq(x, y, rcond=None)[0]
    point = beta[term_idx, :]
    rng = np.random.default_rng(RANDOM_SEED + len(usable) + len(regressors) + sum(ord(ch) for ch in term))
    n = len(usable)
    starts = np.arange(0, max(1, n - block_len + 1))
    boot = np.full((reps, len(target_cols)), np.nan, dtype=float)
    for rep in range(reps):
        take: list[int] = []
        while len(take) < n:
            start = int(rng.choice(starts))
            take.extend(range(start, min(start + block_len, n)))
        idx = take[:n]
        xs = x[idx, :]
        ys = y[idx, :]
        if not np.isfinite(xs).all() or not np.isfinite(ys).all():
            continue
        boot[rep, :] = np.linalg.lstsq(xs, ys, rcond=None)[0][term_idx, :]
    success = np.isfinite(boot).all(axis=1)
    if success.sum() < max(100, int(0.80 * reps)):
        raise RuntimeError(f"LP bootstrap finite-draw rate too low: {success.mean():.3f}")
    return point, boot[success, :]


def full_rank_regressors(usable: pd.DataFrame, regressors: list[str], required_terms: list[str]) -> list[str]:
    active = [
        column
        for column in regressors
        if column in required_terms or usable[column].nunique(dropna=True) > 1
    ]
    while active:
        x = sm.add_constant(usable[active].astype(float), has_constant="add").to_numpy(dtype=float)
        if np.linalg.matrix_rank(x) == x.shape[1] and np.isfinite(x).all():
            return active
        removable = [column for column in active if column not in required_terms]
        if not removable:
            break
        removable.sort(key=lambda column: (not column.startswith("month_"), column))
        active.remove(removable[0])
    raise np.linalg.LinAlgError("LP design remains rank deficient after dropping non-core controls.")


def lp_target(frame: pd.DataFrame, outcome: str, horizon: int) -> pd.Series:
    """Return the preregistered Q2 estimand for one outcome and horizon."""
    if outcome == "china_fx_log_change_pct":
        return frame["fx_level_pct"].shift(-horizon) - frame["fx_level_pct"].shift(1)
    return frame[outcome].shift(-horizon)


def validate_lp_target_contract(frame: pd.DataFrame, outcome: str, horizon: int, target_col: str) -> None:
    expected = lp_target(frame, outcome, horizon)
    observed = frame[target_col]
    mask = expected.notna() | observed.notna()
    if not np.allclose(
```

```
        expected.loc[mask].to_numpy(dtype=float),
        observed.loc[mask].to_numpy(dtype=float),
        equal_nan=True,
        atol=1e-12,
        rtol=1e-12,
    ):
        raise AssertionError(f"Q2 LP target contract failed for {outcome}, h={horizon}.")


def local_projection(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    base = add_month_dummies(monthly)
    base["fx_level_pct"] = np.log(base["cny_per_usd"]) * 100.0
    month_cols = [column for column in base.columns if column.startswith("month_") and column != "month_end"]
    shocks = available_shocks(base, warnings_log)
    for shock_col in shocks:
        base[f"{shock_col}_lag1"] = base[shock_col].shift(1)

    for outcome in outcomes:
        horizons = [0, 3, 6, 12] if outcome == "china_iav_yoy_pct" else list(range(13))
        for shock_col in shocks:
            frame = base.copy()
            frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
            target_cols: list[str] = []
            for horizon in horizons:
                target_col = f"target_h{horizon}"
                target_cols.append(target_col)
                frame[target_col] = lp_target(frame, outcome, horizon)
                validate_lp_target_contract(frame, outcome, horizon, target_col)
            regressors = [
                shock_col,
                f"{outcome}_lag1",
                f"{shock_col}_lag1",
                "usd_broad_index_log_return",
                "GPR_z",
                "covid_phase",
            ] + month_cols
            usable = frame.dropna(subset=target_cols + regressors).copy()
            if len(usable) < len(regressors) + 16:
                fallback_rows = 0
                family_size = len(target_cols)
                simultaneous_alpha = 0.05 / family_size
                for horizon, target_col in zip(horizons, target_cols, strict=False):
                    usable_h = frame.dropna(subset=[target_col] + regressors).copy()
                    if len(usable_h) < len(regressors) + 16:
                        continue
                    active_regressors = full_rank_regressors(usable_h, regressors, [shock_col])
                    fit = fit_ols_hac(
                        usable_h[target_col],
                        usable_h[active_regressors].astype(float),
                        maxlags=horizon + 1,
                    )
                    estimate = float(fit.params[shock_col])
                    boot = block_bootstrap_coefs(
                        usable_h,
                        target_col,
                        active_regressors,
                        shock_col,
                        horizon + 1,
                    )
                    boot = boot[np.isfinite(boot)]
                    if len(boot) < max(100, int(0.80 * BOOTSTRAP_REPS)):
                        continue
                    se = float(np.std(boot, ddof=1))
                    lower_80, upper_80 = np.quantile(boot, [0.10, 0.90])
                    lower_95, upper_95 = np.quantile(boot, [0.025, 0.975])
                    joint_lower, joint_upper = np.quantile(
                        boot,
                        [simultaneous_alpha / 2.0, 1.0 - simultaneous_alpha / 2.0],
                    )
                    p_boot = min(
                        1.0,
                        2.0
                        * min(
                            float((1 + np.sum(boot <= 0.0)) / (len(boot) + 1)),
                            float((1 + np.sum(boot >= 0.0)) / (len(boot) + 1)),
```

```
                        ),
                    )
                    rows.append(
                        {
                            "outcome": outcome,
                            "shock": shock_col,
                            "horizon": horizon,
                            "response": estimate,
                            "std_error": se,
                            "lower_80": float(lower_80),
                            "upper_80": float(upper_80),
                            "lower_95": float(lower_95),
                            "upper_95": float(upper_95),
                            "joint_lower_95": float(joint_lower),
                            "joint_upper_95": float(joint_upper),
                            "pvalue": float(p_boot),
                            "bootstrap_reps": int(len(boot)),
                            "model": "LocalProjection",
                            "specification": "horizon-specific LP samples with moving-block bootstrap and Bonferroni 95% simultaneous intervals; lagged outcome, lagged shock, USD, GPR, month and COVID controls",
                            "inference_band": "bonferroni_95_horizon_specific_samples",
                            "shock_identification": SHOCK_SPECS[shock_col],
                            "response_definition": OUTCOME_RESPONSE_CONTRACT[outcome][0],
                            "response_unit": OUTCOME_RESPONSE_CONTRACT[outcome][1],
                            "sample_start": usable_h["period"].iloc[0],
                            "sample_end": usable_h["period"].iloc[-1],
                            "n": int(len(usable_h)),
                        }
                    )
                    fallback_rows += 1
                if fallback_rows == 0:
                    warnings_log.append(
                        {
                            "code": "q2_lp_skipped",
                            "message": f"{outcome} {shock_col}: too few observations for joint or horizon-specific LP.",
                        }
                    )
                continue
            active_regressors = full_rank_regressors(usable, regressors, [shock_col])
            point, boot = joint_lp_bootstrap(usable, target_cols, active_regressors, shock_col)
            se = np.nanstd(boot, axis=0, ddof=1)
            lower_80, upper_80 = np.quantile(boot, [0.10, 0.90], axis=0)
            lower_95, upper_95 = np.quantile(boot, [0.025, 0.975], axis=0)
            t_boot = (boot - point) / se
            sup_t = np.nanquantile(np.nanmax(np.abs(t_boot), axis=1), 0.95)
            joint_lower = point - sup_t * se
            joint_upper = point + sup_t * se
            for idx, horizon in enumerate(horizons):
                bh = boot[:, idx]
                p_boot = min(
                    1.0,
                    2.0
                    * min(
                        float((1 + np.sum(bh <= 0.0)) / (len(bh) + 1)),
                        float((1 + np.sum(bh >= 0.0)) / (len(bh) + 1)),
                    ),
                )
                rows.append(
                    {
                        "outcome": outcome,
                        "shock": shock_col,
                        "horizon": horizon,
                        "response": float(point[idx]),
                        "std_error": float(se[idx]),
                        "lower_80": float(lower_80[idx]),
                        "upper_80": float(upper_80[idx]),
                        "lower_95": float(lower_95[idx]),
                        "upper_95": float(upper_95[idx]),
                        "joint_lower_95": float(joint_lower[idx]),
                        "joint_upper_95": float(joint_upper[idx]),
                        "pvalue": float(p_boot),
                        "bootstrap_reps": int(len(boot)),
                        "model": "LocalProjection",
                        "specification": "joint h=0..12 LP with one moving-block bootstrap sample source; lagged outcome, lagged shock, USD, GPR, month and COVID controls",
                        "inference_band": "moving_block_bootstrap_sup_t_95",
```

```
                        "shock_identification": SHOCK_SPECS[shock_col],
                        "response_definition": OUTCOME_RESPONSE_CONTRACT[outcome][0],
                        "response_unit": OUTCOME_RESPONSE_CONTRACT[outcome][1],
                        "sample_start": usable["period"].iloc[0],
                        "sample_end": usable["period"].iloc[-1],
                        "n": int(len(usable)),
                    }
                )
    result = pd.DataFrame(rows)
    result = annotate_lp_inference(result)
    save_csv(result, "q2_irf.csv")
    return result


def benjamini_hochberg(pvalues: pd.Series) -> pd.Series:
    p = pd.to_numeric(pvalues, errors="coerce")
    q = pd.Series(np.nan, index=p.index, dtype=float)
    valid = p.dropna().sort_values()
    m = len(valid)
    if m == 0:
        return q
    adjusted = valid * m / np.arange(1, m + 1)
    adjusted = adjusted.iloc[::-1].cummin().iloc[::-1].clip(upper=1.0)
    q.loc[adjusted.index] = adjusted
    return q


def annotate_lp_inference(irf: pd.DataFrame) -> pd.DataFrame:
    if irf.empty:
        return irf
    result = irf.copy()
    result["ci95_contains_zero"] = result["lower_95"].le(0) & result["upper_95"].ge(0)
    if {"joint_lower_95", "joint_upper_95"}.issubset(result.columns):
        result["joint_ci95_contains_zero"] = result["joint_lower_95"].le(0) & result["joint_upper_95"].ge(0)
    else:
        result["joint_ci95_contains_zero"] = result["ci95_contains_zero"]
    result["pre_specified_horizon"] = result["horizon"].isin([0, 3, 6, 12])
    result["fdr_qvalue"] = np.nan
    group_cols = ["outcome", "shock"] if "shock" in result.columns else ["outcome"]
    for _, group in result.groupby(group_cols):
        result.loc[group.index, "fdr_qvalue"] = benjamini_hochberg(group["pvalue"])
    result["fdr_significant_10pct"] = result["fdr_qvalue"].le(0.10)
    result["supports_growth_loss_language"] = (
        result["outcome"].isin(["china_iav_yoy_pct", "china_real_gdp_yoy_pct"])
        & result["response"].lt(0)
        & ~result["joint_ci95_contains_zero"]
    )
    return result


def asymmetry_check(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    base = add_month_dummies(monthly)
    shock_col = main_shock_column(base)
    base["shock_pos"] = base[shock_col].clip(lower=0)
    base["shock_neg"] = base[shock_col].clip(upper=0)
    month_cols = [column for column in base.columns if column.startswith("month_") and column != "month_end"]
    for outcome in outcomes:
        frame = base.copy()
        frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
        regressors = ["shock_pos", "shock_neg", f"{outcome}_lag1", "usd_broad_index_log_return", "GPR_z", "covid_phase"] + month_cols
        usable = frame.dropna(subset=[outcome] + regressors)
        if len(usable) < len(regressors) + 12:
            continue
        fit = fit_ols_hac(usable[outcome], usable[regressors].astype(float), maxlags=6)
        for term in ["shock_pos", "shock_neg"]:
            estimate = float(fit.params[term])
            se = float(fit.bse[term])
            rows.append(
                {
                    "outcome": outcome,
                    "term": term,
                    "shock": shock_col,
                    "estimate": estimate,
                    "std_error": se,
                    "lower_95": estimate - norm.ppf(0.975) * se,
```

```
                    "upper_95": estimate + norm.ppf(0.975) * se,
                    "model": "sign_asymmetry_DL",
                    "specification": f"positive and negative {shock_col} split; not a full NARDL/ECM",
                    "sample_start": usable["period"].iloc[0],
                    "sample_end": usable["period"].iloc[-1],
                    "n": int(len(usable)),
                }
            )
    result = pd.DataFrame(rows)
    save_csv(result, "q2_asymmetry.csv")
    return result


def lag_and_covid_robustness(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    base_all = add_month_dummies(monthly)
    shock_col = main_shock_column(base_all)
    month_cols = [column for column in base_all.columns if column.startswith("month_") and column != "month_end"]
    for lag_max in [3, 6, 12]:
        for exclude_covid in [False, True]:
            base = base_all.loc[~base_all["covid_phase"].eq(1)].copy() if exclude_covid else base_all.copy()
            controls = ["usd_broad_index_log_return", "GPR_z"] + ([] if exclude_covid else ["covid_phase"]) + month_cols
            for outcome in outcomes:
                frame = make_lags(base, shock_col, lag_max, prefix="shock")
                frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
                lag_terms = [f"shock_lag{lag}" for lag in range(lag_max + 1)]
                regressors = lag_terms + [f"{outcome}_lag1"] + controls
                usable = frame.dropna(subset=[outcome] + regressors)
                if len(usable) < len(regressors) + 12:
                    warnings_log.append(
                        {
                            "code": "q2_robustness_skipped",
                            "message": f"{outcome} lag={lag_max} exclude_covid={exclude_covid}: too few observations.",
                        }
                    )
                    continue
                fit = fit_ols_hac(usable[outcome], usable[regressors].astype(float), maxlags=lag_max)
                estimate = float(fit.params[lag_terms].sum())
                cov = fit.cov_params().loc[lag_terms, lag_terms]
                se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
                rows.append(
                    {
                        "outcome": outcome,
                        "shock": shock_col,
                        "lag_max": lag_max,
                        "exclude_covid": exclude_covid,
                        "estimate": estimate,
                        "std_error": se,
                        "lower_95": estimate - norm.ppf(0.975) * se,
                        "upper_95": estimate + norm.ppf(0.975) * se,
                        "model": "ARDL_robustness",
                        "specification": f"{shock_col} lags 0..{lag_max}; exclude_covid={exclude_covid}",
                        "sample_start": usable["period"].iloc[0],
                        "sample_end": usable["period"].iloc[-1],
                        "n": int(len(usable)),
                    }
                )
    result = pd.DataFrame(rows)
    save_csv(result, "q2_robustness.csv")
    return result


def gdp_validation(quarterly: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    frame = quarterly.copy()
    frame["gdp_lag1"] = frame["china_real_gdp_yoy_pct"].shift(1)
    shock_sum_col = "oil_specific_risk_shock_sum" if "oil_specific_risk_shock_sum" in frame.columns and frame["
      oil_specific_risk_shock_sum"].dropna().shape[0] >= 8 else "OilShock_sum"
    usable = frame.dropna(subset=["china_real_gdp_yoy_pct", "gdp_lag1", shock_sum_col, "GPR_z_mean"]) if "GPR_z_mean" in frame.columns
      else pd.DataFrame()
    if "GPR_z_mean" not in frame.columns:
        monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
        monthly["quarter"] = quarter_label(monthly["period"] + "-01")
        gpr_q = monthly.groupby("quarter", as_index=False).agg(GPR_z_mean=("GPR_z", "mean"))
        frame = frame.merge(gpr_q, on="quarter", how="left")
        usable = frame.dropna(subset=["china_real_gdp_yoy_pct", "gdp_lag1", shock_sum_col, "GPR_z_mean"])
    if len(usable) < 16:
```

```
        warnings_log.append({"code": "q2_gdp_validation_limited", "message": "Too few GDP validation observations after lags."})
        result = pd.DataFrame(
            [
                {
                    "outcome": "china_real_gdp_yoy_pct",
                    "model": "quarterly_validation",
                    "estimate": np.nan,
                    "std_error": np.nan,
                    "lower_95": np.nan,
                    "upper_95": np.nan,
                    "evidence_status": "INCONCLUSIVE",
                    "shock": shock_sum_col,
                    "correlation": float(frame[["china_real_gdp_yoy_pct", shock_sum_col]].dropna().corr().iloc[0, 1]) if frame[["
    china_real_gdp_yoy_pct", shock_sum_col]].dropna().shape[0] > 3 else np.nan,
                    "n": int(frame[["china_real_gdp_yoy_pct", shock_sum_col]].dropna().shape[0]),
                    "sample_start": "",
                    "sample_end": "",
                    "specification": "insufficient observations for regression; correlation shown if available",
                }
            ]
        )
        save_csv(result, "q2_gdp_validation.csv")
        return result
    regressors = [shock_sum_col, "gdp_lag1", "GPR_z_mean"]
    fit = fit_ols_hac(usable["china_real_gdp_yoy_pct"], usable[regressors].astype(float), maxlags=2)
    estimate = float(fit.params[shock_sum_col])
    se = float(fit.bse[shock_sum_col])
    lower_95 = estimate - norm.ppf(0.975) * se
    upper_95 = estimate + norm.ppf(0.975) * se
    result = pd.DataFrame(
        [
            {
                "outcome": "china_real_gdp_yoy_pct",
                "model": "quarterly_validation",
                "estimate": estimate,
                "std_error": se,
                "lower_80": estimate - norm.ppf(0.90) * se,
                "upper_80": estimate + norm.ppf(0.90) * se,
                "lower_95": lower_95,
                "upper_95": upper_95,
                "pvalue": float(fit.pvalues[shock_sum_col]),
                "evidence_status": "SUPPORTED" if lower_95 > 0 or upper_95 < 0 else "INCONCLUSIVE",
                "shock": shock_sum_col,
                "correlation": float(usable[["china_real_gdp_yoy_pct", shock_sum_col]].corr().iloc[0, 1]),
                "n": int(len(usable)),
                "sample_start": usable["quarter"].iloc[0],
                "sample_end": usable["quarter"].iloc[-1],
                "specification": f"GDP yoy on quarterly {shock_sum_col}, lagged GDP yoy and GPR",
            }
        ]
    )
    save_csv(result, "q2_gdp_validation.csv")
    return result


def build_transmission_metrics(irf: pd.DataFrame, gdp: pd.DataFrame) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    if not irf.empty:
        for (shock, outcome), group in irf.groupby(["shock", "outcome"]):
            frame = group.sort_values("horizon").copy()
            values = pd.to_numeric(frame["response"], errors="coerce")
            if values.dropna().empty:
                continue
            if outcome == "china_iav_yoy_pct":
                idx = values.idxmin()
                extremum_type = "trough"
            else:
                idx = values.abs().idxmax()
                extremum_type = "peak_abs"
            extreme = frame.loc[idx]
            complete_monthly_path = set(frame["horizon"].astype(int)) >= set(range(13))
            area_h6 = frame.loc[frame["horizon"].le(6), "response"].sum() if complete_monthly_path else np.nan
            area_h12 = frame.loc[frame["horizon"].le(12), "response"].sum() if complete_monthly_path else np.nan
            recovery = frame.loc[
                frame["horizon"].ge(int(extreme["horizon"])) & frame["lower_95"].le(0) & frame["upper_95"].ge(0),
                "horizon",
```

```
            ]
            supported_loss = bool(
                outcome == "china_iav_yoy_pct"
                and float(extreme["response"]) < 0
                and not (float(extreme["joint_lower_95"]) <= 0 <= float(extreme["joint_upper_95"]))
            )
            evidence = "SUPPORTED" if supported_loss else ("INCONCLUSIVE" if bool(frame["joint_ci95_contains_zero"].any()) else "SUPPORTED")
            rows.append(
                {
                    "shock": shock,
                    "outcome": outcome,
                    "extremum_type": extremum_type,
                    "extremum_response": float(extreme["response"]),
                    "extremum_month": int(extreme["horizon"]),
                    "extremum_lower_95": float(extreme["lower_95"]),
                    "extremum_upper_95": float(extreme["upper_95"]),
                    "extremum_joint_lower_95": float(extreme["joint_lower_95"]),
                    "extremum_joint_upper_95": float(extreme["joint_upper_95"]),
                    "response_curve_area_0_6": float(area_h6) if np.isfinite(area_h6) else np.nan,
                    "response_curve_area_0_12": float(area_h12) if np.isfinite(area_h12) else np.nan,
                    "area_unit": "percentage_point_month" if complete_monthly_path else "not_applicable_sparse_horizons",
                    "available_horizons": "|".join(str(int(value)) for value in sorted(frame["horizon"].unique())),
                    "recovery_month": int(recovery.iloc[0]) if not recovery.empty else np.nan,
                    "evidence_status": evidence,
                    "allows_growth_loss_language": supported_loss,
                    "inference_source": str(extreme.get("inference_band", "")),
                    "sample_start": str(frame["sample_start"].dropna().iloc[0]) if not frame["sample_start"].dropna().empty else "",
                    "sample_end": str(frame["sample_end"].dropna().iloc[-1]) if not frame["sample_end"].dropna().empty else "",
                }
            )
    if not gdp.empty:
        for row in gdp.to_dict("records"):
            rows.append(
                {
                    "shock": row.get("shock", ""),
                    "outcome": row.get("outcome", "china_real_gdp_yoy_pct"),
                    "extremum_type": "quarterly_validation",
                    "extremum_response": row.get("estimate", np.nan),
                    "extremum_month": np.nan,
                    "extremum_lower_95": row.get("lower_95", np.nan),
                    "extremum_upper_95": row.get("upper_95", np.nan),
                    "extremum_joint_lower_95": row.get("lower_95", np.nan),
                    "extremum_joint_upper_95": row.get("upper_95", np.nan),
                    "response_curve_area_0_6": np.nan,
                    "response_curve_area_0_12": np.nan,
                    "area_unit": "not_applicable_quarterly_validation",
                    "available_horizons": "",
                    "recovery_month": np.nan,
                    "evidence_status": row.get("evidence_status", "INCONCLUSIVE"),
                    "allows_growth_loss_language": False,
                    "inference_source": "quarterly_HAC_validation",
                    "sample_start": row.get("sample_start", ""),
                    "sample_end": row.get("sample_end", ""),
                }
            )
    result = pd.DataFrame(rows)
    save_csv(result, "q2_transmission_metrics.csv")
    return result


def plot_irf(irf: pd.DataFrame) -> None:
    if irf.empty:
        return
    plot_shock = "oil_specific_risk_shock" if "oil_specific_risk_shock" in set(irf.get("shock", pd.Series(dtype=str))) else str(irf.get("shock", pd.Series(["OilShock"])).dropna().iloc[0])
    irf = irf.loc[irf.get("shock", pd.Series(plot_shock, index=irf.index)).eq(plot_shock)].copy()
    outcomes = irf["outcome"].unique().tolist()
    fig, axes = plt.subplots(len(outcomes), 1, figsize=(8.8, max(4.2, 3.0 * len(outcomes))), sharex=True)
    if len(outcomes) == 1:
        axes = [axes]
    for ax, outcome in zip(axes, outcomes, strict=False):
        frame = irf.loc[irf["outcome"].eq(outcome)].sort_values("horizon")
        ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
        if outcome == "china_iav_yoy_pct":
            ax.errorbar(
```

```
                frame["horizon"],
                frame["response"],
                yerr=[frame["response"] - frame["lower_95"], frame["upper_95"] - frame["response"]],
                fmt="o",
                color=PALETTE["blue"],
                capsize=3,
                label="离散期限响应及95%区间",
            )
        else:
            ax.plot(frame["horizon"], frame["response"], marker="o", color=PALETTE["blue"], label="估计响应")
            ax.fill_between(
                frame["horizon"],
                frame["lower_95"],
                frame["upper_95"],
                color=PALETTE["blue_light"],
                alpha=0.30,
                linewidth=0,
                label="95%置信区间",
            )
        label = OUTCOME_SPECS.get(outcome, outcome)
        ax.text(0.0, 0.91, label, transform=ax.transAxes, ha="left", va="top", fontsize=10.2, color=PALETTE["ink"])
        style_axis(ax, ylabel="响应")
    axes[-1].set_xlabel("油价冲击后月份")
    axes[0].legend(loc="upper right", handlelength=2.4)
    finish_figure(
        fig,
        title="问题二：中国宏观变量对约化形式油价冲击的响应",
        subtitle=f"月度中国变量；主图冲击为 {plot_shock}；阴影为同源移动块bootstrap 95%逐期区间。",
        source="来源：问题一结构冲击接口、国家统计局 IAV/PPI、OECD CPI、FRED 汇率/美元/GPR；由 code/problem2/run_q2.py 生成。",
        rect=(0.09, 0.12, 0.98, 0.84),
    )
    save_figure(fig, FIGURES_DIR / "q2_irf")
    plt.close(fig)


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--plots-only", action="store_true", help="Regenerate Q2 figures from saved result tables.")
    args = parser.parse_args(argv)

    np.random.seed(RANDOM_SEED)
    apply_paper_style()
    ensure_dirs()
    if args.plots_only:
        irf = pd.read_csv(RESULTS_DIR / "q2_irf.csv")
        plot_irf(irf)
        print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 1}, ensure_ascii=False, indent=2))
        return 0

    warnings_log: list[dict[str, Any]] = []
    monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
    quarterly = pd.read_csv(PROCESSED_DIR / "model_quarterly_cn.csv")
    outcomes = available_outcomes(monthly, warnings_log)
    ardl = ardl_baseline(monthly, outcomes, warnings_log)
    irf = local_projection(monthly, outcomes, warnings_log)
    asymmetry = asymmetry_check(monthly, outcomes, warnings_log)
    robustness = lag_and_covid_robustness(monthly, outcomes, warnings_log)
    gdp = gdp_validation(quarterly, warnings_log)
    transmission_metrics = build_transmission_metrics(irf, gdp)
    plot_irf(irf)
    gdp_status = "INCONCLUSIVE"
    if not gdp.empty and "evidence_status" in gdp.columns:
        gdp_status = str(gdp["evidence_status"].dropna().iloc[0]) if not gdp["evidence_status"].dropna().empty else "INCONCLUSIVE"

    summary = pd.DataFrame(
        [
            {
                "component": "ARDL",
                "rows": len(ardl),
                "status": "PASS" if len(ardl) else "WARN",
                "note": "available monthly outcomes only; OilShock is reduced-form innovation",
            },
            {
                "component": "LocalProjection",
                "rows": len(irf),
                "status": "PASS" if len(irf) else "WARN",
```

```
                "note": "horizons 0..12 where estimable; reports FDR and zero-crossing flags",
            },
            {
                "component": "GDPValidation",
                "rows": len(gdp),
                "status": gdp_status,
                "note": "quarterly GDP check; no monthly interpolation",
            },
            {
                "component": "Asymmetry",
                "rows": len(asymmetry),
                "status": "PASS" if len(asymmetry) else "WARN",
                "note": "positive and negative shock split; not a full NARDL",
            },
            {
                "component": "LagCovidRobustness",
                "rows": len(robustness),
                "status": "PASS" if len(robustness) else "WARN",
                "note": "lag 3/6/12 and exclude-COVID specifications",
            },
        ]
    )
    save_csv(summary, "q2_summary.csv")
    payload = {
        "status": "PASS" if len(irf) and len(ardl) else "CONDITIONAL",
        "execution_status": "PASS" if len(irf) or len(ardl) else "WARN",
        "evidence_status": "INCONCLUSIVE",
        "allowed_claims": [
            "Q2 reports direction, magnitude and uncertainty for oil-shock transmission",
            "RMB crude-cost, PPI, CPI, FX and industrial activity responses are model outputs where data permit",
            "GDP is a low-frequency validation and remains inconclusive when intervals cross zero",
        ],
        "forbidden_claims": [
            "oil shocks caused a robust aggregate growth loss unless the industrial/GDP joint interval excludes zero",
            "reduced-form OilShock is a structural supply or war shock",
        ],
        "random_seed": RANDOM_SEED,
        "outcomes": outcomes,
        "shock_identification": "LP main interface estimates supply, aggregate-demand and oil-specific-risk shocks from the historical recursive VAR when available; OilShock remains a reduced-form robustness shock.",
        "conclusion_guardrail": "Current estimates do not support a clear aggregate growth-loss statement unless negative responses jointly exclude zero.",
        "warnings": warnings_log,
        "rows": {
            "q2_ardl_baseline.csv": int(len(ardl)),
            "q2_irf.csv": int(len(irf)),
            "q2_gdp_validation.csv": int(len(gdp)),
            "q2_asymmetry.csv": int(len(asymmetry)),
            "q2_robustness.csv": int(len(robustness)),
            "q2_transmission_metrics.csv": int(len(transmission_metrics)),
        },
    }
    (RESULTS_DIR / "q2_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(payload, ensure_ascii=False, indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 7: run_q2.py

```
"""Question 3: cross-country buffers and China policy counterfactuals."""

from __future__ import annotations

import argparse
import json
import sys
import warnings as py_warnings
from pathlib import Path
from typing import Any

import matplotlib

```

```
matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from linearmodels.panel import PanelOLS
from linearmodels.panel.utility import AbsorbingEffectWarning
from scipy.stats import norm


REPO_ROOT = Path(__file__).resolve().parents[2]
CODE_DIR = REPO_ROOT / "code"
if str(CODE_DIR) not in sys.path:
    sys.path.insert(0, str(CODE_DIR))

from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

PROCESSED_DIR = REPO_ROOT / "data" / "processed"
RESULTS_DIR = REPO_ROOT / "results"
FIGURES_DIR = REPO_ROOT / "figures"
RANDOM_SEED = 20260730
POLICY_BOOTSTRAP_REPS = 2000
POLICY_BOOTSTRAP_BLOCK = 6


COUNTRY_ORDER = ["CHN", "DEU", "FRA", "ITA", "ESP", "JPN", "KOR"]
MAIN_CONTROL_COUNTRIES = ["DEU", "FRA", "ITA", "ESP", "JPN", "KOR"]
COUNTRY_LABEL_ZH = {
    "CHN": "中国",
    "DEU": "德国",
    "FRA": "法国",
    "ITA": "意大利",
    "ESP": "西班牙",
    "JPN": "日本",
    "KOR": "韩国",
}
OUTCOME_LABELS = {
    "fuel": "燃油价格累计对数变化，百分点",
    "cpi": "CPI同比，百分点",
    "ip": "工业生产同比对数变化，百分点",
}


def ensure_dirs() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)


def save_csv(frame: pd.DataFrame, filename: str) -> Path:
    path = RESULTS_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def read_csv_result(filename: str) -> pd.DataFrame:
    path = RESULTS_DIR / filename
    return pd.read_csv(path) if path.exists() else pd.DataFrame()


def log_positive(series: pd.Series) -> pd.Series:
    values = pd.to_numeric(series, errors="coerce")
    result = pd.Series(np.nan, index=series.index, dtype=float)
    mask = values > 0
    result.loc[mask] = np.log(values.loc[mask])
    return result


def add_lags(frame: pd.DataFrame, column: str, max_lag: int) -> pd.DataFrame:
    result = frame.copy()
    for lag in range(max_lag + 1):
        result[f"{column}_lag{lag}"] = result[column].shift(lag)
    return result
```

```python


def bool_series(series: pd.Series) -> pd.Series:
    if pd.api.types.is_bool_dtype(series):
        return series
    return series.astype(str).str.lower().isin(["true", "1", "yes"])


def shock_column(frame: pd.DataFrame) -> str:
    for column in ["oil_specific_risk_shock", "supply_shock", "aggregate_demand_shock", "OilShock"]:
        if column in frame.columns and pd.to_numeric(frame[column], errors="coerce").notna().sum() >= 24:
            return column
    raise ValueError("Q3 panel lacks an auditable oil shock column.")


def fit_country_pass_through(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for country in COUNTRY_ORDER:
        frame = panel.loc[panel["country"].eq(country)].sort_values("period").copy()
        frame = add_lags(frame, "brent_local_log_return", 6)
        regressors = [f"brent_local_log_return_lag{lag}" for lag in range(7)]
        frame["fuel_lag1"] = frame["fuel_log_return"].shift(1)
        usable = frame.dropna(subset=["fuel_log_return", "fuel_lag1"] + regressors)
        if len(usable) < 48:
            warnings_log.append({"code": "q3_pass_through_limited", "message": f"{country}: insufficient fuel observations."})
            continue
        included = bool(bool_series(frame["included_in_main_comparison"]).iloc[0]) if "included_in_main_comparison" in frame else country
       != "CHN"
        measure_type = frame["price_measure_type"].iloc[0] if "price_measure_type" in frame else ""
        observed_or_regulated = frame["observed_or_regulated"].iloc[0] if "observed_or_regulated" in frame else ""
        comparability_note = frame["comparability_note"].iloc[0] if "comparability_note" in frame else ""
        x = sm.add_constant(usable[regressors + ["fuel_lag1"]].astype(float), has_constant="add")
        fit = sm.OLS(usable["fuel_log_return"], x).fit(cov_type="HAC", cov_kwds={"maxlags": 6})
        for horizon in [1, 3, 6]:
            lag_terms = [f"brent_local_log_return_lag{lag}" for lag in range(horizon + 1)]
            estimate = float(fit.params[lag_terms].sum())
            cov = fit.cov_params().loc[lag_terms, lag_terms]
            se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
            rows.append(
                {
                    "country": country,
                    "country_name": frame["country_name"].iloc[0],
                    "horizon": horizon,
                    "response": estimate,
                    "std_error": se,
                    "lower_80": estimate - norm.ppf(0.90) * se,
                    "upper_80": estimate + norm.ppf(0.90) * se,
                    "lower_95": estimate - norm.ppf(0.975) * se,
                    "upper_95": estimate + norm.ppf(0.975) * se,
                    "model": "distributed_lag_pass_through",
                    "specification": "fuel log return on local-currency Brent log-return lags 0..6 and fuel lag",
                    "sample_start": usable["period"].iloc[0],
                    "sample_end": usable["period"].iloc[-1],
                    "n": int(len(usable)),
                    "fuel_source": frame["fuel_source"].iloc[0],
                    "fuel_unit": frame["fuel_unit"].iloc[0],
                    "price_measure_type": measure_type,
                    "observed_or_regulated": observed_or_regulated,
                    "included_in_main_comparison": included,
                    "comparability_note": comparability_note,
                }
            )
    result = pd.DataFrame(rows)
    save_csv(result, "q3_country_pass_through.csv")
    return result


def panel_design(frame: pd.DataFrame, outcome: str, horizon: int) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame, str]:
    data = frame.copy()
    shock = shock_column(data)
    data["shock"] = pd.to_numeric(data[shock], errors="coerce")
    if outcome == "fuel":
        data["target"] = data.groupby("country")["fuel_log"].shift(-horizon) - data.groupby("country")["fuel_log"].shift(1)
        data["target"] = data["target"] * 100.0
        needed = ["target", "shock", "fuel_log"]
    elif outcome == "cpi":
```

```
        data["target"] = data.groupby("country")["cpi_yoy_pct"].shift(-horizon)
        needed = ["target", "shock", "cpi_yoy_pct"]
    else:
        activity_col = "industrial_activity_yoy_pct" if "industrial_activity_yoy_pct" in data.columns else "ip_yoy_log_change_pct"
        data["target"] = data.groupby("country")[activity_col].shift(-horizon)
        needed = ["target", "shock", activity_col]
    data["trend"] = data.groupby("country").cumcount()
    for country in MAIN_CONTROL_COUNTRIES:
        data[f"oil_diff_{country}_vs_CHN"] = np.where(data["country"].eq(country), data["shock"], 0.0)
        data[f"trend_{country}_vs_CHN"] = np.where(data["country"].eq(country), data["trend"], 0.0)
    needed += [f"oil_diff_{country}_vs_CHN" for country in MAIN_CONTROL_COUNTRIES]
    x_cols = [f"oil_diff_{country}_vs_CHN" for country in MAIN_CONTROL_COUNTRIES] + [
        f"trend_{country}_vs_CHN" for country in MAIN_CONTROL_COUNTRIES
    ]
    usable = data.dropna(subset=needed).copy()
    usable["month_index"] = pd.PeriodIndex(usable["period"], freq="M").to_timestamp()
    usable = usable.set_index(["country", "month_index"]).sort_index()
    y = usable["target"].astype(float)
    x = usable[x_cols].astype(float)
    return y, x, usable.reset_index(), shock


def buffer_design(frame: pd.DataFrame, outcome: str, horizon: int, buffer: str) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame, str, str
    ]:
    data = frame.copy()
    shock = shock_column(data)
    data["shock"] = pd.to_numeric(data[shock], errors="coerce")
    if buffer not in data.columns:
        raise ValueError(f"country_policy_buffers_annual.csv lacks {buffer}.")
    data[buffer] = pd.to_numeric(data[buffer], errors="coerce")
    if buffer != "fuel_price_regulation":
        std = data[buffer].std(skipna=True)
        if not np.isfinite(std) or std <= 0:
            raise ValueError(f"{buffer} has zero variation.")
        data[f"{buffer}_standardized"] = (data[buffer] - data[buffer].mean(skipna=True)) / std
        buffer_col = f"{buffer}_standardized"
    else:
        buffer_col = buffer
    if outcome == "fuel":
        data["target"] = data.groupby("country")["fuel_log"].shift(-horizon) - data.groupby("country")["fuel_log"].shift(1)
        data["target"] = data["target"] * 100.0
        needed = ["target", "shock", "fuel_log", buffer_col]
    elif outcome == "cpi":
        data["target"] = data.groupby("country")["cpi_yoy_pct"].shift(-horizon)
        needed = ["target", "shock", "cpi_yoy_pct", buffer_col]
    else:
        activity_col = "industrial_activity_yoy_pct" if "industrial_activity_yoy_pct" in data.columns else "ip_yoy_log_change_pct"
        data["target"] = data.groupby("country")[activity_col].shift(-horizon)
        needed = ["target", "shock", activity_col, buffer_col]
    term = f"shock_x_{buffer}"
    data[term] = data["shock"] * data[buffer_col]
    usable = data.dropna(subset=needed + [term]).copy()
    usable = usable.loc[usable[term].notna()].copy()
    usable["month_index"] = pd.PeriodIndex(usable["period"], freq="M").to_timestamp()
    usable = usable.set_index(["country", "month_index"]).sort_index()
    y = usable["target"].astype(float)
    x = usable[[term]].astype(float)
    return y, x, usable.reset_index(), shock, term


def fit_panel_lp(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    from linearmodels.panel import PanelOLS

    rows: list[dict[str, Any]] = []
    for outcome in ["cpi", "ip"]:
        outcome_panel = panel.copy()
        for horizon in range(13):
            y, x, usable, shock = panel_design(outcome_panel, outcome, horizon)
            if len(y) < x.shape[1] + 20 or y.index.get_level_values(0).nunique() < 3:
                warnings_log.append({"code": "q3_panel_lp_skipped", "message": f"{outcome} h={horizon}: insufficient panel support."})
                continue
            fit = PanelOLS(y, x, entity_effects=True, time_effects=True, drop_absorbed=True, check_rank=False).fit(
                cov_type="kernel",
                kernel="bartlett",
                bandwidth=max(1, horizon + 1),
            )
```

```
            for country in MAIN_CONTROL_COUNTRIES:
                term = f"oil_diff_{country}_vs_CHN"
                if term not in fit.params.index:
                    continue
                estimate = float(fit.params[term])
                se = float(fit.std_errors[term])
                rows.append(
                    {
                        "outcome": outcome,
                        "country": country,
                        "reference_country": "CHN",
                        "horizon": horizon,
                        "response": estimate,
                        "std_error": se,
                        "lower_80": estimate - norm.ppf(0.90) * se,
                        "upper_80": estimate + norm.ppf(0.90) * se,
                        "lower_95": estimate - norm.ppf(0.975) * se,
                        "upper_95": estimate + norm.ppf(0.975) * se,
                        "pvalue": float(fit.pvalues[term]),
                        "shock": shock,
                        "model": "stacked_panel_LP_time_FE_relative_to_CHN",
                        "response_type": "control_country_minus_china_relative_response",
                        "specification": "country FE, full year-month FE, control-country shock interactions relative to China, control-country trends, Driscoll-Kraay covariance",
                        "sample_start": usable["period"].min(),
                        "sample_end": usable["period"].max(),
                        "n": int(len(usable)),
                    }
                )
    result = pd.DataFrame(rows)
    save_csv(result, "q3_panel_irf.csv")
    return result


def fit_buffer_interactions(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    """Fail closed while the annual structural-buffer table remains unaudited."""
    buffers = ["oil_import_dependency", "oil_intensity", "import_source_hhi", "fuel_price_regulation"]
    status = pd.DataFrame(
        [
            {
                "buffer": buffer,
                "data_status": "FAIL_FOR_MAIN_UNAUDITED_PROXY",
                "quantitative_result_released": False,
                "reason": "country_policy_buffers_annual.csv is a normalized proxy pending audited official exports",
                "required_replacement": "country-year raw values, formula, official URL/table id, download date and file hash",
            }
            for buffer in buffers
        ]
    )
    save_csv(status, "q3_buffer_data_status.csv")
    columns = [
        "outcome",
        "buffer",
        "horizon",
        "estimate",
        "std_error",
        "lower_95",
        "upper_95",
        "shock",
        "model",
        "data_status",
    ]
    result = pd.DataFrame(columns=columns)
    save_csv(result, "q3_buffer_interactions.csv")
    warnings_log.append(
        {
            "code": "q3_buffer_quantitative_results_withheld",
            "message": "Continuous buffer interactions were withheld because the country-year table is an unaudited proxy.",
        }
    )
    return result


def china_fuel_to_macro_elasticities(warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    official = pd.read_csv(PROCESSED_DIR / "china_regulated_gasoline_monthly.csv")
    macro = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
```

```
    frame = macro.merge(
        official[["period", "china_regulated_gasoline_cny_per_ton"]],
        on="period",
        how="left",
    ).sort_values("period")
    frame["fuel_log"] = log_positive(frame["china_regulated_gasoline_cny_per_ton"])
    frame["fuel_log_return"] = frame["fuel_log"].diff()
    frame = add_lags(frame, "fuel_log_return", 6)
    outcomes = {
        "china_ppi_yoy_pct": "PPI",
        "china_cpi_yoy_pct": "CPI",
        "china_iav_yoy_pct": "IAV",
    }
    rows: list[dict[str, Any]] = []
    kernel_rows: list[dict[str, Any]] = []
    covariance_rows: list[dict[str, Any]] = []
    for outcome, label in outcomes.items():
        if outcome not in frame.columns:
            warnings_log.append({"code": "q3_policy_macro_outcome_missing", "message": f"{outcome} missing for policy propagation."})
            continue
        frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
        lag_terms = [f"fuel_log_return_lag{lag}" for lag in range(7)]
        regressors = lag_terms + [f"{outcome}_lag1", "GPR"]
        usable = frame.dropna(subset=[outcome] + regressors).copy()
        if len(usable) < 48:
            warnings_log.append({"code": "q3_policy_macro_elasticity_limited", "message": f"{outcome}: only {len(usable)} usable
  observations."})
            continue
        fit = sm.OLS(usable[outcome], sm.add_constant(usable[regressors].astype(float), has_constant="add")).fit(
            cov_type="HAC",
            cov_kwds={"maxlags": 6},
        )
        estimate = float(fit.params[lag_terms].sum())
        cov = fit.cov_params().loc[lag_terms, lag_terms]
        se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
        param_terms = ["const"] + lag_terms + [f"{outcome}_lag1", "GPR"]
        kernel_row = {
            "outcome": outcome,
            "outcome_label": label,
            "sample_start": usable["period"].iloc[0],
            "sample_end": usable["period"].iloc[-1],
            "n": int(len(usable)),
            "model": "China_fuel_to_macro_ARDL",
            "specification": "macro yoy outcome on regulated gasoline log-return lags 0..6, lagged outcome and GPR; HAC(6)",
            "fuel_to_macro_cumulative_elasticity": estimate,
        }
        for lag in range(7):
            term = f"fuel_log_return_lag{lag}"
            kernel_row[f"beta_fuel_lag{lag}"] = float(fit.params[term])
            kernel_row[f"se_fuel_lag{lag}"] = float(fit.bse[term])
        kernel_row["phi_outcome_lag1"] = float(fit.params[f"{outcome}_lag1"])
        kernel_row["se_outcome_lag1"] = float(fit.bse[f"{outcome}_lag1"])
        kernel_row["gpr_coefficient"] = float(fit.params["GPR"])
        kernel_row["constant"] = float(fit.params["const"])
        kernel_rows.append(kernel_row)
        full_cov = fit.cov_params()
        for row_term in param_terms:
            for column_term in param_terms:
                covariance_rows.append(
                    {
                        "outcome": outcome,
                        "outcome_label": label,
                        "row_term": row_term,
                        "column_term": column_term,
                        "covariance": float(full_cov.loc[row_term, column_term]),
                        "model": "China_fuel_to_macro_ARDL",
                        "sample_start": usable["period"].iloc[0],
                        "sample_end": usable["period"].iloc[-1],
                        "n": int(len(usable)),
                    }
                )
        rows.append(
            {
                "outcome": outcome,
                "outcome_label": label,
                "fuel_to_macro_cumulative_elasticity": estimate,
```

```
                "std_error": se,
                "sample_start": usable["period"].iloc[0],
                "sample_end": usable["period"].iloc[-1],
                "n": int(len(usable)),
                "model": "China_fuel_to_macro_ARDL",
                "specification": "macro yoy outcome on regulated gasoline log-return lags 0..6, lagged outcome and GPR; HAC(6)",
            }
        )
    kernel_columns = [
        "outcome",
        "outcome_label",
        "sample_start",
        "sample_end",
        "n",
        "model",
        "specification",
        "fuel_to_macro_cumulative_elasticity",
        "beta_fuel_lag0",
        "beta_fuel_lag1",
        "beta_fuel_lag2",
        "beta_fuel_lag3",
        "beta_fuel_lag4",
        "beta_fuel_lag5",
        "beta_fuel_lag6",
        "phi_outcome_lag1",
        "gpr_coefficient",
        "constant",
        "se_fuel_lag0",
        "se_fuel_lag1",
        "se_fuel_lag2",
        "se_fuel_lag3",
        "se_fuel_lag4",
        "se_fuel_lag5",
        "se_fuel_lag6",
        "se_outcome_lag1",
    ]
    covariance_columns = [
        "outcome",
        "outcome_label",
        "row_term",
        "column_term",
        "covariance",
        "model",
        "sample_start",
        "sample_end",
        "n",
    ]
    save_csv(pd.DataFrame(kernel_rows, columns=kernel_columns), "q3_policy_macro_kernel.csv")
    save_csv(pd.DataFrame(covariance_rows, columns=covariance_columns), "q3_policy_macro_covariance.csv")
    return pd.DataFrame(rows)


def circular_block_indices(n: int, block_length: int, rng: np.random.Generator) -> np.ndarray:
    indices: list[int] = []
    while len(indices) < n:
        start = int(rng.integers(0, n))
        indices.extend((start + offset) % n for offset in range(block_length))
    return np.asarray(indices[:n], dtype=int)


def policy_macro_model_frames() -> dict[str, tuple[pd.DataFrame, list[str]]]:
    official = pd.read_csv(PROCESSED_DIR / "china_regulated_gasoline_monthly.csv")
    macro = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
    frame = macro.merge(
        official[["period", "china_regulated_gasoline_cny_per_ton"]],
        on="period",
        how="left",
    ).sort_values("period")
    frame["fuel_log"] = log_positive(frame["china_regulated_gasoline_cny_per_ton"])
    frame["fuel_log_return"] = frame["fuel_log"].diff()
    frame = add_lags(frame, "fuel_log_return", 6)
    model_frames: dict[str, tuple[pd.DataFrame, list[str]]] = {}
    for outcome in ["china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"]:
        frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
        regressors = [f"fuel_log_return_lag{lag}" for lag in range(7)] + [f"{outcome}_lag1", "GPR"]
        usable = frame.dropna(subset=[outcome] + regressors).copy()
```

```python
        model_frames[outcome] = (usable[["period", outcome] + regressors], regressors)
    return model_frames


def joint_block_bootstrap_macro_draws(warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    """Use the same circular time blocks for the PPI, CPI and IAV equations."""
    model_frames = policy_macro_model_frames()
    common_periods = sorted(
        set.intersection(*(set(frame["period"].astype(str)) for frame, _ in model_frames.values()))
    )
    if len(common_periods) < 48:
        raise ValueError("Q3 joint policy bootstrap has fewer than 48 common monthly observations.")
    aligned: dict[str, tuple[pd.DataFrame, list[str]]] = {}
    for outcome, (frame, regressors) in model_frames.items():
        aligned_frame = frame.loc[frame["period"].isin(common_periods)].set_index("period").loc[common_periods].reset_index()
        aligned[outcome] = (aligned_frame, regressors)
    rng = np.random.default_rng(RANDOM_SEED + 31)
    rows: list[dict[str, Any]] = []
    accepted = 0
    attempts = 0
    max_attempts = int(POLICY_BOOTSTRAP_REPS * 1.20)
    while accepted < POLICY_BOOTSTRAP_REPS and attempts < max_attempts:
        attempts += 1
        sampled = circular_block_indices(len(common_periods), POLICY_BOOTSTRAP_BLOCK, rng)
        draw_rows: list[dict[str, Any]] = []
        valid = True
        for outcome, (frame, regressors) in aligned.items():
            sample = frame.iloc[sampled]
            x = sm.add_constant(sample[regressors].astype(float), has_constant="add")
            try:
                fit = sm.OLS(sample[outcome].astype(float), x).fit()
            except (ValueError, np.linalg.LinAlgError):
                valid = False
                break
            phi = float(fit.params[f"{outcome}_lag1"])
            betas = [float(fit.params[f"fuel_log_return_lag{lag}"]) for lag in range(7)]
            if not np.isfinite([*betas, phi]).all() or abs(phi) >= 0.995:
                valid = False
                break
            draw_rows.append(
                {
                    "draw": accepted,
                    "outcome": outcome,
                    **{f"beta_fuel_lag{lag}": betas[lag] for lag in range(7)},
                    "phi_outcome_lag1": phi,
                    "common_sample_start": common_periods[0],
                    "common_sample_end": common_periods[-1],
                    "common_n": len(common_periods),
                    "block_length": POLICY_BOOTSTRAP_BLOCK,
                }
            )
        if valid:
            rows.extend(draw_rows)
            accepted += 1
    if accepted != POLICY_BOOTSTRAP_REPS:
        raise ValueError(f"Q3 joint policy bootstrap accepted {accepted}/{POLICY_BOOTSTRAP_REPS} required draws.")
    warnings_log.append(
        {
            "code": "q3_policy_joint_bootstrap",
            "message": f"Accepted {accepted} joint three-equation moving-block draws from {attempts} attempts.",
        }
    )
    result = pd.DataFrame(rows)
    save_csv(result, "q3_policy_macro_bootstrap_draws.csv")
    return result


def dynamic_macro_gap(fuel_return_gap: np.ndarray, beta: np.ndarray, phi: float) -> np.ndarray:
    result = np.zeros(len(fuel_return_gap), dtype=float)
    for t in range(len(fuel_return_gap)):
        distributed = 0.0
        for lag in range(7):
            if t - lag >= 0:
                distributed += float(beta[lag]) * float(fuel_return_gap[t - lag])
        result[t] = distributed + (float(phi) * result[t - 1] if t > 0 else 0.0)
    return result
```

```python


def validate_dynamic_macro_identity() -> None:
    fuel = np.array([0.10, 0.0, 0.0, 0.0], dtype=float)
    beta = np.array([1.0, 2.0, 3.0, 4.0, 0.0, 0.0, 0.0], dtype=float)
    phi = 0.5
    observed = dynamic_macro_gap(fuel, beta, phi)
    expected = np.array([0.10, 0.25, 0.425, 0.6125], dtype=float)
    if not np.allclose(observed, expected, atol=1e-12, rtol=1e-12):
        raise AssertionError("Q3 distributed-lag policy identity test failed.")


def policy_counterfactual(country_panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    official_path = PROCESSED_DIR / "china_regulated_gasoline_monthly.csv"
    if not official_path.exists():
        raise FileNotFoundError("china_regulated_gasoline_monthly.csv is required for the regulated-price proxy counterfactual.")
    official = pd.read_csv(official_path)
    policy = pd.read_csv(PROCESSED_DIR / "china_fuel_policy_monthly.csv")
    full_frame = official.merge(
        policy[["period", "gasoline_policy_gap_cny_t", "diesel_policy_gap_cny_t", "cum_gasoline_policy_gap_cny_t", "
      cum_diesel_policy_gap_cny_t"]],
        on="period",
        how="left",
        suffixes=("", "_policy"),
    )
    full_frame = full_frame.sort_values("period").copy()
    elasticities = china_fuel_to_macro_elasticities(warnings_log)
    validate_dynamic_macro_identity()
    bootstrap_draws = joint_block_bootstrap_macro_draws(warnings_log)
    full_frame["actual_log"] = log_positive(full_frame["china_regulated_gasoline_cny_per_ton"])
    full_frame["no_control_log"] = log_positive(full_frame["no_temporary_control_gasoline_cny_per_ton"])
    full_frame["actual_fuel_log_return"] = full_frame["actual_log"].diff()
    full_frame["no_control_fuel_log_return"] = full_frame["no_control_log"].diff()
    full_frame["fuel_return_gap"] = full_frame["no_control_fuel_log_return"] - full_frame["actual_fuel_log_return"]
    full_frame["fuel_return_gap"] = full_frame["fuel_return_gap"].fillna(0.0)
    frame = full_frame.loc[full_frame["period"].between("2026-02", "2026-06")].copy()
    frame["policy_adjusted_official_cny_l"] = frame.get("china_regulated_gasoline_cny_per_l", pd.Series(np.nan, index=frame.index))
    frame["no_temporary_control_official_cny_l"] = frame.get("no_temporary_control_gasoline_cny_per_l", pd.Series(np.nan, index=frame.
      index))
    frame["policy_adjusted_official_cny_t"] = frame["china_regulated_gasoline_cny_per_ton"]
    frame["no_temporary_control_official_cny_t"] = frame["no_temporary_control_gasoline_cny_per_ton"]
    frame["incremental_gasoline_gap_cny_t"] = frame["gasoline_policy_gap_cny_t"]
    frame["cumulative_gasoline_gap_cny_t"] = frame["cum_gasoline_policy_gap_cny_t"]
    frame["actual"] = frame["policy_adjusted_official_cny_t"]
    frame["prediction"] = frame["no_temporary_control_official_cny_t"]
    frame["response"] = frame["cumulative_gasoline_gap_cny_t"]
    frame["fuel_log_gap"] = np.log(frame["prediction"] / frame["actual"])
    frame["fuel_return_gap"] = full_frame.loc[frame.index, "fuel_return_gap"].to_numpy()
    macro_rows: list[dict[str, Any]] = []
    event_gap = frame["fuel_return_gap"].to_numpy(dtype=float)
    kernel = pd.read_csv(RESULTS_DIR / "q3_policy_macro_kernel.csv")
    for elastic in elasticities.to_dict("records"):
        outcome = str(elastic["outcome"])
        kernel_row = kernel.loc[kernel["outcome"].eq(outcome)].iloc[0]
        beta = np.array([float(kernel_row[f"beta_fuel_lag{lag}"]) for lag in range(7)], dtype=float)
        phi = float(kernel_row["phi_outcome_lag1"])
        point_path = dynamic_macro_gap(event_gap, beta, phi)
        outcome_draws = bootstrap_draws.loc[bootstrap_draws["outcome"].eq(outcome)].sort_values("draw")
        draw_paths = np.vstack(
            [
                dynamic_macro_gap(
                    event_gap,
                    np.array([float(row[f"beta_fuel_lag{lag}"]) for lag in range(7)], dtype=float),
                    float(row["phi_outcome_lag1"]),
                )
                for row in outcome_draws.to_dict("records")
            ]
        )
        for t, period_row in enumerate(frame.to_dict("records")):
            gap = float(point_path[t])
            lower, upper = np.quantile(draw_paths[:, t], [0.025, 0.975])
            macro_rows.append(
                {
                    "period": period_row["period"],
                    "outcome": elastic["outcome"],
                    "outcome_label": elastic["outcome_label"],
```

```
                    "policy_adjusted_official_cny_t": period_row["policy_adjusted_official_cny_t"],
                    "no_temporary_control_official_cny_t": period_row["no_temporary_control_official_cny_t"],
                    "incremental_gasoline_gap_cny_t": period_row["incremental_gasoline_gap_cny_t"],
                    "cumulative_gasoline_gap_cny_t": period_row["cumulative_gasoline_gap_cny_t"],
                    "fuel_log_gap": period_row["fuel_log_gap"],
                    "fuel_return_gap": period_row["fuel_return_gap"],
                    "macro_counterfactual_gap_pctpt": gap,
                    "lower_95": float(lower),
                    "upper_95": float(upper),
                    "model": "China_proxy_fuel_dynamic_ARDL_counterfactual",
                    "horizon": t,
                    "specification": "monthly convolution of no-control minus actual proxy fuel returns through lags 0..6 with recursive outcome AR(1); joint three-equation circular block bootstrap",
                    "sample_start": elastic["sample_start"],
                    "sample_end": elastic["sample_end"],
                    "n": elastic["n"],
                    "price_layer_status": "regulated_gasoline_adjustment_index_proxy_with_2026_notice_gaps",
                    "evidence_status": "CONDITIONAL_DYNAMIC_PROXY_SCENARIO",
                    "bootstrap_reps": POLICY_BOOTSTRAP_REPS,
                    "bootstrap_block_length": POLICY_BOOTSTRAP_BLOCK,
                }
            )
    macro_columns = [
        "period",
        "outcome",
        "outcome_label",
        "policy_adjusted_official_cny_t",
        "no_temporary_control_official_cny_t",
        "incremental_gasoline_gap_cny_t",
        "cumulative_gasoline_gap_cny_t",
        "fuel_log_gap",
        "fuel_return_gap",
        "macro_counterfactual_gap_pctpt",
        "lower_95",
        "upper_95",
        "model",
        "horizon",
        "specification",
        "sample_start",
        "sample_end",
        "n",
        "price_layer_status",
        "evidence_status",
        "bootstrap_reps",
        "bootstrap_block_length",
    ]
    macro_counterfactual = pd.DataFrame(macro_rows, columns=macro_columns)
    save_csv(macro_counterfactual, "q3_policy_macro_counterfactual.csv")
    cpi_macro = macro_counterfactual.loc[macro_counterfactual["outcome"].eq("china_cpi_yoy_pct"), ["period", "macro_counterfactual_gap_pctpt", "lower_95", "upper_95"]]
    frame = frame.drop(columns=[column for column in ["lower_95", "upper_95"] if column in frame.columns])
    frame = frame.merge(cpi_macro.rename(columns={"macro_counterfactual_gap_pctpt": "cpi_counterfactual_gap_pctpt"}), on="period", how="left")
    frame["model"] = "China_policy_counterfactual"
    frame["horizon"] = np.arange(len(frame), dtype=int)
    frame["specification"] = "add separately audited 2026 NDRC policy gaps to the reconstructed regulated-gasoline adjustment-index proxy ; dynamic macro propagation is written to q3_policy_macro_counterfactual.csv"
    frame["evidence_status"] = np.where(frame["policy_adjusted_official_cny_t"].notna(), "CONDITIONAL_PROXY_PRICE_LAYER", "CONDITIONAL_DATA_COVERAGE")
    frame["price_layer_status"] = "regulated_gasoline_adjustment_index_proxy_with_2026_notice_gaps"
    frame["sample_start"] = "2013-03"
    frame["sample_end"] = "2026-06"
    result = frame[
        [
            "period",
            "actual",
            "prediction",
            "response",
            "policy_adjusted_official_cny_l",
            "no_temporary_control_official_cny_l",
            "policy_adjusted_official_cny_t",
            "no_temporary_control_official_cny_t",
            "incremental_gasoline_gap_cny_t",
            "cumulative_gasoline_gap_cny_t",
            "fuel_log_gap",
            "fuel_return_gap",
```

```
            "cpi_counterfactual_gap_pctpt",
            "lower_95",
            "upper_95",
            "gasoline_policy_gap_cny_t",
            "diesel_policy_gap_cny_t",
            "cum_gasoline_policy_gap_cny_t",
            "model",
            "horizon",
            "specification",
            "evidence_status",
            "price_layer_status",
            "sample_start",
            "sample_end",
        ]
    ]
    save_csv(result, "q3_policy_counterfactual.csv")
    return result


def classify_better(diff: float, lower: float, upper: float, better_when_positive: bool = True) -> str:
    if not np.isfinite(diff):
        return "INCONCLUSIVE"
    point_better = diff > 0 if better_when_positive else diff < 0
    interval_better = lower > 0 if better_when_positive else upper < 0
    if interval_better:
        return "SUPPORTED"
    if point_better:
        return "PARTIAL"
    return "NOT_SUPPORTED"


def build_resilience_metrics(pass_through: pd.DataFrame, panel_irf: pd.DataFrame, macro_counterfactual: pd.DataFrame) -> pd.DataFrame:
    """Report exactly three economic dimensions and fail closed on invalid joint intervals."""
    rows: list[dict[str, Any]] = []
    fuel_h6 = pass_through.loc[pass_through["horizon"].eq(6)].copy()
    china = fuel_h6.loc[fuel_h6["country"].eq("CHN")]
    controls = fuel_h6.loc[fuel_h6["country"].isin(MAIN_CONTROL_COUNTRIES)]
    rows.append(
        {
            "dimension": "fuel_price",
            "metric": "six_month_pass_through_proxy_sensitivity",
            "horizon": 6,
            "china_value": float(china["response"].iloc[0]) if not china.empty else np.nan,
            "control_median": float(controls["response"].median()) if not controls.empty else np.nan,
            "china_vs_control_median_diff": np.nan,
            "diff_lower_95": np.nan,
            "diff_upper_95": np.nan,
            "judgement": "CONDITIONAL_PROXY",
            "interpretation": "China is reported as an adjustment-index proxy sensitivity and is excluded from the formal retail-price ranking; no invalid median interval is constructed.",
        }
    )
    for outcome, dimension in [("cpi", "consumer_prices"), ("ip", "industrial_activity")]:
        sub = panel_irf.loc[panel_irf["outcome"].eq(outcome)].copy()
        if sub.empty:
            continue
        grouped = sub.groupby("horizon", as_index=False).agg(median_relative_response=("response", "median"))
        row = grouped.loc[grouped["median_relative_response"].abs().idxmax()]
        rows.append(
            {
                "dimension": dimension,
                "metric": f"control_minus_china_{outcome}_relative_response",
                "horizon": int(row["horizon"]),
                "china_value": np.nan,
                "control_median": float(row["median_relative_response"]),
                "china_vs_control_median_diff": float(row["median_relative_response"]),
                "diff_lower_95": np.nan,
                "diff_upper_95": np.nan,
                "judgement": "INCONCLUSIVE",
                "interpretation": "Only the control-minus-China relative point estimate is shown; China is not set to zero and no confidence claim is made without a joint country-time bootstrap.",
            }
        )
    if not macro_counterfactual.empty:
        for outcome, group in macro_counterfactual.groupby("outcome"):
            max_gap = group.loc[group["macro_counterfactual_gap_pctpt"].abs().idxmax()]
```

```
            rows.append(
                {
                    "dimension": "policy_counterfactual_macro",
                    "metric": outcome,
                    "horizon": int(max_gap.get("horizon", 6)),
                    "china_value": float(max_gap["macro_counterfactual_gap_pctpt"]),
                    "control_median": np.nan,
                    "china_vs_control_median_diff": np.nan,
                    "diff_lower_95": float(max_gap["lower_95"]),
                    "diff_upper_95": float(max_gap["upper_95"]),
                    "judgement": "POLICY_SCENARIO",
                    "interpretation": "macro gap if 2026 temporary fuel-price controls are mechanically removed from the official regulated price path",
                }
            )
    result = pd.DataFrame(rows)
    overall = "INCONCLUSIVE"
    if not result.empty:
        result["overall_china_resilience_judgement"] = overall
    save_csv(result, "q3_resilience_metrics.csv")
    return result


def robustness_checks(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    main_panel = panel.loc[bool_series(panel["included_in_main_comparison"])].copy() if "included_in_main_comparison" in panel else panel.copy()
    main_countries = [country for country in COUNTRY_ORDER if country in set(main_panel["country"])]
    warnings_log.append(
        {
            "code": "q3_fuel_panel_leave_one_withheld",
            "message": "Fuel panel leave-one-country inference is withheld because China is a proxy and excluded from the formal price-level panel.",
        }
    )

    monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_q1.csv")
    usd_panel = main_panel.merge(monthly[["period", "brent_usd_bbl_log_return"]], on="period", how="left")
    for country in main_countries:
        frame = usd_panel.loc[usd_panel["country"].eq(country)].sort_values("period").copy()
        frame = add_lags(frame, "brent_usd_bbl_log_return", 6)
        frame["fuel_lag1"] = frame["fuel_log_return"].shift(1)
        regressors = [f"brent_usd_bbl_log_return_lag{lag}" for lag in range(7)] + ["fuel_lag1"]
        usable = frame.dropna(subset=["fuel_log_return"] + regressors)
        if len(usable) < 48:
            continue
        fit = sm.OLS(
            usable["fuel_log_return"],
            sm.add_constant(usable[regressors].astype(float), has_constant="add"),
        ).fit(cov_type="HAC", cov_kwds={"maxlags": 6})
        lag_terms = [f"brent_usd_bbl_log_return_lag{lag}" for lag in range(7)]
        estimate = float(fit.params[lag_terms].sum())
        rows.append(
            {
                "robustness_type": "usd_brent_fx_ignored_pass_through_h6",
                "omitted_country": "",
                "country": country,
                "horizon": 6,
                "estimate": estimate,
                "std_error": np.nan,
                "model": "distributed_lag_pass_through",
                "specification": "uses USD Brent instead of local-currency Brent",
                "n": int(len(usable)),
            }
        )

    policy = pd.read_csv(PROCESSED_DIR / "china_fuel_policy_monthly.csv")
    policy = policy.loc[policy["period"].between("2026-02", "2026-06")].copy()
    for _, row in policy.iterrows():
        rows.append(
            {
                "robustness_type": "diesel_policy_layer_gap",
                "omitted_country": "",
                "country": "CHN",
                "horizon": 0,
                "estimate": float(row["cum_diesel_policy_gap_cny_t"]),
```

```
                "std_error": np.nan,
                "model": "policy_price_layer",
                "specification": f"cumulative diesel policy gap through {row['period']}",
                "n": 1,
            }
        )
    result = pd.DataFrame(rows)
    save_csv(result, "q3_robustness.csv")
    return result


def plot_pass_through(pass_through: pd.DataFrame) -> None:
    if pass_through.empty:
        return
    frame = pass_through.loc[pass_through["horizon"].eq(6)].copy()
    if "included_in_main_comparison" in frame:
        frame = frame.loc[bool_series(frame["included_in_main_comparison"])].copy()
    frame["country_label"] = frame["country"].map(COUNTRY_LABEL_ZH).fillna(frame["country"])
    frame["err_low"] = frame["response"] - frame["lower_95"]
    frame["err_high"] = frame["upper_95"] - frame["response"]
    color_cycle = [PALETTE["gold"], PALETTE["olive"], PALETTE["rose"], PALETTE["slate"], PALETTE["sand"], PALETTE["blue_light"]]
    colors = [color_cycle[idx % len(color_cycle)] for idx in range(len(frame))]
    fig, ax = plt.subplots(figsize=(7.6, 4.8))
    ax.bar(
        frame["country_label"],
        frame["response"],
        color=colors,
        edgecolor="white",
        linewidth=0.8,
        width=0.62,
        yerr=[frame["err_low"], frame["err_high"]],
        error_kw={"ecolor": PALETTE["ink"], "elinewidth": 0.8, "capsize": 3},
    )
    ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
    style_axis(ax, ylabel="累计传导系数")
    finish_figure(
        fig,
        title="问题三：六个月燃油价格传导率",
        subtitle="仅纳入覆盖充分的可观测或官方受管制零售汽油价格；中国覆盖不足时不参与主排名。",
        source="来源：欧盟周度油价公报、日本METI、韩国KOSIS/KNOC、北京市发改委与 FRED；由 code/problem3/run_q3.py 生成。",
        rect=(0.10, 0.13, 0.98, 0.84),
    )
    save_figure(fig, FIGURES_DIR / "q3_pass_through_6m")
    plt.close(fig)


def plot_panel_irf(panel_irf: pd.DataFrame) -> None:
    if panel_irf.empty:
        return
    frame = panel_irf.loc[panel_irf["outcome"].eq("fuel")].copy()
    if frame.empty:
        frame = panel_irf.copy()
    fig, ax = plt.subplots(figsize=(8.8, 5.1))
    style_map = {
        "CHN": (PALETTE["blue"], "solid", "o"),
        "DEU": (PALETTE["gold"], (0, (4, 2)), "s"),
        "FRA": (PALETTE["olive"], (0, (2, 2)), "^"),
        "ITA": (PALETTE["rose"], (0, (1, 2)), "D"),
        "ESP": (PALETTE["slate"], (0, (6, 2)), "v"),
        "JPN": (PALETTE["sand"], (0, (3, 1, 1, 1)), "P"),
        "KOR": (PALETTE["blue_light"], (0, (1, 1)), "X"),
    }
    for country in COUNTRY_ORDER:
        sub = frame.loc[frame["country"].eq(country)].sort_values("horizon")
        if sub.empty:
            continue
        color, linestyle, marker = style_map.get(country, (PALETTE["muted"], "solid", "o"))
        ax.plot(sub["horizon"], sub["response"], color=color, linestyle=linestyle, marker=marker, label=COUNTRY_LABEL_ZH.get(country,
      country))
    ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
    style_axis(ax, xlabel="油价冲击后月份", ylabel="响应")
    ax.legend(loc="upper left", ncol=4, handlelength=2.6)
    finish_figure(
        fig,
        title="问题三：跨国面板 LP 响应",
        subtitle="完整年月固定效应吸收共同冲击，因此曲线表示国家相对响应差异；燃油主图仅纳入覆盖充分国家。",
```

```
        source="来源：model_country_monthly.csv 与 q1_monthly_shocks.csv；由 code/problem3/run_q3.py 生成。",
    )
    save_figure(fig, FIGURES_DIR / "q3_panel_irf")
    plt.close(fig)


def plot_policy(counterfactual: pd.DataFrame) -> None:
    if counterfactual.empty:
        return
    frame = counterfactual.copy()
    x = np.arange(len(frame))
    fig, ax = plt.subplots(figsize=(7.8, 4.8))
    ax.plot(x, frame["actual"], marker="o", color=PALETTE["blue"], label="政策调整后调价指数代理")
    ax.plot(x, frame["prediction"], marker="s", color=PALETTE["gold"], linestyle=(0, (4, 2)), label="无临时调控规则路径")
    ax.fill_between(x, frame["actual"], frame["prediction"], color=PALETTE["gold_light"], alpha=0.26, linewidth=0)
    ax.set_xticks(x)
    ax.set_xticklabels(frame["period"])
    for tick, row in enumerate(frame.itertuples(index=False)):
        if getattr(row, "cumulative_gasoline_gap_cny_t") > 0:
            label = f"累计差额 {row.cumulative_gasoline_gap_cny_t:.0f}"
            if getattr(row, "incremental_gasoline_gap_cny_t") > 0 and row.period == "2026-04":
                label = f"累计差额 {row.cumulative_gasoline_gap_cny_t:.0f}\n4月新增 {row.incremental_gasoline_gap_cny_t:.0f}"
            ax.text(tick, max(row.actual, row.prediction), label, ha="center", va="bottom", fontsize=8.4, color=PALETTE["muted"])
    style_axis(ax, ylabel="元/吨")
    ax.legend(loc="lower center", bbox_to_anchor=(0.5, 1.01), ncol=2, handlelength=2.6)
    finish_figure(
        fig,
        title="问题三：中国成品油调控政策情景",
        subtitle="阴影代表将单独核验的2026政策差额加回历史调价指数代理；宏观传播为条件动态情景。",
        source="来源：国家发展改革委2026调价公告、公开历史调价镜像及处理说明；由 code/problem3/run_q3.py 生成。",
        rect=(0.10, 0.13, 0.98, 0.94),
    )
    save_figure(fig, FIGURES_DIR / "q3_policy_counterfactual")
    plt.close(fig)


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--plots-only", action="store_true", help="Regenerate Q3 figures from saved result tables.")
    args = parser.parse_args(argv)

    np.random.seed(RANDOM_SEED)
    apply_paper_style()
    ensure_dirs()
    if args.plots_only:
        pass_through = read_csv_result("q3_country_pass_through.csv")
        panel_irf = read_csv_result("q3_panel_irf.csv")
        counterfactual = read_csv_result("q3_policy_counterfactual.csv")
        plot_pass_through(pass_through)
        plot_panel_irf(panel_irf)
        plot_policy(counterfactual)
        print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 3}, ensure_ascii=False, indent=2))
        return 0

    from linearmodels.panel.utility import AbsorbingEffectWarning

    warnings_log: list[dict[str, Any]] = []
    panel = pd.read_csv(PROCESSED_DIR / "model_country_monthly.csv")
    panel["fuel_log"] = log_positive(panel["fuel_price_local"])
    py_warnings.filterwarnings("ignore", category=AbsorbingEffectWarning)
    pass_through = fit_country_pass_through(panel, warnings_log)
    panel_irf = fit_panel_lp(panel, warnings_log)
    buffer_irf = fit_buffer_interactions(panel, warnings_log)
    counterfactual = policy_counterfactual(panel, warnings_log)
    macro_counterfactual = read_csv_result("q3_policy_macro_counterfactual.csv")
    resilience = build_resilience_metrics(pass_through, panel_irf, macro_counterfactual)
    robustness = robustness_checks(panel, warnings_log)
    plot_pass_through(pass_through)
    plot_panel_irf(panel_irf)
    plot_policy(counterfactual)

    summary = pd.DataFrame(
        [
            {
                "component": "CountryPassThrough",
                "rows": len(pass_through),
```

```
                "status": "PASS" if len(pass_through) else "WARN",
                "note": "formal ranking covers six comparable control countries; China adjustment-index proxy is retained only as sensitivity",
            },
            {
                "component": "PanelLP",
                "rows": len(panel_irf),
                "status": "PASS" if len(panel_irf) else "WARN",
                "note": "CPI and industrial-activity LP coefficients are control-country minus China relative responses with country and full year-month fixed effects",
            },
            {
                "component": "BufferInteractions",
                "rows": len(buffer_irf),
                "status": "WITHHELD_UNAUDITED_PROXY",
                "note": "continuous buffer interactions are not released until the country-year source table is independently auditable",
            },
            {
                "component": "ChinaPolicyCounterfactual",
                "rows": len(counterfactual),
                "status": "PASS" if len(counterfactual) and len(macro_counterfactual) else "WARN",
                "note": "2026 notice gaps are propagated through a dynamic ARDL kernel on a reconstructed adjustment-index proxy with joint time-block uncertainty",
            },
            {
                "component": "ResilienceMetrics",
                "rows": len(resilience),
                "status": "PASS" if len(resilience) else "WARN",
                "note": "exactly three economic dimensions; overall judgement fails closed because the fuel layer is a proxy and joint cross-country intervals are not claimed",
            },
            {
                "component": "Robustness",
                "rows": len(robustness),
                "status": "PASS" if len(robustness) else "WARN",
                "note": "USD-Brent sensitivity and diesel policy-layer checks; invalid fuel panel leave-one inference is withheld",
            },
        ]
    )
    save_csv(summary, "q3_summary.csv")
    payload = {
        "status": "CONDITIONAL",
        "execution_status": "PASS" if len(pass_through) and len(panel_irf) and len(counterfactual) else "WARN",
        "evidence_status": str(resilience["overall_china_resilience_judgement"].dropna().iloc[0]) if "overall_china_resilience_judgement" in resilience and not resilience["overall_china_resilience_judgement"].dropna().empty else "INCONCLUSIVE",
        "allowed_claims": [
            "China historical fuel pass-through is a proxy sensitivity and is excluded from the formal retail-price ranking",
            "Panel LP coefficients are relative to China under full year-month fixed effects",
            "The 2026 conditional policy scenario propagates proxy fuel-return gaps dynamically through PPI/CPI/IAV kernels with a common time-block bootstrap",
        ],
        "forbidden_claims": [
            "China is better solely because Brent-CNY proxy pass-through is lower",
            "fuel price smoothing is welfare-improving without considering cumulative policy gap costs",
            "unaudited structural-buffer proxies support quantitative policy-interaction effects",
        ],
        "random_seed": RANDOM_SEED,
        "comparability_guardrail": "China uses a reconstructed regulated-gasoline adjustment-index proxy and is excluded from the formal cross-country retail-price ranking; 2026 policy notices are audited separately.",
        "warnings": warnings_log,
        "rows": {
            "q3_country_pass_through.csv": int(len(pass_through)),
            "q3_panel_irf.csv": int(len(panel_irf)),
            "q3_buffer_interactions.csv": int(len(buffer_irf)),
            "q3_buffer_data_status.csv": 4,
            "q3_policy_counterfactual.csv": int(len(counterfactual)),
            "q3_policy_macro_counterfactual.csv": int(len(macro_counterfactual)),
            "q3_policy_macro_bootstrap_draws.csv": POLICY_BOOTSTRAP_REPS * 3,
            "q3_resilience_metrics.csv": int(len(resilience)),
            "q3_robustness.csv": int(len(robustness)),
        },
    }
    (RESULTS_DIR / "q3_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(payload, ensure_ascii=False, indent=2))
    return 0

```

```

if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 8: run_q3.py

```
"""Extension Q4: Brent upper-tail risk and China macro-policy stress tests."""

from __future__ import annotations

import argparse
import json
import math
import sys
import time
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import minimize


REPO_ROOT = Path(__file__).resolve().parents[2]
CODE_DIR = REPO_ROOT / "code"
if str(CODE_DIR) not in sys.path:
    sys.path.insert(0, str(CODE_DIR))

from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

PROCESSED_DIR = REPO_ROOT / "data" / "processed"
RESULTS_DIR = REPO_ROOT / "results"
FIGURES_DIR = REPO_ROOT / "figures"
CUTOFF = pd.Timestamp("2026-06-30")
RANDOM_SEED = 20260730
HORIZON_STEPS = {1: 22, 3: 66, 6: 132}
FORECAST_QUANTILES = [0.05, 0.10, 0.50, 0.90, 0.95]
OUTCOME_LABELS = {
    "china_ppi_yoy_pct": "PPI",
    "china_cpi_yoy_pct": "CPI",
    "china_iav_yoy_pct": "工业增加值",
}


def ensure_dirs() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)


def save_csv(frame: pd.DataFrame, filename: str) -> Path:
    path = RESULTS_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def write_json(payload: dict[str, Any], filename: str) -> Path:
    path = RESULTS_DIR / filename
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8")
    return path


def json_records(frame: pd.DataFrame) -> list[dict[str, Any]]:
    if frame.empty:
        return []
    return json.loads(frame.where(pd.notna(frame), None).to_json(orient="records"))


```

```python
def load_daily() -> pd.DataFrame:
    path = PROCESSED_DIR / "model_daily_q1.csv"
    frame = pd.read_csv(path, parse_dates=["date"])
    required = {"date", "brent_usd_bbl", "brent_usd_bbl_log_return"}
    missing = required.difference(frame.columns)
    if missing:
        raise ValueError(f"Q4 daily input missing columns: {sorted(missing)}")
    frame = frame.loc[frame["date"].le(CUTOFF)].copy()
    frame["brent_usd_bbl"] = pd.to_numeric(frame["brent_usd_bbl"], errors="coerce")
    frame["return_pct"] = pd.to_numeric(frame["brent_usd_bbl_log_return"], errors="coerce") * 100.0
    frame = frame.dropna(subset=["date", "brent_usd_bbl", "return_pct"]).sort_values("date").reset_index(drop=True)
    if len(frame) < 750:
        raise ValueError(f"Q4 needs at least 750 daily returns, got {len(frame)}")
    if frame["date"].max() != CUTOFF:
        raise ValueError(f"Q4 last daily observation must be {CUTOFF.date()}, got {frame['date'].max().date()}")
    return frame


def _variance_path(eps: np.ndarray, params: np.ndarray, sample_var: float) -> np.ndarray:
    omega, alpha, gamma, beta = params
    h = np.empty(len(eps), dtype=float)
    h[0] = sample_var
    for idx in range(1, len(eps)):
        lag = eps[idx - 1]
        h[idx] = omega + alpha * lag**2 + gamma * (lag < 0) * lag**2 + beta * h[idx - 1]
    return h


def fit_gjr_garch(returns_pct: pd.Series) -> dict[str, Any]:
    values = pd.to_numeric(returns_pct, errors="coerce").dropna().to_numpy(dtype=float)
    if len(values) < 500:
        raise ValueError(f"GJR-GARCH needs at least 500 observations, got {len(values)}")
    mu = float(np.mean(values))
    eps = values - mu
    sample_var = float(np.var(eps, ddof=1))
    if not np.isfinite(sample_var) or sample_var <= 0:
        raise ValueError("GJR-GARCH sample variance is not positive.")

    def neg_loglike(params: np.ndarray) -> float:
        omega, alpha, gamma, beta = params
        persistence = alpha + 0.5 * gamma + beta
        if omega <= 0 or min(alpha, gamma, beta) < 0 or persistence >= 0.999:
            return 1e15
        h = _variance_path(eps, params, sample_var)
        if np.any(~np.isfinite(h)) or np.any(h <= 0):
            return 1e15
        return float(0.5 * np.sum(np.log(2 * np.pi) + np.log(h) + eps**2 / h))

    starts = [
        np.array([0.02 * sample_var, 0.05, 0.05, 0.88]),
        np.array([0.03 * sample_var, 0.08, 0.02, 0.86]),
        np.array([0.05 * sample_var, 0.10, 0.00, 0.80]),
    ]
    constraints = ({"type": "ineq", "fun": lambda p: 0.999 - p[1] - 0.5 * p[2] - p[3]},)
    fits = []
    for start in starts:
        fit = minimize(
            neg_loglike,
            x0=start,
            bounds=[(1e-10, None), (0.0, 1.0), (0.0, 1.0), (0.0, 0.999)],
            constraints=constraints,
            method="SLSQP",
            options={"maxiter": 600, "ftol": 1e-9},
        )
        if fit.success and np.isfinite(fit.fun):
            fits.append(fit)
    if not fits:
        raise RuntimeError("All GJR-GARCH optimization starts failed.")
    fit = min(fits, key=lambda item: float(item.fun))
    params = np.asarray(fit.x, dtype=float)
    h = _variance_path(eps, params, sample_var)
    z = eps / np.sqrt(h)
    z = z[np.isfinite(z)]
    z_std = float(np.std(z, ddof=1))
    if len(z) < 400 or not np.isfinite(z_std) or z_std <= 0:
        raise RuntimeError("GJR-GARCH standardized residuals are unusable.")
```

```
    z = (z - float(np.mean(z))) / z_std
    omega, alpha, gamma, beta = params
    return {
        "mu": mu,
        "omega": float(omega),
        "alpha": float(alpha),
        "gamma": float(gamma),
        "beta": float(beta),
        "persistence": float(alpha + 0.5 * gamma + beta),
        "last_eps": float(eps[-1]),
        "last_h": float(h[-1]),
        "standardized_residuals": z,
        "n": int(len(values)),
        "objective": float(fit.fun),
    }


def simulate_fhs(
    start_price: float,
    fit: dict[str, Any],
    steps: int,
    paths: int,
    seed: int,
) -> np.ndarray:
    rng = np.random.default_rng(seed)
    prices = np.empty((paths, steps), dtype=float)
    log_price = np.full(paths, math.log(start_price), dtype=float)
    h_prev = np.full(paths, float(fit["last_h"]), dtype=float)
    eps_prev = np.full(paths, float(fit["last_eps"]), dtype=float)
    residuals = np.asarray(fit["standardized_residuals"], dtype=float)
    omega = float(fit["omega"])
    alpha = float(fit["alpha"])
    gamma = float(fit["gamma"])
    beta = float(fit["beta"])
    mu = float(fit["mu"])
    for step in range(steps):
        h = omega + alpha * eps_prev**2 + gamma * (eps_prev < 0) * eps_prev**2 + beta * h_prev
        h = np.maximum(h, 1e-10)
        z = rng.choice(residuals, size=paths, replace=True)
        eps = np.sqrt(h) * z
        log_price += (mu + eps) / 100.0
        prices[:, step] = np.exp(log_price)
        h_prev = h
        eps_prev = eps
    return prices


def simulate_gaussian(
    start_price: float,
    returns_pct: pd.Series,
    steps: int,
    paths: int,
    seed: int,
) -> np.ndarray:
    values = pd.to_numeric(returns_pct, errors="coerce").dropna().to_numpy(dtype=float)
    mu = float(np.mean(values))
    sigma = float(np.std(values, ddof=1))
    if not np.isfinite(sigma) or sigma <= 0:
        raise ValueError("Gaussian baseline volatility is not positive.")
    rng = np.random.default_rng(seed)
    draws = rng.normal(mu, sigma, size=(paths, steps)) / 100.0
    return start_price * np.exp(np.cumsum(draws, axis=1))


def historical_thresholds(daily: pd.DataFrame) -> dict[str, float]:
    prices = daily["brent_usd_bbl"].dropna()
    return {
        "historical_price_p90": float(prices.quantile(0.90)),
        "historical_price_p95": float(prices.quantile(0.95)),
    }


def summarize_paths(
    paths: np.ndarray,
    model: str,
    origin_date: pd.Timestamp,
```

```python
    origin_price: float,
    thresholds: dict[str, float],
) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for horizon, steps in HORIZON_STEPS.items():
        terminal = paths[:, steps - 1]
        prefix = paths[:, :steps]
        quantiles = np.quantile(terminal, FORECAST_QUANTILES)
        threshold_90 = thresholds["historical_price_p90"]
        threshold_95 = thresholds["historical_price_p95"]
        upper_tail = terminal[terminal > threshold_95]
        rows.append(
            {
                "origin_date": origin_date.strftime("%Y-%m-%d"),
                "origin_price_usd_bbl": origin_price,
                "horizon_months": horizon,
                "trading_days": steps,
                "model": model,
                "paths": int(len(terminal)),
                "p05_price": float(quantiles[0]),
                "p10_price": float(quantiles[1]),
                "median_price": float(quantiles[2]),
                "p90_price": float(quantiles[3]),
                "p95_price": float(quantiles[4]),
                "historical_price_p90": threshold_90,
                "historical_price_p95": threshold_95,
                "terminal_prob_above_hist_p90": float(np.mean(terminal > threshold_90)),
                "terminal_prob_above_hist_p95": float(np.mean(terminal > threshold_95)),
                "path_prob_cross_hist_p90": float(np.mean(np.max(prefix, axis=1) > threshold_90)),
                "path_prob_cross_hist_p95": float(np.mean(np.max(prefix, axis=1) > threshold_95)),
                "upper_tail_mean_above_hist_p95": float(np.mean(upper_tail)) if len(upper_tail) else np.nan,
                "evidence_status": "PROBABILISTIC_STRESS_FORECAST",
            }
        )
    return pd.DataFrame(rows)


def pinball(actual: float, prediction: float, quantile: float) -> float:
    error = actual - prediction
    return float(max(quantile * error, (quantile - 1.0) * error))


def interval_score(actual: float, lower: float, upper: float, alpha: float) -> float:
    score = upper - lower
    if actual < lower:
        score += (2.0 / alpha) * (lower - actual)
    elif actual > upper:
        score += (2.0 / alpha) * (actual - upper)
    return float(score)


def backtest_origins(daily: pd.DataFrame) -> pd.DataFrame:
    candidates = daily.loc[daily["date"].between(pd.Timestamp("2022-01-01"), pd.Timestamp("2025-12-31"))].copy()
    candidates["month"] = candidates["date"].dt.to_period("M")
    candidates = candidates.groupby("month", as_index=False).tail(1)
    max_step = max(HORIZON_STEPS.values())
    candidates = candidates.loc[candidates.index + max_step < len(daily)].copy()
    return candidates


def run_backtest(
    daily: pd.DataFrame,
    paths: int,
    seed: int,
    max_origins: int | None = None,
) -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
    origins = backtest_origins(daily)
    if max_origins is not None:
        origins = origins.tail(max_origins)
    origin_rows: list[dict[str, Any]] = []
    warnings: list[str] = []
    max_step = max(HORIZON_STEPS.values())
    for origin_number, (idx, origin) in enumerate(origins.iterrows()):
        train = daily.iloc[: idx + 1].copy()
        fit = fit_gjr_garch(train["return_pct"])
        fhs_paths = simulate_fhs(
```

```
            float(origin["brent_usd_bbl"]), fit, max_step, paths, seed + origin_number * 101
        )
        gaussian_paths = simulate_gaussian(
            float(origin["brent_usd_bbl"]), train["return_pct"], max_step, paths, seed + origin_number * 101 + 1
        )
        for model, simulated in [("FHS_GJR_GARCH", fhs_paths), ("Gaussian_random_walk", gaussian_paths)]:
            for horizon, steps in HORIZON_STEPS.items():
                target = daily.iloc[idx + steps]
                actual = float(target["brent_usd_bbl"])
                terminal = simulated[:, steps - 1]
                q = np.quantile(terminal, FORECAST_QUANTILES)
                losses = [pinball(actual, float(pred), tau) for pred, tau in zip(q, FORECAST_QUANTILES)]
                origin_rows.append(
                    {
                        "origin_date": origin["date"].strftime("%Y-%m-%d"),
                        "target_date": target["date"].strftime("%Y-%m-%d"),
                        "horizon_months": horizon,
                        "model": model,
                        "actual_price": actual,
                        "p05_price": float(q[0]),
                        "p10_price": float(q[1]),
                        "median_price": float(q[2]),
                        "p90_price": float(q[3]),
                        "p95_price": float(q[4]),
                        "mean_pinball_loss": float(np.mean(losses)),
                        "median_absolute_error": float(abs(actual - q[2])),
                        "covered_80": bool(q[1] <= actual <= q[3]),
                        "covered_90": bool(q[0] <= actual <= q[4]),
                        "interval_width_80": float(q[3] - q[1]),
                        "interval_width_90": float(q[4] - q[0]),
                        "interval_score_80": interval_score(actual, float(q[1]), float(q[3]), 0.20),
                        "interval_score_90": interval_score(actual, float(q[0]), float(q[4]), 0.10),
                        "quantile_score_approx_crps": float(2.0 * np.mean(losses)),
                        "train_end": origin["date"].strftime("%Y-%m-%d"),
                        "no_future_information": bool(target["date"] > origin["date"]),
                    }
                )
    detailed = pd.DataFrame(origin_rows)
    if detailed.empty:
        return detailed, pd.DataFrame(), warnings
    summary = (
        detailed.groupby(["model", "horizon_months"], as_index=False)
        .agg(
            origins=("origin_date", "count"),
            mean_pinball_loss=("mean_pinball_loss", "mean"),
            median_absolute_error=("median_absolute_error", "median"),
            coverage_80=("covered_80", "mean"),
            coverage_90=("covered_90", "mean"),
            mean_width_80=("interval_width_80", "mean"),
            mean_width_90=("interval_width_90", "mean"),
            mean_interval_score_80=("interval_score_80", "mean"),
            mean_interval_score_90=("interval_score_90", "mean"),
            mean_quantile_score_approx_crps=("quantile_score_approx_crps", "mean"),
            all_no_future_information=("no_future_information", "all"),
        )
        .sort_values(["horizon_months", "model"])
        .reset_index(drop=True)
    )
    summary["model_role"] = np.where(summary["model"].eq("FHS_GJR_GARCH"), "main", "baseline")
    summary["coverage_error_80"] = (summary["coverage_80"] - 0.80).abs()
    summary["coverage_error_90"] = (summary["coverage_90"] - 0.90).abs()
    return detailed, summary, warnings


def paired_backtest_summary(detailed: pd.DataFrame, seed: int, block_length: int = 6, reps: int = 2000) -> pd.DataFrame:
    if detailed.empty:
        return pd.DataFrame()
    rng = np.random.default_rng(seed + 909)
    rows: list[dict[str, Any]] = []
    metrics = ["mean_pinball_loss", "interval_score_80", "interval_score_90", "quantile_score_approx_crps"]
    for horizon, group in detailed.groupby("horizon_months"):
        for metric in metrics:
            pivot = group.pivot(index="origin_date", columns="model", values=metric).dropna()
            if not {"FHS_GJR_GARCH", "Gaussian_random_walk"}.issubset(pivot.columns) or len(pivot) < 12:
                continue
            diff = (pivot["FHS_GJR_GARCH"] - pivot["Gaussian_random_walk"]).to_numpy(dtype=float)
```

```
            draws: list[float] = []
            for _ in range(reps):
                indices: list[int] = []
                while len(indices) < len(diff):
                    start = int(rng.integers(0, len(diff)))
                    indices.extend((start + offset) % len(diff) for offset in range(block_length))
                draws.append(float(np.mean(diff[np.asarray(indices[: len(diff)], dtype=int)])))
            lower, upper = np.quantile(draws, [0.025, 0.975])
            rows.append(
                {
                    "horizon_months": int(horizon),
                    "metric": metric,
                    "origins": int(len(diff)),
                    "block_length": block_length,
                    "bootstrap_reps": reps,
                    "mean_difference_fhs_minus_gaussian": float(np.mean(diff)),
                    "lower_95": float(lower),
                    "upper_95": float(upper),
                    "evidence_status": "FHS_BETTER" if upper < 0 else ("GAUSSIAN_BETTER" if lower > 0 else "INCONCLUSIVE"),
                }
            )
    return pd.DataFrame(rows)


def build_macro_stress() -> tuple[pd.DataFrame, dict[str, float]]:
    shocks = pd.read_csv(RESULTS_DIR / "q1_structural_shocks.csv")
    values = pd.to_numeric(shocks["oil_specific_risk_shock"], errors="coerce").dropna()
    if len(values) < 120:
        raise ValueError(f"Q4 needs at least 120 oil-specific risk shocks, got {len(values)}")
    scenario_values = {
        "moderate_q75": float(values.quantile(0.75)),
        "severe_q90": float(values.quantile(0.90)),
        "extreme_q95": float(values.quantile(0.95)),
    }
    irf = pd.read_csv(RESULTS_DIR / "q2_irf.csv")
    irf = irf.loc[
        irf["shock"].eq("oil_specific_risk_shock")
        & irf["outcome"].isin(OUTCOME_LABELS)
        & pd.to_numeric(irf["horizon"], errors="coerce").isin([0, 3, 6, 12])
    ].copy()
    required = {"response", "joint_lower_95", "joint_upper_95", "sample_start", "sample_end"}
    missing = required.difference(irf.columns)
    if missing or irf.empty:
        raise ValueError(f"Q4 macro stress missing Q2 IRF fields: {sorted(missing)}")
    rows: list[dict[str, Any]] = []
    for scenario, shock_value in scenario_values.items():
        for row in irf.itertuples(index=False):
            response = float(row.response) * shock_value
            lower = float(row.joint_lower_95) * shock_value
            upper = float(row.joint_upper_95) * shock_value
            rows.append(
                {
                    "scenario": scenario,
                    "shock_quantile_value": shock_value,
                    "shock": "oil_specific_risk_shock",
                    "outcome": row.outcome,
                    "outcome_label": OUTCOME_LABELS[row.outcome],
                    "horizon": int(row.horizon),
                    "conditional_response_pctpt": response,
                    "joint_lower_95": min(lower, upper),
                    "joint_upper_95": max(lower, upper),
                    "joint_ci95_contains_zero": bool(min(lower, upper) <= 0 <= max(lower, upper)),
                    "row_evidence_status": (
                        "INCONCLUSIVE" if min(lower, upper) <= 0 <= max(lower, upper) else "SUPPORTED_JOINT_95"
                    ),
                    "source_q2_evidence_status": "INCONCLUSIVE",
                    "sample_start": row.sample_start,
                    "sample_end": row.sample_end,
                    "specification": "historical positive structural-shock quantile multiplied by Q2 LP response; conditional scenario,
      not deterministic loss",
                }
            )
    return pd.DataFrame(rows), scenario_values


def build_policy_stress() -> pd.DataFrame:
```

```
    source = pd.read_csv(RESULTS_DIR / "q3_policy_macro_counterfactual.csv")
    source = source.loc[source["outcome"].isin(OUTCOME_LABELS)].copy()
    required = {"period", "outcome", "macro_counterfactual_gap_pctpt", "lower_95", "upper_95"}
    missing = required.difference(source.columns)
    if missing or source.empty:
        raise ValueError(f"Q4 policy stress missing Q3 fields: {sorted(missing)}")
    rows: list[dict[str, Any]] = []
    for row in source.itertuples(index=False):
        gap = float(row.macro_counterfactual_gap_pctpt)
        lower = float(row.lower_95)
        upper = float(row.upper_95)
        if row.outcome == "china_iav_yoy_pct":
            benefit = -gap
            benefit_lower = -upper
            benefit_upper = -lower
            interpretation = "actual policy avoided an industrial-activity loss relative to no temporary control"
        else:
            benefit = gap
            benefit_lower = lower
            benefit_upper = upper
            interpretation = "actual policy avoided a price-index increase relative to no temporary control"
        rows.append(
            {
                "period": row.period,
                "outcome": row.outcome,
                "outcome_label": OUTCOME_LABELS[row.outcome],
                "no_policy_minus_actual_gap_pctpt": gap,
                "policy_buffer_benefit_pctpt": benefit,
                "benefit_lower_95": min(benefit_lower, benefit_upper),
                "benefit_upper_95": max(benefit_lower, benefit_upper),
                "evidence_status": (
                    "SUPPORTED_95"
                    if min(benefit_lower, benefit_upper) > 0
                    else "INCONCLUSIVE"
                ),
                "interpretation": interpretation,
                "price_layer_status": row.price_layer_status,
                "source_evidence_status": row.evidence_status,
                "specification": "Q3 realized 2026 no-temporary-control counterfactual restated as policy buffer benefit; not
      extrapolated to simulated Q4 price paths",
            }
        )
    return pd.DataFrame(rows)


def plot_price_tail_risk(risk: pd.DataFrame) -> None:
    apply_paper_style()
    fig, ax = plt.subplots(figsize=(8.6, 5.0))
    x = np.array([1, 3, 6], dtype=float)
    for model, color, label in [
        ("FHS_GJR_GARCH", PALETTE["blue"], "FHS－GJR-GARCH"),
        ("Gaussian_random_walk", PALETTE["gold"], "高斯随机游走"),
    ]:
        subset = risk.loc[risk["model"].eq(model)].sort_values("horizon_months")
        if subset.empty:
            continue
        xx = subset["horizon_months"].to_numpy(dtype=float)
        median = subset["median_price"].to_numpy(dtype=float)
        ax.plot(xx, median, marker="o", color=color, label=f"{label} 中位数")
        if model == "FHS_GJR_GARCH":
            ax.fill_between(
                xx,
                subset["p05_price"].to_numpy(dtype=float),
                subset["p95_price"].to_numpy(dtype=float),
                color=PALETTE["blue_light"],
                alpha=0.30,
                label="FHS 90%区间",
            )
            ax.fill_between(
                xx,
                subset["p10_price"].to_numpy(dtype=float),
                subset["p90_price"].to_numpy(dtype=float),
                color=PALETTE["blue_light"],
                alpha=0.50,
                label="FHS 80%区间",
            )
```

```python
    threshold_90 = float(risk["historical_price_p90"].iloc[0])
    threshold_95 = float(risk["historical_price_p95"].iloc[0])
    ax.axhline(threshold_90, color=PALETTE["olive"], linestyle="--", linewidth=1.2, label="历史价格90%分位")
    ax.axhline(threshold_95, color=PALETTE["rose"], linestyle=":", linewidth=1.4, label="历史价格95%分位")
    ax.set_xticks(x, ["1个月", "3个月", "6个月"])
    style_axis(ax, ylabel="Brent价格（美元/桶）", xlabel="预测期限")
    ax.legend(ncol=2, loc="upper left")
    finish_figure(
        fig,
        title="国际油价上行尾部风险",
        subtitle="阴影为FHS条件分布区间，虚线为截止日前历史价格阈值。",
        source="数据：EIA/FRED Brent；计算：FHS－GJR-GARCH 与高斯随机游走，信息截止 2026-06-30。",
    )
    save_figure(fig, FIGURES_DIR / "q4_price_tail_risk")
    plt.close(fig)


def plot_macro_policy_stress(macro: pd.DataFrame, policy: pd.DataFrame) -> None:
    apply_paper_style()
    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.8), gridspec_kw={"width_ratios": [1.35, 1.0]})
    left = macro.loc[macro["scenario"].eq("extreme_q95")].copy()
    for outcome, color in [
        ("china_ppi_yoy_pct", PALETTE["gold"]),
        ("china_cpi_yoy_pct", PALETTE["blue"]),
        ("china_iav_yoy_pct", PALETTE["rose"]),
    ]:
        subset = left.loc[left["outcome"].eq(outcome)].sort_values("horizon")
        axes[0].plot(
            subset["horizon"],
            subset["conditional_response_pctpt"],
            marker="o",
            color=color,
            label=OUTCOME_LABELS[outcome],
        )
    axes[0].axhline(0, color=PALETTE["muted"], linewidth=0.8)
    axes[0].set_xticks([0, 3, 6, 12])
    style_axis(axes[0], ylabel="同比增速条件响应（百分点）", xlabel="冲击后月份")
    axes[0].legend(loc="best")

    june = policy.loc[policy["period"].eq("2026-06")].copy()
    order = ["PPI", "CPI", "工业增加值"]
    june["order"] = june["outcome_label"].map({label: idx for idx, label in enumerate(order)})
    june = june.sort_values("order")
    values = june["policy_buffer_benefit_pctpt"].to_numpy(dtype=float)
    lower = june["benefit_lower_95"].to_numpy(dtype=float)
    upper = june["benefit_upper_95"].to_numpy(dtype=float)
    ypos = np.arange(len(june))
    axes[1].barh(ypos, values, color=[PALETTE["gold"], PALETTE["blue"], PALETTE["rose"]])
    axes[1].errorbar(values, ypos, xerr=np.vstack([values - lower, upper - values]), fmt="none", ecolor=PALETTE["ink"], capsize=3)
    axes[1].set_yticks(ypos, june["outcome_label"])
    axes[1].invert_yaxis()
    axes[1].axvline(0, color=PALETTE["muted"], linewidth=0.8)
    style_axis(axes[1], ylabel=None, xlabel="政策缓冲收益（百分点）")
    finish_figure(
        fig,
        title="极端结构冲击的宏观压力与已实现政策缓冲",
        subtitle="左图为95%分位结构冲击条件响应；右图为2026年6月无临时调控相对实际政策路径差额。",
        source="计算：Q1结构冲击分位数、Q2 Local Projection 与 Q3 官方成品油价格层反事实；区间口径见结果表。",
        rect=(0.06, 0.12, 0.99, 0.98),
    )
    save_figure(fig, FIGURES_DIR / "q4_macro_policy_stress")
    plt.close(fig)


def run_probe() -> dict[str, Any]:
    ensure_dirs()
    started = time.perf_counter()
    daily = load_daily()
    fit = fit_gjr_garch(daily["return_pct"])
    thresholds = historical_thresholds(daily)
    max_step = max(HORIZON_STEPS.values())
    start_price = float(daily["brent_usd_bbl"].iloc[-1])
    first = simulate_fhs(start_price, fit, max_step, 500, RANDOM_SEED)
    replay = simulate_fhs(start_price, fit, max_step, 500, RANDOM_SEED)
    alternate = simulate_fhs(start_price, fit, max_step, 500, RANDOM_SEED + 1)
    baseline = simulate_gaussian(start_price, daily["return_pct"], max_step, 500, RANDOM_SEED)
```

```
    first_summary = summarize_paths(first, "FHS_GJR_GARCH", CUTOFF, start_price, thresholds)
    alternate_summary = summarize_paths(alternate, "FHS_GJR_GARCH", CUTOFF, start_price, thresholds)
    baseline_summary = summarize_paths(baseline, "Gaussian_random_walk", CUTOFF, start_price, thresholds)
    _, backtest, backtest_warnings = run_backtest(daily, paths=300, seed=RANDOM_SEED, max_origins=3)
    macro, scenario_values = build_macro_stress()
    policy = build_policy_stress()
    prob_columns = ["terminal_prob_above_hist_p90", "terminal_prob_above_hist_p95"]
    seed_sensitivity = float(
        np.max(
            np.abs(
                first_summary[prob_columns].to_numpy(dtype=float)
                - alternate_summary[prob_columns].to_numpy(dtype=float)
            )
        )
    )
    checks = {
        "daily_rows_at_least_750": len(daily) >= 750,
        "cutoff_exact": daily["date"].max() == CUTOFF,
        "gjr_persistence_below_one": 0 <= fit["persistence"] < 0.999,
        "fhs_distribution_non_degenerate": bool((first_summary["p95_price"] > first_summary["p05_price"]).all()),
        "baseline_distribution_non_degenerate": bool((baseline_summary["p95_price"] > baseline_summary["p05_price"]).all()),
        "fixed_seed_exact_replay": bool(np.array_equal(first, replay)),
        "seed_sensitivity_below_0_15": seed_sensitivity <= 0.15,
        "representative_backtest_complete": len(backtest) == 6 and bool(backtest["all_no_future_information"].all()),
        "macro_three_outcomes_present": set(macro["outcome"]) == set(OUTCOME_LABELS),
        "macro_three_scenarios_present": len(scenario_values) == 3,
        "policy_three_outcomes_present": set(policy["outcome"]) == set(OUTCOME_LABELS),
    }
    status = "PASS" if all(checks.values()) else "FAIL"
    payload = {
        "status": status,
        "method": "FHS_GJR_GARCH",
        "role": "main",
        "baseline": "Gaussian_random_walk",
        "data_range": [daily["date"].min().strftime("%Y-%m-%d"), daily["date"].max().strftime("%Y-%m-%d")],
        "random_seed": RANDOM_SEED,
        "probe_paths": 500,
        "representative_backtest_origins": 3,
        "checks": checks,
        "metrics": {
            "daily_rows": int(len(daily)),
            "gjr_persistence": float(fit["persistence"]),
            "seed_probability_max_abs_difference": seed_sensitivity,
            "runtime_seconds": float(time.perf_counter() - started),
        },
        "warnings": backtest_warnings,
        "pass_rule": "all input, stationarity, non-degeneracy, replay, seed-perturbation, backtest and cross-module field checks pass",
        "failure_action": "stop the Q4 gate and fix the input/model contract; do not substitute fallback estimates",
    }
    write_json(payload, "q4_risk_probe.json")
    print(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False))
    return payload


def run_full(paths: int, backtest_paths: int) -> dict[str, Any]:
    ensure_dirs()
    probe_path = RESULTS_DIR / "q4_risk_probe.json"
    if not probe_path.exists():
        raise FileNotFoundError("Run `python code/problem4/run_q4.py --probe` before the full Q4 implementation.")
    probe = json.loads(probe_path.read_text(encoding="utf-8"))
    if probe.get("status") != "PASS":
        raise RuntimeError("Q4 risk probe did not pass; full implementation is blocked.")

    daily = load_daily()
    thresholds = historical_thresholds(daily)
    start_price = float(daily["brent_usd_bbl"].iloc[-1])
    fit = fit_gjr_garch(daily["return_pct"])
    max_step = max(HORIZON_STEPS.values())
    fhs_paths = simulate_fhs(start_price, fit, max_step, paths, RANDOM_SEED)
    gaussian_paths = simulate_gaussian(start_price, daily["return_pct"], max_step, paths, RANDOM_SEED + 1)
    risk = pd.concat(
        [
            summarize_paths(fhs_paths, "FHS_GJR_GARCH", CUTOFF, start_price, thresholds),
            summarize_paths(gaussian_paths, "Gaussian_random_walk", CUTOFF, start_price, thresholds),
        ],
        ignore_index=True,
```

```
    )
    detailed_backtest, backtest, backtest_warnings = run_backtest(
        daily, paths=backtest_paths, seed=RANDOM_SEED
    )
    paired_backtest = paired_backtest_summary(detailed_backtest, RANDOM_SEED, block_length=6)
    macro, scenario_values = build_macro_stress()
    policy = build_policy_stress()

    save_csv(risk, "q4_price_tail_risk.csv")
    save_csv(detailed_backtest, "q4_risk_backtest_origins.csv")
    save_csv(backtest, "q4_risk_backtest.csv")
    save_csv(paired_backtest, "q4_risk_backtest_paired.csv")
    save_csv(macro, "q4_macro_stress.csv")
    save_csv(policy, "q4_policy_stress.csv")
    plot_price_tail_risk(risk)
    plot_macro_policy_stress(macro, policy)

    overall_pinball = (
        backtest.groupby("model", as_index=False)["mean_pinball_loss"].mean().sort_values("mean_pinball_loss")
        if not backtest.empty
        else pd.DataFrame()
    )
    preferred = str(overall_pinball.iloc[0]["model"]) if not overall_pinball.empty else None
    fhs_key = risk.loc[risk["model"].eq("FHS_GJR_GARCH")].copy()
    june_policy = policy.loc[policy["period"].eq("2026-06")].copy()
    extreme = macro.loc[macro["scenario"].eq("extreme_q95")].copy()
    summary = {
        "status": "PASS",
        "execution_status": "PASS",
        "evidence_status": "CONDITIONAL_STRESS_TEST",
        "problem_status": "SELF_DEFINED_PROBLEM_4_RISK_LAYER",
        "random_seed": RANDOM_SEED,
        "cutoff": CUTOFF.strftime("%Y-%m-%d"),
        "main_method": "FHS_GJR_GARCH",
        "baseline": "Gaussian_random_walk",
        "backtest_preferred_model_by_mean_pinball": preferred,
        "allowed_claims": [
            "future Brent upper-tail probabilities are conditional stress forecasts under the fixed cutoff",
            "Q2 structural-shock quantiles map to conditional PPI/CPI/IAV response ranges",
            "Q3 dynamic proxy scenario conditionally propagates the separately audited 2026 temporary-control gaps",
        ],
        "forbidden_claims": [
            "tail probabilities are certain price outcomes",
            "Q4 macro stress values are deterministic GDP losses",
            "Q2 structural responses and Q3 policy counterfactual gaps can be added as one causal estimate",
            "the stress test proves China is generally optimal or costless",
        ],
        "guardrail": "Price risk, structural-shock macro scenarios and realized policy counterfactuals are reported as three linked but
      non-additive evidence layers.",
        "gjr_parameters": {
            key: float(fit[key]) for key in ["mu", "omega", "alpha", "gamma", "beta", "persistence"]
        },
        "structural_scenarios": scenario_values,
        "rows": {
            "q4_price_tail_risk.csv": int(len(risk)),
            "q4_risk_backtest_origins.csv": int(len(detailed_backtest)),
            "q4_risk_backtest.csv": int(len(backtest)),
            "q4_risk_backtest_paired.csv": int(len(paired_backtest)),
            "q4_macro_stress.csv": int(len(macro)),
            "q4_policy_stress.csv": int(len(policy)),
        },
        "key_price_risk": json_records(fhs_key),
        "key_extreme_macro_rows": json_records(
            extreme.loc[extreme["horizon"].isin([6, 12])].sort_values(["outcome", "horizon"])
        ),
        "key_2026_06_policy_buffer": json_records(june_policy.sort_values("outcome")),
        "backtest_summary": json_records(backtest),
        "paired_backtest_summary": json_records(paired_backtest),
        "warnings": backtest_warnings,
    }
    write_json(summary, "q4_summary.json")
    print(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False))
    return summary


def parse_args() -> argparse.Namespace:
```

```
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--probe", action="store_true", help="run only the pre-registered low-cost risk probe")
    parser.add_argument("--paths", type=int, default=20000, help="Monte Carlo paths for the cutoff forecast")
    parser.add_argument("--backtest-paths", type=int, default=1500, help="paths per rolling backtest origin")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.probe:
        payload = run_probe()
        return 0 if payload["status"] == "PASS" else 1
    run_full(paths=args.paths, backtest_paths=args.backtest_paths)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 9: run_q4.py

```
"""Question 4: SAPR-CVaR adaptive fuel-price smoothing rule.

The model treats the official mechanism-implied adjustment as the primary
pricing signal and optimizes only a transparent temporary smoothing layer.
"""

from __future__ import annotations

import argparse
import json
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


REPO_ROOT = Path(__file__).resolve().parents[2]
CODE_DIR = REPO_ROOT / "code"
if str(CODE_DIR) not in sys.path:
    sys.path.insert(0, str(CODE_DIR))

from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

PROCESSED_DIR = REPO_ROOT / "data" / "processed"
RESULTS_DIR = REPO_ROOT / "results"
FIGURES_DIR = REPO_ROOT / "figures"

RANDOM_SEED = 20260730
CUTOFF = "2026-06-30"
SCENARIO_LENGTH = 6
BOOTSTRAP_DRAWS = 2000
DEVELOPMENT_END = "2021-12"
HOLDOUT_START = "2022-01"
TEMPORARY_2026_RHO = (1160.0 + 420.0) / (2205.0 + 800.0)
RECOVERY_MONTHS = 6
MAX_GAP_RATIO = 0.35
MAX_TERMINAL_GAP_RATIO = 0.20
MAX_RECOVERY_TERMINAL_GAP_RATIO = 0.05
MAX_MONTHLY_ADJUSTMENT_RATIO = 0.25

OUTCOMES = ["china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"]
OUTCOME_LABELS = {
    "china_ppi_yoy_pct": "PPI",
```

```
    "china_cpi_yoy_pct": "CPI",
    "china_iav_yoy_pct": "IAV",
}
REGIME_ORDER = ["normal", "stress", "extreme"]


@dataclass(frozen=True)
class Scenario:
    scenario_id: str
    sample_split: str
    source_start: str
    periods: tuple[str, ...]
    mechanism_adjustment: np.ndarray
    actual_adjustment: np.ndarray
    cumulative_policy_gap: np.ndarray
    regimes: tuple[str, ...]
    is_2026_anchor: bool


@dataclass(frozen=True)
class MacroKernel:
    means: dict[str, np.ndarray]
    covariances: dict[str, np.ndarray]
    labels: dict[str, str]
    valid_outcomes: tuple[str, ...]


def ensure_dirs() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)


def save_csv(frame: pd.DataFrame, filename: str) -> Path:
    path = RESULTS_DIR / filename
    frame.to_csv(path, index=False, encoding="utf-8")
    return path


def read_result(filename: str) -> pd.DataFrame:
    path = RESULTS_DIR / filename
    if not path.exists():
        raise FileNotFoundError(f"Required result file is missing: {path}")
    return pd.read_csv(path)


def read_processed(filename: str) -> pd.DataFrame:
    path = PROCESSED_DIR / filename
    if not path.exists():
        raise FileNotFoundError(f"Required processed file is missing: {path}")
    return pd.read_csv(path)


def clean_value(value: Any) -> Any:
    if isinstance(value, (np.integer,)):
        return int(value)
    if isinstance(value, (np.floating, float)):
        return None if not np.isfinite(value) else round(float(value), 8)
    if pd.isna(value):
        return None
    return value


def stress_regime(value: float, q75: float, q95: float) -> str:
    if not np.isfinite(value) or value <= q75:
        return "normal"
    if value <= q95:
        return "stress"
    return "extreme"


def build_policy_months(threshold_quantiles: tuple[float, float] = (0.75, 0.95)) -> tuple[pd.DataFrame, dict[str, float]]:
    official = read_processed("china_regulated_gasoline_monthly.csv")
    events = read_processed("cn_fuel_adjustment_events.csv")
    macro = read_processed("model_monthly_cn.csv")
    shocks = read_result("q1_monthly_shocks.csv")

```

```
    official = official[
        [
            "period",
            "china_regulated_gasoline_cny_per_ton",
            "source_url",
            "coverage_status",
            "measure_type",
        ]
    ].copy()
    events = events.copy()
    events["period"] = pd.to_datetime(events["effective_date"]).dt.to_period("M").astype(str)
    event_monthly = (
        events.groupby("period", as_index=False)
        .agg(
            gasoline_actual_adjustment_cny_t=("actual_adjustment_cny_per_ton", "sum"),
            gasoline_rule_adjustment_cny_t=("rule_implied_adjustment_cny_per_ton", "sum"),
            event_source_count=("source_url", "count"),
        )
        .sort_values("period")
    )
    event_monthly["gasoline_policy_gap_cny_t"] = (
        event_monthly["gasoline_rule_adjustment_cny_t"] - event_monthly["gasoline_actual_adjustment_cny_t"]
    )
    event_monthly["cum_gasoline_policy_gap_cny_t"] = event_monthly["gasoline_policy_gap_cny_t"].cumsum()

    frame = (
        official.merge(event_monthly, on="period", how="left")
        .merge(
            macro[["period", "brent_usd_bbl", "brent_usd_bbl_log_return", "GPR", "GPR_z"]],
            on="period",
            how="left",
        )
        .merge(
            shocks[
                [
                    "period",
                    "supply_shock",
                    "aggregate_demand_shock",
                    "oil_specific_risk_shock",
                    "reduced_form_shock",
                ]
            ],
            on="period",
            how="left",
        )
        .sort_values("period")
    )
    frame = frame.loc[frame["period"].between("2013-03", "2026-06")].copy()
    frame = frame.dropna(subset=["china_regulated_gasoline_cny_per_ton"]).copy()
    if len(frame) < 48:
        raise ValueError("Q4 requires at least 48 months of the reconstructed regulated-gasoline adjustment proxy.")
    for column in [
        "gasoline_actual_adjustment_cny_t",
        "gasoline_rule_adjustment_cny_t",
        "gasoline_policy_gap_cny_t",
        "cum_gasoline_policy_gap_cny_t",
        "event_source_count",
    ]:
        frame[column] = frame[column].fillna(0.0)

    frame["previous_official_price_cny_t"] = frame["china_regulated_gasoline_cny_per_ton"].shift(1)
    fallback_previous = frame["china_regulated_gasoline_cny_per_ton"] - frame["gasoline_actual_adjustment_cny_t"].fillna(0.0)
    frame["previous_official_price_cny_t"] = frame["previous_official_price_cny_t"].fillna(fallback_previous)
    bad_previous = frame["previous_official_price_cny_t"].le(0) | frame["previous_official_price_cny_t"].isna()
    if bool(bad_previous.any()):
        raise ValueError("Q4 cannot compute mechanism adjustment rates because a previous official fuel price is missing or nonpositive."
    )

    base_price = float(frame.loc[frame["period"].eq("2026-06"), "china_regulated_gasoline_cny_per_ton"].iloc[0])
    frame["mechanism_adjustment_cny_t_original"] = frame["gasoline_rule_adjustment_cny_t"].fillna(0.0)
    frame["actual_adjustment_cny_t_original"] = frame["gasoline_actual_adjustment_cny_t"].fillna(0.0)
    frame["mechanism_adjustment_rate"] = frame["mechanism_adjustment_cny_t_original"] / frame["previous_official_price_cny_t"]
    frame["actual_adjustment_rate"] = frame["actual_adjustment_cny_t_original"] / frame["previous_official_price_cny_t"]
    frame["mechanism_adjustment_cny_t"] = frame["mechanism_adjustment_rate"] * base_price
    frame["actual_adjustment_cny_t"] = frame["actual_adjustment_rate"] * base_price
    frame["cumulative_policy_gap_cny_t_original"] = frame["cum_gasoline_policy_gap_cny_t"].fillna(0.0)
```

```python
    frame["source_vintage"] = "downloaded_2026-07-31_ex_post"

    development = frame.loc[frame["period"].le(DEVELOPMENT_END)].copy()
    positive = pd.to_numeric(development["mechanism_adjustment_cny_t"], errors="coerce")
    positive = positive.loc[positive.gt(0)].dropna()
    if len(positive) < 12:
        raise ValueError("Q4 cannot define stress thresholds because development positive adjustments are too sparse.")
    q75 = float(positive.quantile(threshold_quantiles[0]))
    q95 = float(positive.quantile(threshold_quantiles[1]))
    frame["stress_regime"] = frame["mechanism_adjustment_cny_t"].map(lambda value: stress_regime(float(value), q75, q95))

    meta = {
        "base_price_2026_06_cny_t": base_price,
        "stress_threshold_75_cny_t": q75,
        "stress_threshold_95_cny_t": q95,
        "threshold_quantile_low": threshold_quantiles[0],
        "threshold_quantile_high": threshold_quantiles[1],
    }
    return frame, meta


def build_scenarios(threshold_quantiles: tuple[float, float] = (0.75, 0.95)) -> tuple[pd.DataFrame, list[Scenario], dict[str, float]]:
    frame, meta = build_policy_months(threshold_quantiles)
    scenario_rows: list[dict[str, Any]] = []
    scenarios: list[Scenario] = []
    for start in range(0, len(frame) - SCENARIO_LENGTH + 1):
        window = frame.iloc[start : start + SCENARIO_LENGTH].copy()
        source_start = str(window["period"].iloc[0])
        source_end = str(window["period"].iloc[-1])
        if source_end <= DEVELOPMENT_END:
            split = "development"
        elif source_start >= HOLDOUT_START and source_end <= "2026-06":
            split = "holdout"
        else:
            continue
        scenario_id = f"{split.upper()}_{source_start}"
        is_2026_anchor = source_start == "2026-01" and source_end == "2026-06"
        scenario = Scenario(
            scenario_id=scenario_id,
            sample_split=split,
            source_start=source_start,
            periods=tuple(window["period"].astype(str).tolist()),
            mechanism_adjustment=window["mechanism_adjustment_cny_t"].astype(float).to_numpy(),
            actual_adjustment=window["actual_adjustment_cny_t"].astype(float).to_numpy(),
            cumulative_policy_gap=window["cumulative_policy_gap_cny_t_original"].astype(float).to_numpy(),
            regimes=tuple(window["stress_regime"].astype(str).tolist()),
            is_2026_anchor=is_2026_anchor,
        )
        scenarios.append(scenario)
        for month_index, (_, row) in enumerate(window.iterrows()):
            scenario_rows.append(
                {
                    "scenario_id": scenario_id,
                    "sample_split": split,
                    "month_index": month_index,
                    "period": row["period"],
                    "source_start": source_start,
                    "source_end": source_end,
                    "brent_return": row["brent_usd_bbl_log_return"],
                    "brent_level": row["brent_usd_bbl"],
                    "mechanism_adjustment_rate": row["mechanism_adjustment_rate"],
                    "mechanism_adjustment_cny_t": row["mechanism_adjustment_cny_t"],
                    "mechanism_adjustment_cny_t_original": row["mechanism_adjustment_cny_t_original"],
                    "actual_adjustment_cny_t_original": row["actual_adjustment_cny_t_original"],
                    "cumulative_policy_gap_cny_t_original": row["cumulative_policy_gap_cny_t_original"],
                    "stress_regime": row["stress_regime"],
                    "is_2026_anchor": is_2026_anchor,
                    "GPR": row["GPR"],
                    "GPR_z": row["GPR_z"],
                    "supply_shock": row["supply_shock"],
                    "aggregate_demand_shock": row["aggregate_demand_shock"],
                    "oil_specific_risk_shock": row["oil_specific_risk_shock"],
                    "reduced_form_shock": row["reduced_form_shock"],
                    "source_vintage": row["source_vintage"],
                }
            )
```

```
    scenario_table = pd.DataFrame(scenario_rows)
    return scenario_table, scenarios, meta


def validated_covariance(matrix: np.ndarray, outcome: str) -> np.ndarray:
    if not np.isfinite(matrix).all():
        raise ValueError(f"Q3 covariance for {outcome} contains non-finite values.")
    if not np.allclose(matrix, matrix.T, atol=1e-10, rtol=1e-10):
        raise ValueError(f"Q3 covariance for {outcome} is not symmetric.")
    eigvals = np.linalg.eigvalsh(matrix)
    if float(eigvals.min()) < -1e-10:
        raise ValueError(f"Q3 covariance for {outcome} is not positive semidefinite.")
    return (matrix + matrix.T) / 2.0


def load_macro_kernel() -> MacroKernel:
    kernel = read_result("q3_policy_macro_kernel.csv")
    covariance = read_result("q3_policy_macro_covariance.csv")
    required_kernel = {"outcome", "outcome_label", "n", "phi_outcome_lag1"} | {f"beta_fuel_lag{lag}" for lag in range(7)}
    required_cov = {"outcome", "row_term", "column_term", "covariance"}
    if not required_kernel.issubset(kernel.columns) or not required_cov.issubset(covariance.columns):
        raise ValueError("Q3 macro kernel or covariance file lacks required columns for Q4.")
    means: dict[str, np.ndarray] = {}
    covariances: dict[str, np.ndarray] = {}
    labels: dict[str, str] = {}
    for outcome in OUTCOMES:
        rows = kernel.loc[kernel["outcome"].eq(outcome)]
        if len(rows) != 1:
            raise ValueError(f"Q4 requires exactly one macro-kernel row for {outcome}.")
        row = rows.iloc[0]
        terms = [f"fuel_log_return_lag{lag}" for lag in range(7)] + [f"{outcome}_lag1"]
        means[outcome] = np.array([float(row[f"beta_fuel_lag{lag}"]) for lag in range(7)] + [float(row["phi_outcome_lag1"])])
        if not np.isfinite(means[outcome]).all():
            raise ValueError(f"Q3 macro kernel for {outcome} contains non-finite coefficients.")
        labels[outcome] = str(row["outcome_label"])
        cov_block = covariance.loc[covariance["outcome"].eq(outcome)].copy()
        required_pairs = {(row_term, col_term) for row_term in terms for col_term in terms}
        observed_pairs = set(zip(cov_block["row_term"].astype(str), cov_block["column_term"].astype(str)))
        missing_pairs = required_pairs - observed_pairs
        if missing_pairs:
            raise ValueError(f"Q3 covariance for {outcome} is missing {len(missing_pairs)} required term pairs.")
        matrix = np.zeros((len(terms), len(terms)), dtype=float)
        for i, row_term in enumerate(terms):
            for j, col_term in enumerate(terms):
                hit = cov_block.loc[cov_block["row_term"].eq(row_term) & cov_block["column_term"].eq(col_term), "covariance"]
                if len(hit) != 1:
                    raise ValueError(f"Q3 covariance for {outcome} has duplicate or missing value for {row_term}, {col_term}.")
                matrix[i, j] = float(hit.iloc[0])
        covariances[outcome] = validated_covariance(matrix, outcome)
    valid = tuple(outcome for outcome in OUTCOMES if outcome in means and outcome in covariances)
    if set(valid) != set(OUTCOMES):
        raise ValueError("Q4 requires PPI, CPI and IAV macro kernels from Q3.")
    return MacroKernel(means=means, covariances=covariances, labels=labels, valid_outcomes=valid)


def draw_parameter_sets(
    kernel: MacroKernel,
    rng: np.random.Generator,
    target_draws: int = BOOTSTRAP_DRAWS,
    minimum_valid_rate: float = 0.95,
) -> tuple[list[dict[str, np.ndarray]], float, int]:
    joint_path = RESULTS_DIR / "q3_policy_macro_bootstrap_draws.csv"
    if joint_path.exists():
        joint = pd.read_csv(joint_path)
        required = {"draw", "outcome", "phi_outcome_lag1"} | {f"beta_fuel_lag{lag}" for lag in range(7)}
        if not required.issubset(joint.columns):
            raise ValueError("Q3 joint macro bootstrap file lacks required Q4 kernel columns.")
        available_draws = sorted(pd.to_numeric(joint["draw"], errors="coerce").dropna().astype(int).unique())
        if len(available_draws) < target_draws:
            raise ValueError(f"Q4 requires {target_draws} joint Q3 draws but found {len(available_draws)}.")
        draws: list[dict[str, np.ndarray]] = []
        for draw_id in available_draws[:target_draws]:
            block = joint.loc[pd.to_numeric(joint["draw"], errors="coerce").eq(draw_id)]
            draw: dict[str, np.ndarray] = {}
            for outcome in kernel.valid_outcomes:
                row = block.loc[block["outcome"].eq(outcome)]
```

```
                if len(row) != 1:
                    raise ValueError(f"Q3 joint draw {draw_id} lacks exactly one row for {outcome}.")
                values = np.array(
                    [float(row.iloc[0][f"beta_fuel_lag{lag}"]) for lag in range(7)]
                    + [float(row.iloc[0]["phi_outcome_lag1"])],
                    dtype=float,
                )
                if not np.isfinite(values).all() or abs(float(values[-1])) >= 1.0:
                    raise ValueError(f"Q3 joint draw {draw_id} is invalid for {outcome}.")
                draw[outcome] = values
            draws.append(draw)
        return draws, 1.0, len(draws)
    draws: list[dict[str, np.ndarray]] = []
    max_attempts = int(np.floor(target_draws / minimum_valid_rate))
    attempts_used = 0
    while len(draws) < target_draws and attempts_used < max_attempts:
        attempts_used += 1
        draw: dict[str, np.ndarray] = {}
        valid = True
        for outcome in kernel.valid_outcomes:
            mean = kernel.means[outcome]
            covariance = kernel.covariances[outcome]
            phi = float(mean[-1])
            if abs(phi) >= 1.0:
                raise ValueError(f"Q4 macro kernel for {outcome} has an unstable point-estimate lag coefficient.")
            transformed_mean = mean.copy()
            transformed_mean[-1] = np.arctanh(phi)
            jacobian = np.eye(len(mean), dtype=float)
            jacobian[-1, -1] = 1.0 / (1.0 - phi**2)
            transformed_covariance = validated_covariance(jacobian @ covariance @ jacobian.T, f"{outcome}_stationary_phi")
            transformed_sample = rng.multivariate_normal(transformed_mean, transformed_covariance, method="svd")
            sample = transformed_sample.copy()
            sample[-1] = np.tanh(transformed_sample[-1])
            if not np.isfinite(sample).all() or abs(float(sample[-1])) >= 1.0:
                valid = False
                break
            draw[outcome] = sample
        if valid:
            draws.append(draw)
    valid_rate = len(draws) / float(attempts_used) if attempts_used else 0.0
    return draws, valid_rate, attempts_used


def point_coefficients(kernel: MacroKernel) -> dict[str, np.ndarray]:
    return {outcome: values.copy() for outcome, values in kernel.means.items()}


def historical_macro_stds(weights: dict[str, float]) -> dict[str, float]:
    macro = read_processed("model_monthly_cn.csv")
    frame = macro.loc[macro["period"].between("2013-03", DEVELOPMENT_END)].copy()
    stds: dict[str, float] = {}
    for outcome in OUTCOMES:
        std = float(pd.to_numeric(frame[outcome], errors="coerce").std(skipna=True))
        stds[outcome] = std if np.isfinite(std) and std > 1e-9 else 1.0
    return stds


def enumerate_rules() -> list[dict[str, float]]:
    values = [round(i * 0.05, 2) for i in range(21)]
    rules: list[dict[str, float]] = []
    for rho_normal in values:
        for rho_stress in values:
            for rho_extreme in values:
                if rho_normal >= rho_stress >= rho_extreme:
                    rules.append(
                        {
                            "rho_normal": rho_normal,
                            "rho_stress": rho_stress,
                            "rho_extreme": rho_extreme,
                        }
                    )
    return rules


def rho_for_regime(rule: dict[str, float], regime: str) -> float:
    if regime == "extreme":
```

```python
        return float(rule["rho_extreme"])
    if regime == "stress":
        return float(rule["rho_stress"])
    return float(rule["rho_normal"])


def simulate_rule_path(scenario: Scenario, rule: dict[str, float], base_price: float) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    price = np.empty(SCENARIO_LENGTH, dtype=float)
    actual_adjustment = np.empty(SCENARIO_LENGTH, dtype=float)
    deferred_gap = np.empty(SCENARIO_LENGTH, dtype=float)
    fuel_return = np.empty(SCENARIO_LENGTH, dtype=float)
    previous_price = base_price
    gap = 0.0
    for t in range(SCENARIO_LENGTH):
        desired = float(scenario.mechanism_adjustment[t]) + gap
        if desired > 0:
            adjustment = rho_for_regime(rule, scenario.regimes[t]) * desired
            gap = desired - adjustment
            gap_cap = MAX_TERMINAL_GAP_RATIO * base_price
            if gap > gap_cap:
                catch_up = gap - gap_cap
                adjustment += catch_up
                gap = gap_cap
        else:
            adjustment = desired
            gap = 0.0
        current_price = max(previous_price + adjustment, 1.0)
        actual_adjustment[t] = adjustment
        deferred_gap[t] = gap
        price[t] = current_price
        fuel_return[t] = np.log(current_price / previous_price)
        previous_price = current_price
    return price, actual_adjustment, deferred_gap, fuel_return


def simulate_actual_2026_path(scenario: Scenario, base_price: float) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    price = np.empty(SCENARIO_LENGTH, dtype=float)
    actual_adjustment = np.empty(SCENARIO_LENGTH, dtype=float)
    deferred_gap = np.empty(SCENARIO_LENGTH, dtype=float)
    fuel_return = np.empty(SCENARIO_LENGTH, dtype=float)
    previous_price = base_price
    for t in range(SCENARIO_LENGTH):
        adjustment = float(scenario.actual_adjustment[t])
        current_price = max(previous_price + adjustment, 1.0)
        actual_adjustment[t] = adjustment
        deferred_gap[t] = float(scenario.cumulative_policy_gap[t])
        price[t] = current_price
        fuel_return[t] = np.log(current_price / previous_price)
        previous_price = current_price
    return price, actual_adjustment, deferred_gap, fuel_return


def recovery_terminal_gap(terminal_gap: float, rule: dict[str, float]) -> float:
    gap = max(float(terminal_gap), 0.0)
    rho = float(rule["rho_normal"])
    for _ in range(RECOVERY_MONTHS):
        gap = (1.0 - rho) * gap
    return gap


def macro_path_from_returns(fuel_return: np.ndarray, coefficients: dict[str, np.ndarray]) -> dict[str, np.ndarray]:
    paths = {outcome: np.zeros(SCENARIO_LENGTH, dtype=float) for outcome in OUTCOMES}
    lag_buffer = [0.0] * 7
    previous_y = {outcome: 0.0 for outcome in OUTCOMES}
    for t in range(SCENARIO_LENGTH):
        lag_buffer = [float(fuel_return[t])] + lag_buffer[:6]
        for outcome in OUTCOMES:
            params = coefficients[outcome]
            y = float(np.dot(params[:7], np.array(lag_buffer)) + params[7] * previous_y[outcome])
            paths[outcome][t] = y
            previous_y[outcome] = y
    return paths


def objective_from_scenario_outputs(
```

```
    scenario_losses: list[float],
    gap_burdens: list[float],
    fuel_returns: list[float],
) -> dict[str, float]:
    losses = np.asarray(scenario_losses, dtype=float)
    gaps = np.asarray(gap_burdens, dtype=float)
    returns = np.asarray(fuel_returns, dtype=float)
    if len(losses) == 0:
        raise ValueError("Q4 objective cannot be computed from an empty scenario set.")
    if not np.isfinite(losses).all() or not np.isfinite(gaps).all() or not np.isfinite(returns).all():
        raise ValueError("Q4 objective inputs contain non-finite values.")
    tail_count = max(1, int(np.ceil(0.05 * len(losses))))
    tail = np.sort(losses)[-tail_count:]
    return {
        "J1_macro_loss": float(np.mean(losses)),
        "J2_cvar95_macro_loss": float(np.mean(tail)),
        "J3_avg_gap_month_burden": float(np.mean(gaps)),
        "J4_adjustment_volatility": float(np.std(returns, ddof=0)) if len(returns) else 0.0,
    }


def evaluate_strategy(
    scenarios: list[Scenario],
    coefficients: dict[str, np.ndarray],
    rule: dict[str, float] | None,
    strategy_name: str,
    split: str,
    base_price: float,
    stds: dict[str, float],
    weights: dict[str, float],
    scenario_indices: np.ndarray | None = None,
) -> dict[str, float]:
    candidates = [scenario for scenario in scenarios if scenario.sample_split == split]
    if split == "war_2026":
        candidates = [scenario for scenario in scenarios if scenario.is_2026_anchor]
    if not candidates:
        raise ValueError(f"Q4 strategy evaluation has no candidate scenarios for split={split}.")
    if scenario_indices is not None and len(candidates):
        candidates = [candidates[int(index) % len(candidates)] for index in scenario_indices]
    losses: list[float] = []
    gap_burdens: list[float] = []
    returns_all: list[float] = []
    max_gap_ratios: list[float] = []
    terminal_gap_ratios: list[float] = []
    recovery_gap_ratios: list[float] = []
    max_adjustment_ratios: list[float] = []
    for scenario in candidates:
        if strategy_name == "actual_2026_event_path":
            price, adjustment, gap, fuel_return = simulate_actual_2026_path(scenario, base_price)
        else:
            if rule is None:
                raise ValueError("A SAPR rule is required for simulated strategies.")
            price, adjustment, gap, fuel_return = simulate_rule_path(scenario, rule, base_price)
        macro_paths = macro_path_from_returns(fuel_return, coefficients)
        month_loss = np.zeros(SCENARIO_LENGTH, dtype=float)
        month_loss += weights["china_ppi_yoy_pct"] * np.maximum(macro_paths["china_ppi_yoy_pct"], 0.0) / stds["china_ppi_yoy_pct"]
        month_loss += weights["china_cpi_yoy_pct"] * np.maximum(macro_paths["china_cpi_yoy_pct"], 0.0) / stds["china_cpi_yoy_pct"]
        month_loss += weights["china_iav_yoy_pct"] * np.maximum(-macro_paths["china_iav_yoy_pct"], 0.0) / stds["china_iav_yoy_pct"]
        losses.append(float(np.mean(month_loss)))
        gap_burdens.append(float(np.mean(np.maximum(gap, 0.0)) / base_price))
        max_gap_ratios.append(float(np.max(np.maximum(gap, 0.0)) / base_price))
        terminal_gap_ratios.append(float(max(float(gap[-1]), 0.0) / base_price))
        recovery_gap_ratios.append(
            float(recovery_terminal_gap(float(gap[-1]), rule) / base_price) if rule is not None else float(max(float(gap[-1]), 0.0) /
      base_price)
        )
        max_adjustment_ratios.append(float(np.max(np.abs(adjustment)) / base_price))
        returns_all.extend(fuel_return.tolist())
    result = objective_from_scenario_outputs(losses, gap_burdens, returns_all)
    result.update(
        {
            "scenario_count": len(candidates),
            "max_gap_ratio": float(max(max_gap_ratios)),
            "max_terminal_gap_ratio": float(max(terminal_gap_ratios)),
            "max_recovery_terminal_gap_ratio": float(max(recovery_gap_ratios)),
            "max_monthly_adjustment_ratio": float(max(max_adjustment_ratios)),
```

```
        }
    )
    return result


def scenario_candidates(scenarios: list[Scenario], split: str) -> list[Scenario]:
    candidates = [scenario for scenario in scenarios if scenario.sample_split == split]
    if split == "war_2026":
        candidates = [scenario for scenario in scenarios if scenario.is_2026_anchor]
    if not candidates:
        raise ValueError(f"Q4 has no scenario candidates for split={split}.")
    return candidates


def strategy_path_arrays(
    scenarios: list[Scenario],
    rule: dict[str, float] | None,
    strategy_name: str,
    split: str,
    base_price: float,
) -> dict[str, np.ndarray]:
    candidates = scenario_candidates(scenarios, split)
    fuel_returns = np.empty((len(candidates), SCENARIO_LENGTH), dtype=float)
    gap_burdens = np.empty(len(candidates), dtype=float)
    max_gap_ratios = np.empty(len(candidates), dtype=float)
    terminal_gap_ratios = np.empty(len(candidates), dtype=float)
    recovery_gap_ratios = np.empty(len(candidates), dtype=float)
    max_adjustment_ratios = np.empty(len(candidates), dtype=float)
    for index, scenario in enumerate(candidates):
        if strategy_name == "actual_2026_event_path":
            _, adjustment, gap, fuel_return = simulate_actual_2026_path(scenario, base_price)
        else:
            if rule is None:
                raise ValueError("A SAPR rule is required for simulated strategies.")
            _, adjustment, gap, fuel_return = simulate_rule_path(scenario, rule, base_price)
        fuel_returns[index, :] = fuel_return
        gap_burdens[index] = float(np.mean(np.maximum(gap, 0.0)) / base_price)
        max_gap_ratios[index] = float(np.max(np.maximum(gap, 0.0)) / base_price)
        terminal_gap_ratios[index] = float(max(float(gap[-1]), 0.0) / base_price)
        recovery_gap_ratios[index] = (
            float(recovery_terminal_gap(float(gap[-1]), rule) / base_price)
            if rule is not None
            else terminal_gap_ratios[index]
        )
        max_adjustment_ratios[index] = float(np.max(np.abs(adjustment)) / base_price)
    return {
        "fuel_returns": fuel_returns,
        "gap_burdens": gap_burdens,
        "max_gap_ratios": max_gap_ratios,
        "terminal_gap_ratios": terminal_gap_ratios,
        "recovery_gap_ratios": recovery_gap_ratios,
        "max_adjustment_ratios": max_adjustment_ratios,
    }


def macro_losses_from_returns(
    fuel_returns: np.ndarray,
    coefficients: dict[str, np.ndarray],
    stds: dict[str, float],
    weights: dict[str, float],
) -> np.ndarray:
    if fuel_returns.ndim != 2 or fuel_returns.shape[1] != SCENARIO_LENGTH:
        raise ValueError("Q4 macro loss calculation requires a scenario-by-month fuel-return matrix.")
    month_loss = np.zeros_like(fuel_returns, dtype=float)
    for outcome in OUTCOMES:
        params = coefficients[outcome]
        y = np.zeros_like(fuel_returns, dtype=float)
        for t in range(SCENARIO_LENGTH):
            signal = np.zeros(fuel_returns.shape[0], dtype=float)
            for lag in range(7):
                if t - lag >= 0:
                    signal += float(params[lag]) * fuel_returns[:, t - lag]
            if t > 0:
                signal += float(params[7]) * y[:, t - 1]
            y[:, t] = signal
        if outcome == "china_iav_yoy_pct":
```

```
            contribution = np.maximum(-y, 0.0)
        else:
            contribution = np.maximum(y, 0.0)
        month_loss += weights[outcome] * contribution / stds[outcome]
    return month_loss.mean(axis=1)


def objectives_from_arrays(
    scenario_losses: np.ndarray,
    gap_burdens: np.ndarray,
    fuel_returns: np.ndarray,
    scenario_indices: np.ndarray | None = None,
) -> dict[str, float]:
    if scenario_indices is not None:
        indices = np.asarray(scenario_indices, dtype=int) % len(scenario_losses)
        losses = scenario_losses[indices]
        gaps = gap_burdens[indices]
        returns = fuel_returns[indices, :].reshape(-1)
    else:
        losses = scenario_losses
        gaps = gap_burdens
        returns = fuel_returns.reshape(-1)
    return objective_from_scenario_outputs(losses.tolist(), gaps.tolist(), returns.tolist())


def dominates(a: np.ndarray, b: np.ndarray) -> bool:
    return bool(np.all(a <= b + 1e-12) and np.any(a < b - 1e-12))


def pareto_mask(objectives: np.ndarray) -> np.ndarray:
    n = objectives.shape[0]
    mask = np.ones(n, dtype=bool)
    for i in range(n):
        if not mask[i]:
            continue
        dominated_by_i = np.all(objectives[i] <= objectives + 1e-12, axis=1) & np.any(objectives[i] < objectives - 1e-12, axis=1)
        dominated_by_i[i] = False
        mask[dominated_by_i] = False
        if np.any(np.all(objectives[mask] <= objectives[i] + 1e-12, axis=1) & np.any(objectives[mask] < objectives[i] - 1e-12, axis=1)):
            mask[i] = False
    return mask


def epsilon_pareto_mask(objectives: np.ndarray, epsilon_fraction: float = 0.01) -> np.ndarray:
    """Treat sub-material objective differences as ties to avoid a mechanically wide front."""
    spans = np.ptp(objectives, axis=0)
    epsilon = np.maximum(spans * epsilon_fraction, 1e-12)
    n = objectives.shape[0]
    mask = np.ones(n, dtype=bool)
    for i in range(n):
        candidate = objectives[i]
        for j in range(n):
            if i == j:
                continue
            challenger = objectives[j]
            if np.all(challenger <= candidate + epsilon) and np.any(challenger < candidate - epsilon):
                mask[i] = False
                break
    return mask


def choose_knee(grid: pd.DataFrame) -> pd.DataFrame:
    objective_cols = ["J1_macro_loss", "J2_cvar95_macro_loss", "J3_avg_gap_month_burden", "J4_adjustment_volatility"]
    grid = grid.copy()
    grid["is_feasible"] = (
        grid["max_gap_ratio"].le(MAX_GAP_RATIO)
        & grid["max_terminal_gap_ratio"].le(MAX_TERMINAL_GAP_RATIO)
        & grid["max_recovery_terminal_gap_ratio"].le(MAX_RECOVERY_TERMINAL_GAP_RATIO)
        & grid["max_monthly_adjustment_ratio"].le(MAX_MONTHLY_ADJUSTMENT_RATIO)
    )
    grid["is_pareto"] = False
    grid["is_knee"] = False
    feasible = grid.loc[grid["is_feasible"]].copy()
    if feasible.empty:
        raise ValueError("Q4 has no feasible rule under the preregistered gap, recovery and adjustment constraints.")
    feasible_values = feasible[objective_cols].astype(float).to_numpy()
```

```
    exact_mask = pareto_mask(feasible_values)
    grid["is_pareto_exact"] = False
    grid.loc[feasible.index, "is_pareto_exact"] = exact_mask
    mask = epsilon_pareto_mask(feasible_values, epsilon_fraction=0.01)
    grid.loc[feasible.index, "is_pareto"] = mask
    pareto = grid.loc[grid["is_pareto"]].copy()
    if pareto.empty:
        raise ValueError("Q4 Pareto set is empty.")
    mins = pareto[objective_cols].min()
    maxs = pareto[objective_cols].max()
    denom = (maxs - mins).replace(0.0, 1.0)
    normalized = (pareto[objective_cols] - mins) / denom
    pareto["knee_score"] = np.sqrt((normalized**2).sum(axis=1))
    pareto = pareto.sort_values(["knee_score", "J3_avg_gap_month_burden", "rho_normal"], ascending=[True, True, False])
    knee_rule_id = pareto.iloc[0]["rule_id"]
    grid.loc[grid["rule_id"].eq(knee_rule_id), "is_knee"] = True
    grid["knee_score"] = np.nan
    grid.loc[pareto.index, "knee_score"] = pareto["knee_score"]
    return grid


def run_grid_search(
    scenarios: list[Scenario],
    coefficients: dict[str, np.ndarray],
    base_price: float,
    stds: dict[str, float],
    weights: dict[str, float],
) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for idx, rule in enumerate(enumerate_rules(), start=1):
        objectives = evaluate_strategy(scenarios, coefficients, rule, "SAPR_candidate", "development", base_price, stds, weights)
        rows.append(
            {
                "rule_id": f"R{idx:04d}",
                **rule,
                **objectives,
                "sample_split": "development",
            }
        )
    grid = choose_knee(pd.DataFrame(rows))
    if int(grid["is_knee"].sum()) != 1:
        raise ValueError("Q4 knee selection is not unique.")
    return grid


def strategy_catalog(knee_rule: dict[str, float]) -> dict[str, dict[str, Any]]:
    return {
        "full_mechanism": {"rule": {"rho_normal": 1.0, "rho_stress": 1.0, "rho_extreme": 1.0}, "type": "rule"},
        "uniform_75_smoothing": {"rule": {"rho_normal": 0.75, "rho_stress": 0.75, "rho_extreme": 0.75}, "type": "rule"},
        "temporary_2026_approx": {"rule": {"rho_normal": 1.0, "rho_stress": 1.0, "rho_extreme": round(TEMPORARY_2026_RHO, 6)}, "type": "
      rule"},
        "SAPR_CVaR_knee": {"rule": knee_rule, "type": "rule"},
        "actual_2026_event_path": {"rule": None, "type": "actual_2026"},
    }


def block_bootstrap_indices(n: int, block_length: int, rng: np.random.Generator) -> np.ndarray:
    if n <= 0:
        raise ValueError("Q4 block bootstrap requires a positive scenario count.")
    indices: list[int] = []
    while len(indices) < n:
        start = int(rng.integers(0, n))
        for offset in range(block_length):
            indices.append((start + offset) % n)
            if len(indices) >= n:
                break
    return np.array(indices[:n], dtype=int)


def compare_strategies(
    scenarios: list[Scenario],
    kernel: MacroKernel,
    parameter_draws: list[dict[str, np.ndarray]],
    knee_rule: dict[str, float],
    base_price: float,
    stds: dict[str, float],
```

```
    weights: dict[str, float],
    rng: np.random.Generator,
    block_length: int = 6,
) -> tuple[pd.DataFrame, float, bool]:
    point = point_coefficients(kernel)
    strategies = strategy_catalog(knee_rule)
    rows: list[dict[str, Any]] = []
    interval_store: dict[tuple[str, str], list[dict[str, float]]] = {}
    splits = ["development", "holdout", "war_2026"]
    for split in splits:
        for strategy_name, meta in strategies.items():
            if strategy_name == "actual_2026_event_path" and split != "war_2026":
                continue
            arrays = strategy_path_arrays(scenarios, meta["rule"], strategy_name, split, base_price)
            point_losses = macro_losses_from_returns(arrays["fuel_returns"], point, stds, weights)
            objectives = objectives_from_arrays(point_losses, arrays["gap_burdens"], arrays["fuel_returns"])
            objectives["scenario_count"] = int(len(point_losses))
            objectives.update(
                {
                    "max_gap_ratio": float(np.max(arrays["max_gap_ratios"])),
                    "max_terminal_gap_ratio": float(np.max(arrays["terminal_gap_ratios"])),
                    "max_recovery_terminal_gap_ratio": float(np.max(arrays["recovery_gap_ratios"])),
                    "max_monthly_adjustment_ratio": float(np.max(arrays["max_adjustment_ratios"])),
                }
            )
            row = {
                "sample_split": split,
                "strategy": strategy_name,
                "rho_normal": clean_value(meta["rule"]["rho_normal"]) if meta["rule"] else None,
                "rho_stress": clean_value(meta["rule"]["rho_stress"]) if meta["rule"] else None,
                "rho_extreme": clean_value(meta["rule"]["rho_extreme"]) if meta["rule"] else None,
                **objectives,
            }
            rows.append(row)
            interval_store[(split, strategy_name)] = []
            n = int(len(point_losses))
            for draw in parameter_draws:
                sampled = block_bootstrap_indices(n, block_length, rng)
                draw_losses = macro_losses_from_returns(arrays["fuel_returns"], draw, stds, weights)
                interval_store[(split, strategy_name)].append(
                    objectives_from_arrays(draw_losses, arrays["gap_burdens"], arrays["fuel_returns"], sampled)
                )
    comparison = pd.DataFrame(rows)
    full_by_split = comparison.loc[comparison["strategy"].eq("full_mechanism")].set_index("sample_split")
    for idx, row in comparison.iterrows():
        split = row["sample_split"]
        if split in full_by_split.index:
            full = full_by_split.loc[split]
            for col in ["J1_macro_loss", "J2_cvar95_macro_loss"]:
                denom = abs(float(full[col])) if abs(float(full[col])) > 1e-12 else np.nan
                comparison.loc[idx, f"{col}_improvement_vs_full_pct"] = (
                    100.0 * (float(full[col]) - float(row[col])) / denom if np.isfinite(denom) else np.nan
                )
    for idx, row in comparison.iterrows():
        draws = interval_store.get((row["sample_split"], row["strategy"]), [])
        if not draws:
            continue
        for col in ["J1_macro_loss", "J2_cvar95_macro_loss"]:
            values = np.array([float(draw[col]) for draw in draws], dtype=float)
            comparison.loc[idx, f"{col}_lower_95"] = float(np.quantile(values, 0.025))
            comparison.loc[idx, f"{col}_upper_95"] = float(np.quantile(values, 0.975))

    probability, point_non_dominated = holdout_validation_probability(
        scenarios,
        parameter_draws,
        knee_rule,
        base_price,
        stds,
        weights,
        rng,
        block_length=block_length,
        point_comparison=comparison,
    )
    return comparison, probability, point_non_dominated
```

```python
def holdout_validation_probability(
    scenarios: list[Scenario],
    parameter_draws: list[dict[str, np.ndarray]],
    knee_rule: dict[str, float],
    base_price: float,
    stds: dict[str, float],
    weights: dict[str, float],
    rng: np.random.Generator,
    block_length: int,
    point_comparison: pd.DataFrame | None = None,
    point_values: dict[str, np.ndarray] | None = None,
    save_outputs: bool = True,
) -> tuple[float, bool]:
    objective_cols = ["J1_macro_loss", "J2_cvar95_macro_loss", "J3_avg_gap_month_burden", "J4_adjustment_volatility"]
    strategies = strategy_catalog(knee_rule)
    holdout_strategies = [name for name in strategies if name != "actual_2026_event_path"]
    if point_comparison is None:
        if point_values is None:
            raise ValueError("Q4 holdout validation requires either point comparison rows or explicit point coefficients.")
        point_rows: list[dict[str, Any]] = []
        for strategy_name in holdout_strategies:
            meta = strategies[strategy_name]
            arrays = strategy_path_arrays(scenarios, meta["rule"], strategy_name, "holdout", base_price)
            point_losses = macro_losses_from_returns(arrays["fuel_returns"], point_values, stds, weights)
            objectives = objectives_from_arrays(point_losses, arrays["gap_burdens"], arrays["fuel_returns"])
            objectives["scenario_count"] = int(len(point_losses))
            point_rows.append({"sample_split": "holdout", "strategy": strategy_name, **objectives})
        point_holdout = pd.DataFrame(point_rows)
    else:
        point_holdout = point_comparison.loc[
            point_comparison["sample_split"].eq("holdout") & point_comparison["strategy"].isin(holdout_strategies)
        ].copy()
    if set(point_holdout["strategy"]) != set(holdout_strategies):
        raise ValueError("Q4 holdout validation is missing one or more preregistered strategies.")
    point_knee = point_holdout.loc[point_holdout["strategy"].eq("SAPR_CVaR_knee")]
    if len(point_knee) != 1:
        raise ValueError("Q4 holdout validation requires exactly one SAPR knee row.")
    knee_vec = point_knee[objective_cols].iloc[0].astype(float).to_numpy()
    point_non_dominated = True
    for _, row in point_holdout.loc[point_holdout["strategy"].ne("SAPR_CVaR_knee")].iterrows():
        other_vec = row[objective_cols].astype(float).to_numpy()
        if dominates(other_vec, knee_vec):
            point_non_dominated = False
            break
    holdout_n = len([scenario for scenario in scenarios if scenario.sample_split == "holdout"])
    arrays_by_strategy = {
        strategy_name: strategy_path_arrays(scenarios, strategies[strategy_name]["rule"], strategy_name, "holdout", base_price)
        for strategy_name in holdout_strategies
    }
    point_matrix = point_holdout.set_index("strategy").loc[holdout_strategies, objective_cols].astype(float).to_numpy()
    point_mins = point_matrix.min(axis=0)
    point_denom = np.maximum(point_matrix.max(axis=0) - point_mins, 1e-12)
    point_distances = np.sqrt((((point_matrix - point_mins) / point_denom) ** 2).sum(axis=1))
    point_knee_distance = float(point_distances[holdout_strategies.index("SAPR_CVaR_knee")])
    point_oracle_distance = float(point_distances.min())
    draw_rows: list[dict[str, Any]] = []
    valid_flags: list[bool] = []
    for draw_id, draw in enumerate(parameter_draws):
        sampled = block_bootstrap_indices(holdout_n, block_length, rng)
        vectors: dict[str, np.ndarray] = {}
        for strategy_name in holdout_strategies:
            arrays = arrays_by_strategy[strategy_name]
            draw_losses = macro_losses_from_returns(arrays["fuel_returns"], draw, stds, weights)
            obj = objectives_from_arrays(draw_losses, arrays["gap_burdens"], arrays["fuel_returns"], sampled)
            vectors[strategy_name] = np.array(
                [obj["J1_macro_loss"], obj["J2_cvar95_macro_loss"], obj["J3_avg_gap_month_burden"], obj["J4_adjustment_volatility"]],
                dtype=float,
            )
        sampled_knee_vec = vectors["SAPR_CVaR_knee"]
        non_dominated = not any(dominates(vec, sampled_knee_vec) for name, vec in vectors.items() if name != "SAPR_CVaR_knee")
        valid_flags.append(non_dominated)
        matrix = np.vstack([vectors[name] for name in holdout_strategies])
        mins = matrix.min(axis=0)
        denom = np.maximum(matrix.max(axis=0) - mins, 1e-12)
        distances = np.sqrt((((matrix - mins) / denom) ** 2).sum(axis=1))
        knee_distance = float(distances[holdout_strategies.index("SAPR_CVaR_knee")])
```

```
        oracle_distance = float(distances.min())
        for baseline in [name for name in holdout_strategies if name != "SAPR_CVaR_knee"]:
            delta = sampled_knee_vec - vectors[baseline]
            draw_rows.append(
                {
                    "draw": draw_id,
                    "block_length": block_length,
                    "baseline": baseline,
                    **{f"delta_{column}": float(delta[idx]) for idx, column in enumerate(objective_cols)},
                    "knee_distance_to_ideal": knee_distance,
                    "oracle_distance_to_ideal": oracle_distance,
                    "knee_regret_vs_oracle": knee_distance - oracle_distance,
                    "knee_non_dominated": non_dominated,
                }
            )
    if len(valid_flags) != len(parameter_draws):
        raise ValueError("Q4 holdout validation did not evaluate every valid parameter draw.")
    validation = pd.DataFrame(draw_rows)
    if save_outputs:
        save_csv(validation, "q4_sapr_holdout_validation.csv")
    paired_summary: list[dict[str, Any]] = []
    for baseline, group in validation.groupby("baseline"):
        for column in objective_cols:
            values = group[f"delta_{column}"].to_numpy(dtype=float)
            lower, upper = np.quantile(values, [0.025, 0.975])
            paired_summary.append(
                {
                    "baseline": baseline,
                    "objective": column,
                    "mean_delta_knee_minus_baseline": float(np.mean(values)),
                    "lower_95": float(lower),
                    "upper_95": float(upper),
                    "supported_improvement": bool(upper < 0.0),
                }
            )
    paired_frame = pd.DataFrame(paired_summary)
    if save_outputs:
        save_csv(paired_frame, "q4_sapr_holdout_paired_summary.csv")
    unique_draws = validation.drop_duplicates("draw")
    summary_payload = {
        "block_length": block_length,
        "draws": int(len(unique_draws)),
        "point_non_dominated": point_non_dominated,
        "non_dominated_probability": float(np.mean(valid_flags)),
        "point_knee_distance_to_ideal": point_knee_distance,
        "point_oracle_distance_to_ideal": point_oracle_distance,
        "point_regret_vs_oracle": point_knee_distance - point_oracle_distance,
        "median_regret_vs_oracle": float(unique_draws["knee_regret_vs_oracle"].median()),
        "regret_upper_95": float(unique_draws["knee_regret_vs_oracle"].quantile(0.975)),
        "probability_within_0_10_of_oracle": float(unique_draws["knee_regret_vs_oracle"].le(0.10).mean()),
        "supported_improvement_count": int(paired_frame["supported_improvement"].sum()),
    }
    if save_outputs:
        (RESULTS_DIR / "q4_sapr_holdout_validation_summary.json").write_text(
            json.dumps(summary_payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
            encoding="utf-8",
        )
    return float(np.mean(valid_flags)), point_non_dominated


def aggregate_macro_paths(
    scenarios: list[Scenario],
    kernel: MacroKernel,
    parameter_draws: list[dict[str, np.ndarray]],
    knee_rule: dict[str, float],
    base_price: float,
    split: str = "war_2026",
) -> pd.DataFrame:
    strategies = strategy_catalog(knee_rule)
    point = point_coefficients(kernel)
    rows: list[dict[str, Any]] = []
    target_scenarios = [scenario for scenario in scenarios if scenario.is_2026_anchor] if split == "war_2026" else [scenario for scenario
       in scenarios if scenario.sample_split == split]
    if not target_scenarios:
        raise ValueError(f"Q4 macro-path aggregation has no target scenarios for split={split}.")
    for strategy_name, meta in strategies.items():
```

```
        if strategy_name == "actual_2026_event_path" and split != "war_2026":
            continue
        month_records: list[dict[str, Any]] = []
        for scenario in target_scenarios:
            if strategy_name == "actual_2026_event_path":
                price, adjustment, gap, fuel_return = simulate_actual_2026_path(scenario, base_price)
            else:
                price, adjustment, gap, fuel_return = simulate_rule_path(scenario, meta["rule"], base_price)
            macro_paths = macro_path_from_returns(fuel_return, point)
            draw_paths = {outcome: [] for outcome in OUTCOMES}
            for draw in parameter_draws:
                simulated = macro_path_from_returns(fuel_return, draw)
                for outcome in OUTCOMES:
                    draw_paths[outcome].append(simulated[outcome])
            for t, period in enumerate(scenario.periods):
                record = {
                    "sample_split": split,
                    "scenario_id": scenario.scenario_id,
                    "strategy": strategy_name,
                    "month_index": t,
                    "period": period,
                    "fuel_price_cny_t": price[t],
                    "actual_adjustment_cny_t": adjustment[t],
                    "fuel_log_return": fuel_return[t],
                    "cumulative_deferred_gap_cny_t": gap[t],
                    "PPI_response_pctpt": macro_paths["china_ppi_yoy_pct"][t],
                    "CPI_response_pctpt": macro_paths["china_cpi_yoy_pct"][t],
                    "IAV_response_pctpt": macro_paths["china_iav_yoy_pct"][t],
                }
                for outcome, label in OUTCOME_LABELS.items():
                    values = np.array([path[t] for path in draw_paths[outcome]], dtype=float)
                    record[f"{label}_lower_95"] = float(np.quantile(values, 0.025))
                    record[f"{label}_upper_95"] = float(np.quantile(values, 0.975))
                month_records.append(record)
        if not month_records:
            raise ValueError(f"Q4 macro-path aggregation produced no rows for strategy={strategy_name}, split={split}.")
        rows.extend(month_records)
    return pd.DataFrame(rows)


def policy_identity_checks(scenarios: list[Scenario], base_price: float, kernel: MacroKernel) -> dict[str, Any]:
    scenario = next(item for item in scenarios if item.sample_split == "development")
    _, full_adjustment, full_gap, _ = simulate_rule_path(
        scenario,
        {"rho_normal": 1.0, "rho_stress": 1.0, "rho_extreme": 1.0},
        base_price,
    )
    synthetic = Scenario(
        scenario_id="IDENTITY_ZERO",
        sample_split="development",
        source_start="2099-01",
        periods=tuple(f"2099-0{i + 1}" for i in range(SCENARIO_LENGTH)),
        mechanism_adjustment=np.zeros(SCENARIO_LENGTH),
        actual_adjustment=np.zeros(SCENARIO_LENGTH),
        cumulative_policy_gap=np.zeros(SCENARIO_LENGTH),
        regimes=tuple(["normal"] * SCENARIO_LENGTH),
        is_2026_anchor=False,
    )
    _, _, _, zero_return = simulate_rule_path(
        synthetic,
        {"rho_normal": 0.35, "rho_stress": 0.2, "rho_extreme": 0.1},
        base_price,
    )
    zero_macro = macro_path_from_returns(zero_return, point_coefficients(kernel))
    accumulation = Scenario(
        scenario_id="IDENTITY_ACCUMULATION",
        sample_split="development",
        source_start="2099-01",
        periods=tuple(f"2099-0{i + 1}" for i in range(SCENARIO_LENGTH)),
        mechanism_adjustment=np.array([100.0, 100.0, 0.0, 0.0, 0.0, 0.0]),
        actual_adjustment=np.zeros(SCENARIO_LENGTH),
        cumulative_policy_gap=np.zeros(SCENARIO_LENGTH),
        regimes=tuple(["normal"] * SCENARIO_LENGTH),
        is_2026_anchor=False,
    )
    _, _, gap_accum, _ = simulate_rule_path(
```

```
        accumulation,
        {"rho_normal": 0.0, "rho_stress": 0.0, "rho_extreme": 0.0},
        base_price,
    )
    offset = Scenario(
        scenario_id="IDENTITY_OFFSET",
        sample_split="development",
        source_start="2099-01",
        periods=tuple(f"2099-0{i + 1}" for i in range(SCENARIO_LENGTH)),
        mechanism_adjustment=np.array([100.0, -40.0, -100.0, 0.0, 0.0, 0.0]),
        actual_adjustment=np.zeros(SCENARIO_LENGTH),
        cumulative_policy_gap=np.zeros(SCENARIO_LENGTH),
        regimes=tuple(["normal"] * SCENARIO_LENGTH),
        is_2026_anchor=False,
    )
    _, _, gap_offset, _ = simulate_rule_path(
        offset,
        {"rho_normal": 0.0, "rho_stress": 0.0, "rho_extreme": 0.0},
        base_price,
    )
    policy = read_processed("china_fuel_policy_monthly.csv")
    march = policy.loc[policy["period"].eq("2026-03")].iloc[0]
    april = policy.loc[policy["period"].eq("2026-04")].iloc[0]
    q3_repro = reproduce_q3_policy_counterfactual(kernel)
    return {
        "full_rule_max_abs_gap": float(np.max(np.abs(full_gap))),
        "full_rule_adjustment_identity_max_abs_error": float(np.max(np.abs(full_adjustment - scenario.mechanism_adjustment))),
        "zero_policy_macro_max_abs_response": float(max(np.max(np.abs(values)) for values in zero_macro.values())),
        "zero_upward_pass_through_terminal_gap": float(gap_accum[2]),
        "negative_adjustment_offsets_gap": bool(abs(float(gap_offset[2])) < 1e-9),
        "official_2026_03_gap": float(march["gasoline_policy_gap_cny_t"]),
        "official_2026_04_gap": float(april["gasoline_policy_gap_cny_t"]),
        "official_2026_04_cumulative_gap": float(april["cum_gasoline_policy_gap_cny_t"]),
        "q3_policy_counterfactual_reproduction_max_abs_error": q3_repro,
    }


def reproduce_q3_policy_counterfactual(kernel: MacroKernel) -> float:
    q3 = read_result("q3_policy_macro_counterfactual.csv")
    max_error = 0.0
    for outcome, group in q3.groupby("outcome"):
        group = group.sort_values("horizon")
        fuel = group["fuel_return_gap"].to_numpy(dtype=float)
        params = kernel.means[str(outcome)]
        expected = np.zeros(len(group), dtype=float)
        for t in range(len(group)):
            for lag in range(7):
                if t - lag >= 0:
                    expected[t] += float(params[lag]) * float(fuel[t - lag])
            if t > 0:
                expected[t] += float(params[7]) * expected[t - 1]
        observed = group["macro_counterfactual_gap_pctpt"].to_numpy(dtype=float)
        max_error = max(max_error, float(np.max(np.abs(expected - observed))))
    return float(max_error)


def run_sensitivity(
    default_grid: pd.DataFrame,
    scenarios: list[Scenario],
    kernel: MacroKernel,
    parameter_draws: list[dict[str, np.ndarray]],
    meta: dict[str, float],
    stds: dict[str, float],
    rng: np.random.Generator,
) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    variants = [
        ("threshold_70_90", (0.70, 0.90), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 3),
        ("threshold_80_975", (0.80, 0.975), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 3),
        ("cpi_weight_plus20", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.2, "china_iav_yoy_pct": 1.0}, 3),
        ("iav_weight_plus20", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.2}, 3),
        ("bootstrap_block_3", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 3),
        ("bootstrap_block_6", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 6),
        ("bootstrap_block_12", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 12),
    ]
    point = point_coefficients(kernel)
```

```
    for variant_name, quantiles, weights, block_length in variants:
        if variant_name.startswith("threshold"):
            _, variant_scenarios, variant_meta = build_scenarios(quantiles)
        else:
            variant_scenarios = scenarios
            variant_meta = meta
        if variant_name.startswith("bootstrap_block"):
            grid = default_grid.copy()
        else:
            grid = run_grid_search(variant_scenarios, point, float(variant_meta["base_price_2026_06_cny_t"]), stds, weights)
        knee = grid.loc[grid["is_knee"]].iloc[0]
        knee_rule = {
            "rho_normal": float(knee["rho_normal"]),
            "rho_stress": float(knee["rho_stress"]),
            "rho_extreme": float(knee["rho_extreme"]),
        }
        prob, point_nd = holdout_validation_probability(
            variant_scenarios,
            parameter_draws,
            knee_rule,
            float(variant_meta["base_price_2026_06_cny_t"]),
            stds,
            weights,
            rng,
            block_length=block_length,
            point_values=point,
            save_outputs=False,
        )
        rows.append(
            {
                "variant": variant_name,
                "threshold_low_quantile": quantiles[0],
                "threshold_high_quantile": quantiles[1],
                "bootstrap_block_length": block_length,
                "ppi_weight": weights["china_ppi_yoy_pct"],
                "cpi_weight": weights["china_cpi_yoy_pct"],
                "iav_weight": weights["china_iav_yoy_pct"],
                "rho_normal": knee_rule["rho_normal"],
                "rho_stress": knee_rule["rho_stress"],
                "rho_extreme": knee_rule["rho_extreme"],
                "pareto_rule_count": int(grid["is_pareto"].sum()),
                "holdout_non_dominated_probability": prob,
                "holdout_point_non_dominated": point_nd,
                "changed_from_default": bool(
                    knee_rule["rho_normal"] != float(default_grid.loc[default_grid["is_knee"], "rho_normal"].iloc[0])
                    or knee_rule["rho_stress"] != float(default_grid.loc[default_grid["is_knee"], "rho_stress"].iloc[0])
                    or knee_rule["rho_extreme"] != float(default_grid.loc[default_grid["is_knee"], "rho_extreme"].iloc[0])
                ),
            }
        )
    return pd.DataFrame(rows)


def plot_pareto(grid: pd.DataFrame) -> None:
    fig, ax = plt.subplots(figsize=(7.6, 4.6))
    pareto = grid.loc[grid["is_pareto"]].copy()
    if pareto.empty:
        raise ValueError("Q4 Pareto plot requires a non-empty Pareto set.")
    volatility = pareto["J4_adjustment_volatility"].astype(float)
    size = 24.0 + 72.0 * (volatility - volatility.min()) / max(float(volatility.max() - volatility.min()), 1e-12)
    sc = ax.scatter(
        pareto["J1_macro_loss"],
        pareto["J3_avg_gap_month_burden"],
        c=pareto["J2_cvar95_macro_loss"],
        cmap="cividis",
        s=size,
        alpha=0.82,
    )
    knee = pareto.loc[pareto["is_knee"]]
    if len(knee) != 1:
        raise ValueError("Q4 Pareto plot requires exactly one knee rule.")
    ax.scatter(knee["J1_macro_loss"], knee["J3_avg_gap_month_burden"], marker="*", s=180, color=PALETTE["rose"], edgecolor="white",
      linewidth=0.8, label="膝点规则")
    style_axis(ax, xlabel="期望宏观损失", ylabel="平均缺口月负担")
    ax.legend(loc="upper right")
    cbar = fig.colorbar(sc, ax=ax)
```

```
    cbar.set_label("CVaR95宏观损失")
    finish_figure(
        fig,
        title="问题四：SAPR-CVaR 四目标Pareto前沿",
        subtitle="横纵轴为J1/J3，颜色为J2，点大小为J4；仅绘制通过硬约束的1%epsilon-非支配规则，星形为膝点。",
        source="来源：results/q4_sapr_policy_grid.csv；作者计算。",
    )
    save_figure(fig, FIGURES_DIR / "q4_sapr_pareto_front")
    plt.close(fig)


def plot_policy_heatmap(knee_rule: dict[str, float]) -> None:
    regimes = ["normal", "stress", "extreme"]
    gap_bins = ["低缺口", "中缺口", "高缺口"]
    values = np.array([[rho_for_regime(knee_rule, regime) for regime in regimes] for _ in gap_bins])
    fig, ax = plt.subplots(figsize=(6.4, 3.8))
    image = ax.imshow(values, cmap="YlGnBu", vmin=0, vmax=1)
    ax.set_xticks(np.arange(len(regimes)), labels=["普通", "压力", "极端"])
    ax.set_yticks(np.arange(len(gap_bins)), labels=gap_bins)
    for i in range(values.shape[0]):
        for j in range(values.shape[1]):
            ax.text(j, i, f"{values[i, j]:.2f}", ha="center", va="center", color=PALETTE["ink"], fontsize=10)
    ax.set_xlabel("机制应调额状态")
    ax.set_ylabel("累计未调价缺口")
    cbar = fig.colorbar(image, ax=ax)
    cbar.set_label("最优传导率")
    finish_figure(
        fig,
        title="问题四：冲击强度与最优传导率",
        subtitle="SAPR规则按冲击状态调整传导率，累计缺口通过状态转移进入下一期应调额。",
        source="来源：results/q4_sapr_optimal_rule.csv；作者计算。",
    )
    save_figure(fig, FIGURES_DIR / "q4_sapr_policy_heatmap")
    plt.close(fig)


def plot_strategy_comparison(comparison: pd.DataFrame) -> None:
    frame = comparison.loc[comparison["sample_split"].eq("holdout") & comparison["strategy"].ne("actual_2026_event_path")].copy()
    if frame.empty:
        raise ValueError("Q4 strategy comparison plot requires holdout strategy rows.")
    labels = {
        "full_mechanism": "完全传导",
        "uniform_75_smoothing": "固定75%平滑",
        "temporary_2026_approx": "2026近似",
        "SAPR_CVaR_knee": "SAPR膝点",
    }
    frame["label"] = frame["strategy"].map(labels).fillna(frame["strategy"])
    metrics = ["J1_macro_loss", "J2_cvar95_macro_loss", "J3_avg_gap_month_burden", "J4_adjustment_volatility"]
    metric_labels = ["平均宏观损失", "CVaR95宏观损失", "平均缺口月负担", "月度调价波动"]
    fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.4))
    colors = [PALETTE["slate"], PALETTE["gold"], PALETTE["olive"], PALETTE["blue"]]
    for ax, metric, label in zip(axes.ravel(), metrics, metric_labels):
        ax.bar(frame["label"], frame[metric].astype(float), color=colors, width=0.62)
        ax.tick_params(axis="x", labelrotation=18)
        style_axis(ax, ylabel=label)
    finish_figure(
        fig,
        title="问题四：检验期四种策略原始目标对比",
        subtitle="四个子图保留各自原始量纲；数值越低表示该目标表现越好，不能跨子图直接比较高度。",
        source="来源：results/q4_sapr_strategy_comparison.csv；作者计算。",
        rect=(0.08, 0.08, 0.98, 0.90),
    )
    save_figure(fig, FIGURES_DIR / "q4_sapr_strategy_comparison")
    plt.close(fig)


def plot_2026_paths(paths: pd.DataFrame) -> None:
    frame = paths.loc[paths["sample_split"].eq("war_2026")].copy()
    if frame.empty:
        raise ValueError("Q4 2026 path plot requires war_2026 rows.")
    labels = {
        "full_mechanism": "完全传导",
        "uniform_75_smoothing": "固定75%平滑",
        "temporary_2026_approx": "2026近似",
        "SAPR_CVaR_knee": "SAPR膝点",
        "actual_2026_event_path": "2026实际",
```

```
    }
    fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.2), sharex=True)
    panels = [
        ("PPI_response_pctpt", "PPI响应"),
        ("CPI_response_pctpt", "CPI响应"),
        ("IAV_response_pctpt", "IAV响应"),
        ("cumulative_deferred_gap_cny_t", "累计缺口"),
    ]
    for ax, (column, ylabel) in zip(axes.ravel(), panels):
        for strategy, sub in frame.groupby("strategy"):
            sub = sub.sort_values("month_index")
            ax.plot(sub["month_index"], sub[column], marker="o", label=labels.get(strategy, strategy))
        ax.axhline(0, color=PALETTE["muted"], linewidth=0.7)
        style_axis(ax, ylabel=ylabel)
    axes[-1, 0].set_xlabel("冲击后月份")
    axes[-1, 1].set_xlabel("冲击后月份")
    handles, legend_labels = axes[0, 0].get_legend_handles_labels()
    fig.legend(handles, legend_labels, loc="upper center", ncol=5, frameon=False)
    finish_figure(
        fig,
        title="问题四：2026冲击下策略路径对比",
        subtitle="宏观路径由Q3燃油价格响应核传播，累计缺口为价格平滑延期负担代理。",
        source="来源：results/q4_sapr_macro_paths.csv；作者计算。",
        rect=(0.08, 0.08, 0.98, 0.92),
    )
    save_figure(fig, FIGURES_DIR / "q4_sapr_2026_macro_paths")
    plt.close(fig)


def write_summary(
    scenario_table: pd.DataFrame,
    grid: pd.DataFrame,
    optimal: pd.DataFrame,
    comparison: pd.DataFrame,
    sensitivity: pd.DataFrame,
    identity: dict[str, Any],
    valid_draw_rate: float,
    total_draw_attempts: int,
    valid_draw_count: int,
    holdout_probability: float,
    holdout_point_non_dominated: bool,
) -> dict[str, Any]:
    validation_path = RESULTS_DIR / "q4_sapr_holdout_validation_summary.json"
    validation = json.loads(validation_path.read_text(encoding="utf-8")) if validation_path.exists() else {}
    knee_rows = comparison.loc[comparison["strategy"].eq("SAPR_CVaR_knee")]
    knee_feasible_all_splits = bool(
        not knee_rows.empty
        and knee_rows["max_gap_ratio"].le(MAX_GAP_RATIO).all()
        and knee_rows["max_terminal_gap_ratio"].le(MAX_TERMINAL_GAP_RATIO).all()
        and knee_rows["max_recovery_terminal_gap_ratio"].le(MAX_RECOVERY_TERMINAL_GAP_RATIO).all()
        and knee_rows["max_monthly_adjustment_ratio"].le(MAX_MONTHLY_ADJUSTMENT_RATIO).all()
    )
    supported_improvements = int(validation.get("supported_improvement_count", 0))
    near_oracle_probability = float(validation.get("probability_within_0_10_of_oracle", 0.0))
    if knee_feasible_all_splits and supported_improvements >= 1 and near_oracle_probability >= 0.80:
        evidence_status = "SUPPORTED"
    elif knee_feasible_all_splits and holdout_point_non_dominated:
        evidence_status = "PARTIAL"
    else:
        evidence_status = "NOT_SUPPORTED"
    required_interval_cols = [
        "J1_macro_loss_lower_95",
        "J1_macro_loss_upper_95",
        "J2_cvar95_macro_loss_lower_95",
        "J2_cvar95_macro_loss_upper_95",
    ]
    has_complete_intervals = bool(
        set(required_interval_cols).issubset(comparison.columns)
        and np.isfinite(comparison[required_interval_cols].astype(float).to_numpy()).all()
    )
    execution_gates = {
        "q4_sapr_scenario_no_leakage_gate": bool(
            not scenario_table.empty
            and scenario_table.loc[scenario_table["sample_split"].eq("development"), "source_end"].max() <= DEVELOPMENT_END
            and scenario_table.loc[scenario_table["sample_split"].eq("holdout"), "source_start"].min() >= HOLDOUT_START
        ),
```

```
        "q4_sapr_macro_kernel_gate": bool(valid_draw_count == BOOTSTRAP_DRAWS and valid_draw_rate >= 0.95),
        "q4_sapr_policy_identity_gate": bool(
            identity["full_rule_max_abs_gap"] < 1e-8
            and identity["full_rule_adjustment_identity_max_abs_error"] < 1e-8
            and identity["zero_policy_macro_max_abs_response"] < 1e-10
            and abs(identity["official_2026_03_gap"] - 1045.0) < 1e-8
            and abs(identity["official_2026_04_gap"] - 380.0) < 1e-8
            and abs(identity["official_2026_04_cumulative_gap"] - 1425.0) < 1e-8
            and identity["q3_policy_counterfactual_reproduction_max_abs_error"] < 1e-8
            and identity["negative_adjustment_offsets_gap"]
        ),
        "q4_sapr_pareto_selection_gate": bool(len(grid) == 1771 and int(grid["is_pareto"].sum()) > 0 and int(grid["is_knee"].sum()) == 1),
        "q4_sapr_policy_feasibility_gate": knee_feasible_all_splits,
        "q4_sapr_holdout_validation_gate": bool(
            comparison["sample_split"].astype(str).eq("holdout").any()
            and "SAPR_CVaR_knee" in set(comparison["strategy"])
            and evidence_status in {"SUPPORTED", "PARTIAL", "NOT_SUPPORTED"}
        ),
        "q4_sapr_uncertainty_gate": bool(valid_draw_count == BOOTSTRAP_DRAWS and valid_draw_rate >= 0.95 and has_complete_intervals),
        "q4_sapr_claim_strength_gate": True,
    }
    execution_status = "PASS" if all(execution_gates.values()) else "FAIL"
    knee = optimal.iloc[0].to_dict()
    payload = {
        "status": execution_status,
        "execution_status": execution_status,
        "evidence_status": evidence_status,
        "model_name": "SAPR-CVaR",
        "cutoff": CUTOFF,
        "random_seed": RANDOM_SEED,
        "bootstrap_attempts": BOOTSTRAP_DRAWS,
        "bootstrap_total_draw_attempts": total_draw_attempts,
        "valid_parameter_draw_count": valid_draw_count,
        "valid_parameter_draw_rate": valid_draw_rate,
        "parameter_sampling": "joint three-equation circular moving-block bootstrap inherited from Q3; identical time blocks for PPI, CPI and IAV",
        "holdout_non_dominated_probability": holdout_probability,
        "holdout_point_non_dominated": holdout_point_non_dominated,
        "holdout_validation": validation,
        "scenario_counts": scenario_table.groupby("sample_split")["scenario_id"].nunique().to_dict(),
        "rule_count": int(len(grid)),
        "pareto_rule_count": int(grid["is_pareto"].sum()),
        "optimal_rule": {key: clean_value(value) for key, value in knee.items()},
        "identity_checks": identity,
        "execution_gates": execution_gates,
        "allowed_claims": [
            "SAPR-CVaR is optimal only within the registered three-regime smoothing family, six-month scenario library and four-objective criterion.",
            "The selected rule can be compared with full pass-through, fixed smoothing and a 2026 temporary-control approximation under the frozen Q3 macro kernel.",
            "J3 is the average monthly outstanding adjustment gap divided by the base price; terminal and recovery gaps are separate hard constraints.",
        ],
        "forbidden_claims": [
            "global optimum across all possible Chinese fuel pricing policies",
            "complete welfare gain or fiscal cost estimate",
            "the deferred gap equals government subsidy spending or a complete welfare loss",
            "Q4 uses post-2021 data to select stress thresholds or the optimal rule",
        ],
        "sensitivity_rows": int(len(sensitivity)),
        "figure_files": [
            "figures/q4_sapr_pareto_front.png",
            "figures/q4_sapr_policy_heatmap.png",
            "figures/q4_sapr_strategy_comparison.png",
            "figures/q4_sapr_2026_macro_paths.png",
        ],
    }
    (RESULTS_DIR / "q4_sapr_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8")
    return payload


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--plots-only", action="store_true", help="Regenerate Q4 figures from saved result tables.")
```

```
    args = parser.parse_args(argv)
    apply_paper_style()
    ensure_dirs()

    if args.plots_only:
        grid = read_result("q4_sapr_policy_grid.csv")
        optimal = read_result("q4_sapr_optimal_rule.csv")
        comparison = read_result("q4_sapr_strategy_comparison.csv")
        paths = read_result("q4_sapr_macro_paths.csv")
        knee_rule = {
            "rho_normal": float(optimal["rho_normal"].iloc[0]),
            "rho_stress": float(optimal["rho_stress"].iloc[0]),
            "rho_extreme": float(optimal["rho_extreme"].iloc[0]),
        }
        plot_pareto(grid)
        plot_policy_heatmap(knee_rule)
        plot_strategy_comparison(comparison)
        plot_2026_paths(paths)
        print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 4}, ensure_ascii=False, indent=2))
        return 0

    rng = np.random.default_rng(RANDOM_SEED)
    scenario_table, scenarios, meta = build_scenarios()
    save_csv(scenario_table, "q4_sapr_scenarios.csv")
    kernel = load_macro_kernel()
    parameter_draws, valid_draw_rate, total_draw_attempts = draw_parameter_sets(kernel, rng)
    if len(parameter_draws) != BOOTSTRAP_DRAWS:
        raise ValueError(
            f"Q4 generated {len(parameter_draws)} valid macro-parameter draws from {total_draw_attempts} attempts; "
            f"{BOOTSTRAP_DRAWS} valid draws are required."
        )
    point = point_coefficients(kernel)
    weights = {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}
    stds = historical_macro_stds(weights)
    base_price = float(meta["base_price_2026_06_cny_t"])
    grid = run_grid_search(scenarios, point, base_price, stds, weights)
    for key, value in meta.items():
        grid[key] = value
    save_csv(grid, "q4_sapr_policy_grid.csv")
    knee = grid.loc[grid["is_knee"]].iloc[0]
    knee_rule = {
        "rho_normal": float(knee["rho_normal"]),
        "rho_stress": float(knee["rho_stress"]),
        "rho_extreme": float(knee["rho_extreme"]),
    }
    optimal = pd.DataFrame(
        [
            {
                "model": "SAPR-CVaR",
                "rule_id": knee["rule_id"],
                "rho_normal": knee_rule["rho_normal"],
                "rho_stress": knee_rule["rho_stress"],
                "rho_extreme": knee_rule["rho_extreme"],
                "stress_threshold_75_cny_t": meta["stress_threshold_75_cny_t"],
                "stress_threshold_95_cny_t": meta["stress_threshold_95_cny_t"],
                "base_price_2026_06_cny_t": base_price,
                "J1_macro_loss": knee["J1_macro_loss"],
                "J2_cvar95_macro_loss": knee["J2_cvar95_macro_loss"],
                "J3_avg_gap_month_burden": knee["J3_avg_gap_month_burden"],
                "J4_adjustment_volatility": knee["J4_adjustment_volatility"],
                "max_gap_ratio": knee["max_gap_ratio"],
                "max_terminal_gap_ratio": knee["max_terminal_gap_ratio"],
                "max_recovery_terminal_gap_ratio": knee["max_recovery_terminal_gap_ratio"],
                "max_monthly_adjustment_ratio": knee["max_monthly_adjustment_ratio"],
                "constraint_max_gap_ratio": MAX_GAP_RATIO,
                "constraint_max_terminal_gap_ratio": MAX_TERMINAL_GAP_RATIO,
                "constraint_max_recovery_terminal_gap_ratio": MAX_RECOVERY_TERMINAL_GAP_RATIO,
                "constraint_max_monthly_adjustment_ratio": MAX_MONTHLY_ADJUSTMENT_RATIO,
                "recovery_months": RECOVERY_MONTHS,
                "pareto_rule_count": int(grid["is_pareto"].sum()),
                "feasible_rule_count": int(grid["is_feasible"].sum()),
                "candidate_rule_count": int(len(grid)),
                "selection_rule": "minimum normalized Euclidean distance to the feasible Pareto ideal; ties by smaller average gap-month burden and higher normal pass-through",
            }
        ]
```

```
    )
    save_csv(optimal, "q4_sapr_optimal_rule.csv")
    comparison, holdout_probability, holdout_point_non_dominated = compare_strategies(
        scenarios,
        kernel,
        parameter_draws,
        knee_rule,
        base_price,
        stds,
        weights,
        rng,
        block_length=6,
    )
    save_csv(comparison, "q4_sapr_strategy_comparison.csv")
    paths = aggregate_macro_paths(scenarios, kernel, parameter_draws, knee_rule, base_price, split="war_2026")
    save_csv(paths, "q4_sapr_macro_paths.csv")
    sensitivity = run_sensitivity(grid, scenarios, kernel, parameter_draws[:500], meta, stds, rng)
    save_csv(sensitivity, "q4_sapr_sensitivity.csv")
    identity = policy_identity_checks(scenarios, base_price, kernel)
    summary = write_summary(
        scenario_table,
        grid,
        optimal,
        comparison,
        sensitivity,
        identity,
        valid_draw_rate,
        total_draw_attempts,
        len(parameter_draws),
        holdout_probability,
        holdout_point_non_dominated,
    )
    plot_pareto(grid)
    plot_policy_heatmap(knee_rule)
    plot_strategy_comparison(comparison)
    plot_2026_paths(paths)
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    return 0 if summary["execution_status"] == "PASS" else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 10: run_q4_sapr.py

3.3 数值冻结与绘图工具

```
"""Paper-oriented Matplotlib style helpers for the oil-shock figures."""

from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path

import matplotlib.pyplot as plt
from cycler import cycler


INK = "#232323"
MUTED = "#606060"
GRID = "#d9d4ca"
SPINE = "#4a4a4a"

PALETTE = {
    "ink": INK,
    "muted": MUTED,
    "grid": GRID,
    "blue": "#355c7d",
    "blue_light": "#9fb4c9",
    "gold": "#b98b3d",
    "gold_light": "#d8c49a",
    "olive": "#6f7f4f",
```

```
    "olive_light": "#b8c1a3",
    "rose": "#a65d6a",
    "rose_light": "#d4a7af",
    "slate": "#6f7582",
    "sand": "#c9b99a",
}

SERIES_COLORS = [
    PALETTE["blue"],
    PALETTE["gold"],
    PALETTE["olive"],
    PALETTE["rose"],
    PALETTE["slate"],
]


def apply_paper_style() -> None:
    plt.rcParams.update(
        {
            "figure.facecolor": "white",
            "axes.facecolor": "white",
            "savefig.facecolor": "white",
            "savefig.edgecolor": "white",
            "font.family": ["Times New Roman", "SimSun", "FangSong", "DejaVu Serif"],
            "font.serif": ["Times New Roman", "SimSun", "FangSong", "STIXGeneral", "DejaVu Serif"],
            "mathtext.fontset": "stix",
            "axes.unicode_minus": False,
            "text.color": INK,
            "axes.labelcolor": INK,
            "axes.edgecolor": SPINE,
            "axes.linewidth": 0.8,
            "axes.spines.top": False,
            "axes.spines.right": False,
            "axes.grid": True,
            "axes.axisbelow": True,
            "grid.color": GRID,
            "grid.linewidth": 0.55,
            "grid.alpha": 0.75,
            "xtick.color": MUTED,
            "ytick.color": MUTED,
            "xtick.major.width": 0.65,
            "ytick.major.width": 0.65,
            "xtick.labelsize": 9.5,
            "ytick.labelsize": 9.5,
            "axes.labelsize": 10.5,
            "axes.titlesize": 13,
            "legend.fontsize": 9.2,
            "legend.frameon": False,
            "lines.linewidth": 1.65,
            "lines.markersize": 4.2,
            "patch.edgecolor": "white",
            "axes.prop_cycle": cycler(color=SERIES_COLORS),
            "pdf.fonttype": 42,
            "ps.fonttype": 42,
        }
    )


def style_axis(ax: plt.Axes, *, ylabel: str | None = None, xlabel: str | None = None) -> None:
    if ylabel:
        ax.set_ylabel(ylabel)
    if xlabel:
        ax.set_xlabel(xlabel)
    ax.grid(True, axis="y")
    ax.grid(False, axis="x")
    ax.tick_params(length=3.2, width=0.65)
    for side in ["left", "bottom"]:
        ax.spines[side].set_color(SPINE)
        ax.spines[side].set_linewidth(0.8)


def finish_figure(
    fig: plt.Figure,
    *,
    title: str,
    subtitle: str,
```

```
    source: str,
    rect: tuple[float, float, float, float] = (0.08, 0.11, 0.98, 0.98),
) -> None:
    """Finish a paper figure while leaving its title to the LaTeX caption.

    ``title`` and ``subtitle`` remain required as generation metadata for the
    calling code, but they are deliberately not painted into the figure.
    """
    fig.tight_layout(rect=rect)
    fig._shu_caption = title  # type: ignore[attr-defined]
    fig._shu_subtitle = subtitle  # type: ignore[attr-defined]
    fig.text(rect[0], 0.035, source, ha="left", va="bottom", fontsize=8.4, color=MUTED)


def save_figure(fig: plt.Figure, path_stem: Path, *, dpi: int = 300) -> None:
    fig.savefig(path_stem.with_suffix(".png"), dpi=dpi, bbox_inches="tight", pad_inches=0.08)
    fixed_time = datetime(2026, 7, 30, tzinfo=timezone.utc)
    pdf_metadata = {
        "Creator": "SHU-OilShock-CN",
        "Producer": "Matplotlib",
        "CreationDate": fixed_time,
        "ModDate": fixed_time,
    }
    fig.savefig(
        path_stem.with_suffix(".pdf"),
        dpi=dpi,
        bbox_inches="tight",
        pad_inches=0.08,
        metadata=pdf_metadata,
    )
```

Listing 11: plot_style.py

```
"""Generate paper handoff tables and supplemental paper-ready figures.

This script reads the frozen modeling outputs and creates artifacts intended
for the writing stage. It does not estimate models or change any numerical
result used by the freeze gate.
"""

from __future__ import annotations

import json
import sys
from pathlib import Path

import matplotlib.patches as patches
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[2]
RESULTS_DIR = REPO_ROOT / "results"
REPORTS_DIR = REPO_ROOT / "reports"
FIGURES_DIR = REPO_ROOT / "figures"

sys.path.append(str(REPO_ROOT / "code" / "utils"))
from plot_style import PALETTE, apply_paper_style, finish_figure, save_figure  # noqa: E402


COUNTRY_LABEL_ZH = {
    "CHN": "中国",
    "DEU": "德国",
    "FRA": "法国",
    "ITA": "意大利",
    "ESP": "西班牙",
    "JPN": "日本",
    "KOR": "韩国",
}

OUTCOME_LABEL_ZH = {
    "brent_cny_cost_log_change_pct": "人民币原油成本",
    "china_ppi_yoy_pct": "PPI",
    "china_cpi_yoy_pct": "CPI",
    "china_iav_yoy_pct": "工业增加值",
```

```
    "china_fx_log_change_pct": "汇率",
}

SHOCK_LABEL_ZH = {
    "supply_shock": "供给冲击",
    "aggregate_demand_shock": "全球需求冲击",
    "oil_specific_risk_shock": "油价特定风险冲击",
    "OilShock": "约化油价创新",
}


def read_csv(name: str) -> pd.DataFrame:
    return pd.read_csv(RESULTS_DIR / name)


def read_json(name: str) -> dict:
    return json.loads((RESULTS_DIR / name).read_text(encoding="utf-8"))


def zh_outcome(value: str) -> str:
    return OUTCOME_LABEL_ZH.get(value, value)


def write_paper_numbers() -> None:
    risk = read_json("risk_probe_summary.json")
    q1_origin = read_csv("q1_origin_forecast.csv")
    q1_event = read_csv("q1_event_effects.csv")
    q1_structural = read_csv("q1_structural_shocks.csv")
    q1_vol = read_csv("q1_volatility_summary.csv")
    q2_metrics = read_csv("q2_transmission_metrics.csv")
    q2_gdp = read_csv("q2_gdp_validation.csv")
    q3_pass = read_csv("q3_country_pass_through.csv")
    q3_policy = read_csv("q3_policy_counterfactual.csv")
    q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
    q3_resilience = read_csv("q3_resilience_metrics.csv")
    q4_risk = read_csv("q4_price_tail_risk.csv")
    q4_backtest = read_csv("q4_risk_backtest.csv")
    q4_macro = read_csv("q4_macro_stress.csv")
    q4_policy = read_csv("q4_policy_stress.csv")
    q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
    q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
    q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
    q4_sapr_summary = read_json("q4_sapr_summary.json")

    rows: list[dict[str, object]] = []

    def add(section: str, number_id: str, value: object, unit: str, source_file: str, allowed: str, forbidden: str) -> None:
        rows.append(
            {
                "section": section,
                "number_id": number_id,
                "value": value,
                "unit": unit,
                "rounding": "论文正文保留2到3位有效数字；表格按来源文件精度",
                "source_file": source_file,
                "allowed_claim": allowed,
                "forbidden_claim": forbidden,
            }
        )

    add(
        "overall",
        "paper_finalize_allowed",
        str(risk["paper_finalize_allowed"]).lower(),
        "boolean",
        "results/risk_probe_summary.json",
        "所有核心门禁通过后可进入论文撰写",
        "不得忽略后续新增数据或代码变动后的重新冻结",
    )
    add(
        "overall",
        "blocking_probe_count",
        len(risk["blocking_probe_ids"]),
        "count",
        "results/risk_probe_summary.json",
        "当前无阻塞门禁",
```

```
        "不得把单个模型的显著性等同于全部结论成立",
    )

    for _, row in q1_origin.sort_values("horizon_months").iterrows():
        add(
            "Q1",
            f"origin_2026_02_forecast_h{int(row['horizon_months'])}",
            round(float(row["prediction_price"]), 3),
            "美元/桶",
            "results/q1_origin_forecast.csv",
            f"{row['target_period']} 的 no-change 预测价格",
            "不得把未到期的2026-08预测写成有实际误差",
        )
    for _, row in q1_event[q1_event["model"].eq("brent_usd_bbl_event_car")].iterrows():
        add(
            "Q1",
            f"{row['stage_id']}_estimate",
            round(float(row["estimate_log_return"]), 6),
            "log return",
            "results/q1_event_effects.csv",
            "交易日CAR与经验placebo可作为事件相关异常证据",
            "不得解释为严格战争净因果贡献",
        )
        add(
            "Q1",
            f"{row['stage_id']}_empirical_p",
            round(float(row["pvalue_empirical"]), 6),
            "probability",
            "results/q1_event_effects.csv",
            "经验p值使用有限样本add-one修正",
            "不得报告0或只引用常规HAC显著性",
        )
    add(
        "Q1",
        "structural_shock_nonmissing_months_min",
        int(q1_structural[["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]].notna().sum().min()),
        "months",
        "results/q1_structural_shocks.csv",
        "三类SVAR结构冲击满足不少于120个月覆盖",
        "不得再使用52期STEO残差作为主结构识别",
    )
    for _, row in q1_vol.iterrows():
        add(
            "Q1",
            f"vol_multiple_{row['war_stage']}",
            round(float(row["vol_multiple_vs_pre_median"]), 3),
            "倍",
            "results/q1_volatility_summary.csv",
            "条件波动率可描述战争阶段油价波动变化",
            "不得解释为福利损失或实体经济损失",
        )

    focus = q2_metrics[
        q2_metrics["shock"].eq("oil_specific_risk_shock")
        & q2_metrics["outcome"].isin(["brent_cny_cost_log_change_pct", "china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"])
    ]
    for _, row in focus.iterrows():
        add(
            "Q2",
            f"oil_specific_{row['outcome']}_extremum",
            round(float(row["extremum_response"]), 6),
            "百分点或对数百分点",
            "results/q2_transmission_metrics.csv",
            f"{zh_outcome(row['outcome'])} 的响应方向和量级可报告为估计结果",
            "联合区间未排除零时不得写成稳健增长损失",
        )
        if pd.notna(row.get("response_curve_area_0_12")):
            add(
                "Q2",
                f"oil_specific_{row['outcome']}_area0_12",
                round(float(row["response_curve_area_0_12"]), 6),
                "百分点·月",
                "results/q2_transmission_metrics.csv",
                f"{zh_outcome(row['outcome'])} 的0到12月响应曲线面积可报告为描述性量级",
                "不得称为累计百分点、累计产出损失或累计增长损失",
            )
```

```
    if not q2_gdp.empty:
        gdp = q2_gdp.iloc[0]
        add(
            "Q2",
            "quarterly_gdp_validation_estimate",
            round(float(gdp["estimate"]), 6),
            "百分点",
            "results/q2_gdp_validation.csv",
            "季度GDP仅作低频验证",
            "不得插值为月度主增长指标",
        )

    h6 = q3_pass[q3_pass["horizon"].eq(6)]
    for _, row in h6.sort_values("country").iterrows():
        add(
            "Q3",
            f"fuel_pass_through_h6_{row['country']}",
            round(float(row["response"]), 6),
            "累计传导率",
            "results/q3_country_pass_through.csv",
            f"{COUNTRY_LABEL_ZH.get(row['country'], row['country'])}可进入主燃油比较",
            "不得混用中国Brent-CNY代理值与他国观测零售价排名",
        )
    apr = q3_policy[q3_policy["period"].eq("2026-04")]
    if not apr.empty:
        apr0 = apr.iloc[0]
        add(
            "Q3",
            "policy_gap_2026_04_incremental",
            round(float(apr0["incremental_gasoline_gap_cny_t"]), 3),
            "元/吨",
            "results/q3_policy_counterfactual.csv",
            "4月新增调控差额可写为380元/吨",
            "不得把新增差额误写成累计差额",
        )
        add(
            "Q3",
            "policy_gap_2026_04_cumulative",
            round(float(apr0["cumulative_gasoline_gap_cny_t"]), 3),
            "元/吨",
            "results/q3_policy_counterfactual.csv",
            "4月累计规则缺口可写为1425元/吨",
            "不得把价格平滑解释为无成本福利改善",
        )
    june_macro = q3_policy_macro[q3_policy_macro["period"].eq("2026-06")]
    for _, row in june_macro.sort_values("outcome").iterrows():
        add(
            "Q3",
            f"policy_macro_gap_2026_06_{row['outcome']}",
            round(float(row["macro_counterfactual_gap_pctpt"]), 6),
            "百分点",
            "results/q3_policy_macro_counterfactual.csv",
            f"无临时调控路径对{row['outcome_label']}的情景差额可作为政策关闭反事实",
            "不得解释为已发生事实或完整福利评价",
        )
    overall = q3_resilience["overall_china_resilience_judgement"].dropna().iloc[0]
    add(
        "Q3",
        "overall_china_resilience_judgement",
        overall,
        "categorical",
        "results/q3_resilience_metrics.csv",
        "中国综合韧性当前判断为PARTIAL",
        "不得写成无条件SUPPORTED",
    )

    for _, row in q4_risk[q4_risk["model"].eq("FHS_GJR_GARCH")].sort_values("horizon_months").iterrows():
        horizon = int(row["horizon_months"])
        add(
            "Q4",
            f"fhs_median_price_h{horizon}",
            round(float(row["median_price"]), 6),
            "美元/桶",
            "results/q4_price_tail_risk.csv",
            "可作为2026-06-30信息集下的条件分布中位数",
            "不得写成确定性油价路径",
```

```
        )
        add(
            "Q4",
            f"fhs_terminal_prob_above_hist_p95_h{horizon}",
            round(float(row["terminal_prob_above_hist_p95"]), 6),
            "probability",
            "results/q4_price_tail_risk.csv",
            "可报告期末价格超过历史95%分位阈值的条件概率",
            "不得解释为事件必然发生概率或因果概率",
        )
    for _, row in q4_backtest.sort_values(["horizon_months", "model"]).iterrows():
        add(
            "Q4",
            f"backtest_pinball_{row['model']}_h{int(row['horizon_months'])}",
            round(float(row["mean_pinball_loss"]), 6),
            "美元/桶分位损失",
            "results/q4_risk_backtest.csv",
            "主方法与基线按相同滚动原点和期限比较",
            "不得只报告有利期限或隐藏基线",
        )
    for _, row in q4_macro[
        q4_macro["scenario"].eq("extreme_q95") & q4_macro["horizon"].isin([6, 12])
    ].sort_values(["outcome", "horizon"]).iterrows():
        add(
            "Q4",
            f"extreme_q95_{row['outcome']}_h{int(row['horizon'])}",
            round(float(row["conditional_response_pctpt"]), 6),
            "百分点",
            "results/q4_macro_stress.csv",
            "可作为历史95%分位油价特定风险冲击下的条件响应",
            "联合区间跨零时不得写成确定性宏观损失",
        )
    for _, row in q4_policy[q4_policy["period"].eq("2026-06")].sort_values("outcome").iterrows():
        add(
            "Q4",
            f"policy_buffer_benefit_2026_06_{row['outcome']}",
            round(float(row["policy_buffer_benefit_pctpt"]), 6),
            "百分点",
            "results/q4_policy_stress.csv",
            "可作为Q3已实现临时调控关闭反事实的政策缓冲收益重述",
            "不得外推到Q4模拟油价路径或写成完整福利收益",
        )

    if not q4_sapr_optimal.empty:
        opt = q4_sapr_optimal.iloc[0]
        for key, unit, claim in [
            ("rho_normal", "pass-through ratio", "普通状态最优传导率"),
            ("rho_stress", "pass-through ratio", "压力状态最优传导率"),
            ("rho_extreme", "pass-through ratio", "极端状态最优传导率"),
            ("J2_cvar95_macro_loss", "standardized loss", "训练样本膝点规则的95%尾部宏观损失"),
            ("J3_avg_gap_month_burden", "ratio", "训练样本膝点规则的平均标准化缺口月负担"),
        ]:
            add(
                "Q4",
                f"sapr_optimal_{key}",
                round(float(opt[key]), 6),
                unit,
                "results/q4_sapr_optimal_rule.csv",
                claim,
                "不得写成全局福利最优或不受规则族限制的最优政策",
            )
    sapr_holdout = q4_sapr_comparison[
        q4_sapr_comparison["sample_split"].eq("holdout")
        & q4_sapr_comparison["strategy"].eq("SAPR_CVaR_knee")
    ]
    if not sapr_holdout.empty:
        row = sapr_holdout.iloc[0]
        add(
            "Q4",
            "sapr_holdout_macro_loss_cvar95",
            round(float(row["J2_cvar95_macro_loss"]), 6),
            "standardized loss",
            "results/q4_sapr_strategy_comparison.csv",
            "可作为2022—2026隔离检验样本下SAPR尾部宏观损失",
            "不得声称2026以后仍必然占优",
        )
```

```
        add(
            "Q4",
            "sapr_holdout_gap_burden_ratio",
            round(float(row["J3_avg_gap_month_burden"]), 6),
            "ratio",
            "results/q4_sapr_strategy_comparison.csv",
            "可作为隔离检验样本下平均标准化缺口月负担",
            "不得解释为财政支出或完整社会成本",
        )
    add(
        "Q4",
        "sapr_holdout_non_dominated_probability",
        round(float(q4_sapr_summary["holdout_non_dominated_probability"]), 6),
        "probability",
        "results/q4_sapr_summary.json",
        "可报告训练期选出规则在检验期保持非支配的bootstrap概率",
        "不得改写成真实世界政策成功概率",
    )
    for _, row in q4_sapr_sensitivity.sort_values("variant").iterrows():
        add(
            "Q4",
            f"sapr_sensitivity_{row['variant']}_rule",
            f"({row['rho_normal']:.2f},{row['rho_stress']:.2f},{row['rho_extreme']:.2f})",
            "rule tuple",
            "results/q4_sapr_sensitivity.csv",
            "可用于说明阈值、块长和权重变动下规则是否稳定",
            "不得事后删除不利敏感性结果",
        )

    pd.DataFrame(rows).to_csv(REPORTS_DIR / "paper_numbers.csv", index=False, encoding="utf-8-sig")


def plot_q1_structural_shocks() -> None:
    df = read_csv("q1_structural_shocks.csv")
    df["period"] = pd.to_datetime(df["period"])
    columns = ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
    labels = ["供给冲击", "全球需求冲击", "油价特定风险冲击"]
    fig, ax = plt.subplots(figsize=(8.4, 4.2))
    for col, label in zip(columns, labels):
        series = df[col].rolling(3, min_periods=1).mean()
        ax.plot(df["period"], series, label=label)
    ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
    ax.set_ylabel("标准化冲击")
    ax.legend(ncol=3, loc="upper left")
    finish_figure(
        fig,
        title="问题一：历史递归SVAR结构冲击",
        subtitle="三个月滚动均值，仅用于识别与传导链输入展示",
        source="来源：EIA International、Dallas Fed IGREA、Brent、美国CPI；作者计算。",
    )
    save_figure(fig, FIGURES_DIR / "q1_structural_shocks")
    plt.close(fig)


def plot_q2_transmission_chain() -> None:
    df = read_csv("q2_transmission_metrics.csv")
    focus = df[
        df["shock"].eq("oil_specific_risk_shock")
        & df["outcome"].isin(
            [
                "brent_cny_cost_log_change_pct",
                "china_ppi_yoy_pct",
                "china_cpi_yoy_pct",
                "china_iav_yoy_pct",
                "china_fx_log_change_pct",
            ]
        )
    ].copy()
    focus["label"] = focus["outcome"].map(OUTCOME_LABEL_ZH)
    focus = focus.sort_values("extremum_response")
    fig, ax = plt.subplots(figsize=(7.8, 4.6))
    colors = [PALETTE["rose"] if v < 0 else PALETTE["blue"] for v in focus["extremum_response"]]
    ax.barh(focus["label"], focus["extremum_response"], color=colors, alpha=0.88)
    ax.axvline(0, color=PALETTE["muted"], linewidth=0.8)
    for _, row in focus.iterrows():
        ax.text(
```

```
            row["extremum_response"],
            row["label"],
            f"  {row['extremum_response']:.2f}, h={int(row['extremum_month'])}",
            va="center",
            fontsize=9,
            color=PALETTE["ink"],
        )
    ax.set_xlabel("峰值/谷值响应（百分点或对数百分点）")
    finish_figure(
        fig,
        title="问题二：油价特定风险冲击的传导摘要",
        subtitle="显示各变量峰值或谷值响应；完整区间以结果表为准",
        source="来源：results/q2_transmission_metrics.csv。",
    )
    save_figure(fig, FIGURES_DIR / "q2_transmission_chain")
    plt.close(fig)


def plot_q3_resilience_metrics() -> None:
    df = read_csv("q3_resilience_metrics.csv")
    focus = df[df["dimension"].isin(["fuel_pass_through", "cpi_peak_response", "industrial_activity_trough"])].copy()
    label_map = {
        "fuel_1m_cumulative_pass_through": "燃油1月传导",
        "fuel_3m_cumulative_pass_through": "燃油3月传导",
        "fuel_6m_cumulative_pass_through": "燃油6月传导",
        "cpi_relative_to_china": "CPI相对响应",
        "ip_relative_to_china": "工业活动相对响应",
    }
    focus["label"] = focus["metric"].map(label_map)
    focus = focus.iloc[::-1]
    y = np.arange(len(focus))
    err_left = focus["china_vs_control_median_diff"] - focus["diff_lower_95"]
    err_right = focus["diff_upper_95"] - focus["china_vs_control_median_diff"]
    fig, ax = plt.subplots(figsize=(8.2, 4.3))
    ax.barh(y, focus["china_vs_control_median_diff"], color=PALETTE["olive"], alpha=0.82)
    ax.errorbar(
        focus["china_vs_control_median_diff"],
        y,
        xerr=[err_left, err_right],
        fmt="none",
        ecolor=PALETTE["ink"],
        elinewidth=0.8,
        capsize=3,
    )
    ax.axvline(0, color=PALETTE["muted"], linewidth=0.8)
    ax.set_yticks(y)
    ax.set_yticklabels(focus["label"])
    ax.set_xlabel("中国相对六国中位数差异")
    finish_figure(
        fig,
        title="问题三：中国综合韧性指标",
        subtitle="点估计与95%区间共同决定SUPPORTED/PARTIAL/NOT_SUPPORTED",
        source="来源：results/q3_resilience_metrics.csv。",
    )
    save_figure(fig, FIGURES_DIR / "q3_resilience_metrics")
    plt.close(fig)


def plot_q3_policy_macro_counterfactual() -> None:
    df = read_csv("q3_policy_macro_counterfactual.csv")
    df["period_dt"] = pd.to_datetime(df["period"], format="%Y-%m")
    label_map = {"PPI": "PPI", "CPI": "CPI", "IAV": "工业增加值"}
    fig, ax = plt.subplots(figsize=(7.8, 4.5))
    for label, color in zip(["PPI", "CPI", "IAV"], [PALETTE["blue"], PALETTE["gold"], PALETTE["rose"]]):
        sub = df[df["outcome_label"].eq(label)].sort_values("period_dt")
        ax.plot(sub["period_dt"], sub["macro_counterfactual_gap_pctpt"], marker="o", color=color, label=label_map[label])
        ax.fill_between(sub["period_dt"], sub["lower_95"], sub["upper_95"], color=color, alpha=0.12, linewidth=0)
    ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
    ax.set_ylabel("无临时调控相对实际路径差额（百分点）")
    ax.legend(loc="upper left")
    finish_figure(
        fig,
        title="问题三：政策关闭宏观反事实",
        subtitle="在官方成品油价格层上移除临时调控缺口后传播至PPI/CPI/IAV",
        source="来源：results/q3_policy_macro_counterfactual.csv。",
    )
```

```
    save_figure(fig, FIGURES_DIR / "q3_policy_macro_counterfactual")
    plt.close(fig)


def draw_box(ax: plt.Axes, xy: tuple[float, float], text: str, width: float = 0.18, height: float = 0.16, color: str = "#f4efe6") -> None
    :
    box = patches.FancyBboxPatch(
        xy,
        width,
        height,
        boxstyle="round,pad=0.018,rounding_size=0.02",
        linewidth=0.9,
        edgecolor=PALETTE["slate"],
        facecolor=color,
    )
    ax.add_patch(box)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=10)


def arrow(ax: plt.Axes, start: tuple[float, float], end: tuple[float, float]) -> None:
    ax.annotate(
        "",
        xy=end,
        xytext=start,
        arrowprops={"arrowstyle": "->", "linewidth": 1.1, "color": PALETTE["muted"]},
    )


def save_flow(fig: plt.Figure, stem: str, title: str, subtitle: str) -> None:
    fig.subplots_adjust(left=0.04, right=0.98, bottom=0.18, top=0.92)
    fig._shu_caption = title  # type: ignore[attr-defined]
    fig._shu_subtitle = subtitle  # type: ignore[attr-defined]
    fig.text(0.04, 0.045, "来源：作者根据冻结模型流程绘制。", ha="left", va="bottom", fontsize=8.4, color=PALETTE["muted"])
    save_figure(fig, FIGURES_DIR / stem)
    plt.close(fig)


def plot_route_map() -> None:
    fig, ax = plt.subplots(figsize=(10.2, 3.6))
    ax.set_axis_off()
    boxes = [
        ((0.02, 0.55), "问题一\n预测与事件识别"),
        ((0.21, 0.55), "结构冲击\nSVAR输出"),
        ((0.40, 0.55), "问题二\n中国宏观传导"),
        ((0.59, 0.55), "问题三\n跨国韧性与政策情景"),
        ((0.78, 0.55), "问题四\n尾部风险与SAPR优化"),
        ((0.40, 0.18), "冻结校验\n风险门禁PASS"),
        ((0.64, 0.18), "论文撰写\n数字台账引用"),
    ]
    for xy, text in boxes:
        draw_box(ax, xy, text, width=0.15, color="#f4efe6")
    arrow(ax, (0.17, 0.63), (0.21, 0.63))
    arrow(ax, (0.36, 0.63), (0.40, 0.63))
    arrow(ax, (0.55, 0.63), (0.59, 0.63))
    arrow(ax, (0.74, 0.63), (0.78, 0.63))
    arrow(ax, (0.86, 0.55), (0.55, 0.34))
    arrow(ax, (0.55, 0.26), (0.64, 0.26))
    save_flow(fig, "paper_route_map", "四问统一技术路线", "模型链由预测、宏观传导和政策比较延伸到尾部风险压力测试与自适应调价优化")


def plot_event_timeline() -> None:
    fig, ax = plt.subplots(figsize=(8.6, 3.0))
    ax.set_axis_off()
    xs = [0.12, 0.36, 0.64, 0.84]
    labels = ["2026-02-28\n周末事件发生", "2026-03-02\nE1首个交易日", "2026-03-05\nE2持续中断", "2026-06-17\nE3缓和阶段"]
    ax.plot([xs[0], xs[-1]], [0.5, 0.5], color=PALETTE["slate"], linewidth=1.2)
    for x, label in zip(xs, labels):
        ax.scatter([x], [0.5], s=70, color=PALETTE["blue"], zorder=3)
        ax.text(x, 0.67, label, ha="center", va="bottom", fontsize=10)
    ax.text(0.5, 0.25, "正式推断使用CAR[0]、CAR[0,+1]、CAR[0,+2]与周末匹配placebo经验分布", ha="center", fontsize=10, color=PALETTE["
      muted"])
    save_flow(fig, "paper_event_timeline", "战争事件时间线", "周末事件统一映射到首个共同交易日")


def plot_transmission_mechanism() -> None:
    fig, ax = plt.subplots(figsize=(8.8, 3.8))
```

```
    ax.set_axis_off()
    boxes = [
        ((0.04, 0.58), "结构性\n油价冲击"),
        ((0.27, 0.58), "人民币\n原油成本"),
        ((0.50, 0.58), "PPI\n生产成本"),
        ((0.73, 0.66), "CPI\n居民价格"),
        ((0.73, 0.42), "工业活动\n增长响应"),
        ((0.50, 0.18), "季度GDP\n低频验证"),
    ]
    for xy, text in boxes:
        draw_box(ax, xy, text, color="#eef3ee")
    arrow(ax, (0.22, 0.66), (0.27, 0.66))
    arrow(ax, (0.45, 0.66), (0.50, 0.66))
    arrow(ax, (0.68, 0.66), (0.73, 0.74))
    arrow(ax, (0.68, 0.58), (0.73, 0.50))
    arrow(ax, (0.59, 0.58), (0.59, 0.34))
    save_flow(fig, "paper_transmission_mechanism", "油价向中国经济传导链", "主结果报告方向、量级与不确定性而非制造显著性")


def plot_policy_counterfactual_flow() -> None:
    fig, ax = plt.subplots(figsize=(8.8, 3.5))
    ax.set_axis_off()
    boxes = [
        ((0.04, 0.58), "官方调价\n实际路径"),
        ((0.04, 0.26), "无临时调控\n规则路径"),
        ((0.31, 0.42), "1045/380/1425\n政策缺口"),
        ((0.57, 0.42), "燃油价格—宏观\n分布滞后模型"),
        ((0.79, 0.42), "PPI/CPI/IAV\n反事实区间"),
    ]
    for xy, text in boxes:
        draw_box(ax, xy, text, color="#f3eef0")
    arrow(ax, (0.22, 0.66), (0.31, 0.53))
    arrow(ax, (0.22, 0.34), (0.31, 0.50))
    arrow(ax, (0.49, 0.50), (0.57, 0.50))
    arrow(ax, (0.75, 0.50), (0.79, 0.50))
    save_flow(fig, "paper_policy_counterfactual_flow", "中国临时调控关闭反事实流程", "情景结果代表价格平滑成本/延期负担代理而非福利判断")


def write_handoff_markdown() -> None:
    risk = read_json("risk_probe_summary.json")
    q1_origin = read_csv("q1_origin_forecast.csv")
    q1_event = read_csv("q1_event_effects.csv")
    q2_metrics = read_csv("q2_transmission_metrics.csv")
    q3_resilience = read_csv("q3_resilience_metrics.csv")
    q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
    q4_risk = read_csv("q4_price_tail_risk.csv")
    q4_backtest = read_csv("q4_risk_backtest.csv")
    q4_macro = read_csv("q4_macro_stress.csv")
    q4_policy = read_csv("q4_policy_stress.csv")
    q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
    q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
    q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
    q4_sapr_summary = read_json("q4_sapr_summary.json")

    q1_rows = []
    for _, row in q1_origin.sort_values("horizon_months").iterrows():
        actual = "未到期" if row["forecast_status"] == "FORECAST_ONLY" else f"{row['actual_price']:.2f}"
        q1_rows.append(f"- h={int(row['horizon_months'])}: 预测 {row['prediction_price']:.2f} 美元/桶，实际 {actual}。")
    q1_car = q1_event[q1_event["model"].eq("brent_usd_bbl_event_car")]
    car_rows = [
        f"- {row['stage_id']}: CAR={row['estimate_log_return']:.4f}，经验 p={row['pvalue_empirical']:.4f}。"
        for _, row in q1_car.iterrows()
    ]
    q2_focus = q2_metrics[
        q2_metrics["shock"].eq("oil_specific_risk_shock")
        & q2_metrics["outcome"].isin(["brent_cny_cost_log_change_pct", "china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"])
    ]
    q2_rows = []
    for _, row in q2_focus.iterrows():
        area_text = (
            f"，0—12月响应曲线面积 {row['response_curve_area_0_12']:.3f} 百分点·月"
            if pd.notna(row.get("response_curve_area_0_12"))
            else "，稀疏期限不计算曲线面积"
        )
        q2_rows.append(
            f"- {zh_outcome(row['outcome'])}: 峰值/谷值 {row['extremum_response']:.3f}，h={int(row['extremum_month'])}{area_text}，证据状
```

```
        态 {row['evidence_status']}。"
649         )
650     resilience_overall = q3_resilience["overall_china_resilience_judgement"].dropna().iloc[0]
651     q3_rows = []
652     for _, row in q3_resilience[q3_resilience["dimension"].isin(["fuel_price", "consumer_prices", "industrial_activity"])].iterrows():
653         china_text = f"{row['china_value']:.3f}" if pd.notna(row["china_value"]) else "不以0代替"
654         control_text = f"{row['control_median']:.3f}" if pd.notna(row["control_median"]) else "不可得"
655         q3_rows.append(f"- {row['metric']}: 中国值 {china_text}, 对照量 {control_text}, 判断 {row['judgement']}。")
656     q3_macro_rows = [
657         f"- 2026-06 {row['outcome_label']}: 无临时调控相对实际路径差额 {row['macro_counterfactual_gap_pctpt']:.3f} 个百分点, 95%区间 [{
        row['lower_95']:.3f}, {row['upper_95']:.3f}]。"
658         for _, row in q3_policy_macro[q3_policy_macro["period"].eq("2026-06")].sort_values("outcome").iterrows()
659     ]
660     q4_price_rows = [
661         f"- h={int(row['horizon_months'])}: 中位数 {row['median_price']:.2f} 美元/桶, 90%区间 [{row['p05_price']:.2f}, {row['p95_price
        ']:.2f}], 期末超过历史95%价格分位的条件概率 {100 * row['terminal_prob_above_hist_p95']:.2f}%。"
662         for _, row in q4_risk[q4_risk["model"].eq("FHS_GJR_GARCH")].sort_values("horizon_months").iterrows()
663     ]
664     q4_backtest_rows = [
665         f"- {row['model']}, h={int(row['horizon_months'])}: 平均分位损失 {row['mean_pinball_loss']:.3f}, 80%/90%覆盖率 {row['coverage_80
        ']:.3f}/{row['coverage_90']:.3f}。"
666         for _, row in q4_backtest.sort_values(["horizon_months", "model"]).iterrows()
667     ]
668     q4_macro_rows = [
669         f"- {row['outcome_label']}, h={int(row['horizon'])}: 95%分位结构冲击条件响应 {row['conditional_response_pctpt']:.3f} 个百分点, 联
        合95%区间 [{row['joint_lower_95']:.3f}, {row['joint_upper_95']:.3f}], {row['row_evidence_status']}。"
670         for _, row in q4_macro[q4_macro["scenario"].eq("extreme_q95") & q4_macro["horizon"].isin([6, 12])]
671         .sort_values(["outcome", "horizon"]).iterrows()
672     ]
673     q4_policy_rows = [
674         f"- 2026-06 {row['outcome_label']}: 政策缓冲收益 {row['policy_buffer_benefit_pctpt']:.3f} 个百分点, 95%区间 [{row['
        benefit_lower_95']:.3f}, {row['benefit_upper_95']:.3f}]。"
675         for _, row in q4_policy[q4_policy["period"].eq("2026-06")].sort_values("outcome").iterrows()
676     ]
677     q4_sapr_opt = q4_sapr_optimal.iloc[0]
678     q4_sapr_rule_text = (
679         f"- SAPR-CVaR 膝点规则: 普通/压力/极端传导率 = "
680         f"({q4_sapr_opt['rho_normal']:.2f}, {q4_sapr_opt['rho_stress']:.2f}, {q4_sapr_opt['rho_extreme']:.2f}); "
681         f"压力阈值 {q4_sapr_opt['stress_threshold_75_cny_t']:.1f} 元/吨, 极端阈值 {q4_sapr_opt['stress_threshold_95_cny_t']:.1f} 元/吨。"
682     )
683     q4_sapr_holdout_rows = [
684         f"- {row['strategy']}: 检验样本宏观损失均值 {row['J1_macro_loss']:.3f}, 95%CVaR {row['J2_cvar95_macro_loss']:.3f}, "
685         f"平均缺口月负担 {row['J3_avg_gap_month_burden']:.3f}, 调价波动率 {row['J4_adjustment_volatility']:.3f}。"
686         for _, row in q4_sapr_comparison[q4_sapr_comparison["sample_split"].eq("holdout")]
687         .sort_values("strategy").iterrows()
688     ]
689     q4_sapr_2026_rows = [
690         f"- {row['strategy']}: 2026情景宏观损失均值 {row['J1_macro_loss']:.3f}, 95%CVaR {row['J2_cvar95_macro_loss']:.3f}, "
691         f"平均缺口月负担 {row['J3_avg_gap_month_burden']:.3f}。"
692         for _, row in q4_sapr_comparison[q4_sapr_comparison["sample_split"].eq("war_2026")]
693         .sort_values("strategy").iterrows()
694     ]
695     q4_sapr_sensitivity_rows = [
696         f"- {row['variant']}: 规则=({row['rho_normal']:.2f},{row['rho_stress']:.2f},{row['rho_extreme']:.2f}), "
697         f"检验期非支配概率 {row['holdout_non_dominated_probability']:.3f}。"
698         for _, row in q4_sapr_sensitivity.sort_values("variant").iterrows()
699     ]
700
701     text = f"""# 建模到论文移交说明

[PAPER_READY]

原题三问与自拟问题四的建模、检验、数值冻结和论文移交材料已完成。当前 `overall_status={risk['overall_status']}`, `paper_finalize_allowed={
        str(risk['paper_finalize_allowed']).lower()}`, 阻塞门禁数为 {len(risk['blocking_probe_ids'])}。

## 验证命令

```powershell
python code\\utils\\freeze_results.py
python code\\utils\\verify_freeze.py
python code\\utils\\verify_freeze.py --require-final
```

最后一次运行结果均为 `PASS`。论文所有数字优先引用 `results/frozen_numbers.json`、`results/final_numbers.json` 和 `reports/paper_numbers.
        csv`。

716
```

```
## 问题一：预测与战争影响

主线：`no_change` 作为诚实胜出的主预测模型，交易日 CAR 与周末匹配 placebo 作为事件相关异常证据，递归 SVAR 提供三类结构冲击。

核心写法：

{chr(10).join(q1_rows)}
{chr(10).join(car_rows)}

可用图表：`figures/q1_forecast_1m.png`、`figures/q1_war_counterfactual.png`、`figures/q1_structural_shocks.png`、`figures/
    paper_event_timeline.png`。

禁止写法：不得把 `ARBaselineGap` 称作严格战争净贡献；不得把 ARIMA/SARIMAX 未胜出解释成建模失败。

## 问题二：中国经济增长传导

主线：结构性油价冲击 → 人民币原油成本 → PPI → CPI/工业增加值 → 季度 GDP 验证。

油价特定风险冲击下的摘要：

{chr(10).join(q2_rows)}

结论边界：当前总体仍应写成“尚未发现稳健的总体增长损失证据”。若正文使用“增长损失”，必须同时满足工业活动负响应且联合区间排除零。

可用图表：`figures/q2_irf.png`、`figures/q2_transmission_chain.png`、`figures/paper_transmission_mechanism.png`。

## 问题三：中国政策与跨国比较

主线：中国历史燃油序列统一作为受管制汽油调价指数代理并退出正式价格水平排名；CPI/IAV面板只估计六个对照国相对中国的响应差；待审计缓冲交互不
    发布定量结果；2026政策关闭情景在明确标注的代理层上动态传播至PPI/CPI/IAV。

综合韧性判断：`{resilience_overall}`。

{chr(10).join(q3_rows)}

政策关闭宏观反事实：

{chr(10).join(q3_macro_rows)}

可用图表：`figures/q3_pass_through_6m.png`、`figures/q3_panel_irf.png`、`figures/q3_resilience_metrics.png`、`figures/
    q3_policy_macro_counterfactual.png`、`figures/paper_policy_counterfactual_flow.png`。

禁止写法：不得仅凭价格传导或中国单一价格监管变量宣称“中国显著更好”；不得把价格平滑写成无成本福利改善。

## 问题四：极端油价尾部风险、政策压力测试与自适应调价规则优化

定位：该部分作为论文正式自拟问题四，不再设置其他自拟问题。问题四由两个互补模块构成：同事已有的 FHS－GJR-GARCH 尾部风险与宏观政策压力测试
    负责回答“极端冲击有多大概率、会形成多大压力”；本轮 SAPR-CVaR 自适应调价优化负责回答“在现行机制约束上应如何设置状态依赖的临时平滑
    层”。油价概率、Q2 结构冲击宏观情景、Q3 已实现政策反事实和 SAPR 策略优化是递进关系，不可简单相加为单一因果贡献。

FHS－GJR-GARCH 条件尾部预测：

{chr(10).join(q4_price_rows)}

与高斯随机游走的同口径滚动回测：

{chr(10).join(q4_backtest_rows)}

95%分位油价特定风险结构冲击的宏观压力：

{chr(10).join(q4_macro_rows)}

2026年已实现临时调控的政策缓冲重述：

{chr(10).join(q4_policy_rows)}

SAPR-CVaR 自适应调价规则：

{q4_sapr_rule_text}

隔离检验样本策略比较：

{chr(10).join(q4_sapr_holdout_rows)}

2026实际冲击情景策略比较：

```

```
{chr(10).join(q4_sapr_2026_rows)}

敏感性检验：

{chr(10).join(q4_sapr_sensitivity_rows)}

证据状态：`{q4_sapr_summary['evidence_status']}`；检验期非支配 bootstrap 概率为 `{q4_sapr_summary['holdout_non_dominated_probability']:.3f}`。

可用图表：`figures/q4_price_tail_risk.png`、`figures/q4_macro_policy_stress.png`、`figures/q4_sapr_pareto_front.png`、`figures/q4_sapr_policy_heatmap.png`、`figures/q4_sapr_strategy_comparison.png`、`figures/q4_sapr_2026_macro_paths.png`。

禁止写法：不得把尾部概率写成确定结果，不得把宏观情景写成确定性 GDP 损失，不得把 Q2 条件响应与 Q3 政策差额相加为单一因果贡献；不得把 SAPR 的注册规则族最优写成全球最优、完整福利收益或财政成本估计。

## 论文接手顺序

1. 先按 `reports/paper_numbers.csv` 抽取数值，填入摘要、问题重述、模型假设和结果表。
2. 四个问题均按“目标—数据—公式—估计—结果—检验—解释边界”写；问题四单列为自拟问题，明确从前三问结果递进而来。
3. 所有图表从 `figures/` 选择 PNG 入文，PDF 留作高清备份。
4. 写作期间如改动模型代码、输入数据或核心结果，必须重新运行冻结和 `--require-final`。
"""
    (REPORTS_DIR / "MODELING_TO_PAPER_HANDOFF.md").write_text(text, encoding="utf-8")


def main() -> None:
    apply_paper_style()
    REPORTS_DIR.mkdir(exist_ok=True)
    FIGURES_DIR.mkdir(exist_ok=True)
    write_paper_numbers()
    plot_q1_structural_shocks()
    plot_q2_transmission_chain()
    plot_q3_resilience_metrics()
    plot_q3_policy_macro_counterfactual()
    plot_route_map()
    plot_event_timeline()
    plot_transmission_mechanism()
    plot_policy_counterfactual_flow()
    write_handoff_markdown()
    print("[PAPER_READY] handoff artifacts generated.")


if __name__ == "__main__":
    main()
```

Listing 12: build_paper_handoff.py

```python
"""Freeze stage outputs and run paper-finalization risk probes."""

from __future__ import annotations

import hashlib
import importlib.metadata
import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


REPO_ROOT = Path(__file__).resolve().parents[2]
CODE_DIR = REPO_ROOT / "code"
if str(CODE_DIR) not in sys.path:
    sys.path.insert(0, str(CODE_DIR))

```

```
from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

RESULTS_DIR = REPO_ROOT / "results"
FIGURES_DIR = REPO_ROOT / "figures"
REPORTS_DIR = REPO_ROOT / "reports"
CUTOFF = "2026-06-30"
RISK_SCHEMA_VERSION = "1.0"
MANIFEST_SCHEMA_VERSION = "1.0"
STANDARD_FREEZE_SCHEMA_VERSION = 1
RANDOM_SEED = 20260730

CORE_RESULT_FILES = [
    "q1_forecast_metrics.csv",
    "q1_event_effects.csv",
    "q1_placebo_distribution.csv",
    "q1_daily_counterfactual.csv",
    "q1_monthly_shocks.csv",
    "q1_structural_shocks.csv",
    "q1_svar_diagnostics.csv",
    "q1_stationarity_diagnostics.csv",
    "q1_origin_forecast.csv",
    "q1_volatility.csv",
    "q1_volatility_summary.csv",
    "q1_robustness.csv",
    "q1_summary.json",
    "q2_ardl_baseline.csv",
    "q2_irf.csv",
    "q2_gdp_validation.csv",
    "q2_asymmetry.csv",
    "q2_robustness.csv",
    "q2_transmission_metrics.csv",
    "q2_summary.csv",
    "q2_summary.json",
    "q3_country_pass_through.csv",
    "q3_panel_irf.csv",
    "q3_buffer_interactions.csv",
    "q3_buffer_data_status.csv",
    "q3_policy_counterfactual.csv",
    "q3_policy_macro_counterfactual.csv",
    "q3_resilience_metrics.csv",
    "q3_robustness.csv",
    "q3_summary.csv",
    "q3_summary.json",
    "q3_policy_macro_kernel.csv",
    "q3_policy_macro_covariance.csv",
    "q3_policy_macro_bootstrap_draws.csv",
    "q4_risk_probe.json",
    "q4_price_tail_risk.csv",
    "q4_risk_backtest_origins.csv",
    "q4_risk_backtest.csv",
    "q4_risk_backtest_paired.csv",
    "q4_macro_stress.csv",
    "q4_policy_stress.csv",
    "q4_summary.json",
    "q4_sapr_scenarios.csv",
    "q4_sapr_policy_grid.csv",
    "q4_sapr_optimal_rule.csv",
    "q4_sapr_strategy_comparison.csv",
    "q4_sapr_macro_paths.csv",
    "q4_sapr_sensitivity.csv",
    "q4_sapr_holdout_validation.csv",
    "q4_sapr_holdout_paired_summary.csv",
    "q4_sapr_holdout_validation_summary.json",
    "q4_sapr_summary.json",
]

PROCESSED_INPUT_FILES = [
    "p0_daily_market.csv",
    "p0_monthly_market.csv",
    "global_oil_svar_monthly.csv",
    "model_daily_q1.csv",
    "model_monthly_q1.csv",
    "model_monthly_cn.csv",
```

```
    "model_quarterly_cn.csv",
    "model_country_monthly.csv",
    "oecd_g20_cpi_monthly.csv",
    "oecd_kei_ip_monthly.csv",
    "eu_eurosuper95_monthly.csv",
    "france_eurosuper95_monthly.csv",
    "italy_eurosuper95_monthly.csv",
    "spain_eurosuper95_monthly.csv",
    "germany_eurosuper95_monthly.csv",
    "japan_regular_gasoline_monthly.csv",
    "korea_regular_gasoline_monthly.csv",
    "cn_fuel_adjustment_events.csv",
    "cn_fuel_policy_events.csv",
    "china_fuel_policy_monthly.csv",
    "china_fuel_proxy_monthly.csv",
    "china_regulated_gasoline_monthly.csv",
    "eurostat_spain_hicp_yoy_monthly.csv",
    "country_policy_buffers_annual.csv",
    "nbs_iav_monthly.csv",
    "nbs_ppi_monthly.csv",
    "nbs_manual_import_metadata.json",
    "release_date_matrix.csv",
]

CODE_FILES = [
    "code/data_download/download_p0.py",
    "code/data_download/p0_sources.json",
    "code/data_processing/build_p0_datasets.py",
    "code/data_processing/build_model_panels.py",
    "code/data_processing/import_nbs_manual.py",
    "code/problem1/run_q1.py",
    "code/problem2/run_q2.py",
    "code/problem3/run_q3.py",
    "code/problem4/run_q4.py",
    "code/problem4/run_q4_sapr.py",
    "code/schemas/frozen_numbers.schema.json",
    "code/schemas/reproducibility_manifest.schema.json",
    "code/schemas/risk_probe_summary.schema.json",
    "code/utils/build_paper_handoff.py",
    "code/utils/freeze_results.py",
    "code/utils/plot_style.py",
    "code/utils/verify_freeze.py",
    "data/DATA_DICTIONARY.csv",
    "data/SOURCE_REGISTRY.csv",
    "requirements.in",
    "requirements.lock.txt",
]

COUNTRY_LABEL_ZH = {
    "CHN": "中国",
    "DEU": "德国",
    "FRA": "法国",
    "ITA": "意大利",
    "ESP": "西班牙",
    "JPN": "日本",
    "KOR": "韩国",
}
CORE_PACKAGES = ["numpy", "pandas", "scipy", "statsmodels", "linearmodels", "matplotlib", "requests", "openpyxl", "xlrd", "jsonschema"]


def read_csv(filename: str) -> pd.DataFrame:
    path = RESULTS_DIR / filename
    if not path.exists():
        return pd.DataFrame()
    return pd.read_csv(path)


def load_json(filename: str) -> dict[str, Any]:
    path = RESULTS_DIR / filename
    if not path.exists():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))


def sha256_bytes(payload: bytes) -> str:
    return hashlib.sha256(payload).hexdigest()
```

```python


def sha256_file(path: Path) -> str:
    payload = path.read_bytes()
    if path.suffix.lower() in {".csv", ".json", ".md", ".py", ".txt"}:
        payload = payload.replace(b"\r\n", b"\n").replace(b"\r", b"\n")
    return hashlib.sha256(payload).hexdigest()


def sha256_json(payload: dict[str, Any]) -> str:
    return sha256_bytes(json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":")).encode("utf-8"))


def clean_value(value: Any) -> Any:
    if isinstance(value, (np.integer,)):
        return int(value)
    if isinstance(value, (np.floating, float)):
        if not np.isfinite(value):
            return None
        return round(float(value), 8)
    if pd.isna(value):
        return None
    return value


def records(frame: pd.DataFrame) -> list[dict[str, Any]]:
    return [{key: clean_value(value) for key, value in row.items()} for row in frame.to_dict("records")]


def hash_existing(base_dir: Path, filenames: list[str]) -> dict[str, str]:
    hashes: dict[str, str] = {}
    for filename in filenames:
        path = base_dir / filename
        if path.exists():
            hashes[filename] = sha256_file(path)
    return hashes


def collect_figures() -> list[str]:
    if not FIGURES_DIR.exists():
        return []
    return sorted(path.name for path in FIGURES_DIR.glob("*.png") if path.name.startswith(("q", "data_")))


def git_commit() -> str:
    result = subprocess.run(["git", "rev-parse", "HEAD"], cwd=REPO_ROOT, text=True, capture_output=True, check=False)
    return result.stdout.strip() if result.returncode == 0 else "UNKNOWN"


def environment_snapshot() -> dict[str, Any]:
    packages: dict[str, str] = {}
    for package in CORE_PACKAGES:
        try:
            packages[package] = importlib.metadata.version(package)
        except importlib.metadata.PackageNotFoundError:
            packages[package] = "NOT_INSTALLED"
    lock_path = REPO_ROOT / "requirements.lock.txt"
    return {
        "python_version": platform.python_version(),
        "python_executable": sys.executable,
        "platform": platform.platform(),
        "packages": packages,
        "requirements_lock_sha256": sha256_file(lock_path) if lock_path.exists() else None,
    }


def replay_environment_snapshot() -> dict[str, Any]:
    lock_path = REPO_ROOT / "requirements.lock.txt"
    lock_text = lock_path.read_text(encoding="utf-8") if lock_path.exists() else ""
    python_version = None
    packages: dict[str, str] = {}
    for line in lock_text.splitlines():
        stripped = line.strip()
        if stripped.startswith("# Python:"):
            python_version = stripped.split(":", 1)[1].strip()
        elif stripped and not stripped.startswith("#") and "==" in stripped:
```

```
            package, version = stripped.split("==", 1)
            if package.strip().lower() in CORE_PACKAGES:
                packages[package.strip().lower()] = version.strip()
    return {
        "python_version": python_version,
        "packages": packages,
        "requirements_lock_sha256": sha256_file(lock_path) if lock_path.exists() else None,
    }


def probe(
    probe_id: str,
    status: str,
    metric: Any,
    threshold: str,
    evidence_files: list[str],
    message: str,
    severity: str = "blocking",
) -> dict[str, Any]:
    return {
        "probe_id": probe_id,
        "status": status,
        "severity": severity,
        "metric": metric,
        "threshold": threshold,
        "evidence_files": evidence_files,
        "message": message,
    }


def combined_status(probes: list[dict[str, Any]]) -> str:
    statuses = {row["status"] for row in probes}
    if "FAIL" in statuses:
        return "FAIL"
    if "CONDITIONAL" in statuses:
        return "CONDITIONAL"
    return "PASS"


def has_nonmissing(path: Path, column: str, minimum: int) -> bool:
    if not path.exists():
        return False
    frame = pd.read_csv(path)
    return column in frame.columns and int(pd.to_numeric(frame[column], errors="coerce").notna().sum()) >= minimum


def bool_series(series: pd.Series) -> pd.Series:
    if pd.api.types.is_bool_dtype(series):
        return series
    return series.astype(str).str.lower().isin(["true", "1", "yes"])


def build_risk_probe_summary(output_hashes: dict[str, str], input_hashes: dict[str, str], code_hashes: dict[str, str]) -> dict[str, Any]:
    probes: list[dict[str, Any]] = []

    metrics = read_csv("q1_forecast_metrics.csv")
    if metrics.empty or "model_status" not in metrics.columns:
        probes.append(probe("q1_forecast_baseline_gate", "FAIL", "missing model_status", "all forecast metrics carry model_status", ["
      results/q1_forecast_metrics.csv"], "Q1 forecast metrics cannot prove that advanced models were downgraded."))
    else:
        advanced_non_pass = metrics.loc[metrics["model"].ne("no_change") & metrics["model_status"].ne("PASS")]
        no_change_rows = metrics.loc[metrics["model"].eq("no_change")]
        no_change_pass = not no_change_rows.empty and no_change_rows["model_status"].eq("PASS").all()
        status = "PASS" if no_change_pass else "FAIL"
        if no_change_rows.empty:
            status = "FAIL"
        probes.append(
            probe(
                "q1_forecast_baseline_gate",
                status,
                {
                    "selected_model": "no_change",
                    "advanced_non_pass_rows": int(len(advanced_non_pass)),
                    "no_change_rows": int(len(no_change_rows)),
                    "no_change_pass": bool(no_change_pass),
                },
```

```
                "no-change rows exist and pass the pre-registered rolling backtest; advanced model failures are recorded but non-blocking",
                ["results/q1_forecast_metrics.csv", "results/q1_summary.json"],
                "No-change is selected as the main forecast. ARIMA/SARIMAX/ETS/Theta failures are empirical findings, not a paper-finalization blocker when the baseline wins honestly.",
            )
        )

    q1_summary = load_json("q1_summary.json")
    event_calendar = q1_summary.get("event_calendar", {})
    event_ok = event_calendar.get("e1_trading_start") == "2026-03-02" and event_calendar.get("e2_trading_start") == "2026-03-05"
    effects = read_csv("q1_event_effects.csv")
    stage_e1 = effects.loc[
        effects.get("model", pd.Series(dtype=str)).eq("brent_usd_bbl_stage_dummy")
        & effects.get("stage_id", pd.Series(dtype=str)).eq("E1")
    ] if not effects.empty else pd.DataFrame()
    car_rows = effects.loc[
        effects.get("model", pd.Series(dtype=str)).eq("brent_usd_bbl_event_car")
        & effects.get("stage_id", pd.Series(dtype=str)).isin(["E1_CAR_0", "E1_CAR_0_1", "E1_CAR_0_2"])
        & effects.get("identification_status", pd.Series(dtype=str)).eq("PASS")
    ] if not effects.empty else pd.DataFrame()
    stage_e1_descriptive = not stage_e1.empty and stage_e1["identification_status"].eq("DESCRIPTIVE_ONLY").all()
    car_p_nonzero = not car_rows.empty and pd.to_numeric(car_rows.get("pvalue_empirical", pd.Series(dtype=float)), errors="coerce").dropna().gt(0).all()
    probes.append(
        probe(
            "q1_event_trading_day_gate",
            "PASS" if event_ok and stage_e1_descriptive and len(car_rows) == 3 and car_p_nonzero else "FAIL",
            {
                "event_calendar": event_calendar,
                "stage_e1_descriptive_only": bool(stage_e1_descriptive),
                "brent_car_rows": int(len(car_rows)),
                "car_empirical_p_nonzero": bool(car_p_nonzero),
            },
            "E1 maps to 2026-03-02; stage dummy is descriptive only; CAR[0], CAR[0,+1], CAR[0,+2] carry formal event inference",
            ["results/q1_event_effects.csv", "results/q1_summary.json"],
            "E1 weekend event has been remapped to the first common trading day. Single-window conventional significance is blocked; formal evidence uses recursive normal-return CAR and matched placebo p-values.",
        )
    )

    placebos = read_csv("q1_placebo_distribution.csv")
    empirical_p = q1_summary.get("placebo_min_empirical_pvalue")
    if placebos.empty or empirical_p is None or not 0.0 < float(empirical_p) <= 1.0:
        placebo_status = "FAIL"
    elif float(empirical_p) <= 0.0:
        placebo_status = "FAIL"
    else:
        placebo_status = "PASS"
    probes.append(
        probe(
            "q1_placebo_distribution_gate",
            placebo_status,
            {"placebo_rows": int(len(placebos)), "min_empirical_pvalue": empirical_p},
            "matched weekend placebo distribution exists and finite-sample corrected empirical p-values lie in (0, 1]",
            ["results/q1_placebo_distribution.csv", "results/q1_summary.json"],
            "Matched weekend placebo evidence uses the add-one correction, so a finite pseudo-event sample cannot report an empirical p-value of exactly zero.",
        )
    )

    shocks = read_csv("q1_monthly_shocks.csv")
    language_ok = not shocks.empty and "ARBaselineGap" in shocks.columns and "WarPremium" not in shocks.columns
    probes.append(
        probe(
            "q1_counterfactual_language_gate",
            "PASS" if language_ok else "FAIL",
            {"has_ARBaselineGap": bool("ARBaselineGap" in shocks.columns), "has_WarPremium": bool("WarPremium" in shocks.columns)},
            "descriptive AR baseline field exists and old causal premium field is absent",
            ["results/q1_monthly_shocks.csv", "results/q1_daily_counterfactual.csv"],
            "The event-path gap is labeled as ARBaselineGap rather than a war premium or causal no-war contribution.",
        )
    )

    structural_cols = {"supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock", "reduced_form_shock", "source_vintage"}
    structural_ok = not shocks.empty and structural_cols.issubset(shocks.columns)
```

```
    core_structural_cols = ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
    core_structural_counts = (
        {
            column: int(pd.to_numeric(shocks[column], errors="coerce").notna().sum())
            for column in core_structural_cols
        }
        if structural_ok
        else {}
    )
    core_structural_months = min(core_structural_counts.values()) if core_structural_counts else 0
    reduced_form_months = int(pd.to_numeric(shocks.get("reduced_form_shock", pd.Series(dtype=float)), errors="coerce").notna().sum()) if
      structural_ok else 0
    probes.append(
        probe(
            "q1_structural_shock_coverage_gate",
            "PASS" if structural_ok and core_structural_months >= 120 else "CONDITIONAL",
            {
                "has_columns": bool(structural_ok),
                "core_structural_months": core_structural_months,
                "core_structural_counts": core_structural_counts,
                "reduced_form_months": reduced_form_months,
                "minimum_core_structural_months": 120,
            },
            "supply, aggregate-demand and oil-specific-risk shocks each have at least 120 nonmissing monthly observations",
            ["results/q1_monthly_shocks.csv", "results/q1_structural_shocks.csv"],
            "Reduced-form oil returns do not substitute for full structural coverage. Q2/Q3 structural interpretation remains conditional
       until the three structural shock series cover enough history.",
        )
    )
    probes.append(
        probe(
            "q2_structural_shock_identification",
            "PASS" if structural_ok and core_structural_months >= 120 else "CONDITIONAL",
            {
                "has_columns": bool(structural_ok),
                "core_structural_months": core_structural_months,
                "core_structural_counts": core_structural_counts,
                "reduced_form_months": reduced_form_months,
            },
            "Q1 exports supply, aggregate-demand and oil-specific-risk shocks for Q2/Q3; each core structural shock has at least 120
      months",
            ["results/q1_monthly_shocks.csv", "results/q1_structural_shocks.csv"],
            "Q2/Q3 use structural shock columns when available and retain reduced-form OilShock only as robustness. Current EIA STEO
      decomposition is too short for full-history structural claims.",
        )
    )

    svar_diag = read_csv("q1_svar_diagnostics.csv")
    selected_svar = svar_diag.loc[svar_diag.get("is_selected", pd.Series(dtype=bool)).astype(str).str.lower().isin(["true", "1"])] if not
       svar_diag.empty else pd.DataFrame()
    svar_ok = (
        not selected_svar.empty
        and bool(selected_svar["is_stable"].astype(str).str.lower().isin(["true", "1"]).all())
        and core_structural_months >= 120
    )
    probes.append(
        probe(
            "q1_svar_identification_gate",
            "PASS" if svar_ok else "FAIL",
            {
                "selected_rows": int(len(selected_svar)),
                "selected_lag": clean_value(selected_svar["candidate_lags"].iloc[0]) if not selected_svar.empty else None,
                "selected_is_stable": bool(selected_svar["is_stable"].astype(str).str.lower().isin(["true", "1"]).all()) if not
      selected_svar.empty else False,
                "core_structural_months": core_structural_months,
            },
            "Q1 historical recursive VAR has one selected stable specification and enough structural-shock coverage",
            ["results/q1_svar_diagnostics.csv", "results/q1_structural_shocks.csv"],
            "SVAR shocks must be generated from the full-history ex-post input rather than the short STEO residual proxy.",
        )
    )

    origin_forecast = read_csv("q1_origin_forecast.csv")
    origin_ok = (
        not origin_forecast.empty
        and sorted(pd.to_numeric(origin_forecast.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int).
```

```
      tolist()) == [1, 3, 6]
473       and origin_forecast.loc[origin_forecast["horizon_months"].eq(6), "forecast_status"].astype(str).eq("FORECAST_ONLY").all()
474   )
475   probes.append(
476       probe(
477           "q1_origin_forecast_completeness_gate",
478           "PASS" if origin_ok else "FAIL",
479           {
480               "rows": int(len(origin_forecast)),
481               "horizons": sorted(pd.to_numeric(origin_forecast.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna()
      .astype(int).tolist()) if not origin_forecast.empty else [],
482           },
483           "Q1 2026-02 origin forecast table contains 1, 3 and 6 month horizons, with 2026-08 marked forecast-only",
484           ["results/q1_origin_forecast.csv"],
485           "The competition cutoff is 2026-06-30, so the six-month target from 2026-02 must be retained as a forecast-only row rather
      than removed.",
486       )
487   )
488
489   nbs_iav_ok = has_nonmissing(
490       REPO_ROOT / "data" / "processed" / "nbs_iav_monthly.csv",
491       "china_iav_yoy_pct",
492       120,
493   )
494   nbs_ppi_ok = has_nonmissing(
495       REPO_ROOT / "data" / "processed" / "nbs_ppi_monthly.csv",
496       "china_ppi_yoy_pct",
497       120,
498   )
499   probes.append(
500       probe(
501           "q2_nbs_macro_completeness_gate",
502           "PASS" if nbs_iav_ok and nbs_ppi_ok else "CONDITIONAL",
503           {"nbs_iav_monthly": nbs_iav_ok, "nbs_ppi_monthly": nbs_ppi_ok},
504           "NBS IAV and PPI monthly histories are present with enough observations",
505           ["data/processed/nbs_iav_monthly.csv", "data/processed/nbs_ppi_monthly.csv", "results/q2_summary.json"],
506           "Q2 remains conditional until official IAV and PPI histories enter the processed layer; no interpolation is used to fill the
      gap.",
507       )
508   )
509
510   q2_summary = load_json("q2_summary.json")
511   q2_irf = read_csv("q2_irf.csv")
512   q2_required_cols = {
513       "shock",
514       "joint_lower_95",
515       "joint_upper_95",
516       "ci95_contains_zero",
517       "joint_ci95_contains_zero",
518       "fdr_qvalue",
519       "supports_growth_loss_language",
520   }
521   q2_structural_shocks_present = not q2_irf.empty and bool(
522       {"supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"}.intersection(set(q2_irf.get("shock", pd.Series(dtype=str)).
      dropna())))
523   )
524   q2_guard_ok = (
525       not q2_irf.empty
526       and q2_required_cols.issubset(q2_irf.columns)
527       and q2_structural_shocks_present
528       and bool(q2_summary.get("conclusion_guardrail"))
529   )
530   probes.append(
531       probe(
532           "q2_claim_strength_gate",
533           "PASS" if q2_guard_ok else "FAIL",
534           {
535               "irf_rows": int(len(q2_irf)),
536               "has_required_inference_columns": bool(q2_required_cols.issubset(q2_irf.columns)),
537               "structural_shocks_present": bool(q2_structural_shocks_present),
538               "guardrail_present": bool(q2_summary.get("conclusion_guardrail")),
539           },
540           "LP inference flags, structural-shock labels and conclusion guardrails are present",
541           ["results/q2_irf.csv", "results/q2_summary.json"],
542           "Q2 output distinguishes structural shock categories from reduced-form robustness and blocks unsupported growth-loss wording.
      ",
```

```
        )
    )

    q3_pass = read_csv("q3_country_pass_through.csv")
    china_regulated = REPO_ROOT / "data" / "processed" / "china_regulated_gasoline_monthly.csv"
    china_official_months = 0
    if china_regulated.exists():
        china_regulated_frame = pd.read_csv(china_regulated)
        china_official_months = int(
            pd.to_numeric(
                china_regulated_frame.get("china_regulated_gasoline_cny_per_ton", pd.Series(dtype=float)),
                errors="coerce",
            )
            .notna()
            .sum()
        )
    if q3_pass.empty or "included_in_main_comparison" not in q3_pass.columns:
        q3_comp_status = "FAIL"
        q3_metric: Any = "missing comparability fields"
    else:
        chn_rows = q3_pass.loc[q3_pass["country"].eq("CHN")]
        chn_main = bool_series(chn_rows["included_in_main_comparison"]) if not chn_rows.empty else pd.Series(dtype=bool)
        main_countries = sorted(q3_pass.loc[bool_series(q3_pass["included_in_main_comparison"]), "country"].dropna().unique().tolist())
        china_proxy_excluded = (
            not chn_rows.empty
            and not bool(chn_main.any())
            and chn_rows["price_measure_type"].astype(str).str.contains("proxy", case=False, na=False).all()
        )
        q3_comp_status = "PASS" if china_proxy_excluded and len(main_countries) >= 6 else "FAIL"
        q3_metric = {
            "main_countries": main_countries,
            "china_rows": chn_rows[["horizon", "included_in_main_comparison", "price_measure_type", "observed_or_regulated"]].to_dict("
      records"),
            "china_proxy_excluded": bool(china_proxy_excluded),
            "china_official_price_months": china_official_months,
            "minimum_official_price_months": 48,
        }
    probes.append(
        probe(
            "q3_china_comparability",
            q3_comp_status,
            q3_metric,
            "China reconstructed adjustment-index proxy is excluded and at least six comparable control countries remain in the formal
      ranking",
            ["results/q3_country_pass_through.csv", "data/processed/model_country_monthly.csv", "data/processed/
      china_regulated_gasoline_monthly.csv"],
            "Q3 reports China only as a proxy sensitivity and does not overstate it as an official retail-price level.",
        )
    )

    q2_metrics = read_csv("q2_transmission_metrics.csv")
    iav_horizons = (
        sorted(pd.to_numeric(q2_irf.loc[q2_irf.get("outcome", pd.Series(dtype=str)).eq("china_iav_yoy_pct"), "horizon"], errors="coerce")
      .dropna().astype(int).unique().tolist())
        if not q2_irf.empty and "outcome" in q2_irf.columns
        else []
    )
    q2_inference_ok = (
        not q2_irf.empty
        and not q2_metrics.empty
        and iav_horizons == [0, 3, 6, 12]
        and q2_irf.get("inference_band", pd.Series(dtype=str)).astype(str).str.contains("bootstrap", case=False, na=False).all()
        and {"evidence_status", "allows_growth_loss_language", "response_curve_area_0_6", "response_curve_area_0_12", "area_unit"}.
      issubset(q2_metrics.columns)
        and not q2_metrics.loc[q2_metrics["outcome"].eq("china_iav_yoy_pct"), ["response_curve_area_0_6", "response_curve_area_0_12"]].
      notna().any().any()
    )
    probes.append(
        probe(
            "q2_inference_consistency_gate",
            "PASS" if q2_inference_ok else "FAIL",
            {
                "irf_rows": int(len(q2_irf)),
                "transmission_metric_rows": int(len(q2_metrics)),
                "iav_horizons": iav_horizons,
                "all_irf_bootstrap": bool(q2_irf.get("inference_band", pd.Series(dtype=str)).astype(str).str.contains("bootstrap", case=
```

```
        False, na=False).all()) if not q2_irf.empty else False,
            },
            "Q2 LP intervals, p-values/FDR family and transmission metrics use the bootstrap interface, and IAV uses the fixed 0/3/6/12 month grid",
            ["results/q2_irf.csv", "results/q2_transmission_metrics.csv"],
            "Q2 may remain substantively inconclusive, but the inference source and response horizons must be internally consistent.",
        )
    )

    q3_buffer = read_csv("q3_buffer_interactions.csv")
    q3_buffer_status = read_csv("q3_buffer_data_status.csv")
    q3_buffer_ok = (
        q3_buffer.empty
        and len(q3_buffer_status) == 4
        and q3_buffer_status.get("data_status", pd.Series(dtype=str)).eq("FAIL_FOR_MAIN_UNAUDITED_PROXY").all()
    )
    probes.append(
        probe(
            "q3_policy_buffer_interaction_gate",
            "PASS" if q3_buffer_ok else "FAIL",
            {"buffer_rows": int(len(q3_buffer)), "withheld_status_rows": int(len(q3_buffer_status))},
            "Unaudited country-year buffer proxies are rejected from the formal quantitative result",
            ["results/q3_buffer_interactions.csv", "results/q3_buffer_data_status.csv", "data/processed/country_policy_buffers_annual.csv"],
            "Q3 fails closed instead of producing interaction estimates from normalized proxy sequences.",
        )
    )

    q3_policy = read_csv("q3_policy_counterfactual.csv")
    april = q3_policy.loc[q3_policy.get("period", pd.Series(dtype=str)).eq("2026-04")] if not q3_policy.empty else pd.DataFrame()
    policy_ok = False
    policy_metric: dict[str, Any] = {}
    if not april.empty:
        inc = float(april["incremental_gasoline_gap_cny_t"].iloc[0])
        cum = float(april["cumulative_gasoline_gap_cny_t"].iloc[0])
        response = float(april["response"].iloc[0])
        policy_metric = {"april_incremental": inc, "april_cumulative": cum, "april_response": response}
        policy_ok = abs(inc - 380.0) < 1e-6 and abs(cum - 1425.0) < 1e-6 and abs(response - 1425.0) < 1e-6
    probes.append(
        probe(
            "q3_policy_gap_annotation_gate",
            "PASS" if policy_ok else "FAIL",
            policy_metric,
            "April annotation separates incremental 380 from cumulative 1425 CNY/tonne",
            ["results/q3_policy_counterfactual.csv", "figures/q3_policy_counterfactual.png"],
            "Policy scenario fields distinguish incremental and cumulative gaps, matching the revised chart annotation.",
        )
    )

    policy_proxy = not q3_policy.empty and q3_policy.get("price_layer_status", pd.Series(dtype=str)).astype(str).str.contains("proxy", case=False, na=False).all()
    probes.append(
        probe(
            "q3_policy_counterfactual_price_layer",
            "PASS" if policy_proxy else "FAIL",
            {
                "policy_rows": int(len(q3_policy)),
                "uses_labeled_proxy_price_layer": bool(policy_proxy),
            },
            "China policy scenario explicitly labels the historical adjustment-index proxy and separately audits 2026 notice gaps",
            ["results/q3_policy_counterfactual.csv", "data/processed/china_regulated_gasoline_monthly.csv"],
            "The historical layer cannot be called a fixed-product official retail cap; scenario claims remain conditional.",
        )
    )

    q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
    macro_outcomes = set(q3_policy_macro.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) if not q3_policy_macro.empty else set()
    macro_policy_ok = (
        not q3_policy_macro.empty
        and {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}.issubset(macro_outcomes)
        and q3_policy_macro.get("price_layer_status", pd.Series(dtype=str)).astype(str).str.contains("proxy", case=False, na=False).all()
        and {"lower_95", "upper_95", "macro_counterfactual_gap_pctpt", "fuel_return_gap", "bootstrap_reps", "bootstrap_block_length"}.issubset(q3_policy_macro.columns)
        and pd.to_numeric(q3_policy_macro["bootstrap_block_length"], errors="coerce").eq(6).all()
    )
```

```
    probes.append(
        probe(
            "q3_policy_macro_propagation_gate",
            "PASS" if macro_policy_ok else "FAIL",
            {
                "rows": int(len(q3_policy_macro)),
                "outcomes": sorted(macro_outcomes),
                "has_interval_columns": bool({"lower_95", "upper_95", "macro_counterfactual_gap_pctpt"}.issubset(q3_policy_macro.columns)
      ) if not q3_policy_macro.empty else False,
            },
            "Q3 policy-close scenario dynamically propagates proxy fuel-return gaps to PPI, CPI and IAV with joint time-block intervals",
            ["results/q3_policy_macro_counterfactual.csv", "results/q3_policy_counterfactual.csv"],
            "The conditional proxy scenario must use the 0..6 distributed-lag kernel and recursive outcome state, not static elasticity
      multiplication.",
        )
    )

    buffer_table_path = REPO_ROOT / "data" / "processed" / "country_policy_buffers_annual.csv"
    buffer_table = pd.read_csv(buffer_table_path) if buffer_table_path.exists() else pd.DataFrame()
    required_buffers = ["oil_import_dependency", "oil_intensity", "import_source_hhi", "fuel_price_regulation"]
    source_text = " ".join(buffer_table.get("source_url", pd.Series(dtype=str)).astype(str).tolist()).lower()
    buffer_data_ok = q3_buffer.empty and len(q3_buffer_status) == 4 and ("proxy" in source_text or "pending audited" in source_text)
    probes.append(
        probe(
            "q3_buffer_data_completeness_gate",
            "PASS" if buffer_data_ok else "FAIL",
            {
                "buffer_table_rows": int(len(buffer_table)),
                "buffer_interaction_rows": int(len(q3_buffer)),
                "buffers_in_results": [],
                "required_buffers": required_buffers,
            },
            "Q3 detects the unaudited proxy buffer table and withholds all four interaction results",
            ["data/processed/country_policy_buffers_annual.csv", "results/q3_buffer_interactions.csv", "results/q3_buffer_data_status.csv
      "],
            "No quantitative policy-buffer claim is released from normalized or pending-audit inputs.",
        )
    )

    q3_panel = read_csv("q3_panel_irf.csv")
    panel_ok = (
        not q3_panel.empty
        and q3_panel.get("reference_country", pd.Series(dtype=str)).astype(str).eq("CHN").all()
        and q3_panel.get("response_type", pd.Series(dtype=str)).astype(str).str.contains("control_country_minus_china", na=False).all()
        and len(set(q3_panel.get("country", pd.Series(dtype=str)).dropna())) >= 6
    )
    probes.append(
        probe(
            "q3_panel_identification_gate",
            "PASS" if panel_ok else "FAIL",
            {
                "panel_rows": int(len(q3_panel)),
                "countries": sorted(q3_panel.get("country", pd.Series(dtype=str)).dropna().unique().tolist()) if not q3_panel.empty else
      [],
                "reference_country_values": sorted(q3_panel.get("reference_country", pd.Series(dtype=str)).dropna().unique().tolist()) if
       not q3_panel.empty else [],
            },
            "Q3 panel LP identifies control-country responses relative to China under full time fixed effects",
            ["results/q3_panel_irf.csv"],
            "With year-month fixed effects, country absolute shock slopes are not separately interpretable; the formal panel output must
      be relative to a reference country.",
        )
    )

    q3_resilience = read_csv("q3_resilience_metrics.csv")
    resilience_ok = (
        not q3_resilience.empty
        and {"fuel_price", "consumer_prices", "industrial_activity", "policy_counterfactual_macro"}.issubset(
            set(q3_resilience.get("dimension", pd.Series(dtype=str)).dropna().unique().tolist())
        )
        and "overall_china_resilience_judgement" in q3_resilience.columns
        and q3_resilience["overall_china_resilience_judgement"].astype(str).eq("INCONCLUSIVE").all()
    )
    probes.append(
        probe(
            "q3_resilience_output_gate",
```

```
            "PASS" if resilience_ok else "FAIL",
            {
                "rows": int(len(q3_resilience)),
                "dimensions": sorted(q3_resilience.get("dimension", pd.Series(dtype=str)).dropna().unique().tolist()) if not q3_resilience.empty else [],
                "overall": q3_resilience.get("overall_china_resilience_judgement", pd.Series(dtype=str)).dropna().iloc[0] if not q3_resilience.empty and "overall_china_resilience_judgement" in q3_resilience else None,
            },
            "Q3 exports exactly three economic resilience dimensions plus a separate policy scenario and fails closed overall",
            ["results/q3_resilience_metrics.csv"],
            "The third question must not turn three fuel horizons into three votes or use invalid median confidence intervals.",
        )
    )

    q4_probe = load_json("q4_risk_probe.json")
    q4_probe_checks = q4_probe.get("checks", {})
    q4_probe_ok = q4_probe.get("status") == "PASS" and bool(q4_probe_checks) and all(q4_probe_checks.values())
    probes.append(
        probe(
            "q4_method_risk_probe_gate",
            "PASS" if q4_probe_ok else "FAIL",
            {
                "probe_status": q4_probe.get("status"),
                "checks": q4_probe_checks,
                "gjr_persistence": q4_probe.get("metrics", {}).get("gjr_persistence"),
                "seed_probability_max_abs_difference": q4_probe.get("metrics", {}).get("seed_probability_max_abs_difference"),
            },
            "Q4 main method and Gaussian baseline pass input, stationarity, non-degeneracy, replay, perturbation and interface probes",
            ["results/q4_risk_probe.json", "code/problem4/run_q4.py"],
            "The self-defined extension may run fully only after its pre-registered low-cost probe passes.",
        )
    )

    q4_risk = read_csv("q4_price_tail_risk.csv")
    q4_backtest = read_csv("q4_risk_backtest.csv")
    q4_backtest_paired = read_csv("q4_risk_backtest_paired.csv")
    probability_cols = [
        "terminal_prob_above_hist_p90",
        "terminal_prob_above_hist_p95",
        "path_prob_cross_hist_p90",
        "path_prob_cross_hist_p95",
    ]
    q4_models = set(q4_risk.get("model", pd.Series(dtype=str)).dropna().unique().tolist())
    q4_horizons = sorted(pd.to_numeric(q4_risk.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int).unique().tolist())
    q4_probability_finite = bool(
        not q4_risk.empty
        and all(column in q4_risk.columns for column in probability_cols)
        and np.isfinite(q4_risk[probability_cols].apply(pd.to_numeric, errors="coerce").to_numpy()).all()
        and ((q4_risk[probability_cols].apply(pd.to_numeric, errors="coerce") >= 0) & (q4_risk[probability_cols].apply(pd.to_numeric, errors="coerce") <= 1)).all().all()
    )
    q4_non_degenerate = bool(
        not q4_risk.empty
        and {"p05_price", "p95_price"}.issubset(q4_risk.columns)
        and (pd.to_numeric(q4_risk["p95_price"], errors="coerce") > pd.to_numeric(q4_risk["p05_price"], errors="coerce")).all()
    )
    q4_backtest_ok = bool(
        not q4_backtest.empty
        and {"origins", "all_no_future_information", "mean_pinball_loss"}.issubset(q4_backtest.columns)
        and pd.to_numeric(q4_backtest["origins"], errors="coerce").ge(36).all()
        and bool_series(q4_backtest["all_no_future_information"]).all()
        and np.isfinite(pd.to_numeric(q4_backtest["mean_pinball_loss"], errors="coerce")).all()
        and not q4_backtest_paired.empty
        and pd.to_numeric(q4_backtest_paired.get("block_length", pd.Series(dtype=float)), errors="coerce").ge(6).all()
    )
    q4_probability_ok = (
        q4_models == {"FHS_GJR_GARCH", "Gaussian_random_walk"}
        and q4_horizons == [1, 3, 6]
        and q4_probability_finite
        and q4_non_degenerate
        and q4_backtest_ok
    )
    probes.append(
        probe(
            "q4_probability_calibration_gate",
```

```
            "PASS" if q4_probability_ok else "FAIL",
            {
                "models": sorted(q4_models),
                "horizons": q4_horizons,
                "probabilities_finite_and_bounded": q4_probability_finite,
                "distributions_non_degenerate": q4_non_degenerate,
                "backtest_rows": int(len(q4_backtest)),
                "backtest_min_origins": clean_value(pd.to_numeric(q4_backtest.get("origins", pd.Series(dtype=float)), errors="coerce").min()) if not q4_backtest.empty else None,
                "backtest_no_future_information": bool(q4_backtest_ok),
            },
            "FHS and Gaussian baseline each produce 1/3/6-month non-degenerate bounded tail forecasts with at least ten leakage-free backtest origins",
            ["results/q4_price_tail_risk.csv", "results/q4_risk_backtest.csv", "results/q4_risk_backtest_origins.csv"],
            "Q4 probability claims require a comparable baseline, rolling calibration evidence and no future information at any origin.",
        )
    )

    q4_macro = read_csv("q4_macro_stress.csv")
    q4_policy = read_csv("q4_policy_stress.csv")
    q4_summary = load_json("q4_summary.json")
    q4_layers_ok = bool(
        not q4_macro.empty
        and set(q4_macro.get("scenario", pd.Series(dtype=str)).dropna().unique().tolist()) == {"moderate_q75", "severe_q90", "extreme_q95"}
        and set(q4_macro.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) == {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}
        and "joint_ci95_contains_zero" in q4_macro.columns
        and not q4_policy.empty
        and set(q4_policy.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) == {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}
        and q4_summary.get("evidence_status") == "CONDITIONAL_STRESS_TEST"
        and bool(q4_summary.get("guardrail"))
    )
    probes.append(
        probe(
            "q4_evidence_layer_guardrail",
            "PASS" if q4_layers_ok else "FAIL",
            {
                "macro_rows": int(len(q4_macro)),
                "macro_scenarios": sorted(q4_macro.get("scenario", pd.Series(dtype=str)).dropna().unique().tolist()) if not q4_macro.empty else [],
                "macro_outcomes": sorted(q4_macro.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) if not q4_macro.empty else [],
                "policy_rows": int(len(q4_policy)),
                "summary_evidence_status": q4_summary.get("evidence_status"),
                "guardrail_present": bool(q4_summary.get("guardrail")),
            },
            "Q4 keeps price probabilities, structural-shock macro scenarios and realized policy counterfactuals as three linked but non-additive evidence layers",
            ["results/q4_macro_stress.csv", "results/q4_policy_stress.csv", "results/q4_summary.json"],
            "The extension cannot turn conditional Q2 responses or Q3 policy scenarios into deterministic losses or one combined causal estimate.",
        )
    )

    q4_sapr_scenarios = read_csv("q4_sapr_scenarios.csv")
    q4_sapr_grid = read_csv("q4_sapr_policy_grid.csv")
    q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
    q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
    q4_sapr_paths = read_csv("q4_sapr_macro_paths.csv")
    q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
    q4_sapr_summary = load_json("q4_sapr_summary.json")
    q3_kernel = read_csv("q3_policy_macro_kernel.csv")
    q3_covariance = read_csv("q3_policy_macro_covariance.csv")
    q3_joint_draws = read_csv("q3_policy_macro_bootstrap_draws.csv")
    sapr_outcomes = {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}

    q4_sapr_dev = q4_sapr_scenarios.loc[q4_sapr_scenarios.get("sample_split", pd.Series(dtype=str)).eq("development")] if not q4_sapr_scenarios.empty else pd.DataFrame()
    q4_sapr_holdout = q4_sapr_scenarios.loc[q4_sapr_scenarios.get("sample_split", pd.Series(dtype=str)).eq("holdout")] if not q4_sapr_scenarios.empty else pd.DataFrame()
    q4_sapr_anchor_count = int(q4_sapr_scenarios.loc[bool_series(q4_sapr_scenarios.get("is_2026_anchor", pd.Series(dtype=bool))), "scenario_id"].nunique()) if not q4_sapr_scenarios.empty else 0
    q4_sapr_scenario_ok = (
        not q4_sapr_dev.empty
```

```
        and not q4_sapr_holdout.empty
        and str(q4_sapr_dev["source_end"].max()) <= "2021-12"
        and str(q4_sapr_holdout["source_start"].min()) >= "2022-01"
        and q4_sapr_anchor_count == 1
    )
    probes.append(
        probe(
            "q4_sapr_scenario_no_leakage_gate",
            "PASS" if q4_sapr_scenario_ok else "FAIL",
            {
                "development_scenarios": int(q4_sapr_dev["scenario_id"].nunique()) if not q4_sapr_dev.empty else 0,
                "holdout_scenarios": int(q4_sapr_holdout["scenario_id"].nunique()) if not q4_sapr_holdout.empty else 0,
                "development_max_source_end": str(q4_sapr_dev["source_end"].max()) if not q4_sapr_dev.empty else None,
                "holdout_min_source_start": str(q4_sapr_holdout["source_start"].min()) if not q4_sapr_holdout.empty else None,
                "anchor_2026_scenarios": q4_sapr_anchor_count,
            },
            "SAPR thresholds and rule selection use only 2013-03 to 2021-12 development scenarios; 2022-2026 remains holdout",
            ["results/q4_sapr_scenarios.csv"],
            "The adaptive rule must not use the 2026 conflict period or post-2021 holdout data to tune thresholds or choose the rule.",
        )
    )

    q4_sapr_kernel_ok = (
        sapr_outcomes.issubset(set(q3_kernel.get("outcome", pd.Series(dtype=str)).dropna().tolist()))
        and sapr_outcomes.issubset(set(q3_covariance.get("outcome", pd.Series(dtype=str)).dropna().tolist()))
        and q3_joint_draws.get("draw", pd.Series(dtype=float)).nunique() >= 2000
        and int(q4_sapr_summary.get("valid_parameter_draw_count", 0)) == 2000
        and float(q4_sapr_summary.get("valid_parameter_draw_rate", 0.0)) >= 0.95
    )
    probes.append(
        probe(
            "q4_sapr_macro_kernel_gate",
            "PASS" if q4_sapr_kernel_ok else "FAIL",
            {
                "kernel_outcomes": sorted(set(q3_kernel.get("outcome", pd.Series(dtype=str)).dropna().tolist())) if not q3_kernel.empty else [],
                "covariance_outcomes": sorted(set(q3_covariance.get("outcome", pd.Series(dtype=str)).dropna().tolist())) if not q3_covariance.empty else [],
                "valid_parameter_draw_count": q4_sapr_summary.get("valid_parameter_draw_count"),
                "valid_parameter_draw_rate": q4_sapr_summary.get("valid_parameter_draw_rate"),
                "parameter_sampling": q4_sapr_summary.get("parameter_sampling"),
            },
            "SAPR reads the frozen Q3 macro kernel and propagates 2000 stable joint parameter samples",
            ["results/q3_policy_macro_kernel.csv", "results/q3_policy_macro_covariance.csv", "results/q3_policy_macro_bootstrap_draws.csv", "results/q4_sapr_summary.json"],
            "SAPR is the policy-optimization layer of Q4 and must reuse Q3 macro kernels rather than choosing a new macro model.",
        )
    )

    q4_sapr_identity = q4_sapr_summary.get("identity_checks", {})
    q4_sapr_identity_ok = bool(
        q4_sapr_identity
        and abs(float(q4_sapr_identity.get("full_rule_max_abs_gap", 1.0))) < 1e-8
        and abs(float(q4_sapr_identity.get("full_rule_adjustment_identity_max_abs_error", 1.0))) < 1e-8
        and abs(float(q4_sapr_identity.get("zero_policy_macro_max_abs_response", 1.0))) < 1e-10
        and bool(q4_sapr_identity.get("negative_adjustment_offsets_gap"))
        and abs(float(q4_sapr_identity.get("official_2026_03_gap", 0.0)) - 1045.0) < 1e-8
        and abs(float(q4_sapr_identity.get("official_2026_04_gap", 0.0)) - 380.0) < 1e-8
        and abs(float(q4_sapr_identity.get("official_2026_04_cumulative_gap", 0.0)) - 1425.0) < 1e-8
        and abs(float(q4_sapr_identity.get("q3_policy_counterfactual_reproduction_max_abs_error", 1.0))) < 1e-8
    )
    probes.append(
        probe(
            "q4_sapr_policy_identity_gate",
            "PASS" if q4_sapr_identity_ok else "FAIL",
            q4_sapr_identity,
            "SAPR accounting identities hold and official 2026 policy gaps reproduce 1045, 380 and 1425 yuan/ton",
            ["results/q4_sapr_summary.json", "data/processed/china_fuel_policy_monthly.csv"],
            "The adaptive optimization is invalid unless full pass-through, deferred-gap accumulation, downward offset and Q3 macro propagation identities are exact.",
        )
    )

    q4_sapr_pareto_ok = (
        len(q4_sapr_grid) == 1771
        and "is_pareto" in q4_sapr_grid.columns
```

```
        and "is_feasible" in q4_sapr_grid.columns
        and "is_knee" in q4_sapr_grid.columns
        and int(bool_series(q4_sapr_grid["is_pareto"]).sum()) > 0
        and int(bool_series(q4_sapr_grid["is_knee"]).sum()) == 1
        and bool_series(q4_sapr_grid.loc[bool_series(q4_sapr_grid["is_knee"]), "is_feasible"]).all()
        and len(q4_sapr_optimal) == 1
    )
    probes.append(
        probe(
            "q4_sapr_pareto_selection_gate",
            "PASS" if q4_sapr_pareto_ok else "FAIL",
            {
                "rule_rows": int(len(q4_sapr_grid)),
                "pareto_rows": int(bool_series(q4_sapr_grid.get("is_pareto", pd.Series(dtype=bool))).sum()) if not q4_sapr_grid.empty
      else 0,
                "knee_rows": int(bool_series(q4_sapr_grid.get("is_knee", pd.Series(dtype=bool))).sum()) if not q4_sapr_grid.empty else 0,
                "optimal_rows": int(len(q4_sapr_optimal)),
            },
            "SAPR enumerates 1771 monotone rules, enforces gap/recovery/adjustment constraints, and selects one 1%-epsilon Pareto knee",
            ["results/q4_sapr_policy_grid.csv", "results/q4_sapr_optimal_rule.csv"],
            "The policy rule must be selected by deterministic enumeration and Pareto-knee screening, not by an unstable heuristic.",
        )
    )

    q4_sapr_holdout_strategies = set(q4_sapr_comparison.loc[q4_sapr_comparison.get("sample_split", pd.Series(dtype=str)).eq("holdout"), "
      strategy"].dropna().tolist()) if not q4_sapr_comparison.empty else set()
    holdout_validation = load_json("q4_sapr_holdout_validation_summary.json")
    q4_sapr_holdout_ok = (
        {"full_mechanism", "uniform_75_smoothing", "temporary_2026_approx", "SAPR_CVaR_knee"}.issubset(q4_sapr_holdout_strategies)
        and q4_sapr_summary.get("evidence_status") in {"SUPPORTED", "PARTIAL", "NOT_SUPPORTED"}
        and "holdout_non_dominated_probability" in q4_sapr_summary
        and int(holdout_validation.get("block_length", 0)) >= 6
        and int(holdout_validation.get("supported_improvement_count", 0)) >= 1
        and (RESULTS_DIR / "q4_sapr_holdout_paired_summary.csv").exists()
    )
    probes.append(
        probe(
            "q4_sapr_holdout_validation_gate",
            "PASS" if q4_sapr_holdout_ok else "FAIL",
            {
                "holdout_strategies": sorted(q4_sapr_holdout_strategies),
                "evidence_status": q4_sapr_summary.get("evidence_status"),
                "holdout_non_dominated_probability": q4_sapr_summary.get("holdout_non_dominated_probability"),
                "block_length": holdout_validation.get("block_length"),
                "supported_improvement_count": holdout_validation.get("supported_improvement_count"),
                "point_regret_vs_oracle": holdout_validation.get("point_regret_vs_oracle"),
            },
            "SAPR reports preregistered holdout comparisons without using holdout rows for rule selection",
            ["results/q4_sapr_strategy_comparison.csv", "results/q4_sapr_holdout_paired_summary.csv", "results/
      q4_sapr_holdout_validation_summary.json", "results/q4_sapr_summary.json"],
            "A negative or partial holdout result is acceptable; the gate checks evaluation integrity rather than forcing improvement.",
        )
    )

    q4_sapr_interval_cols = [
        "J1_macro_loss_lower_95",
        "J1_macro_loss_upper_95",
        "J2_cvar95_macro_loss_lower_95",
        "J2_cvar95_macro_loss_upper_95",
    ]
    q4_sapr_uncertainty_ok = bool(
        int(q4_sapr_summary.get("valid_parameter_draw_count", 0)) == 2000
        and set(q4_sapr_interval_cols).issubset(q4_sapr_comparison.columns)
        and np.isfinite(q4_sapr_comparison[q4_sapr_interval_cols].astype(float).to_numpy()).all()
        and not q4_sapr_paths.empty
        and not q4_sapr_sensitivity.empty
    )
    probes.append(
        probe(
            "q4_sapr_uncertainty_gate",
            "PASS" if q4_sapr_uncertainty_ok else "FAIL",
            {
                "valid_parameter_draw_count": q4_sapr_summary.get("valid_parameter_draw_count"),
                "comparison_rows": int(len(q4_sapr_comparison)),
                "macro_path_rows": int(len(q4_sapr_paths)),
                "sensitivity_rows": int(len(q4_sapr_sensitivity)),
```

```
                "has_interval_columns": bool(set(q4_sapr_interval_cols).issubset(q4_sapr_comparison.columns)),
            },
            "SAPR comparison, macro paths and sensitivity outputs carry the registered uncertainty results",
            ["results/q4_sapr_strategy_comparison.csv", "results/q4_sapr_macro_paths.csv", "results/q4_sapr_sensitivity.csv", "results/q4_sapr_summary.json"],
            "SAPR uncertainty must come from the frozen 2000-draw parameter propagation and registered sensitivity variants.",
        )
    )

    q4_sapr_forbidden = " ".join(q4_sapr_summary.get("forbidden_claims", []))
    q4_sapr_claim_ok = (
        q4_sapr_summary.get("execution_status") == "PASS"
        and bool(q4_sapr_summary.get("allowed_claims"))
        and "global optimum" in q4_sapr_forbidden
        and "fiscal cost" in q4_sapr_forbidden
        and "post-2021" in q4_sapr_forbidden
    )
    probes.append(
        probe(
            "q4_sapr_claim_strength_gate",
            "PASS" if q4_sapr_claim_ok else "FAIL",
            {
                "execution_status": q4_sapr_summary.get("execution_status"),
                "evidence_status": q4_sapr_summary.get("evidence_status"),
                "allowed_claims": q4_sapr_summary.get("allowed_claims", []),
                "forbidden_claims": q4_sapr_summary.get("forbidden_claims", []),
            },
            "SAPR restricts claims to the registered rule family and forbids global/welfare/fiscal overclaims",
            ["results/q4_sapr_summary.json"],
            "The optimization layer should add decision value without overstating policy causality or welfare conclusions.",
        )
    )

    chart_sources = [
        REPO_ROOT / "code" / "problem1" / "run_q1.py",
        REPO_ROOT / "code" / "problem2" / "run_q2.py",
        REPO_ROOT / "code" / "problem3" / "run_q3.py",
        REPO_ROOT / "code" / "problem4" / "run_q4.py",
        REPO_ROOT / "code" / "problem4" / "run_q4_sapr.py",
    ]
    old_terms = [
        "Actual Brent",
        "No-war counterfactual",
        "Q1 war-premium counterfactual",
        "USD per barrel",
        "95% interval",
        "Months after oil shock",
        "Cumulative coefficient",
        "Actual proxy",
        "No-control proxy",
        "Data overview:",
    ]
    leftovers: list[str] = []
    for path in chart_sources:
        text = path.read_text(encoding="utf-8")
        leftovers.extend([term for term in old_terms if term in text])
    figures = collect_figures()
    style_source = (REPO_ROOT / "code" / "utils" / "plot_style.py").read_text(encoding="utf-8")
    title_painted = "fig.text(rect[0], 0.965, title" in style_source or "fig.suptitle(" in style_source
    probes.append(
        probe(
            "figure_localization_gate",
            "PASS" if len(figures) >= 10 and not leftovers and not title_painted else "FAIL",
            {
                "figure_pngs": figures,
                "old_visible_terms": sorted(set(leftovers)),
                "large_title_painted_inside_figure": title_painted,
            },
            "at least ten figure PNGs exist, visible labels are localized, and large titles are left to LaTeX captions",
            ["figures", "code/problem1/run_q1.py", "code/problem2/run_q2.py", "code/problem3/run_q3.py", "code/problem4/run_q4.py", "code/utils/plot_style.py"],
            "Figure axes and legends are localized to Chinese; large titles are not painted into the figure canvas.",
        )
    )

    manifest_path = REPO_ROOT / "data" / "raw" / "source_manifest.json"
```

```
    manifest_metric: dict[str, Any] = {"manifest_exists": manifest_path.exists()}
    manifest_ok = False
    if manifest_path.exists():
        manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
        cached_missing = []
        status_counts: dict[str, int] = {}
        for row in manifest:
            status_counts[row.get("status", "")] = status_counts.get(row.get("status", ""), 0) + 1
            if row.get("status") in {"DOWNLOADED", "CACHED"}:
                local_path = row.get("local_path", "")
                if not local_path or not (REPO_ROOT / local_path).exists():
                    cached_missing.append(row.get("artifact_id", ""))
        manifest_metric = {"status_counts": status_counts, "cached_missing": cached_missing}
        manifest_ok = not cached_missing
    probes.append(
        probe(
            "source_provenance_integrity",
            "PASS" if manifest_ok else "FAIL",
            manifest_metric,
            "source_manifest has no CACHED/DOWNLOADED rows whose local snapshot is absent",
            ["data/raw/source_manifest.json", "data/raw/source_manifest.csv"],
            "Raw-source provenance cannot claim a cache exists unless the file is present and hashable. REMOTE_ONLY records keep missing
      browser/official snapshots visible.",
        )
    )

    lock_path = REPO_ROOT / "requirements.lock.txt"
    env = environment_snapshot()
    lock_text = lock_path.read_text(encoding="utf-8") if lock_path.exists() else ""
    pinned_lines = [line.strip() for line in lock_text.splitlines() if line.strip() and not line.lstrip().startswith("#")]
    env_ok = "# Python: 3.11.9" in lock_text and bool(pinned_lines) and all("==" in line for line in pinned_lines)
    probes.append(
        probe(
            "environment_lock_gate",
            "PASS" if env_ok else "CONDITIONAL",
            {
                "target_python_version": "3.11.9",
                "generation_python_version": env["python_version"],
                "exact_package_pins": len(pinned_lines),
                "requirements_lock_sha256": env["requirements_lock_sha256"],
            },
            "Python 3.11.9 and exact dependency lock are recorded",
            ["requirements.in", "requirements.lock.txt"],
            "The replay target is pinned independently of the interpreter used only to package and verify artifacts.",
        )
    )

    hash_ok = len(output_hashes) == len(CORE_RESULT_FILES) and len(input_hashes) >= 10 and len(code_hashes) == len(CODE_FILES)
    probes.append(
        probe(
            "freeze_hash_coverage_gate",
            "PASS" if hash_ok else "FAIL",
            {"output_hash_count": len(output_hashes), "input_hash_count": len(input_hashes), "model_code_hash_count": len(code_hashes)},
            "all core result files, processed model inputs and model code files are hashed",
            ["results/reproducibility_manifest.json", "results/frozen_numbers.json"],
            "The reproducibility manifest records model outputs, processed inputs and model code so raw snapshots that cannot be
      redistributed do not prevent reproducibility checks and later code edits invalidate stale freezes.",
        )
    )

    overall = combined_status(probes)
    return {
        "schema_version": RISK_SCHEMA_VERSION,
        "overall_status": overall,
        "paper_finalize_allowed": overall == "PASS",
        "cutoff": CUTOFF,
        "random_seed": RANDOM_SEED,
        "probes": probes,
        "blocking_probe_ids": [row["probe_id"] for row in probes if row["status"] != "PASS" and row["severity"] == "blocking"],
        "core_results_present": {filename: (RESULTS_DIR / filename).exists() for filename in CORE_RESULT_FILES},
        "figure_pngs": collect_figures(),
        "output_hash_count": len(output_hashes),
        "input_hash_count": len(input_hashes),
        "model_code_hash_count": len(code_hashes),
    }

```

```python

def extract_final_numbers() -> dict[str, Any]:
    q1_metrics = read_csv("q1_forecast_metrics.csv")
    q1_events = read_csv("q1_event_effects.csv")
    q1_shocks = read_csv("q1_monthly_shocks.csv")
    q1_origin = read_csv("q1_origin_forecast.csv")
    q1_svar = read_csv("q1_svar_diagnostics.csv")
    q1_vol_summary = read_csv("q1_volatility_summary.csv")
    q1_robust = read_csv("q1_robustness.csv")
    q2_ardl = read_csv("q2_ardl_baseline.csv")
    q2_irf = read_csv("q2_irf.csv")
    q2_gdp = read_csv("q2_gdp_validation.csv")
    q2_transmission = read_csv("q2_transmission_metrics.csv")
    q2_robust = read_csv("q2_robustness.csv")
    q3_pass = read_csv("q3_country_pass_through.csv")
    q3_irf = read_csv("q3_panel_irf.csv")
    q3_buffer = read_csv("q3_buffer_interactions.csv")
    q3_policy = read_csv("q3_policy_counterfactual.csv")
    q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
    q3_resilience = read_csv("q3_resilience_metrics.csv")
    q4_risk = read_csv("q4_price_tail_risk.csv")
    q4_backtest = read_csv("q4_risk_backtest.csv")
    q4_macro = read_csv("q4_macro_stress.csv")
    q4_policy = read_csv("q4_policy_stress.csv")
    q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
    q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
    q4_sapr_paths = read_csv("q4_sapr_macro_paths.csv")
    q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
    q3_robust = read_csv("q3_robustness.csv")
    q4_summary = load_json("q4_summary.json")
    q4_sapr_summary = load_json("q4_sapr_summary.json")

    final: dict[str, Any] = {
        "cutoff": CUTOFF,
        "status_note": "three original tasks plus one self-defined extension; not a paper-final causal freeze unless risk gate passes",
        "q1": {},
        "q2": {},
        "q3": {},
        "q4": {},
    }
    if not q1_metrics.empty:
        final["q1"]["selected_forecast_model"] = "no_change"
        final["q1"]["forecast_best_by_rmse"] = records(q1_metrics.sort_values(["RMSE", "MAE"]).head(3))
        final["q1"]["forecast_metrics"] = records(q1_metrics.sort_values(["horizon", "model"]))
    if not q1_events.empty:
        final["q1"]["brent_event_stage_effects"] = records(
            q1_events.loc[q1_events["model"].eq("brent_usd_bbl_stage_dummy")].sort_values("stage_id")
        )
        final["q1"]["brent_event_car"] = records(
            q1_events.loc[q1_events["model"].eq("brent_usd_bbl_event_car")].sort_values("stage_id")
        )
        final["q1"]["wti_robustness_effects"] = records(
            q1_events.loc[q1_events["model"].eq("wti_usd_bbl_stage_dummy")].sort_values("stage_id")
        )
    if not q1_shocks.empty:
        event_months = q1_shocks.loc[q1_shocks["ARBaselineGap"].abs().gt(0)]
        final["q1"]["ar_baseline_gap_months"] = records(event_months[["period", "ARBaselineGap", "ARBaselineGap_days"]])
    if not q1_origin.empty:
        final["q1"]["origin_2026_02_forecast"] = records(q1_origin.sort_values("horizon_months"))
    if not q1_svar.empty:
        final["q1"]["svar_diagnostics"] = records(q1_svar.sort_values("candidate_lags"))
    if not q1_vol_summary.empty:
        final["q1"]["volatility_summary"] = records(q1_vol_summary)
    if not q1_robust.empty:
        final["q1"]["robustness_rows"] = int(len(q1_robust))
        final["q1"]["robustness_types"] = sorted(q1_robust["robustness_type"].dropna().unique().tolist()) if "robustness_type" in
      q1_robust else []

    if not q2_ardl.empty:
        final["q2"]["ardl_main"] = records(q2_ardl.sort_values(["outcome", "term"]))
    if not q2_irf.empty:
        final["q2"]["lp_selected_horizons"] = records(
            q2_irf.loc[q2_irf["horizon"].isin([0, 6, 12])].sort_values(["outcome", "horizon"])
        )
    if not q2_gdp.empty:
        final["q2"]["gdp_validation"] = records(q2_gdp)
```

```
    if not q2_transmission.empty:
        final["q2"]["transmission_metrics"] = records(q2_transmission.sort_values(["shock", "outcome"]))
    if not q2_robust.empty:
        final["q2"]["robustness"] = records(q2_robust.sort_values(["outcome", "lag_max", "exclude_covid"]))

    if not q3_pass.empty:
        main_pass = q3_pass.loc[bool_series(q3_pass["included_in_main_comparison"])] if "included_in_main_comparison" in q3_pass else
      q3_pass
        final["q3"]["pass_through_h6_main_comparison"] = records(main_pass.loc[main_pass["horizon"].eq(6)].sort_values("country"))
        final["q3"]["china_proxy_sensitivity_h6"] = records(q3_pass.loc[q3_pass["country"].eq("CHN") & q3_pass["horizon"].eq(6)])
    if not q3_irf.empty:
        final["q3"]["panel_lp_selected_horizons"] = records(
            q3_irf.loc[q3_irf["horizon"].isin([0, 6, 12])].sort_values(["outcome", "country", "horizon"])
        )
    if not q3_buffer.empty:
        final["q3"]["buffer_interactions_selected_horizons"] = records(
            q3_buffer.loc[q3_buffer["horizon"].isin([0, 6, 12])].sort_values(["outcome", "buffer", "horizon"])
        )
    if not q3_policy.empty:
        final["q3"]["policy_counterfactual"] = records(q3_policy)
        final["q3"]["policy_max_cumulative_gap_cny_t"] = clean_value(q3_policy["cumulative_gasoline_gap_cny_t"].max())
        final["q3"]["policy_max_cpi_gap_pctpt"] = clean_value(q3_policy["cpi_counterfactual_gap_pctpt"].max())
    if not q3_policy_macro.empty:
        final["q3"]["policy_macro_counterfactual"] = records(q3_policy_macro.sort_values(["period", "outcome"]))
    if not q3_resilience.empty:
        final["q3"]["resilience_metrics"] = records(q3_resilience)
    if not q3_robust.empty:
        final["q3"]["robustness_rows"] = int(len(q3_robust))
        final["q3"]["robustness_types"] = sorted(q3_robust["robustness_type"].dropna().unique().tolist()) if "robustness_type" in
      q3_robust else []
    if not q4_risk.empty:
        final["q4"]["price_tail_risk"] = records(q4_risk.sort_values(["model", "horizon_months"]))
    if not q4_backtest.empty:
        final["q4"]["probabilistic_backtest"] = records(q4_backtest.sort_values(["horizon_months", "model"]))
    if not q4_macro.empty:
        final["q4"]["extreme_q95_macro_stress"] = records(
            q4_macro.loc[q4_macro["scenario"].eq("extreme_q95") & q4_macro["horizon"].isin([6, 12])]
            .sort_values(["outcome", "horizon"])
        )
    if not q4_policy.empty:
        final["q4"]["policy_buffer_2026_06"] = records(
            q4_policy.loc[q4_policy["period"].eq("2026-06")].sort_values("outcome")
        )
    if q4_summary:
        final["q4"]["evidence_status"] = q4_summary.get("evidence_status")
        final["q4"]["tail_risk_status"] = q4_summary.get("problem_status")
        final["q4"]["backtest_preferred_model_by_mean_pinball"] = q4_summary.get("backtest_preferred_model_by_mean_pinball")
        final["q4"]["guardrail"] = q4_summary.get("guardrail")
    if not q4_sapr_optimal.empty:
        final["q4"]["sapr_optimal_rule"] = records(q4_sapr_optimal)
    if not q4_sapr_comparison.empty:
        final["q4"]["sapr_holdout_strategy_comparison"] = records(
            q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("holdout")].sort_values("strategy")
        )
        final["q4"]["sapr_2026_strategy_comparison"] = records(
            q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("war_2026")].sort_values("strategy")
        )
    if not q4_sapr_paths.empty:
        final["q4"]["sapr_2026_macro_paths"] = records(q4_sapr_paths.sort_values(["strategy", "month_index"]))
    if not q4_sapr_sensitivity.empty:
        final["q4"]["sapr_sensitivity"] = records(q4_sapr_sensitivity.sort_values("variant"))
    if q4_sapr_summary:
        final["q4"]["sapr_execution_status"] = q4_sapr_summary.get("execution_status")
        final["q4"]["sapr_evidence_status"] = q4_sapr_summary.get("evidence_status")
        final["q4"]["sapr_holdout_non_dominated_probability"] = q4_sapr_summary.get("holdout_non_dominated_probability")
        final["q4"]["sapr_allowed_claims"] = q4_sapr_summary.get("allowed_claims", [])
        final["q4"]["sapr_forbidden_claims"] = q4_sapr_summary.get("forbidden_claims", [])
    return final


def build_data_overview_figures() -> None:
    apply_paper_style()
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    monthly_path = REPO_ROOT / "data" / "processed" / "model_monthly_q1.csv"
    country_path = REPO_ROOT / "data" / "processed" / "model_country_monthly.csv"
    if monthly_path.exists():
```

```
        monthly = pd.read_csv(monthly_path, parse_dates=["month_end"])
        fig, ax1 = plt.subplots(figsize=(9.2, 5.1))
        ax1.plot(monthly["month_end"], monthly["brent_usd_bbl"], color=PALETTE["blue"], label="Brent", linewidth=1.8)
        style_axis(ax1, ylabel="Brent，美元/桶")
        ax2 = ax1.twinx()
        ax2.plot(monthly["month_end"], monthly["GPR_z"], color=PALETTE["rose"], alpha=0.82, label="GPR标准化值", linewidth=1.35,
      linestyle=(0, (4, 2)))
        ax2.set_ylabel("GPR标准化值")
        ax2.tick_params(colors=PALETTE["muted"], length=3.2, width=0.65)
        ax2.spines["right"].set_color(PALETTE["muted"])
        ax2.spines["right"].set_linewidth(0.7)
        ax2.grid(False)
        lines, labels = ax1.get_legend_handles_labels()
        lines2, labels2 = ax2.get_legend_handles_labels()
        ax1.legend(lines + lines2, labels + labels2, loc="upper left", ncol=2, handlelength=2.8)
        finish_figure(
            fig,
            title="数据概览：Brent 油价与地缘政治风险",
            subtitle="月度 Brent 现货价与标准化 GPR 指数，2010-01 至 2026-06。",
            source="来源：FRED Brent 与 Caldara-Iacoviello GPR 处理面板；由 code/utils/freeze_results.py 生成。",
        )
        save_figure(fig, FIGURES_DIR / "data_overview_oil_gpr")
        plt.close(fig)
    if country_path.exists():
        panel = pd.read_csv(country_path, parse_dates=["month_end"])
        panel = panel.dropna(subset=["fuel_price_local"]).copy()
        panel["fuel_index"] = panel.groupby("country")["fuel_price_local"].transform(lambda s: 100.0 * s / s.dropna().iloc[0])
        fig, ax = plt.subplots(figsize=(9.2, 5.1))
        style_map = {
            "CHN": (PALETTE["blue"], "solid"),
            "DEU": (PALETTE["gold"], (0, (4, 2))),
            "FRA": (PALETTE["olive"], (0, (2, 2))),
            "ITA": (PALETTE["rose"], (0, (1, 2))),
            "ESP": (PALETTE["slate"], (0, (6, 2))),
            "JPN": (PALETTE["sand"], (0, (3, 1, 1, 1))),
            "KOR": (PALETTE["blue_light"], (0, (1, 1))),
        }
        for country, group in panel.groupby("country"):
            color, linestyle = style_map.get(country, (PALETTE["slate"], "solid"))
            ax.plot(group["month_end"], group["fuel_index"], label=COUNTRY_LABEL_ZH.get(country, country), color=color, linestyle=
      linestyle, linewidth=1.55)
        style_axis(ax, ylabel="指数，2010-01=100")
        ax.legend(loc="upper left", ncol=4, handlelength=2.8)
        finish_figure(
            fig,
            title="数据概览：燃油价格指数",
            subtitle="各国燃油价格序列按各自首个可用月归一；中国序列为调价指数代理，仅作敏感性展示。",
            source="来源：欧盟周度油价公报、日本METI、韩国KOSIS/KNOC、北京市发改委；由 code/utils/freeze_results.py 生成。",
        )
        save_figure(fig, FIGURES_DIR / "data_overview_fuel_panel")
        plt.close(fig)


def summarize_warnings() -> list[dict[str, Any]]:
    warnings_list: list[dict[str, Any]] = []
    seen: dict[tuple[str, str], dict[str, Any]] = {}
    for filename in ["data_warnings.json", "q1_summary.json", "q2_summary.json", "q3_summary.json", "q4_summary.json", "q4_sapr_summary.
      json", "model_panel_summary.json"]:
        payload = load_json(filename)
        for warning in payload.get("warnings", []):
            row = dict(warning)
            key = (str(row.get("code", "warning")), str(row.get("message", "")))
            if key not in seen:
                row["sources"] = [filename]
                seen[key] = row
                warnings_list.append(row)
            elif filename not in seen[key]["sources"]:
                seen[key]["sources"].append(filename)
        for stage, stage_warnings in payload.get("stage_warnings", {}).items():
            for warning in stage_warnings:
                row = dict(warning)
                source = f"{filename}:{stage}"
                key = (str(row.get("code", "warning")), str(row.get("message", "")))
                if key not in seen:
                    row["sources"] = [source]
                    seen[key] = row
```

```
                    warnings_list.append(row)
                elif source not in seen[key]["sources"]:
                    seen[key]["sources"].append(source)
    return warnings_list


def markdown_table(frame: pd.DataFrame, columns: list[str], limit: int = 12) -> str:
    if frame.empty:
        return "_暂无可用结果。_"
    subset = frame.copy()
    for column in columns:
        if column not in subset.columns:
            subset[column] = ""
    subset = subset[columns].head(limit).copy()
    for column in subset.columns:
        if pd.api.types.is_numeric_dtype(subset[column]):
            subset[column] = subset[column].map(lambda value: "" if pd.isna(value) else f"{float(value):.4f}")
        else:
            subset[column] = subset[column].fillna("").astype(str)
    header = "| " + " | ".join(subset.columns) + " |"
    divider = "| " + " | ".join(["---"] * len(subset.columns)) + " |"
    body = []
    for row in subset.to_dict("records"):
        body.append("| " + " | ".join(str(row[column]).replace("\n", " ") for column in subset.columns) + " |")
    return "\n".join([header, divider] + body)


def build_report(risk_summary: dict[str, Any], final_numbers: dict[str, Any], warnings_list: list[dict[str, Any]]) -> str:
    q1_metrics = read_csv("q1_forecast_metrics.csv")
    q1_events = read_csv("q1_event_effects.csv")
    q1_origin = read_csv("q1_origin_forecast.csv")
    q1_svar = read_csv("q1_svar_diagnostics.csv")
    q1_vol_summary = read_csv("q1_volatility_summary.csv")
    q2_irf = read_csv("q2_irf.csv")
    q2_gdp = read_csv("q2_gdp_validation.csv")
    q2_metrics = read_csv("q2_transmission_metrics.csv")
    q3_pass = read_csv("q3_country_pass_through.csv")
    q3_policy = read_csv("q3_policy_counterfactual.csv")
    q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
    q3_resilience = read_csv("q3_resilience_metrics.csv")
    q4_risk = read_csv("q4_price_tail_risk.csv")
    q4_backtest = read_csv("q4_risk_backtest.csv")
    q4_macro = read_csv("q4_macro_stress.csv")
    q4_policy = read_csv("q4_policy_stress.csv")
    q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
    q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
    q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
    figures = collect_figures()
    nbs_iav = pd.read_csv(REPO_ROOT / "data" / "processed" / "nbs_iav_monthly.csv")
    nbs_ppi = pd.read_csv(REPO_ROOT / "data" / "processed" / "nbs_ppi_monthly.csv")
    nbs_iav_count = int(pd.to_numeric(nbs_iav.get("china_iav_yoy_pct"), errors="coerce").notna().sum())
    nbs_ppi_count = int(pd.to_numeric(nbs_ppi.get("china_ppi_yoy_pct"), errors="coerce").notna().sum())

    q3_main = q3_pass.loc[bool_series(q3_pass["included_in_main_comparison"])] if not q3_pass.empty and "included_in_main_comparison" in
      q3_pass else q3_pass
    warning_lines = []
    for warning in warnings_list[:20]:
        warning_lines.append(f"- `{warning.get('code', 'warning')}`: {warning.get('message', '')}")
    if not warning_lines:
        warning_lines.append("- 暂无模型执行 warning。")
    blocker_lines = [f"- `{probe_id}`" for probe_id in risk_summary["blocking_probe_ids"]]
    if not blocker_lines:
        blocker_lines.append("- 无。")

    if risk_summary.get("paper_finalize_allowed"):
        report_title = "国际油价四问建模封板结果报告"
        execution_mode = "paper-ready modeling freeze"
        conclusion_text = (
            "本轮结果已通过风险探针、Schema 校验、哈希冻结与 `--require-final` 验证，"
            "可以作为论文定稿数字来源。论文仍需保持证据边界：Q2 的宏观增长损失证据为 "
            "`INCONCLUSIVE`，Q3 对“我国应对更好”的综合判断为 `PARTIAL`。"
        )
        boundary_text = (
            "当前可写入论文的主线是：问题一采用 `no_change` 主预测、交易日 CAR/placebo 事件证据、"
            "历史递归 SVAR 三类结构冲击和描述性 `ARBaselineGap`；问题二采用结构冲击主规格与 "
            "约化形式稳健性，报告人民币原油成本—PPI—CPI/工业活动—GDP 传导链；问题三使用中国"
```

```
            "中国调价指数代理退出正式价格水平排名，待审计缓冲交互不发布，并输出条件动态政策情景；"
            "问题四使用 FHS-GJR-GARCH 报告尾部概率，将 Q2 结构冲击压力与 Q3 已实现政策反事实分层展示，"
            "并进一步用 SAPR-CVaR 在现行调价机制约束上优化临时平滑规则。"
        )
        paper_advice = (
            "论文正文可把当前结果作为封板数值使用，但必须区分预测表现、事件相关异常、结构冲击传导"
            "和政策情景模拟。第一问不得把 `ARBaselineGap` 写成严格战争净贡献；第二问不得在联合区间"
            "未排除零时写“显著增长损失”；第三问不得仅凭价格传导或单一价格监管变量断言中国政策具有"
            "一般因果优势；问题四不得把尾部概率写成确定结果，不得把 Q2 与 Q3 的不同证据层直接相加，"
            "也不得把 SAPR 的注册规则族最优写成全球福利最优。"
        )
    else:
        report_title = "国际油价四问阶段性建模结果报告"
        execution_mode = "staged modeling audit"
        conclusion_text = (
            "本轮结果仍是 `CONDITIONAL` 阶段快照。代码、图表和结果可继续作为建模推进基础，"
            "但论文正文不能把当前输出写成定稿结论。"
        )
        boundary_text = (
            "当前需要先处理下方阻塞门禁，再重新冻结结果。已通过的模型输出只能作为阶段性证据，"
            "不得越过门禁写入最终论文数字。"
        )
        paper_advice = "只有在风险门禁全部 `PASS` 后，才可把冻结文件作为定稿数值来源。"

    return f"""# {report_title}

## Material Passport

- Origin Skill: `math-modeling-solver + math-modeling-paper`
- Execution Mode: `{execution_mode}`
- Verification Status: `{risk_summary['overall_status']}`
- Paper Finalize Allowed: `{str(risk_summary['paper_finalize_allowed']).lower()}`
- Cutoff: `{CUTOFF}`
- Random Seed: `{RANDOM_SEED}`

## 1. 总体结论

{conclusion_text}

{boundary_text}

当前阻塞定稿的门禁：

{chr(10).join(blocker_lines)}

## 2. 问题一：预测与事件窗口

月度预测表已经加入相对基线指标和逐模型状态。当前主预测模型是 `no_change`。

{markdown_table(q1_metrics.sort_values(['horizon', 'model']) if not q1_metrics.empty else q1_metrics, ['model', 'horizon', 'RMSE', '
    relative_RMSE_vs_no_change', 'dm_hln_pvalue_rmse_loss', 'model_status'])}

以 2026-02 为原点的竞赛最终预测表如下。2026-08 在数据截止日之后，保留为 `FORECAST_ONLY`。

{markdown_table(q1_origin.sort_values('horizon_months') if not q1_origin.empty else q1_origin, ['origin_period', 'target_period', '
    horizon_months', 'model', 'forecast_status', 'origin_price', 'prediction_price', 'actual_price', 'forecast_error_price'])}

历史结构冲击使用递归 VAR/Cholesky 分解，变量排序为“全球供给增长 → 全球真实经济活动 → 实际 Brent 收益”。被选规格如下：

{markdown_table(q1_svar.loc[bool_series(q1_svar['is_selected'])] if not q1_svar.empty and 'is_selected' in q1_svar else q1_svar, ['
    candidate_lags', 'is_selected', 'bic', 'is_stable', 'whiteness_pvalue', 'sample_start', 'sample_end', 'nobs'])}

GJR-GARCH 条件波动率用于描述战争阶段油价波动变化，不作为战争净因果效应。

{markdown_table(q1_vol_summary, ['war_stage', 'trading_days', 'conditional_vol_pct_median', 'vol_multiple_vs_pre_median', '
    evidence_status'])}

E1 已从 2026-02-28 周末映射到 2026-03-02 交易日，同时输出 CAR[0]、CAR[0,+1]、CAR[0,+2] 和匹配周末 placebo 经验 p 值。经验 p 值采用 \\((b
    +1)/(B+1)\\) 的有限样本修正，不报告严格的 0。

{markdown_table(q1_events.loc[q1_events['model'].isin(['brent_usd_bbl_stage_dummy', 'brent_usd_bbl_event_car'])].sort_values(['model', '
    stage_id']) if not q1_events.empty else q1_events, ['model', 'stage_id', 'estimate_log_return', 'std_error', 'lower_95', 'upper_95
    ', 'pvalue', 'pvalue_empirical', 'event_observations'])}

## 3. 问题二：中国宏观传导
```

```
国家统计局 IAV/PPI 官方历史已经进入处理层，其中工业增加值有 {nbs_iav_count} 个真实观测，PPI 有 {nbs_ppi_count} 个真实观测；官方 1—2 月结构性空值保持为空，不做插值。Q2 当前只能写为“尚未发现稳健的总体增长损失证据”，所有结果仍需带区间与识别 caveat 报告。

{markdown_table(q2_irf.loc[q2_irf['horizon'].isin([0, 6, 12])].sort_values(['outcome', 'shock', 'horizon']) if not q2_irf.empty else q2_irf, ['outcome', 'shock', 'horizon', 'response', 'lower_95', 'upper_95', 'joint_lower_95', 'joint_upper_95', 'fdr_qvalue', 'inference_band'])}

传导链摘要按冲击—变量输出峰值/谷值、累计响应和证据状态。当前即使部分点估计方向符合直觉，也不自动升级为“增长损失”结论。

{markdown_table(q2_metrics.loc[q2_metrics['shock'].isin(['supply_shock', 'aggregate_demand_shock', 'oil_specific_risk_shock'])].sort_values(['shock', 'outcome']) if not q2_metrics.empty else q2_metrics, ['shock', 'outcome', 'extremum_type', 'extremum_response', 'extremum_month', 'response_curve_area_0_6', 'response_curve_area_0_12', 'area_unit', 'evidence_status', 'allows_growth_loss_language'])}

季度 GDP 只作低频验证，不插值成月度变量。

{markdown_table(q2_gdp, ['outcome', 'estimate', 'lower_95', 'upper_95', 'pvalue', 'n', 'sample_start', 'sample_end'])}

## 4. 问题三：政策缓冲与跨国比较

跨国燃油主排名只纳入覆盖充分且价格层可解释的观测零售汽油价格。中国历史路径是受管制汽油调价指数代理，只保留为敏感性材料；2026政策公告差额单独核验。

{markdown_table(q3_main.loc[q3_main['horizon'].eq(6)].sort_values('country') if not q3_main.empty else q3_main, ['country', 'horizon', 'response', 'lower_95', 'upper_95', 'price_measure_type', 'included_in_main_comparison'])}

综合韧性指标把燃油传导、CPI 相对响应、工业活动相对响应和政策关闭情景放在同一口径下判断。当前总体判断为 `PARTIAL`，不是无条件支持“中国显著更好”。

{markdown_table(q3_resilience, ['dimension', 'metric', 'horizon', 'china_value', 'control_median', 'china_vs_control_median_diff', 'diff_lower_95', 'diff_upper_95', 'judgement', 'overall_china_resilience_judgement'])}

中国政策图与数据表已区分新增差额和累计差额，2026-04 的累计差额为 1425 元/吨，4月新增为 380 元/吨。

{markdown_table(q3_policy, ['period', 'policy_adjusted_official_cny_t', 'no_temporary_control_official_cny_t', 'incremental_gasoline_gap_cny_t', 'cumulative_gasoline_gap_cny_t', 'cpi_counterfactual_gap_pctpt', 'price_layer_status'])}

政策关闭宏观情景在明确标注的调价指数代理层上，将2026临时调控收益差通过0—6阶核和结果变量递归传播到PPI、CPI与IAV；区间来自三方程共同时间块重采样。

{markdown_table(q3_policy_macro.loc[q3_policy_macro['period'].eq('2026-06')].sort_values('outcome') if not q3_policy_macro.empty else q3_policy_macro, ['period', 'outcome_label', 'macro_counterfactual_gap_pctpt', 'lower_95', 'upper_95', 'price_layer_status', 'evidence_status'])}

## 5. 问题四：极端油价尾部风险、政策压力测试与自适应调价规则优化

问题四把同事已完成的“尾部风险与压力测试”作为风险输入层，并把本轮 SAPR-CVaR 自适应调价规则作为政策优化层：先判断极端油价路径概率与宏观压力，再回答我国成品油调价机制在极端冲击下应如何状态依赖地平滑传导。油价概率层以 2026-06-30 最后交易日为原点，比较 FHS－GJR-GARCH 与恒定波动高斯随机游走；历史 90%/95%价格分位只作为上行压力阈值。

{markdown_table(q4_risk.sort_values(['model', 'horizon_months']) if not q4_risk.empty else q4_risk, ['model', 'horizon_months', 'median_price', 'p05_price', 'p95_price', 'terminal_prob_above_hist_p90', 'terminal_prob_above_hist_p95', 'path_prob_cross_hist_p95'])}

滚动回测在同一原点、期限和评价口径下比较主方法与基线；主方法是否胜出由分位损失和覆盖率共同判断，不因预设方法而更换回测区间。

{markdown_table(q4_backtest.sort_values(['horizon_months', 'model']) if not q4_backtest.empty else q4_backtest, ['model', 'horizon_months', 'origins', 'mean_pinball_loss', 'median_absolute_error', 'coverage_80', 'coverage_90', 'mean_width_90'])}

宏观压力层使用 Q1 油价特定风险冲击的历史正向分位数缩放 Q2 IRF。下表只展示 95%分位冲击的 6、12 月条件响应；联合区间跨零时继续标记为 `INCONCLUSIVE`。

{markdown_table(q4_macro.loc[q4_macro['scenario'].eq('extreme_q95') & q4_macro['horizon'].isin([6, 12])].sort_values(['outcome', 'horizon']) if not q4_macro.empty else q4_macro, ['scenario', 'outcome_label', 'horizon', 'conditional_response_pctpt', 'joint_lower_95', 'joint_upper_95', 'row_evidence_status'])}

政策压力层只重述 Q3 已实现的 2026 年临时调控关闭反事实，不外推到模拟油价路径。正的“政策缓冲收益”分别表示避免的 PPI/CPI 增幅或避免的工业活动损失。

{markdown_table(q4_policy.loc[q4_policy['period'].eq('2026-06')].sort_values('outcome') if not q4_policy.empty else q4_policy, ['period', 'outcome_label', 'policy_buffer_benefit_pctpt', 'benefit_lower_95', 'benefit_upper_95', 'evidence_status'])}

SAPR-CVaR 优化层只在 2013-03—2021-12 开发样本上确定阈值和规则，2022-01—2026-06 完全作为隔离检验样本；2026 年战争冲击只用于检验和展示，不参与规则选择。三档传导率满足 `普通 ≥ 压力 ≥ 极端`，目标函数同时考虑宏观损失、95% CVaR、累计未调价负担和国内调价波动。

{markdown_table(q4_sapr_optimal, ['rule_id', 'rho_normal', 'rho_stress', 'rho_extreme', 'stress_threshold_75_cny_t', '
```

```
      stress_threshold_95_cny_t', 'J1_macro_loss', 'J2_cvar95_macro_loss', 'J3_avg_gap_month_burden', 'J4_adjustment_volatility', '
      max_gap_ratio', 'max_recovery_terminal_gap_ratio'])}
1599
1600 隔离检验样本与 2026 实际冲击情景的策略比较如下。若 SAPR 在检验样本被预注册基线支配，论文必须报告负结果：当前证据状态以 `q4_sapr_summary.
      json` 为准。
1601
1602 {markdown_table(q4_sapr_comparison.loc[q4_sapr_comparison['sample_split'].isin(['holdout', 'war_2026'])].sort_values(['sample_split', '
      strategy']) if not q4_sapr_comparison.empty else q4_sapr_comparison, ['sample_split', 'strategy', 'rho_normal', 'rho_stress', '
      rho_extreme', 'J1_macro_loss', 'J2_cvar95_macro_loss', 'J3_avg_gap_month_burden', 'J4_adjustment_volatility', 'max_gap_ratio', '
      max_recovery_terminal_gap_ratio'])}
1603
1604 敏感性检验固定改变阈值、bootstrap 块长和 CPI/IAV 权重，不为追求结论改动样本或目标函数。
1605
1606 {markdown_table(q4_sapr_sensitivity, ['variant', 'rho_normal', 'rho_stress', 'rho_extreme', 'pareto_rule_count', '
      holdout_non_dominated_probability', 'changed_from_default'])}
1607
1608 ## 6. 图表与冻结文件
1609
1610 核心 PNG 图表：{', '.join(figures) if figures else '暂无图表'}。
1611
1612 冻结文件：
1613
1614 - `results/final_numbers.json`
1615 - `results/frozen_numbers.json`
1616 - `results/risk_probe_summary.json`
1617 - `results/reproducibility_manifest.json`
1618
1619 其中 `frozen_numbers.json` 遵循 `3coding-visual` 标准冻结格式；风险门禁以及代码、输入、输出文件哈希保存在独立的 reproducibility manifest
      中。数值一致性使用标准 skill 脚本检查，项目级文件与环境检查使用 `python code/utils/verify_freeze.py`。
1620
1621 ## 7. Warnings
1622
1623 {chr(10).join(warning_lines)}
1624
1625 ## 8. 论文使用建议
1626
1627 {paper_advice}
1628 """
1629
1630
1631 def write_json(path: Path, payload: dict[str, Any]) -> None:
1632     path.write_text(
1633         json.dumps(payload, ensure_ascii=False, indent=2, sort_keys=True, allow_nan=False) + "\n",
1634         encoding="utf-8",
1635     )
1636
1637
1638 def standard_freeze_payload(final_numbers: dict[str, Any]) -> dict[str, Any]:
1639     return {
1640         "schema_version": STANDARD_FREEZE_SCHEMA_VERSION,
1641         "frozen_at": datetime.now(timezone.utc).isoformat().replace("+00:00", "Z"),
1642         "source_file": "results/final_numbers.json",
1643         "source_sha256": sha256_json(final_numbers),
1644         "values": final_numbers,
1645     }
1646
1647
1648 def reproducibility_manifest(
1649     risk_summary: dict[str, Any],
1650     final_numbers: dict[str, Any],
1651     risk_hash: str,
1652     output_hashes: dict[str, str],
1653     input_hashes: dict[str, str],
1654     code_hashes: dict[str, str],
1655 ) -> dict[str, Any]:
1656     source = {
1657         "schema_version": MANIFEST_SCHEMA_VERSION,
1658         "snapshot_mode": "paper_final" if risk_summary["paper_finalize_allowed"] else "stage_snapshot_not_paper_final",
1659         "overall_status": risk_summary["overall_status"],
1660         "paper_finalize_allowed": risk_summary["paper_finalize_allowed"],
1661         "git_commit": git_commit(),
1662         "source_commit": git_commit(),
1663         "cutoff": CUTOFF,
1664         "random_seed": RANDOM_SEED,
1665         "replay_environment": replay_environment_snapshot(),
1666         "packaging_environment": environment_snapshot(),
```

```
        "code_hashes": code_hashes,
        "processed_input_hashes": input_hashes,
        "core_result_hashes": output_hashes,
        "risk_gate_hash": risk_hash,
        "final_numbers_source_sha256": sha256_json(final_numbers),
        "standard_freeze_file": "results/frozen_numbers.json",
    }
    manifest = dict(source)
    manifest["manifest_hash"] = sha256_json(source)
    return manifest


def main() -> int:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    REPORTS_DIR.mkdir(parents=True, exist_ok=True)
    build_data_overview_figures()

    output_hashes = hash_existing(RESULTS_DIR, CORE_RESULT_FILES)
    input_hashes = hash_existing(REPO_ROOT / "data" / "processed", PROCESSED_INPUT_FILES)
    code_hashes = hash_existing(REPO_ROOT, CODE_FILES)
    risk_summary = build_risk_probe_summary(output_hashes, input_hashes, code_hashes)
    risk_path = RESULTS_DIR / "risk_probe_summary.json"
    write_json(risk_path, risk_summary)
    risk_hash = sha256_file(risk_path)

    final_numbers = extract_final_numbers()
    final_path = RESULTS_DIR / "final_numbers.json"
    write_json(final_path, final_numbers)

    frozen_path = RESULTS_DIR / "frozen_numbers.json"
    frozen = standard_freeze_payload(final_numbers)
    write_json(frozen_path, frozen)

    manifest = reproducibility_manifest(risk_summary, final_numbers, risk_hash, output_hashes, input_hashes, code_hashes)
    manifest_path = RESULTS_DIR / "reproducibility_manifest.json"
    write_json(manifest_path, manifest)

    warnings_list = summarize_warnings()
    report = build_report(risk_summary, final_numbers, warnings_list)
    (REPORTS_DIR / "RESULTS_REPORT.md").write_text(report.rstrip() + "\n", encoding="utf-8")

    print(
        json.dumps(
            {
                "status": risk_summary["overall_status"],
                "paper_finalize_allowed": risk_summary["paper_finalize_allowed"],
                "source_sha256": frozen["source_sha256"],
                "manifest_hash": manifest["manifest_hash"],
                "blocking_probe_ids": risk_summary["blocking_probe_ids"],
            },
            ensure_ascii=False,
            indent=2,
        )
    )
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 13: freeze_results.py

```
"""Verify the standard numerical freeze and the project reproducibility manifest."""

from __future__ import annotations

import argparse
import hashlib
import importlib.metadata
import json
import platform
import subprocess
import sys
from pathlib import Path
from typing import Any
```

```

import numpy as np
import pandas as pd
from jsonschema import Draft202012Validator


REPO_ROOT = Path(__file__).resolve().parents[2]
RESULTS_DIR = REPO_ROOT / "results"
PROCESSED_DIR = REPO_ROOT / "data" / "processed"
SCHEMA_DIR = REPO_ROOT / "code" / "schemas"

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

STANDARD_FROZEN_REQUIRED = [
    "schema_version",
    "frozen_at",
    "source_file",
    "source_sha256",
    "values",
]
MANIFEST_REQUIRED = [
    "schema_version",
    "snapshot_mode",
    "overall_status",
    "paper_finalize_allowed",
    "git_commit",
    "source_commit",
    "cutoff",
    "random_seed",
    "replay_environment",
    "packaging_environment",
    "code_hashes",
    "processed_input_hashes",
    "core_result_hashes",
    "risk_gate_hash",
    "final_numbers_source_sha256",
    "standard_freeze_file",
    "manifest_hash",
]
RISK_REQUIRED = [
    "schema_version",
    "overall_status",
    "paper_finalize_allowed",
    "cutoff",
    "random_seed",
    "probes",
    "blocking_probe_ids",
    "core_results_present",
    "figure_pngs",
    "output_hash_count",
    "input_hash_count",
    "model_code_hash_count",
]


def sha256_file(path: Path) -> str:
    payload = path.read_bytes()
    if path.suffix.lower() in {".csv", ".json", ".md", ".py", ".txt"}:
        payload = payload.replace(b"\r\n", b"\n").replace(b"\r", b"\n")
    return hashlib.sha256(payload).hexdigest()


def sha256_json(payload: Any) -> str:
    text = json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":"), allow_nan=False)
    return hashlib.sha256(text.encode("utf-8")).hexdigest()


def load_json(path: Path) -> dict[str, Any]:
    return json.loads(path.read_text(encoding="utf-8"))


def require_keys(payload: dict[str, Any], keys: list[str], label: str, errors: list[str]) -> None:
    missing = [key for key in keys if key not in payload]
    if missing:
        errors.append(f"{label} missing required keys: {missing}")
```

```

def validate_schema(payload: dict[str, Any], schema_filename: str, label: str, errors: list[str]) -> None:
    schema_path = SCHEMA_DIR / schema_filename
    if not schema_path.exists():
        errors.append(f"{label} schema missing: {schema_filename}")
        return
    schema = load_json(schema_path)
    validator = Draft202012Validator(schema)
    schema_errors = sorted(validator.iter_errors(payload), key=lambda err: list(err.path))
    for error in schema_errors:
        location = ".".join(str(part) for part in error.path) or "<root>"
        errors.append(f"{label} schema error at {location}: {error.message}")


def verify_source_commit_lineage(frozen: dict[str, Any], errors: list[str]) -> None:
    source_commit = frozen.get("source_commit")
    if not source_commit or source_commit == "UNKNOWN":
        errors.append("frozen source_commit is missing or UNKNOWN")
        return
    result = subprocess.run(
        ["git", "merge-base", "--is-ancestor", source_commit, "HEAD"],
        cwd=REPO_ROOT,
        text=True,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        errors.append(f"current HEAD does not contain frozen source_commit {source_commit}")


def verify_hashes(base_dir: Path, hashes: dict[str, str], label: str, errors: list[str]) -> None:
    for filename, expected in hashes.items():
        path = base_dir / filename
        if not path.exists():
            errors.append(f"{label} file missing: {filename}")
            continue
        actual = sha256_file(path)
        if actual != expected:
            errors.append(f"{label} hash mismatch: {filename}")


def bool_series(series: pd.Series) -> pd.Series:
    if pd.api.types.is_bool_dtype(series):
        return series
    return series.astype(str).str.lower().isin(["true", "1", "yes"])


def verify_environment(manifest: dict[str, Any], messages: list[str]) -> None:
    env = manifest.get("replay_environment", {})
    if env.get("python_version") != platform.python_version():
        messages.append(f"replay python mismatch: required={env.get('python_version')} current={platform.python_version()}")
    packages = env.get("packages", {})
    for package, manifest_version in packages.items():
        try:
            current = importlib.metadata.version(package)
        except importlib.metadata.PackageNotFoundError:
            current = "NOT_INSTALLED"
        if current != manifest_version:
            messages.append(f"replay package mismatch: {package} required={manifest_version} current={current}")


def verify_key_result_shapes(errors: list[str]) -> None:
    q1_metrics = pd.read_csv(RESULTS_DIR / "q1_forecast_metrics.csv")
    required_q1 = {"model", "horizon", "relative_RMSE_vs_no_change", "model_status"}
    if not required_q1.issubset(q1_metrics.columns):
        errors.append("q1_forecast_metrics.csv lacks forecast gate columns")
    else:
        advanced_pass = q1_metrics.loc[q1_metrics["model"].ne("no_change") & q1_metrics["model_status"].eq("PASS")]
        advanced_worse = q1_metrics.loc[q1_metrics["model"].ne("no_change") & q1_metrics["relative_RMSE_vs_no_change"].gt(1.0)]
        if not advanced_pass.merge(advanced_worse[["model", "horizon"]], on=["model", "horizon"], how="inner").empty:
            errors.append("advanced Q1 model with RMSE worse than no-change is still marked PASS")

    q1_events = pd.read_csv(RESULTS_DIR / "q1_event_effects.csv")
    empirical = pd.to_numeric(
        q1_events.loc[q1_events["model"].eq("brent_usd_bbl_event_car"), "pvalue_empirical"],
```

```
        errors="coerce",
    ).dropna()
    if empirical.empty or not empirical.between(0.0, 1.0, inclusive="right").all() or empirical.eq(0.0).any():
        errors.append("Q1 empirical placebo p-values are missing, outside (0, 1], or still report zero")

    q1_origin = pd.read_csv(RESULTS_DIR / "q1_origin_forecast.csv")
    origin_horizons = sorted(pd.to_numeric(q1_origin.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int)
      .tolist())
    if origin_horizons != [1, 3, 6]:
        errors.append("q1_origin_forecast.csv must contain exactly 1/3/6 month horizons")
    six_month = q1_origin.loc[q1_origin["horizon_months"].eq(6)]
    if six_month.empty or not six_month["forecast_status"].astype(str).eq("FORECAST_ONLY").all():
        errors.append("Q1 2026-08 six-month origin forecast must be retained and marked FORECAST_ONLY")

    q1_svar = pd.read_csv(RESULTS_DIR / "q1_svar_diagnostics.csv")
    selected = q1_svar.loc[q1_svar.get("is_selected", pd.Series(dtype=bool)).astype(str).str.lower().isin(["true", "1"])]
    if len(selected) != 1 or not selected["is_stable"].astype(str).str.lower().isin(["true", "1"]).all():
        errors.append("q1_svar_diagnostics.csv must identify exactly one selected stable VAR specification")
    q1_stationarity = pd.read_csv(RESULTS_DIR / "q1_stationarity_diagnostics.csv")
    if set(q1_stationarity.get("variable", pd.Series(dtype=str))) != {"supply_growth", "global_real_activity", "real_brent_return"}:
        errors.append("q1_stationarity_diagnostics.csv must cover all three SVAR inputs")

    q2_irf = pd.read_csv(RESULTS_DIR / "q2_irf.csv")
    q2_metrics = pd.read_csv(RESULTS_DIR / "q2_transmission_metrics.csv")
    iav_horizons = sorted(pd.to_numeric(q2_irf.loc[q2_irf["outcome"].eq("china_iav_yoy_pct"), "horizon"], errors="coerce").dropna().
      astype(int).unique().tolist())
    if iav_horizons != [0, 3, 6, 12]:
        errors.append("Q2 IAV response horizons must be the pre-specified 0/3/6/12 grid")
    required_q2_metrics = {"evidence_status", "allows_growth_loss_language", "response_curve_area_0_6", "response_curve_area_0_12", "
      area_unit", "available_horizons"}
    if q2_metrics.empty or not required_q2_metrics.issubset(q2_metrics.columns):
        errors.append("q2_transmission_metrics.csv lacks required transmission-metric columns")
    iav_metrics = q2_metrics.loc[q2_metrics["outcome"].eq("china_iav_yoy_pct")]
    if not iav_metrics.empty and iav_metrics[["response_curve_area_0_6", "response_curve_area_0_12"]].notna().any().any():
        errors.append("Q2 IAV sparse horizons must not be mechanically summed into a curve area")

    q3_pass = pd.read_csv(RESULTS_DIR / "q3_country_pass_through.csv")
    required_q3 = {"price_measure_type", "observed_or_regulated", "included_in_main_comparison", "comparability_note"}
    if not required_q3.issubset(q3_pass.columns):
        errors.append("q3_country_pass_through.csv lacks comparability columns")
    else:
        chn_rows = q3_pass.loc[q3_pass["country"].eq("CHN")]
        chn_included = bool_series(chn_rows["included_in_main_comparison"]) if not chn_rows.empty else pd.Series(dtype=bool)
        chn_proxy = chn_rows["price_measure_type"].astype(str).str.contains("proxy", case=False, na=False) if not chn_rows.empty else pd.
      Series(dtype=bool)
        if bool((chn_included & chn_proxy).any()):
            errors.append("China proxy is included in the main Q3 fuel comparison")
        if chn_rows.empty or bool(chn_included.any()) or not bool(chn_proxy.all()):
            errors.append("China must be present only as an excluded adjustment-index proxy sensitivity in Q3")

    q3_buffer = pd.read_csv(RESULTS_DIR / "q3_buffer_interactions.csv")
    required_buffer = {"outcome", "buffer", "shock", "estimate", "lower_95", "upper_95", "data_status"}
    if not required_buffer.issubset(q3_buffer.columns):
        errors.append("q3_buffer_interactions.csv lacks policy-buffer interaction columns")
    if not q3_buffer.empty:
        errors.append("q3_buffer_interactions.csv must remain empty while country-year buffers are unaudited proxies")
    q3_buffer_status = pd.read_csv(RESULTS_DIR / "q3_buffer_data_status.csv")
    required_buffers = {"fuel_price_regulation", "oil_import_dependency", "oil_intensity", "import_source_hhi"}
    if set(q3_buffer_status.get("buffer", pd.Series(dtype=str))) != required_buffers or not q3_buffer_status["data_status"].eq("
      FAIL_FOR_MAIN_UNAUDITED_PROXY").all():
        errors.append("q3_buffer_data_status.csv must fail closed for all four unaudited buffer variables")

    q3_panel = pd.read_csv(RESULTS_DIR / "q3_panel_irf.csv")
    if "reference_country" not in q3_panel.columns or not q3_panel["reference_country"].astype(str).eq("CHN").all():
        errors.append("q3_panel_irf.csv must use China as the explicit reference country")
    if "response_type" not in q3_panel.columns or not q3_panel["response_type"].astype(str).str.contains("control_country_minus_china",
      na=False).all():
        errors.append("q3_panel_irf.csv must report control-country-minus-China relative responses")

    q3_resilience = pd.read_csv(RESULTS_DIR / "q3_resilience_metrics.csv")
    required_dimensions = {"fuel_price", "consumer_prices", "industrial_activity", "policy_counterfactual_macro"}
    if q3_resilience.empty or not required_dimensions.issubset(set(q3_resilience.get("dimension", pd.Series(dtype=str)).dropna().unique()
      )):
        errors.append("q3_resilience_metrics.csv lacks required resilience dimensions")
    if not q3_resilience.get("overall_china_resilience_judgement", pd.Series(dtype=str)).astype(str).eq("INCONCLUSIVE").all():
        errors.append("Q3 overall resilience must fail closed while China fuel is proxy-only and joint country intervals are withheld")
```

```
235
236     policy = pd.read_csv(RESULTS_DIR / "q3_policy_counterfactual.csv")
237     april = policy.loc[policy["period"].eq("2026-04")]
238     if april.empty:
239         errors.append("q3_policy_counterfactual.csv lacks 2026-04 row")
240     else:
241         incremental = float(april["incremental_gasoline_gap_cny_t"].iloc[0])
242         cumulative = float(april["cumulative_gasoline_gap_cny_t"].iloc[0])
243         if abs(incremental - 380.0) > 1e-6 or abs(cumulative - 1425.0) > 1e-6:
244             errors.append("Q3 April policy gap no longer matches incremental 380 and cumulative 1425")
245     if "price_layer_status" not in policy.columns:
246         errors.append("q3_policy_counterfactual.csv lacks price_layer_status")
247     elif not policy["price_layer_status"].astype(str).str.contains("proxy", case=False, na=False).all():
248         errors.append("q3_policy_counterfactual.csv must identify the historical price layer as a proxy")
249     policy_macro = pd.read_csv(RESULTS_DIR / "q3_policy_macro_counterfactual.csv")
250     required_dynamic = {"fuel_return_gap", "bootstrap_reps", "bootstrap_block_length", "macro_counterfactual_gap_pctpt"}
251     if not required_dynamic.issubset(policy_macro.columns) or not pd.to_numeric(policy_macro["bootstrap_block_length"], errors="coerce").
        eq(6).all():
252         errors.append("Q3 policy macro counterfactual lacks the dynamic joint-block interface")
253
254     policy_macro = pd.read_csv(RESULTS_DIR / "q3_policy_macro_counterfactual.csv")
255     macro_outcomes = set(policy_macro.get("outcome", pd.Series(dtype=str)).dropna().unique())
256     if not {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}.issubset(macro_outcomes):
257         errors.append("q3_policy_macro_counterfactual.csv must include PPI/CPI/IAV propagated paths")
258     if not {"lower_95", "upper_95", "macro_counterfactual_gap_pctpt"}.issubset(policy_macro.columns):
259         errors.append("q3_policy_macro_counterfactual.csv lacks macro interval columns")
260
261     q3_kernel = pd.read_csv(RESULTS_DIR / "q3_policy_macro_kernel.csv")
262     q3_cov = pd.read_csv(RESULTS_DIR / "q3_policy_macro_covariance.csv")
263     kernel_outcomes = set(q3_kernel.get("outcome", pd.Series(dtype=str)).dropna().unique())
264     if not {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}.issubset(kernel_outcomes):
265         errors.append("q3_policy_macro_kernel.csv must expose PPI/CPI/IAV macro kernels for Q4 SAPR")
266     required_kernel_cols = {f"beta_fuel_lag{i}" for i in range(7)} | {"phi_outcome_lag1"}
267     if q3_kernel.empty or not required_kernel_cols.issubset(q3_kernel.columns):
268         errors.append("q3_policy_macro_kernel.csv lacks usable coefficient columns")
269     if q3_cov.empty or not {"outcome", "row_term", "column_term", "covariance"}.issubset(q3_cov.columns):
270         errors.append("q3_policy_macro_covariance.csv lacks covariance matrix columns")
271
272     q4_risk = pd.read_csv(RESULTS_DIR / "q4_price_tail_risk.csv")
273     q4_backtest = pd.read_csv(RESULTS_DIR / "q4_risk_backtest.csv")
274     if set(q4_risk.get("model", pd.Series(dtype=str)).dropna().unique()) != {"FHS_GJR_GARCH", "Gaussian_random_walk"}:
275         errors.append("q4_price_tail_risk.csv must include both FHS_GJR_GARCH and Gaussian_random_walk")
276     q4_horizons = sorted(pd.to_numeric(q4_risk.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int).
      unique().tolist())
277     if q4_horizons != [1, 3, 6]:
278         errors.append("q4_price_tail_risk.csv must report 1/3/6 month horizons")
279     if q4_backtest.empty or not {"origins", "all_no_future_information", "mean_pinball_loss"}.issubset(q4_backtest.columns):
280         errors.append("q4_risk_backtest.csv lacks backtest integrity columns")
281     elif not bool_series(q4_backtest["all_no_future_information"]).all():
282         errors.append("Q4 tail-risk backtest uses future information")
283     elif pd.to_numeric(q4_backtest["origins"], errors="coerce").min() < 36:
284         errors.append("Q4 tail-risk backtest must use at least 36 monthly origins")
285     q4_paired = pd.read_csv(RESULTS_DIR / "q4_risk_backtest_paired.csv")
286     if q4_paired.empty or not {"interval_score_80", "interval_score_90", "mean_pinball_loss"}.issubset(set(q4_paired.get("metric", pd.
      Series(dtype=str)))):
287         errors.append("q4_risk_backtest_paired.csv lacks paired block-bootstrap distributional scores")
288
289     q4_sapr_scenarios = pd.read_csv(RESULTS_DIR / "q4_sapr_scenarios.csv")
290     q4_sapr_grid = pd.read_csv(RESULTS_DIR / "q4_sapr_policy_grid.csv")
291     q4_sapr_optimal = pd.read_csv(RESULTS_DIR / "q4_sapr_optimal_rule.csv")
292     q4_sapr_comparison = pd.read_csv(RESULTS_DIR / "q4_sapr_strategy_comparison.csv")
293     q4_sapr_paths = pd.read_csv(RESULTS_DIR / "q4_sapr_macro_paths.csv")
294     q4_sapr_sensitivity = pd.read_csv(RESULTS_DIR / "q4_sapr_sensitivity.csv")
295     q4_sapr_summary = load_json(RESULTS_DIR / "q4_sapr_summary.json")
296
297     q4_dev = q4_sapr_scenarios.loc[q4_sapr_scenarios["sample_split"].eq("development")]
298     q4_holdout = q4_sapr_scenarios.loc[q4_sapr_scenarios["sample_split"].eq("holdout")]
299     if q4_dev.empty or q4_holdout.empty:
300         errors.append("q4_sapr_scenarios.csv must include development and holdout scenarios")
301     else:
302         if str(q4_dev["source_end"].max()) > "2021-12":
303             errors.append("Q4 SAPR development scenarios leak post-2021 data")
304         if str(q4_holdout["source_start"].min()) < "2022-01":
305             errors.append("Q4 SAPR holdout scenarios overlap the development period")
306     if int(bool_series(q4_sapr_scenarios.get("is_2026_anchor", pd.Series(dtype=bool))).sum()) == 0:
307         errors.append("q4_sapr_scenarios.csv lacks the 2026 anchor scenario")
```

```
    if len(q4_sapr_grid) != 1771:
        errors.append("q4_sapr_policy_grid.csv must contain exactly 1771 monotone rules")
    if int(bool_series(q4_sapr_grid.get("is_knee", pd.Series(dtype=bool))).sum()) != 1:
        errors.append("q4_sapr_policy_grid.csv must identify exactly one knee rule")
    if "is_feasible" not in q4_sapr_grid.columns or not bool_series(q4_sapr_grid.loc[bool_series(q4_sapr_grid["is_knee"]), "is_feasible"
      ]).all():
        errors.append("Q4 SAPR knee rule must belong to the preregistered feasible set")
    if q4_sapr_optimal.empty or len(q4_sapr_optimal) != 1:
        errors.append("q4_sapr_optimal_rule.csv must contain exactly one optimal rule")
    else:
        opt = q4_sapr_optimal.iloc[0]
        if not (float(opt["rho_normal"]) >= float(opt["rho_stress"]) >= float(opt["rho_extreme"])):
            errors.append("Q4 SAPR optimal pass-through rates violate monotonicity")
        required_constraints = {"max_gap_ratio", "max_terminal_gap_ratio", "max_recovery_terminal_gap_ratio", "
      max_monthly_adjustment_ratio", "J3_avg_gap_month_burden"}
        if not required_constraints.issubset(q4_sapr_optimal.columns):
            errors.append("Q4 SAPR optimal rule lacks gap, recovery or adjustment feasibility metrics")
    required_sapr_strategies = {"full_mechanism", "uniform_75_smoothing", "temporary_2026_approx", "SAPR_CVaR_knee"}
    holdout_strategies = set(q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("holdout"), "strategy"].dropna())
    war_strategies = set(q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("war_2026"), "strategy"].dropna())
    if not required_sapr_strategies.issubset(holdout_strategies):
        errors.append("q4_sapr_strategy_comparison.csv lacks required holdout strategies")
    if not required_sapr_strategies.issubset(war_strategies):
        errors.append("q4_sapr_strategy_comparison.csv lacks required 2026-war strategies")
    interval_cols = [
        "J1_macro_loss_lower_95",
        "J1_macro_loss_upper_95",
        "J2_cvar95_macro_loss_lower_95",
        "J2_cvar95_macro_loss_upper_95",
    ]
    if not set(interval_cols).issubset(q4_sapr_comparison.columns):
        errors.append("q4_sapr_strategy_comparison.csv lacks uncertainty interval columns")
    elif not np.isfinite(q4_sapr_comparison[interval_cols].apply(pd.to_numeric, errors="coerce").to_numpy()).all():
        errors.append("Q4 SAPR strategy intervals contain non-finite values")
    if q4_sapr_paths.empty or not {"strategy", "sample_split", "month_index", "cumulative_deferred_gap_cny_t"}.issubset(q4_sapr_paths.
      columns):
        errors.append("q4_sapr_macro_paths.csv lacks required macro-path columns")
    if len(q4_sapr_sensitivity) < 6:
        errors.append("q4_sapr_sensitivity.csv must include threshold, block-length, CPI-weight and IAV-weight variants")
    q4_holdout_summary = load_json(RESULTS_DIR / "q4_sapr_holdout_validation_summary.json")
    if int(q4_holdout_summary.get("block_length", 0)) < 6 or int(q4_holdout_summary.get("supported_improvement_count", 0)) < 1:
        errors.append("Q4 SAPR holdout validation must use at least six-month blocks and support at least one paired improvement")
    if q4_sapr_summary.get("execution_status") != "PASS":
        errors.append("q4_sapr_summary.json execution_status must be PASS")
    if q4_sapr_summary.get("evidence_status") not in {"SUPPORTED", "PARTIAL", "NOT_SUPPORTED"}:
        errors.append("q4_sapr_summary.json has invalid evidence_status")
    if int(q4_sapr_summary.get("valid_parameter_draw_count", 0)) != 2000:
        errors.append("Q4 SAPR must retain exactly 2000 valid parameter draws")
    if float(q4_sapr_summary.get("valid_parameter_draw_rate", 0.0)) < 0.95:
        errors.append("Q4 SAPR valid parameter draw rate is below 95%")
    identity = q4_sapr_summary.get("identity_checks", {})
    if abs(float(identity.get("official_2026_03_gap", 0.0)) - 1045.0) > 1e-6:
        errors.append("Q4 SAPR official 2026-03 policy gap must equal 1045 CNY/t")
    if abs(float(identity.get("official_2026_04_gap", 0.0)) - 380.0) > 1e-6:
        errors.append("Q4 SAPR official 2026-04 incremental policy gap must equal 380 CNY/t")
    if abs(float(identity.get("official_2026_04_cumulative_gap", 0.0)) - 1425.0) > 1e-6:
        errors.append("Q4 SAPR official 2026-04 cumulative policy gap must equal 1425 CNY/t")


def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--require-final", action="store_true", help="Return nonzero unless the risk gate allows paper finalization.")
    parser.add_argument("--strict-environment", action="store_true", help="Treat replay-environment differences as errors.")
    args = parser.parse_args()

    errors: list[str] = []
    environment_messages: list[str] = []
    final_path = RESULTS_DIR / "final_numbers.json"
    frozen_path = RESULTS_DIR / "frozen_numbers.json"
    risk_path = RESULTS_DIR / "risk_probe_summary.json"
    manifest_path = RESULTS_DIR / "reproducibility_manifest.json"

    final_numbers = load_json(final_path)
    frozen = load_json(frozen_path)
    risk = load_json(risk_path)
    manifest = load_json(manifest_path)
```

```
381
382     require_keys(frozen, STANDARD_FROZEN_REQUIRED, "frozen_numbers.json", errors)
383     require_keys(manifest, MANIFEST_REQUIRED, "reproducibility_manifest.json", errors)
384     require_keys(risk, RISK_REQUIRED, "risk_probe_summary.json", errors)
385     validate_schema(frozen, "frozen_numbers.schema.json", "frozen_numbers.json", errors)
386     validate_schema(manifest, "reproducibility_manifest.schema.json", "reproducibility_manifest.json", errors)
387     validate_schema(risk, "risk_probe_summary.schema.json", "risk_probe_summary.json", errors)
388
389     expected_source_hash = sha256_json(final_numbers)
390     if frozen.get("schema_version") != 1:
391         errors.append(f"frozen_numbers.json schema_version must be 1, got {frozen.get('schema_version')!r}")
392     if frozen.get("source_file") != "results/final_numbers.json":
393         errors.append("frozen_numbers.json source_file is not results/final_numbers.json")
394     if frozen.get("source_sha256") != expected_source_hash:
395         errors.append("frozen_numbers.json source_sha256 does not match final_numbers.json")
396     if frozen.get("values") != final_numbers:
397         errors.append("frozen_numbers.json values do not match final_numbers.json")
398
399     manifest_source = {key: manifest[key] for key in MANIFEST_REQUIRED if key != "manifest_hash" and key in manifest}
400     if manifest.get("manifest_hash") != sha256_json(manifest_source):
401         errors.append("manifest_hash does not match canonical reproducibility manifest")
402     if manifest.get("risk_gate_hash") != sha256_file(risk_path):
403         errors.append("risk_gate_hash does not match risk_probe_summary.json")
404     if manifest.get("final_numbers_source_sha256") != expected_source_hash:
405         errors.append("manifest final_numbers_source_sha256 does not match final_numbers.json")
406
407     verify_hashes(RESULTS_DIR, manifest.get("core_result_hashes", {}), "result", errors)
408     verify_hashes(PROCESSED_DIR, manifest.get("processed_input_hashes", {}), "processed input", errors)
409     verify_hashes(REPO_ROOT, manifest.get("code_hashes", {}), "code", errors)
410     verify_environment(manifest, errors if args.strict_environment else environment_messages)
411     verify_source_commit_lineage(manifest, errors)
412     verify_key_result_shapes(errors)
413
414     probe_status = {probe.get("probe_id"): probe.get("status") for probe in risk.get("probes", [])}
415     required_probe_ids = {
416         "q4_method_risk_probe_gate",
417         "q4_probability_calibration_gate",
418         "q4_evidence_layer_guardrail",
419         "q4_sapr_scenario_no_leakage_gate",
420         "q4_sapr_macro_kernel_gate",
421         "q4_sapr_policy_identity_gate",
422         "q4_sapr_pareto_selection_gate",
423         "q4_sapr_holdout_validation_gate",
424         "q4_sapr_uncertainty_gate",
425         "q4_sapr_claim_strength_gate",
426     }
427     missing_probe_ids = sorted(required_probe_ids - set(probe_status))
428     if missing_probe_ids:
429         errors.append(f"risk_probe_summary.json lacks Q4 probe ids: {missing_probe_ids}")
430     failed_probe_ids = sorted(pid for pid in required_probe_ids if probe_status.get(pid) != "PASS")
431     if failed_probe_ids:
432         errors.append(f"Q4 probe ids are not PASS: {failed_probe_ids}")
433
434     if risk.get("paper_finalize_allowed") != (risk.get("overall_status") == "PASS"):
435         errors.append("risk paper_finalize_allowed is inconsistent with overall_status")
436     if manifest.get("paper_finalize_allowed") != risk.get("paper_finalize_allowed"):
437         errors.append("manifest paper_finalize_allowed differs from risk gate")
438     if manifest.get("overall_status") != risk.get("overall_status"):
439         errors.append("manifest overall_status differs from risk gate")
440
441     if errors:
442         print(json.dumps({"status": "FAIL", "errors": errors, "environment_warnings": environment_messages}, ensure_ascii=False, indent
      =2))
443         return 1
444     if args.require_final and not risk.get("paper_finalize_allowed"):
445         print(
446             json.dumps(
447                 {
448                     "status": risk.get("overall_status"),
449                     "paper_finalize_allowed": False,
450                     "blocking_probe_ids": risk.get("blocking_probe_ids", []),
451                     "environment_warnings": environment_messages,
452                 },
453                 ensure_ascii=False,
454                 indent=2,
455             )
```

```
        )
        return 2
    print(
        json.dumps(
            {
                "status": risk.get("overall_status"),
                "paper_finalize_allowed": risk.get("paper_finalize_allowed"),
                "environment_warnings": environment_messages,
            },
            ensure_ascii=False,
            indent=2,
        )
    )
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 14: verify_freeze.py

Listing 15: utils/__init__.py

附录 D 证据解释边界

- $ARBaselineGap$是战前AR规律的描述性差额，不是战争净因果贡献。
- Q2联合区间跨零时，不将点估计写成稳健总体增长损失；响应曲线面积不是累计损失。
- Q3跨国韧性总体为结论不充分；中国燃油序列是调价指数代理。
- Q3政策结果是局部动态代理情景；未经审计年度缓冲代理没有定量结论。
- Q4尾部概率是固定信息集下的条件概率；结构压力和政策反事实不可相加。
- SAPR只在注册规则族内最优；平均缺口月负担不等于财政支出或完整福利损失。